WebTV™ Network Operations and Maintenance Guide

Part Number 980-01003-A20

Copyright 1997 WebTV Networks, Incorporated. All rights reserved.

This material constitutes confidential and proprietary information of WebTV Networks Incorporated. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior written consent of the publisher.

WebTV and WebTV Network are trademarks of WebTV Networks, Inc.

This publication and the information herein is furnished AS IS, is subject to change without notice, and should not be construed as a commitment by WebTV Networks Incorporated. WebTV Networks Incorporated assumes no responsibility or liability for any errors or inaccuracies, makes no warranty of any kind (express, implied or statutory) with respect to this publication, and expressly disclaims any and all warranties of merchantability, fitness for particular purposes and noninfringement of third party right.

Contents

Preface

This Guide Contains .....

Conventions Used in This Guide .....

1 Overview

Duties of Customer Care.....

Duties of Service Operations.....

Duties of Network Systems .....

Delegation policy and schematic diagram.....

2 Staffing

Customer Care .....

Positions in Customer Care.....

Director of Customer Care..... 6

Customer Care Specialists..... 7

Customer Care Project Managers..... 7

Expansion Formula .....

Service Operations .....

Positions in the Service Operations Group .....

Director of Service Operations 8

IAP/ISP Manager..... 8

Toolsmith..... 8

Service Operations Specialists..... 8

Expansion Formula .....

Network Systems .....

Positions in the Network Systems group.....

Manager of Network Systems 9

Network Systems 9

3 Customer Care

Introduction 14
Organization 14
Location 14
Hours 14
Training 14
Facilities 15
Nature of Contacts 15
Contact Volume 15
Customer Support Duties 16
Customer Service Skills 16
Customer Questions 16
POP listings 16
Customer Requests 16
Customer Problems 16
Billing and Administrative Issues 17
Troubleshooting 17
Analyzing installation problems 17
Configuring for the Customer's Phone 18
General Problems with Connecting 18
Dialing Options 18
Audible Dialing 19
Workaround for Calling Cards, etc. 19
Connections from Phone Systems Using Access Codes 20
Analyzing Connection Problems 20
Normal Preregistration Connection Messages (Client 1.1) 21
Normal Connection Messages (Client 1.1) 21
Customer Fails to Connect to the Service At All -- Version 1.1 of the Client 21
Customer complains of an infinite loop while dialing 24
WebTV Networks ANI-Troubleshooting Protocols 25
What is ANI, When Is It Used, and Why? 25
Connectivity Schematic 25
Diagnosing Potential Problems 26
Analyzing error messages 28
Keyboard Problems 28
Keyboard Modifier Keys Stuck 28
Explanation and Additional Troubleshooting 29
IR Keyboard Shortcuts* 29
Remote Control 29
Surfwatch and Kidfriendly filter maintenance 30
Accounts Used by KidsProtection Administrators 30
Procedure 30
Step 1: All Customer Care agents 31
Step 2: KidsProtection Administrators (designated mid-tier agents) 31

Step 3: Surfwatch Customer Care Contact (designated mid-tier agents) 31
Step 4: Any Customer Care agent 31
Step 5: KidsProtection Administrators (designated mid-tier agents) 32
Interactions with problem users 32

4 Service Operations

Monitoring 34
Monitoring Production Unix Environment 34
Cabletron SPECTRUM 34
system activity reporter 34
Monitoring ISP Performance and Connectivity 34
Ping based tools 34
Using WebTV Network Log data to monitor Internet Connectivity 34
Monitoring IAP Performance and Connectivity 35
Verification of IAP Billing/Accounting Information 35
Using WebTV Network Logs to monitor IAP Performance 35
Using IAP-provided information 35
Using delegations from Customer Care 35
Monitoring the WebTV Network 35
Monitord 35
Maintenance 35
Maintenance of Production Unix Environment 35
System Backups 35
User Account Management 36
Unix Configuration Management 36
Maintenance of IAP and ISP Connectivity 36
Maintenance of WebTV Networks Software 36
Software Distribution 36
PhoneDB Maintenance 36
POP Balancing 37
Console Resets 37
WebTV Networks Configuration Maintenance 38
Troubleshooting 38
Troubleshooting: Production Unix Environment 38
Wedged servers 38
Full partitions 38
Rebooting after crashes 38
Correcting for overload 38
Console settings 38
Freeing Up Inodes on NetApps 39
Shutting down the service 39
Jumpstarting replacement machines 40
Troubleshooting: WebTV Network 40
Troubleshooting: IAP and ISP Connectivity 40

5 Network Systems

Database Administrator 41

Overview ... 41
Database Structure ... 45
WebTV Networks Database Components ... 46
Database Backups and Restores ... 46
Billing and Payment Verification Issues ... 46
Postmaster ... 46
Service Delivery Engineer ... 46
WebTV Network Delivery ... 47
Installing new flashrom images ... 50
Delivering New Content ... 50
Changing the WebTV Tricks password ... 51
System Administrator ... 51
Jumpstarting New Servers ... 51
Replacing Production Machines Without Downtime ... 51
Rebuilding Proxy Cache Disks ... 52

6 Tools

bulkmail ... 56
Usage ... 56
cat-event ... 58
Usage ... 58
clientinfo ... 60
Usage ... 60
clientpopedit ... 62
Overview ... 62
Usage ... 62
Extracting tellyscripts ... 63
CS-Ticket, the Service Request System ... 66
Overview ... 66
Workflow in the Service Request System ... 67
Escalations ... 69
Designing Applications of the Remedy Toolkit ... 69
Objects ... 69
Schemas ... 69
Customer Care Knowledge Base ... 70
Overview ... 70
Usage and Options ... 71
Command Overview ... 71
Commands in Detail ... 72
distill-log ... 73
dumpcrash ... 74
Etherpage/Pager software ... 75
Overview ... 75
The Control Script ... 76
Compressing ROM Images ... 77
Verifying ROM Images ... 78

Displaying Headers....78
Message Files....78
Other Features....79
Control Script Definitions....79
Harvester Statistics....81
Overview....81
Using idzify....81
Installation Tools....82
Scripts Overview....82
Primary Scripts....82
Primitives (InstallBuild)....82
Primitives (InstallContent)....82
Libraries and Classes....82
Host specification:....83
Other Notes....84
Mtree....84
InstallBuild....84
InstallContent....84
Misc....84
keytool....85
Overview....85
monitord Configuration....85
Monitord Message Analysis....86
Where Mail Goes....87
Messages from monitord Tests....87
Typical Alphanumeric Page Message....87
Typical Email Message....88
Log Messages from monitord Tests....88
Log Messages from Specific Tests....88
Monitord and External Content Partners....88
move-store....89
Overview....89
Normal Use....89
Options....89
Warnings....90
Example Use....91
PhoneDB Tools....92
Index....92
PhoneDB Overview....93
Useful Terms and Definitions....93
Tools - Summary and Syntax....93
lookuppop and showpop....94
popcoverage....94
popusage....94
listlocal....94
dumpnpas....95

exportnpas 95
dumppops 95
changepop 95
eainfo 95
distcompare 95
localreport 96
testlookup 96
balancereport 96
Special dumpnpas Fields 96
Raw PhoneDB Tools 98
Introduction 98
Index 99
V&H Database Overview 99
CCMI Database Overview 99
POP Files 99
Tools 101
rgenphonedb 102
rgenphonedb Error Messages 103
querydb 105
Real-Time Service Overview and overviewd 106
Service Logs and Log Analysis 107
How to Interpret a Typical Log Entry 109
The error message 109
The error type 109
The reporting function 109
The client id 109
The subscriber id 109
The reporting file and line number 109
The service 109
The pid (or process id) 109
Date and time 109
The flag 110
Usage of the log files 110
Service Manipulation Tools 110
check-servers 111
list-services 111
servicelauncher 111
start-personal-servers 111
start-servers 111
stop-one-server 111
stop-servers 111
switch-services 111
test-services 111
setdb 111
Status Report and statusd 112
tellygen 113

Usage....113
top 115
userStoreLockAndDo 116
userStoreName 116
WebTV Tricks Service 118

A M-Codes

Index of Error Messages (By M-Number) 120
Index of Error Messages (By Error Text) 122
Index of Error Messages (By Category of Error) 123
Normal Conditions 123
MessageWatch Errors 123
Physical Problems that the Customer Can Address 123
Sound Errors....123
The Publisher Didn't Serve a Page at All....123
The Publisher Didn't Serve the Page Normally....124
There's More Information About a Page the User Asked For 124
Unexpected Conditions that Need to Be Escalated to WebTV Networks, Inc. 124
Unexpected Conditions that the Customer Needs to Address....125
Error Codes 125
The M0 Errors....125
The M-2 Error -- Low Memory....125
The M-4 Error -- Host Not Found....126
The M-7 Errors -- Response Errors 126
The M-8 Error -- Cache Full....128
The M-9 Error -- Cannot Parse 128
The M-11 Error -- Unknown Service 129
The M-12 Error -- File Not Found....129
The M-14 Error -- Resource Not Found....129
The M-15 Error -- Page Not Found....129
The M-20 Error -- Timed Out 129
The M-21 Error -- Unknown Data Type....130
The M-22 Error -- Lost Connection 130
The M-23 Error -- Authorization Error 130
The M-25 Error -- Inactive Connection....130
The M-26 Error -- Generic Phone Error 131
The M-27 Error -- Phone Handshake Failure....131
The M-28 Error -- Phone No Answer 132
The M-29 Error -- Phone No Dialtone....132
The M-30 Error -- Phone Busy....133
The M-31 Error -- Phone Unplugged 133
The M-32 Error -- Power Off 133
The M-33 Error -- Endless Loop 133
The M-34 Error -- Too Many Redirects....135
The M-36 Error -- Call Waiting Disconnect 135
M-37 Error -- [***M-37 descriptor***]....135

M-38 Error -- [***M-38 descriptor***] 135
M-39 Error -- Compression Error 135
M-40 Error -- MessageWatch Failed 135
M-41 Error -- RealAudio Error 136
M-42 Error -- [***M-42 descriptor***] 136
M-43 Error -- [***M-43 descriptor***] 136
M-44 Error -- Inactivity Timeout Before Login Complete 136
M-45 Error -- Disconnection Before Login Complete 137
Other Errors 137

B The Service Log Files

Crashrecord.log 139
dumpcrash 140
Flashromd.log 141
Examples 141
Routine download entry (split by comments) 141
Asking for a version of the client that doesn't exist or can't be found 142
An example of a test log (uncommented) 144
Headwaiterd.log 147
Anatomy of a Log Entry 147
Typical Log Entry 147
Typical Errors and Analysis 149
Couldn't find the customer in the database 149
Caused by simulator 150
Expired ticket 150
Using client newer than current production client 151
Unexpected EOF 152
Simulator login record (uncommented) 152
Logserverd.log 155
Anatomy of a Typical Three-line Log Entry 155
Common Error Messages 155
Client error 155
Less common error—used code tounregister client 157
Anatomy of a "Simple" Phone Log Entry from Logserverd.log 157
Values for ResultCode field in a phone log 160
DisconnectionType value listing 161
Obtaining Client Information 162
Uncommented "Long" Phone Log Entry 164
Mailtod.log 169
Anatomy of a Typical Log Entry 170
An Example 170
Or 170
Anatomy of a More Complex Log Entry 170
Examples of "Safe" Warnings and Error Messages 171
A Not-So-Safe Warning 173
A Serious Problem 173

Preregistrationd.log 174
Anatomy of a Preregistration Log Entry 174
General Errors 174
Startup and its typical messages 174
Reclaiming idle file descriptors 175
Oracle errors 175
Server dying errors 176
A Typical Successful Preregistration Sequence 177
Discussion of individual parts 178
ANI and Associated Problems 180
No matching ANI server 180
“Bad” ANI 181
Internal user, probably using “xyzzy” 182
Dialup on local 415-329-0511 or equivalent 183
ANI server failing 184
Missing Tellyscript Fragment 185
Client Problems 185
POP Tours 186
Proxyd.log 187
Proxyd Log Entries 187
Typical user-events.log Entries 188
Descriptions of the fields of a typical log entry 189
Things to note when analyzing a log entry 191
Typical Error Examples 192
Proxy response codes and messages 193
Proxy 'Not Cached' reasons 194

C Choosing the Right POP

Dialing Patterns 196
Choosing the Right POP to Dial 196
Learning What to Do 197
Remembering What to Do 197
Deciding if the Client Needs a New Script 198
Script Descriptions 199
Access Number 200
Panic Mode 200
1-800 Addicts 201
Load Balancing 202

Glossary

Index 211

Preface

This book describes the maintenance and support of the WebTV Network. In addition, it outlines the suggested organization of the Operations group, the component of any replication of the WebTV Network that is assigned to maintain and support the service. The organization of this book is designed to reflect the division of support tasks among the three branches of the Operations group.

The goal of this book is to deliver a plan for staffing and organizing an Operations group, and then to provide a reference guide that permits the new Operations group access to the experience of the Operations group at WebTV Networks, Inc. in Palo Alto. While by no means all-encompassing, this book does include answers to a wide range of previously-encountered problems.

This guide is written for version 1.1 of the WebTV Internet terminal client, and version 1.3.5 of the Service. As the service and client continue to evolve, old problems will be resolved and new problems will doubtless arise to replace them. However, WebTV Networks will make every effort with each release to maintain compatibility with previous releases. As new releases affect the Operations group, this guide will be updated with information to best assist its users in their tasks.

This Guide Contains

Chapter 1, "Overview," describes the three branches of the Operations group, and defines how they interact in delegating problems.

Chapter 2, "Staffing," details the employees required to staff the three branches of the Operations group, including organizational charts and a by-position description of duties and ideal skills.

Chapter 3, "Customer Care," delves into the responsibilities and duties of Customer Care, the branch of the Operations group that deals directly with customer issues. In addition, it contains reference material designed to assist Customer Care agents in solving problems and answering questions.

Chapter 4, "Service Operations," defines the responsibilities and duties of Service Operations, the branch of the Operations group assigned to solve routine problems with the WebTV Network. It also contains reference material on solving some common challenges to the service.

Chapter 5, "Network Systems," contains a description of the activities typically performed by Network Systems, the branch of the Operations group responsible for solving unique problems with the service. Some of the typical tasks performed by Network Systems are described within this section.

Chapter 6, "Tools," describes the tools to operate, support, and monitor the WebTV Network. These include both internally-developed and 3rd-party packages. .

Appendix A, "M-Codes," contains a list of error codes that the box may display to a customer, and includes troubleshooting guidelines for understanding and solving the problems these error messages may represent.

Appendix B, "The Service Log Files," describes the log files produced by the WebTV Network, and offers additional information on analyzing them. This section also contains examples of real log entries, with explanations.

Appendix C, "Choosing the Right POP," describes the steps involved in sending the right tellyscript to the client. This includes assigning the least expensive POP, finding the right dialing pattern, and details on how load balancing is achieved.

Conventions Used in This Guide

The following typographical conventions are used in this book:

Typeface	Description	Example
`AaBbCcDd`	The names of tags, attributes, directories, on-screen computer output	The WebTV interface implements the `<!-->` comment tag.
AaBbCcDd	Placeholder: replace with a real name or value	`<a href="`*attribute*`">`
AaBbCcDd	Book titles, new words or terms, or words to be emphasized	Using the *WebTV Internet terminal*, consumers...

Chapter 1

Overview

This chapter provides an overview of the Operations group at WebTV Networks, Inc., and how its subgroups interact in delegating problems. The following topics are covered:

- Duties of Customer Care
- Duties of Network Systems
- Delegation policy and schematic diagram
- Duties of Service Operations

Duties of the Operations Group

The overall duty of the Operations Group is to ensure the successful operation of the WebTV Network. This includes both maintaining machines and software, and ensuring customer satisfaction. In many cases, the first notification of a problem with the WebTV Network will come from customer contact; it is therefore essential that all three components of the WebTV Networks Operations Group function together to delegate and solve problems appropriately.

Duties of Customer Care

Customer Care functions as the front line of the WebTV Network. WebTV Networks, as a high-quality product, experiences relatively few problems, and therefore Customer Care's primary duty is to help customers learn to use the WebTV Internet terminal and Network. This group fields questions, requests, and problems, acting as the customers' voice within the corporate structure. In addition, Customer Care handles problem customers who violate the terms of service, and tests and suggests additions to the parental access control filters.

Duties of Service Operations

The Service Operations Center is chartered with managing and maintaining the quality of the WebTV Network on a day-to-day basis. This group performs proactively, monitoring to identify problems before they impact customers, correcting routine problems as quickly as possible, and delegating non-routine problems to those who can resolve them.

Duties of Network Systems

Network Systems is a high-level group specializing in addressing unique problems with the WebTV Network. They work closely with the Internet service providers to ensure smooth service performance, and deal with serious technical difficulties. Ops Engineering is also in charge of distributing service and content changes, and takes the lead role in expanding the service over time. Ops Engineering will work closely with their counterparts in other locations to ensure that serious problems are communicated and solved promptly.

Delegation policy and schematic diagram

The WebTV Networks Operations Group functions on a delegation model, wherein the responsibility for problems is retained by the first agent to encounter the problem. Customer Care is responsible for concerns originating from the customer base, and may delegate to any component of the Operations Group, as well as a number of components of the development and support teams at WebTV Networks, Inc. in Palo Alto. Service Operations is responsible for concerns originating from the hardware and software infrastructure of the WebTV Network, and may delegate to the same groups as Customer Care. In addition, Service Operations may also receive delegations from Customer Care. Network Systems will generally receive delegated problems from Service Operations, and may further delegate issues as necessary to ensure successful resolution.

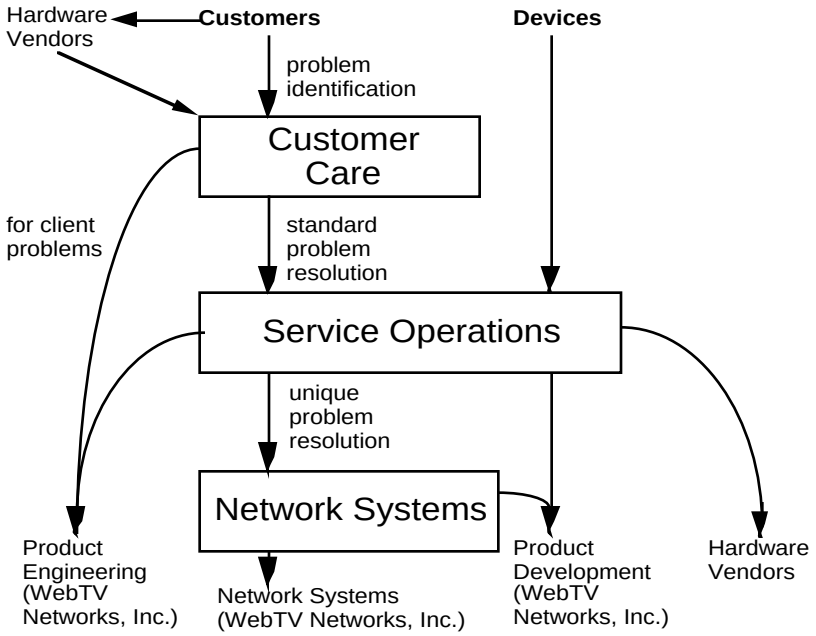

Figure 1-1 Schematic for Operations Group Delegations

Typically, problems relating to the operation of the WebTV Network and supporting hardware and software can be delegated to Ops Engineering at WebTV Networks, Inc. in Palo Alto. Requests for new features can be delegated to the Product Development group at WebTV Networks, Inc. in Palo Alto, while reports of bugs and problems with existing services should be delegated to the Product Engineering group at WebTV Networks, Inc. in Palo Alto.

Lastly, problems with the hardware itself should be redirected to the appropriate hardware vendor.

Chapter 2

Staffing

This chapter describes the organization and staffing needs of the three components of the Operations group. Each section also includes a description of the skills and duties of each type of employee.

This chapter covers the following topics:

- Operations Group Organization
- Customer Care
- Service Operations
- Network Systems

Operations Group Organization

The overall direction of the Operations Group should be determined to best fit the rest of the corporate model. This may mean that the Operations Group directors report directly to the Chief Operations Officer, that one of the directors is in charge of managing the entire Operations Group as well as a branch of the group, that an additional employee exists to fill the position of Coordinator of the Operations Group, or some other organizational scheme.

At WebTV Networks, the Director of Service Operations reports to the Vice President of Network Systems, who also serves as the Director of Network Systems. Both the Director of Network Systems and the Director of Customer Care then report to the Chief Operations Officer. See Figure 2-1 for the WebTV Networks organizational chart.

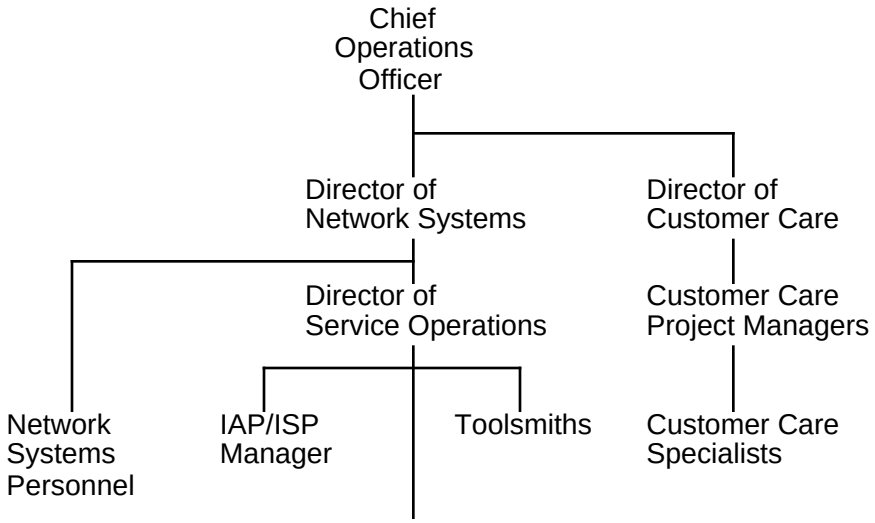

Figure 2-1 Organizational chart of the Operations group.

Customer Care

At WebTV Networks, Inc., customer service is provided by an organization centered on meeting the needs of the customers. As such, the Customer Care plan at WebTV utilizes a three-tier organization capable of expanding with a growing customer base.

Positions in Customer Care

Director of Customer Care

Duties The Director of Customer Care is a single individual tasked with leading the Customer Care department. He or she will have the ultimate say in the expansion and hiring policies of Customer Care, and will be responsible for organizing the Customer Care group as a whole. The Director of Customer Care possesses the authority to refund customers for unexpected charges or problems with the service, and the authority to transfer customers from one service group to another. The Director of Customer Care should also have the final decision on the release of new software to the service and clients. Because the group functions as the first line of contact, the Director needs to be certain that his group is prepared to address all contacts about the new release before it goes out to customers. Additionally, the director must be prepared to take a lead role in addressing the activities of problem users.

The Director of Customer Care should be prepared to be on-call during all hours of Customer Care Center operation to deal with problems and policy decisions.

Skills The job requires prior management experience, preferably within a customer service organization. Experience with customer support for a technical product is a must.

Customer Care Specialists

Duties Front-line Customer Care specialists handle primary contact with customers. They answer email, phones, and voice mail, and use the CS-Ticket service request system (see "Service Manipulation Tools" on page 110) to track problems and record solutions. In addition, they have access to the Customer Master Record tool (see "Customer Master Record tool" on page 70), and can use it to make changes to customer records in the customer database. Customer Care specialists perform shift work, generally for eight hours per day.

Skills At WebTV Networks, Inc., the front-line Customer Care staff consists of junior agents, generally without a specific technical background. They are hired based on both experience in customer support and natural tendencies towards assisting others. While phone support experience is useful, any experience in customer service is considered to be of value, including background in sales, receptionist work, and other varieties of retail assistance.

Customer Care Project Managers

Duties Mid-tier agents handle administration of Customer Care problems, as well as technical areas including maintenance of the CS-ticket service request system (see "Service Manipulation Tools" on page 110) and the Customer Master Record tool (see "Customer Master Record tool" on page 70). They handle the KidsProtection Administration, maintaining and checking the content restriction filters (see "Surfwatch and Kidfriendly filter maintenance" on page 30). At WebTV Networks, Inc., there is approximately one mid-tier agent for every five front-line agents. Customer Care Project Managers need to be on-call throughout Customer Care Center hours to handle problems and policy decisions.

Skills Qualified mid-tier agents should have some experience with technical support or project management, but may still be relatively junior within their fields.

Expansion Formula

The following estimates of work for Customer Care specialists are based on fully trained agents with a good knowledge base, fully-deployed and functional tools, and access to the rest of the resources outlined in this document. When provided with less optimal work conditions, agents will work at a lower level of efficiency. In a normal eight-hour shift, a Customer Care specialist can typically handle 70 phone calls, 70 email responses, or 35 voice mail responses. The Customer Care Center should be staffed with sufficient specialists to meet the contact volume over the duration of coverage each day.

For every five Customer Care specialists, one Customer Care Project Manager should be hired.

Service Operations

While Service Operations as a group deals with service problems rather than customers, the group's role as a first level of delegation from Customer Care requires it to be able to mirror or exceed Customer Care's hours. Ultimately, Service Operations should be manned 24 hours, seven days a week. All WebTV Networks Operations employees should have a broad-based technical background, and good problem-solving ability.

Positions in the Service Operations Group

Director of Service Operations

Duties The Directory of Service Operations is responsible for managing the Service Operations group and working closely alongside the Director of Customer Care and the Manager of Network Systems to coordinate the Operations group. He will be responsible for hiring personnel, organizing staffing schedules, and ensuring that the Service Operations group provides the customers with a high-quality experience.

Skills [Information to be provided in a future release of this document]

IAP/ISP Manager

Duties The IAP/ISP manager is responsible for developing and maintaining lines of communication with the ISPs and IAPs associated with the WebTV Network. He will need to work independently to develop and deploy new business associations, and be capable of providing the necessary technical information to potential partners to ensure compatibility of service.

Skills [Information to be provided in a future release of this document]

Toolsmith

Duties WebTV Networks, Inc. employs a number of tool developers to create specific applications for the support of the Service Operations group in particular and the Operations group as a whole. These individuals develop software solutions to specific tasks frequently performed by the Service Operations group. Toolsmiths may or may not be necessary in a replication of the WebTV Network.

Skills Toolsmiths should be considered for their experience with software development, and for their ability to handle projects with durations in the range of one or two weeks.

Service Operations Specialists

Duties The front-line agents of the Service Operations group, these employees cover the problems with the service that arise over normal operations. They should have a background in UNIX system administration.

Skills [Information to be provided in a future release of this document]

Expansion Formula

One Director of Service Operations and one IAP/ISP Manager should be designated to handle management and project coordination responsibilities.

To provide coverage twenty-four hours a day, seven days a week (24 x 7), at least nine Service Operations Specialists are suggested. Depending upon the specific needs of the replication of the WebTV Network, this number may need to be increased. If the Service Operations group will provide less than 24x7 coverage, then this number can be decreased.

Network Systems

Positions in the Network Systems group

Manager of Network Systems

Duties In addition to serving as the manager of Network Systems, this individual should be prepared to take a leadership role in coordinating the activities of all three branches of the Operations group.

Skills The Manager of Network Systems should have several years of experience managing a data center.

Network Systems

Unlike the Service Operations and Customer Care groups, Network Systems's makeup does not depend as directly upon being able staff positions throughout normal business hours or support hours. Because Network Systems deals primarily with unique problems or with specific technical activities, its staffing needs deal more with filling areas of expertise than with covering a range of operating hours.

Therefore, at WebTV Networks, Inc., the Network Systems group is composed of sufficient people to cover a number of critical areas. For a replication of the WebTV Network, these duties should be divided up among no less than four people.

Depending upon the resources available and the technical needs of the replication of the service, it may even be possible to combine the Service Operations and Network Systems groups into one unit charged with maintaining the WebTV Network software and hardware.

Database Administration The WebTV Network makes extensive use of Oracle databases for the purposes of maintaining customer records. An individual with 2-5 years experience as an Oracle database administrator is therefore essential. A candidate for this position should have experience with Oracle version 7.1 or higher, as well as experience operating Oracle in a UNIX environment. WebTV uses the development tools "Pro C" and "Developer 2000" to customize tools for accessing the database, so experience with these two toolkits is highly desirable. Finally, perl experience is a plus. Because the billing system and credit card verification also run through an Oracle database, a familiarity with billing systems and their communications with databases is also suggested.

WebTV Networks, Inc. can then provide further training to the individual on the database structures and procedures used by the WebTV Network. The visitor should then be prepared to function as the Database Administrator for the trial service, and possibly also for the production service.

Network Administration The WebTV Network relies on a number of internal networks, all of which have to communicate with each other, and ultimately to the client. The Network Systems Network Administrators should be familiar with routing and routers, SNMP monitoring, internal and external network functions, and standard LAN/WAN functions. Specific experience with the Cisco 7200 and Cisco 2514 routers and Cisco 1900 switches would be preferred. An employee with experience maintaining radius servers or other user authentication services

is strongly recommended. Depending on how preregistration is structured, and whether a fallover POP is to be made available, someone with expertise in current modem technology becomes another requirement.

Postmaster The Postmaster should know the relevant RFCs, sendmail version 8 or later, SMTP, MIME, UUCP, TCP/IP networking, C, PERL, and UNIX shell tools, and basic UNIX/Solaris 2.5 system administration. Knowledge of other mail systems like qmail and smail would also be useful. A background in security will be useful for identifying and dealing with mail-related security issues. They should be familiar with creating and maintaining mailing lists. They will handle determining a response to mailbombing incidents, and will work with the Director of Customer Care to handle reports of problem users. Furthermore, they should be prepared to deal with or delegate user concerns raised in electronic mail, including defining a way to divide or track inquiries to the postmaster.

Production Support Toolsmith WebTV Networks, Inc. employs a Production Support toolsmith responsible for managing working with a large number of service machines. His duties include developing new scripts and tools to ease the challenges of operating a large service. Because many or all of the tools produced by WebTV Networks, Inc. will be included with distributed service improvements, it may not be necessary to fill this position for replications of the WebTV Network.

Remedy Manager WebTV Networks, Inc. uses the Remedy Toolkit to build and maintain a number of service request systems, including one used by Customer Care. Because this problem tracking application is intimately tied to an Oracle forms application used by Customer Care to modify and view customer records, WebTV Networks needs to determine what sort of customizations to the service request system will be required for any external localization of the service. Additionally, because WebTV Networks, Inc. uses the development tools “Pro C” and “Developer 2000” to customize tools for accessing the database, experience with these two toolkits is highly desirable.

WebTV Networks, Inc. uses the Remedy toolkit to build and maintain a number of service request systems, including the one used by Customer Care and one used to track internal support requests. In that any replication of the WebTV Network is likely to use some sort of service request system for problem tracking, access to a knowledgeable developer of the underlying system is strongly recommended.

Security Administrator WebTV Networks, Inc. strongly suggests designating a single individual to monitor possible security holes in the service. The individual bearing this duty should have a broad familiarity with network security in general, and should be prepared to identify and implement patches and software packages as necessary to maintain the system’s integrity.

Service Delivery Engineer The Service Delivery Engineer is principally responsible for ensuring that changes to the WebTV Network binaries and source are correctly rolled out into the Production service. Binaries for the service itself will be provided by WebTV Networks, Inc.; content may originate with either WebTV Networks, Inc. or with the partnering venture.

The Service Delivery Engineer should have 2 or more years of experience as a UNIX system administrator, preferably with prior experience with Solaris. They should have some experience with managing software distribution to a large number of machines on a network, and a firm understanding of Domain Name

Service (DNS). They should possess sufficient english language skills to be able to contact WebTV Networks, Inc. for assistance when required. Any candidate for the Service Delivery Engineer position should be a good self-starter, able to work independently and at flexible hours.

System Administration The WebTV Network requires the support of a number of UNIX system administrators. Familiarity with the Solaris 2.5.1 operating system is absolutely essential. Furthermore, experience with Sun Ultra-2 and Ultra-170 machines is highly desirable. UNIX system administrators should have had prior experience installing software in a network environment, and understand machine set up and configuration. They should have experience in the maintenance and operation of dedicated external disk storage, including an understanding of SunSwift disk controller cards or their equivalent. WebTV Networks, Inc. uses NFS file servers made by the Network Appliance Corporation, so an understanding of these specific machines or at least NFS file servers in general is desirable.

Typical duties would include jump starting machines (for addition to the service, or as replacements for failed machines), diagnosis and replacement of server hardware. Additionally, the ability to “watchdog” the operating system and support software is necessary, as well as a familiarity with issues in system security.

WebTV Networks, Inc. can optionally provide training in the installation of the WebTV Network, including the initial jump starting of machines, providing the necessary skills to construct another localization of the service. Additionally, WebTV Networks, Inc. can train system administrators in monitoring and troubleshooting techniques for the WebTV Network software.

Telecommunications Engineer Telecommunications engineers should have a strong background in communications between IAPs and services, including a familiarity with network-telecommunications interfaces, radius authentication, and alternate methods of user authentication. Furthermore, they should have background in the installation and maintenance of modem banks and other telecommunications hardware, because the WebTV Network depends upon this telecommunications hardware to communicate with the customer. Specific experience in US Robotics “Total Control 16” racks configured with PRI NetServer Gateway cards would be preferred.

Two main types of telecommunications hardware are required:

- Preregistration telecommunications
- IAP telecommunications

1. Preregistration telecommunications

   The WebTV Network attempts to identify new customers and provide them with appropriate dialing scripts through an initial call to a preregistration number. In the United States, this number is a toll-free eight hundred number. In another localization of the service, it may be a number that connects to modem banks provided either by an IAP or by the partnering venture.

2. IAP telecommunications

   Customers normally connect to the WebTV Network through local IAP phone numbers. To negotiate authentication protocols and the interactions between the WebTV Network and the IAP, the Network Systems group requires an IAP telecommunications expert.

Webmaster The Webmaster should be familiar with compiling and maintaining HTTP servers, and have a strong background in supporting a web site. Their duties will include maintaining any internal web content, and may extend to handling any non-service corporate web content.

Chapter 3

Customer Care

This chapter describes the Customer Care organization of WebTV Networks, Inc. In addition, it contains reference material designed to assist Customer Care agents in solving problems and answering questions. The following topics are covered:

- Introduction
- Organization
- Customer Support Duties
- Troubleshooting
- Surfwatch and Kidfriendly filter maintenance
- Interactions with problem users

Introduction

Customer Care functions as the front line of the WebTV Network. The service and client, as high-quality products, experience relatively few problems, and therefore Customer Care's primary duty is to help customers learn to use the WebTV Internet terminal and service. This group fields questions, requests, and problems, acting as the customer's voice within the corporate structure. In addition, Customer Care handles problem customers who violate the Terms of Service agreement, and tests and suggests additions to the parental access control filters.

Under the arrangements WebTV Networks, Inc. has developed with its licensing partners, each licensee is responsible for providing customer service for problems between the purchase and connection of the WebTV Internet terminal to the WebTV Network. Once the customer has physically installed the WebTV Internet terminal, WebTV Networks Customer Care accepts responsibility for handling problems with connecting to the service.

Organization

Location

At WebTV Networks, Inc., the Customer Care Center is co-localized with the Service Operations and Network Systems groups. Because the support groups are physically near to each other, they are able to share information and quickly alert each other to problems. For full benefit, the Customer Care Center should either be co-localized with the rest of Operations, or failing that, be established with strong lines of communication to the other Operations groups.

Hours

The Customer Care organization serves a customer population stretching across more than three time zones. Despite the widely-dispersed customer population, WebTV Networks, Inc. does not currently believe that it is necessary to provide 7x24 Customer Care coverage. On weekdays, the Customer Care Center is staffed from 8 am in the earliest time zone the WebTV Network serves, to 11 pm in the latest time zone with a large user population. On weekends, the center is staffed from 9 am in the easternmost time zone, to 9 pm in the westernmost. While specific staffing needs will vary with the area of localization, the service's usage patterns lend themselves to an 8 am to 11 pm coverage pattern on weekdays, and a 9 am to 9 pm coverage pattern on weekends.

When Customer Care is closed, WebTV Networks provides a voice mail service allowing customers to leave messages. Customer Care then attempts to return the customers' calls the following day.

Training

Customer Care Specialists can typically be trained to be productive at answering phone calls and email within a week of their start date. This training includes a basic familiarity with the WebTV Internet terminal, the WebTV Network, and the tools and materials available to Customer Care.

Customer Care Project Managers should also receive a week's worth of initial training on the product and support environment before being directed into specific projects and general mid-tier functions.

Facilities

Each Customer Care Specialist needs to be provided with a personal computer with access to the Customer Care Knowledge Base, the Service Request System, and the CMR tool. Each agent should have shared access to a WebTV Internet terminal and a television for testing solutions. Every agent answering phone calls will also need a telephone. Because Customer Care Specialists perform shift work, it is possible to share a limited number of work stations among different Customer Care Specialists.

Customer Care Project Managers will need access to the same resources as Customer Care Specialists. However, each should have his own personal work space, separate from the rotating Customer Care Specialist work spaces. Additionally, Customer Care Project Managers should also be provided with cellular phones or pagers to allow them to be reached as necessary.

Nature of Contacts

As a high-quality product, WebTV Networks' products experience relatively few problems after a subscriber successfully connects the first time. Because of this quality level, the typical make up of WebTV Networks Customer Care contacts is:

75% questions

10% requests

15% problems

**% billing issues

Of the problems, approximately 95% relate to the initial connection of the WebTV Internet terminal to the WebTV Network. Many of these problems are caused by the environment on the customer's side, requiring Customer Care agents with good problem-solving skills.

Generally, customers will call when they first purchase the terminal, and then again as they learn how to use it. Some customers tend to use the Customer Care Center as a resource for learning about the Internet. WebTV Networks, Inc. is currently considering the possibility of a revenue-providing Internet consulting service.

Contact Volume

Currently, Customer Care at WebTV Networks handles customers through three mediums: phone, email, and voicemail. Of the three media, Customer Care receives the majority of contacts through phone calls, followed closely by email. Voice mail is much less heavily-utilized by those who contact Customer Care.

WebTV Networks Customer Care handles phone calls through an 800 number. The typical phone call takes approximately four minutes. Currently, WebTV Networks, Inc. handles over 6000 phone calls per week. Optimally, one Customer Care agent staffing the phone lines for an eight-hour shift can handle approximately 70 phone calls in a single day.

An increasing component of WebTV Networks Customer Care contact comes through email. As of January of 1997, WebTV Networks, Inc. handles approximately 4500 pieces of email per week, a number that is steadily increasing over time. Email is tracked using a Service Request System (see "Service Manipulation Tools" on page 110). WebTV uses a service request system

developed using the Remedy toolkit. Optimally, one Customer Care agent can process one piece of email in six minutes, and approximately 70 pieces of email in an eight-hour work day.

Voice mail volume is much lower; WebTV Networks, Inc. currently requires approximately one man-hour of work for every five calls received in a day.

Customer Support Duties

Customer Service Skills

Note: Information to be provided in a future release of this document.

Customer Questions

At WebTV Networks, Inc., the principal function of Customer Care has proven to be educating customers and potential customers about the WebTV Networks product. Over 75% of contacts to Customer Care can be defined as questions about WebTV Networks, the service, or the client. To answer these questions, WebTV Networks, Inc. uses a set of internal web pages as a knowledge base, supplemented by educational talks given by the development team. Much of the technical information in these classes has been included in this book and its companion volumes. By familiarizing themselves with corporate, service, and client function at a basic level, all Customer Care specialists will quickly gain the ability to answer the majority of customer questions.

POP listings

At WebTV Networks, Inc., Customer Care agents are frequently called upon to address questions about local phone coverage. Customer Care agents use phonetool, described in "PhoneDB Tools" on page 92, to determine if callers can contact the WebTV Network through local or toll calls. Depending upon the area of localization of the WebTV Network, phonetool or an equivalent tool may or may not be available. Additionally, localized tools will be required to determine if any particular call in another localization of the service will be toll or local.

For areas where local coverage is not complete and the localized corporation is interested in expanding coverage using regional providers, Customer Care should be provided with a list of internal contact information on the employees handling negotiations with new regional providers.

Customer Requests

One of Customer Care's most important functions is to serve as the customer's voice into the corporation. WebTV Networks, Inc. prides itself on its ability to address customer suggestions for improvements to the WebTV Internet terminal and WebTV Network. Customer Care specialists should track all customer suggestions for improvements, escalating frequently-requested or logical suggestions to WebTV Networks, Inc. for potential implementation.

Customer Problems

Some customers do encounter problems either during the initial set-up of the device, or at some point after initial connection. Many of the most common problems Customer Care specialists will be faced with are addressed in "Troubleshooting" on page 17.

Billing and Administrative Issues

Customer Care currently handles addressing billing issues, including requesting replacement credit card numbers, and communicating with customers with billing problems. Additionally, Customer Care also deals with administrative issues, including cancelling accounts and transferring users from one terminal to another.

Troubleshooting

The WebTV Network is designed to be reliable and easy for the customer to use. However, like any technical service, problems can arise from a variety of sources. This section is designed to help diagnose many of the problems customers are likely to report, and offer assistance in finding solutions for problems that do not need to be delegated to Service Operations. Failing that, much of this information is intended to be useful in narrowing down the scope of problems to better assist Service Operations personnel in solving problems.

Analyzing installation problems

Some portion of customers will encounter problems with the initial connection of the WebTV Internet terminal. Frequently, these problems are simply a question of not choosing the proper video input channel on the TV. At WebTV Networks, Inc., the Internet terminal distributors handle the majority of installation issues. Nevertheless, Customer Care agents should be prepared to answer questions on installation.

To function, the WebTV Internet terminal and the attached television need to be installed properly. This includes checking to make certain that both devices are plugged in and connected to each other, that the WebTV Internet terminal is connected to a live analog phone line, and that the television is set to display video input. While some of these problems may seem very obvious, it never hurts to check the basics. Some possible problems include:

- The TV and/or the terminal won't turn on when the customer presses POWER.

First, ask the customer if the green POWER light is on to determine if the terminal is powering on at all. If it is, go to “The POWER light comes on when the customer turns the terminal on, but they can't see it actually doing anything:” on page 17. Otherwise, check to make certain that both devices are plugged in. If they are using a remote, make certain that the batteries are correctly installed. If they have a Sony terminal, their universal remote control may be misconfigured -- see the manuals for how to set up the universal remote. If they have a Philips terminal, the remote can't turn on their TV -- if they had a friend with a Sony terminal, they may be unhappy about this.

- The POWER light comes on when the customer turns the terminal on, but they can't see it actually doing anything:

First, make certain that the WebTV Internet terminal is connected to the television. The manuals provided with the Internet terminal contain instructions on connecting it to a television, as well as to a television with a VCR. All Customer Care agents should make themselves familiar with these manuals and have them on hand for customers who need to be walked through them. If the terminal and TV appear to be properly connected, then the television may not be on the right channel or setting to display video input. Customers with older

televisions may need to turn the television to a specific channel. In some cases, the channel the terminal sends its video to may be very obscure, requiring the customer to flip through many channels. Fortunately, instances of odd channels being the destination of the terminal's video are rare. For newer TV's, the customer will probably need to press the TV/Video button until the “Connecting to WebTV” Road screen comes up. If the customer has a VCR or other electronics connected to their television, these may interfere with the terminal's video -- the customer should probably check his manuals to ensure that the device is properly installed.

Configuring for the Customer's Phone

General Problems with Connecting

If the customer gets the “Connecting to WebTV” Road screen, but cannot connect to the service itself, generally they will get some sort of error message in a grey box.

The most frequent message will probably be: “The phone line seems to be in use or disconnected. Check the line and try again.” The customer needs to check that the terminal is plugged into a phone line, and that the phone line is live. The best way to test a phone jack is to plug in a telephone and listen for a dial tone. If the phone line is live and plugged into the terminal, then the specific configuration of the customer's phone may be the cause of their problem. Most of these problems can be solved by changing the terminal's *Dialing Options.*

Dialing Options

There are two possible ways to get a customer to the *Dialing Options* menu. The fastest way uses secret code 217 to call up the Dialing Options screen when the Internet terminal is next turned on. See“WebTV Internet Terminal Secret Codes” on page 116 for more details on using secret codes. Alternately, ask the customer to turn the terminal off, unplug the phone line, and then turn it on. They can always do this while talking to a Customer Care agent, since it does not require the terminal to call out. They should get a grey box with a button labeled Dialing Options. Tell them to choose this button. For new users, keep in mind that a Customer Care agent may have to explain how to use the remote control to get from one button to another. Once they get to Dialing Options, be sure to remind them that they should plug the phone jack back in!

Ask the customer about the phone they are using. If any of the following apply, they may need to go to Dialing Options and change their settings.

If they need to dial a number to call outside lines (for example, dialing 9 before dialing out), they need to enter that number on the “Basic dialing options” page.

If their phone only allows pulse dialing, they need to choose the “pulse dialing” check box on the “Basic dialing options” page, filling it with a check mark.

If they have voice mail on the line, it may produce an odd tone that interferes with dialing. They should remove the check mark from the “Wait for a dial tone” box on the “Advanced dialing options,” removing the check mark. Then, they should add two commas in the “Prefix:” box on “Basic dialing options” page.

If they have recently moved the terminal (or somehow bought a demo terminal from a store), they may need to go into Dialing Options and choose the “Reset” button. This will clear any odd settings, taking them back to the factory default settings.

If they have a toll call as their closest POP, and have to use a calling card or similar manner to get access (or any other reason requiring such), and that series of digits (including the actual POP number) is greater than 27 digits, see the "Workaround for Calling Cards, etc." on page 19. If the total is under 27 digits (including commas for pauses), just have them enter it all in the "Access Number" field in the dialing options.

Currently, WebTV Internet terminals do not work on digital lines, and ultimately this may be the problem. If the customer has a fax line or a dedicated modem line, they may wish to use that line instead, at least to test the Internet terminal.

Audible Dialing

Turning on audible dialing is one of the first tests an agent should use to diagnose a connection problem. A Customer Care agent may want to suggest that the customer turn on "Audible Dialing" under "Advanced Dialing Options" if the customer has persistent problems obtaining a connection. By listening to the call's progress and watching the status bar message, the customer may be able to tell if the terminal is calling numbers with problems. For example, if the preregistration number or the customer's local POP is down, then the terminal will fail to make modem noises after making that call, and the green bar will retreat in length as the Internet terminal falls back to another POP. Note that this method is best employed when the customer has two phone lines, so he can have the terminal attempt a connection while reading status bar messages to the Customer Care agent, describing the audible dialing noises.

Workaround for Calling Cards, etc.

For users who have to dial more than 27 digits and thus can't enter the phone number and whatever else they have to dial in the normal access code field under dialing options.

- In Dialing Options, under the Advanced Options menu, remove the requirement to "Wait for Dialtone" and set "Audible Dialing" on.
- Most times, if they require a calling card or similar, it won't be a local call, and hence, the toll screen will come up. This is the easier case.
- Set the terminal up to connect, and it should flash the toll warning message.
- Once it shows this message, dial in everything, including the POP number (i.e., do all the dialing manually, instead of the terminal).
- Just after finishing this, click on "continue" (or whatever the appropriate button is to acknowledge the warning and dial anyway).
- The extra digits the terminal dials on the end shouldn't make any difference. If they cause a problem, enter a false access number (a single digit is a good idea here, and even a comma will do).
- The terminal should connect properly this way.

Caution:Don't forget to hang up the phone, once the terminal has started its own dialing sequence! It may require a little experimentation to get the timing on this correct!

- If it isn't a local call, there are three solutions to the problem.

- Use “clientpopedit” (see “clientpopedit” on page 62) to send down a tellyscript that forces the terminal to think it's dialing what might be a toll number. Then follow directions as above.
- Go to dialing options every time before trying to connect (secret code 217). See“WebTV Internet Terminal Secret Codes” on page 116 for more details on secret codes. Dial in everything needed to connect, including phone number and calling card numbers, and then click on the “Connect” button at the bottom of the screen. Follow directions as above.
- Dial in everything (as usual), and turn on the terminal just as you finish dialing. Timing here may be somewhat tricky and require a little bit of experimentation, both for turning the terminal on at the correct time, and for hanging up the phone. You still have to set it up to not wait for dialtone, and audible dialing is still a good idea.

Connections from Phone Systems Using Access Codes

The following process details how to handle phone systems that require the user to input an access code of some sort when making calls outside of the local system. This includes US military bases, many colleges, and probably a number of other systems.

- Find out their phone number and look up their POP's in Pop-O-Rama.
- Get to the dialing setup screen.
- Under the Basic Dialing screen, put in whatever dialing prefix is required to get to an outside number (usually 9). It’s a good idea to put a comma after the dialing prefix, which inserts a pause before the Internet terminal dials the access number (see below).
- Then in the access number, enter the number for preregistration.
- Power on the terminal and watch it call. When it gets to “Preparing to Call” the second time, have the customer power the terminal off. If they miss that and get to the screen that says their Internet terminal needs to be reset, they should power off instead of pressing the “Continue” button.
- With the terminal powered off, enter 217 and power up to get to the dialing setup screen. Go to the access number screen and delete the preregistration number, replacing it with:

<The number for their first POP><Their access code>

This should do the trick. Have them power the terminal back up, it should connect normally now. Of course, moving the terminal off the system is a problem to be aware of in the future. It may be necessary to explain to the customer how to reset their dialing options. See “Dialing Options” on page 18 for more details on resetting dialing options to the factory default settings.

Analyzing Connection Problems

There are a number of points in the WebTV Network where connection errors may occur. Each phone line, each provider, and each link between machines on the WebTV Network is a potential point where connectivity or information can be lost. Since the heart of the WebTV Network is information transmission, connectivity problems are often the most potentially damaging and thus the most serious in nature.

A normal connection to the WebTV Network can begin with a connection to the preregistration server, or may proceed directly to the WebTV Network. Depending on whether or not preregistration is contacted, the messages displayed on the status bar near the bottom of the screen while the terminal is attempting to connect to the WebTV Network will vary.

Normal Preregistration Connection Messages (Client 1.1)

When preregistration is contacted before connection to the WebTV Network, the following messages are sequentially displayed:

- Preparing to Call -- *May appear briefly or not at all*
- Dialing Toll-free
- Waiting for Answer
- WebTV Answered
- Starting Communication
- Connected to WebTV
- Preparing to Call -- *May appear briefly or not at all*
- Dialing WebTV
- Waiting to Connect
- WebTV Answering
- WebTV Answered -- *May not appear*
- Connecting
- Connected to WebTV
- [status bar gone]

Normal Connection Messages (Client 1.1)

When the WebTV Network is contacted directly, the following messages are displayed:

- Preparing to Call -- *May appear only briefly or not at all*
- Dialing WebTV
- Waiting to Connect
- WebTV Answering
- WebTV Answered -- *May not appear*
- Connecting
- Connected to WebTV
- [status bar gone]

Customer Fails to Connect to the Service At All -- Version 1.1 of the Client

If the client is unable to connect to the service, the messages the customer sees near the bottom of the screen while the terminal is attempting to connect to the WebTV Network are often the key to determining where connectivity has been lost. By querying a customer as to the last message they observed during the connection process, it is frequently possible to gain additional insight into a problem and thereby find a solution.

The messages that the customer may see, along with the problems that may induce failure at that stage of the connection process, are listed below.

Connected to WebTV This message indicates that the terminal has successfully negotiated PPP, and is in the process of opening its first TCP connection. It should appear during normal usage, or twice during the pre-registration procedure. It does not appear during a reconnection.

Actions in Response to a Failure at this Stage: The terminal has successfully connected with the WebTV Network, received the login prompt, and successfully logged in. It is likely failing to make the first TCP connection with the service. This suggests a problem with the service, and should be delegated to Service Operations for further investigation.

Connecting This message indicates that the client has received the login prompt, and is about to login.

Actions in Response to a Failure at this Stage: The terminal has negotiated the normal handshake sequence, and is failing to login. Either the "shared secret" has been corrupted, or something similar has happened which is not allowing the client to login properly. It also may be having problems with the password authentication protocol (PAP), if this is being used. Have the customer try again, to see if a transient error in transmitting the "shared secret" has occurred. If this fails, refer the problem to Service Operations.

Dialing Toll-free This message should only be seen during pre-registration procedures. It indicates that the terminal is in the process of detecting a dialtone and dialing the preregistration number.

Actions in Response to a Failure at this Stage: The problem is similar in nature to a failure at "Waiting to Connect". There is likely a problem with the phone line: either there is no dialtone, or there is a problem dialing. First, test the phone line for presence of dialtone, and attempt to make a call on it. If this succeeds, set the terminal to audible dialing. This can be accomplished by disconnecting the phone line or, with the power off, dialing secret code 217 and turning the terminal on, which will put the customer into the phone settings menu. See "WebTV Internet Terminal Secret Codes" on page 116 for more details on secret codes. If a full dialing sequence is audible, and the system still fails to proceed further, check to see if the customer has a voice mail system on the line, or if they need to use a dialing prefix to call outside lines. If none of these tests yield an answer, it is possible that a hardware problem is involved. Direct the customer to contact the manufacturer of the Internet terminal.

Dialing WebTV This message appears after "Preparing to Call", or sometimes instead of "Preparing to Call" during a normal connection or reconnection process. It indicates that the modem is reading the tellyscript and dialing the number. It also appears during preregistration after "Preparing to Call", or, if "Preparing to Call" doesn't appear, after "Connected to WebTV".

Actions in Response to a Failure at this Stage: If the terminal fails here, the first diagnostic would be to set up audible dialing. Use secret code 217 to get to the dialing options, and have the customer turn audible dialing on to see if the terminal is actually dialing. See "WebTV Internet Terminal Secret Codes" on page 116 for more details on secret codes. If it is dialing, and the terminal never makes it to "Waiting to Connect", the customer should carefully count the number of tones the terminal dials. Check to see if the customer has mistakenly

input an access code, or whether the customer requires a dialing prefix (9+ to get an external line, for example on a PBX). Failing all this, refer the problem to Service Operations.

Preparing to Call During either preregistration (twice) or normal use (once), this message is the first message that should be seen by the customer. It indicates that the terminal is testing and configuring the modem for use. Alternately, if this is the second time it's been seen during a pre-registration sequence, it indicates that the terminal has successfully loaded a new tellyscript, and is testing and configuring the modem to call out again using the local number.

Actions in Response to a Failure at this Stage: If it fails at this point, it is likely not receiving a dialtone from the phone line, or is for some reason unable to dial. First, have the customer check the phone line for a dialtone with a normal telephone. Verify that they can successfully make outgoing calls. If this works, ask if they must dial something specific for an outside line (i.e., dial 9+number typical of a PBX). This may indicate a digital, not analog, line; the WebTV Internet terminal currently requires an analog line to function properly. Verify that they do not have a voice mail system on the phone line that changes the dial tone. Another useful diagnostic is to turn on audible dialing using secret code 217. See “WebTV Internet Terminal Secret Codes” on page 116 for more details on secret codes. If the PBX is analog, try turning off the “Wait for dialtone” option, and using slow dialing, both of which are available from the Dialing Options (see “Dialing Options” on page 18) screen. Alternately, adding two commas as a dialing prefix may also help solve the problem. To test the hardware itself, try hooking it up to an analog line (fax lines are good for this). If it works there, it is not a problem with the hardware, but with the other phone line.

Starting Communication This message indicates that the terminal has completed the handshake, and is in the process of negotiating PPP and PAP. This message should only be seen during a pre-registration sequence.

Actions in Response to a Failure at this Stage: There has been a problem somewhere in the PPP negotiation or the PAP (password authentication protocol). Have the customer try to connect again, and if this fails, refer the problem to Service Operations.

Waiting to Connect or Waiting for Answer This message indicates that the terminal is currently listening for busy, ringback, and modem tones, and awaiting the completion of the entire handshake sequence. This message should typically be seen once during a pre-registration sequence, and once during a normal connection. “Waiting for Answer” should be seen near the beginning of a pre-registration sequence, and it means the same as “Waiting to Connect”.

Actions in Response to a Failure at this Stage: The terminal has failed to receive the login prompt from the WebTV Network. This may be a problem with the provider, the service, or it may be a problem with how the terminal negotiates the handshake sequence. Other than trying to connect again, no easy solutions exist, although multiple complaints of this type would tend to indicate a problem with the local provider or the WebTV Network, rather than with the terminal.

WebTV Answered This message should appear during a pre-registration sequence, and may appear during a normal connection. It indicates that the terminal has gotten through the first part of the handshake, and that it hasn't yet gotten result codes for a couple other things yet. Since these codes usually arrive all at once, this message is brief, and may be easily missed.

Actions in Response: An incomplete handshake can be caused by a variety of things, including a defective terminal, which fails to negotiate the handshake sequence as expected. The problem should be referred to Service Operations, who may wish to consider forcing the terminal back through pre-registration to see if it alleviates the problem.

WebTV Answering: This message should appear during both pre-registration and normal connections, and indicates that the terminal is currently negotiating the handshake procedures.

Actions in Response: Similar to “WebTV Answered”, the terminal hasn't gotten through the handshake very far at all. Have the customer try again, this time with audible dialing on. If it fails, but the customer hears “modem sounds” refer it to Service Operations, as this may well be a defective terminal. If the customer doesn't hear modem sounds, Service Operations still needs to be consulted, as one potential course of action is to force the terminal back through pre-registration to get a new tellyscript. However, before notifying Service Operations, it may be worthwhile to test their local POP using Vend-A-Telly, under the WebTV Tricks menu. See “WebTV Tricks Service” on page 118 for more details on WebTV Tricks.

[status bar gone]: The status bar should disappear, and the WebTV Networks logo should replace the highway graphic on screen, before the customer sees the standard opening menu.

Actions in Response to a Failure at this Stage: The disappearance of the status bar indicates that the client has cleared the POP, and the first connect has succeeded. From this point onward, all errors are likely on the service’s end, and should be referred to Service Operations.

[none]: The customer never sees even the first of the connection messages on the screen. The modem has failed to configure properly, and nothing is working.

Possible Actions: First, verify that the unit is connected properly. The unit should be displaying the highway graphic, and the normal music accompaniment. Failure to observe either of these points to either a defective terminal, or an improperly connected unit. Make certain the television is set to the appropriate input (usually Video 1), and that all connections are secure. Make certain the power for both the television and the terminal is on. If the terminal is run through an external stereo input, power may not be reaching it.

Customer complains of an infinite loop while dialing

Likely Cause: Terminals set to default to dial 9+the number, as if on a PBX, will run into an infinite dialing loop if taken to a system which doesn't require a dialing prefix.

Solution: Have the customer input the secret code 217. See “WebTV Internet Terminal Secret Codes” on page 116 for more details on secret codes. This will take them to the phone settings menu. From there, have them delete the extraneous 9 in the appropriate field. This should alleviate the problem.

WebTV Networks ANI-Troubleshooting Protocols

What is ANI, When Is It Used, and Why?

ANI is **automatic number identification**; in the United States, it is encoded into the signal of all incoming calls to a tollfree number, and it can be used to determine the caller's phone number. ANI is similar in principle to Caller-ID services, except it cannot be blocked by the consumer.

In short, the WebTV Network uses the phone number obtained through ANI to search a database and find the cheapest POP (and a short list of alternates) for the terminal to dial into. This information is downloaded into the unit for use in all subsequent calls, alleviating the need for the consumer to wade through a list of phone numbers.

The tollfree number is contacted the first time a unit is activated. Power interruptions or a failure to reach a local POP may also funnel non-first-time calls into the fallover tollfree service.

Connectivity Schematic

Figure 3-1 shows the general connectivity of the machines involved in ANI. Normally, the external call comes in on a DS3 line, where one of a number of ANI-servers (Dianatel machines, running SCO-UNIX, called **anid** collectively) stores the ANI information, and the channel of the call (DS0).

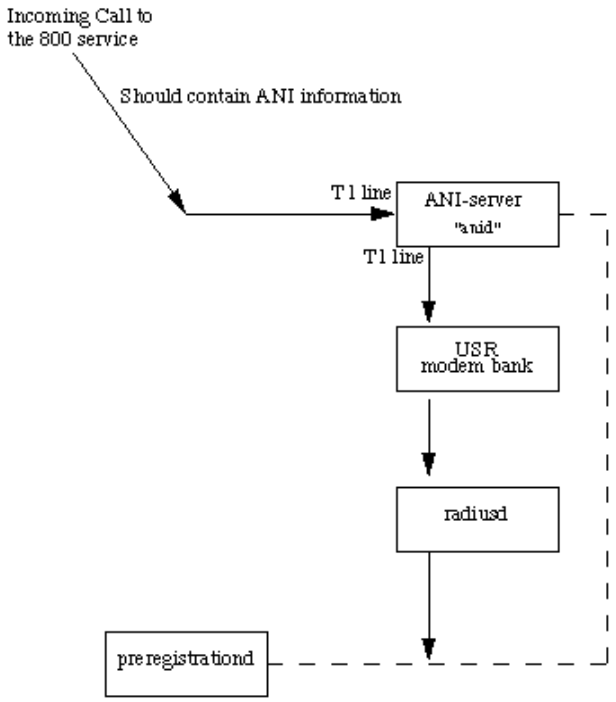

Figure 3-1 ANI general connectivity. Calls originate from clients contacting the 800 service, and proceed through verification and ANI servers to the preregistrationd service.

The incoming call is then processed through the USR modem bank, which queries **radiusd**, which in turn maps the call to a specific IP address, based on the DS0, or "channel" the call was received on.

At this point, the incoming call has its ANI information linked to both an IP address and a DS0. This precise mapping of IP address to DS0 (and associatively, to the desired ANI information) enables **preregistrationd**, the Pre-Registration Server, to send a GET to the appropriate ANI-server (on port 2000), receiving as a reply an indicator of the call's status, as well as the desired phone number.

Diagnosing Potential Problems

The following are all symptoms of problems with ANI. Each is further described, along with solutions to the problem, below.

- Customer complains of having to input his phone number.
- Customer complains of overly long connect times.
- Customer complains of an odd phone number on their phone bill.
- Customer complains of inability to connect

Symptom: Customers call in complaining of having to input their phone number.

Probable Cause: Failure of ANI.

Potential Trouble Spots:

Provider Error -- In certain regions, providers may regularly fail to provide ANI information in real-time (backwater providers). Alternately, a reliable provider may have a transient failure to transmit ANI information.

Line Problems -- If the line which handles the incoming call experiences a failure, the ANI information can be lost.

ANI-server Problems -- The ANI-server can conceivably cause failure through a number of modes.

- Erroneous or failed storage of the ANI information.
- Erroneous or failed storage of DS0, or "crossed channels."
- Failure to respond appropriately to a GET from preregistrationd.

Note: Monitors are in place on the ANI-servers to notify of general failures.

USR Modem Problems -- The modem bank can fail to query **radiusd** for an appropriate IP address, which would have the effect of "unlinking" the IP and the ANI information. The Pre-Registration Server would no longer be able to get the correct ANI from the ANI-Server in this case.

Radius Server Problems -- The Radius Server (radiusd) can fail to appropriately assign an IP address, based on the DS0 ('channel') data it receives. This would also have the net effect of unlinking the IP and ANI information.

Pre-Registration Server Problems -- The Pre-Registration Server (**preregistrationd**) can potentially fail in a number of different modes.

- Failure to frame an appropriate GET for the ANI-Server.
- Failure to receive information in reply to a GET to the ANI-Server.
- Failure to receive the appropriate IP address from radiusd.

Actions to take when an ANI failure is suspected: Instruct a Customer Care terminal to connect to preregistration after clearing its flash NVRAM with code 32768. See "WebTV Internet Terminal Secret Codes" on page 116 for more details on secret codes. If the test is **successful**, and the terminal does not prompt the Customer Care agent to input a telephone number, the problem is either a transient error in the WebTV Network's ANI system, or an error attributable to an outside source, such as a provider. If the test is **not successful**, delegate the problem to Service Operations for further investigation.

Symptom: Customer complains of overly long connect times.

Possible Cause: Incorrect ANI: Incorrect ANI may lead to a terminal calling a local number which it has been told is a POP, but which in truth is not. Incorrect area code on the original ANI would be one cause of such an error. The terminal attempts to call its first POP and fails (usually due to time-out in this case), eventually falling back to the POP + area code, failing that, to a secondary POP, and if that fails, to the backup tollfree service. This sort of connect problem can be diagnosed by having the customer watch the status bar at the bottom of the screen. It should go backwards when it redials, and the messages seen should repeat themselves.

Actions in Response: First, have the customer set up audible dialing. See "Audible Dialing" on page 19 for more details on enabling audible dialing. Have the customer attempt to dial in again. By listening to the answer the terminal receives (is it a voice, a busy signal, modem noises, etc.), a Customer Care agent can more easily figure out what went wrong. If the customer hears a voice, the likely candidate is either bad ANI or customer error (see"Possible Cause: Customer Error" on page 27), and may need to be referred to Service Operations if the terminal was not moved. If the problem is anything but customer error, notify Service Operations of the problem, and have the customer use the 7264 secret code to clear the tellyscript and get a new one. See "WebTV Internet Terminal Secret Codes" on page 116 for more details on secret codes. However, be certain to let Service Operations make this decision, as once this is done, the previous tellyscript is gone, and it may then become impossible to identify the original source of the problem. Knowing the source of such problems is frequently as important as correcting the customer's problem, as errors with unknown causes may be prone to occur again.

Possible Cause: Customer Error: The customer has moved the terminal to a different phone number, and then failed to input it when the terminal prompts for it, the effect is the same as having incorrect ANI information.

Actions in Response: Have the customer unplug the terminal for at least 10 seconds, and then plug it in again. When asked whether their phone number has changed, have them select "yes", and the terminal should go through pre-registration again. If the problem persists, it is likely a problem with the ANI, and you should follow the same procedures as above in notifying Service Operations, and allowing them to determine an appropriate course of action. In the interim, the Customer Care specialist can use the CMR tool to manually input the correct ANI for the customer. See "Customer Master Record tool" on page 70 for more information on the CMR tool.

Symptom: Customer complains of odd numbers on a phone bill.

Possible Cause: Incorrect ANI If an incorrect ANI is assigned, such that the closest POP is in a different area code, and the area code is necessary to dial that number (i.e., customer is in one area code, ANI assigns a different area code, and that number's closest POP is in a third area code, or similar circumstances), the customer may be seeing a non-local phone number on their phone bill.

Actions in Response Check to see what the local POP for their number should be on POP-o-Rama (see "lookuppop and showpop" on page 94) to determine if the number on their bill is from the correct POP. Note that some customers may have as their closest POP a locally billable number, or a toll call. If the number from the phone bill is not that of a local POP, first ask if they have changed phone lines from where they originally registered the terminal. If the terminal has been moved, have them unplug the terminal for 10 seconds, and restart it. When asked if the terminal has been moved to a new phone number, have them enter "yes", which will cause the terminal to go through preregistration again, hopefully alleviating the problem. If it has not been moved, notify Service Operations, so that they may find out how the error occurred, and determine a further course of action. This will likely include following the procedures for clearing the tellyscript using secret code 7264. See "WebTV Internet Terminal Secret Codes" on page 116 for more details on secret codes. Also, verify that the customer has not moved the terminal from where it was originally pre-registered. If it has been moved, and the customer fails to notify the terminal that it is on another number when prompted, it will likely be calling the wrong local POP. In either case, clearing the flash NV RAM should alleviate the problem.)

Symptom: Customer is unable to connect.

Courses of Action When the customer fails to connect, refer to "Analyzing Connection Problems" on page 20. This section discusses connection problems in greater detail.

Analyzing error messages

During the course of normal use, a number of messages in grey boxes may appear on the customer's screen, for a variety of reasons. These messages represent a number of standard errors that may occur.

For each of these error messages, there is a corresponding "M-number". These M-numbers map each error message to a particular type of problem. By asking the customer to hold down the OPTIONS key when a grey error window comes up on screen, in the bottom left corner of that grey window, the text "M-(some number)" will appear. Appendix A, "M-Codes," lists and explains the error messages that may occur during a customer's connection to the WebTV Network.

Keyboard Problems

This problem has been fixed for client 1.1, and is included for completeness.

Keyboard Modifier Keys Stuck

Symptoms:

- The customer complains about always typing capital letters
- The customer complains about getting strange characters (=87=89=DF=8F, etc.)
- The customer complains that no keys but the arrow keys work at all

Solution: One of the modifier keys (Shift, Alt, Ctrl, or Cmd) has gotten stuck. The symptoms listed above respectively correspond to:

- The Shift key being stuck.
- The Alt key being stuck.
- The Ctrl or Cmd key being stuck.

In each case, instruct the customer to press the affected keys while pointing the keyboard at the terminal. Because the Ctrl and Cmd keys produce identical symptoms, a Customer Care agent should probably instruct the customer to press both.

Explanation and Additional Troubleshooting

When something breaks the connection between the keyboard and terminal while one of the modifier keys is pressed, the modifier key gets "stuck" on. If the customer is using the keyboard and nothing is coming out, then the agent should ask them to check that there is a clear IR path between the keyboard and the terminal (less than 25 feet). If there is a clear path, then the agent should ask the customer to check their arrow keys. If the arrow keys work, then the problem is the Ctrl or Cmd key is stuck. If the arrow keys do not work, then a battery problem may be causing keyboard failure. If they have batteries that can be demonstrated to work, then the keyboard is probably bad and should be returned to the dealer.

IR Keyboard Shortcuts*

cmd delete: delete an entire line from a text field

cmd left-arrow: go to the beginning of a line within a text field

cmd right-arrow: go to the end of a line within a text field

cmd scroll-up: go to the top of the page

cmd scroll-down: go to the bottom of the page

cmd options: temporarily show the advanced options

cmd f: slide up the find panel

cmd g: find the next occurrence of the last word

cmd b: look backwards for the last word

cmd r: reload the current page

ctrl power: turn the RF modulator on/off

F1-F7: go directly to one of your first seven favorites

* ctrl-alt can be used on an attached (PC) keyboard in place of the cmd key.

Remote Control

(Sony Remote) When switched to NUMBERS instead of CH, the digits can be used to enter numbers into text fields. The small button labeled enter can be used as the return key. This is especially useful for entering a password.

When the on-screen keyboard is shown, scrolling is disabled. Instead, the scroll-up button can be used to enter spaces, and the scroll-down button can be used to delete characters.

Keyboard command questions can be directed to: Chris White chris@corp.webtv.net

Surfwatch and Kidfriendly filter maintenance

This section describes WebTV Networks Customer Care's procedure for maintaining the service's "black list" and "white list" filters. These accounts refer, respectively, to sites blocked by SurfWatch-restricted users and to sites permitted by Kid-friendly users.

Accounts Used by KidsProtection Administrators

wecare@webtv.net is the central Customer Care address, jointly managed by the Customer Care group.

whitelist@webtv.net and *blacklist@webtv.net* refer to two POP mail accounts. Only designated KidsProtection Administrators have access to these accounts. Each is configured identically, with three folders:

Pending: for requests reported to SurfWatch, currently awaiting addition to the filters.

Investigate: for requests with problems, or requiring investigation.

Completed: for requests that have been processed and verified as added to the filters.

white@trial.webtv.net and *black@trial.webtv.net* refer to two accounts on a WebTV Internet terminal, used to test, respectively, "white list" and "black list" links. *white@trial.webtv.net* should be a Yahooligans-restricted account with the name "Whitelist Approved". *black@trial.webtv.net* should be a SurfWatch-restricted account with the name "Blacklist Approved". Both accounts should also include signature text indicating that the URL's in messages sent from that account have been verified to filter correctly.

sites@surfwatch.com is the electronic mail address at SurfWatch that additions to the "white list" and "black list" should be addressed to.

(Surfwatch contact)*@webtv.net* is the address of the Customer Care agent to whom reports of new filters are addressed.

Procedure

Dealing with requests for additions to the "black list" and "white list" involves a five-step process. For each step, the Customer Care agents who need to be familiar with the procedure are listed at the beginning of the step.

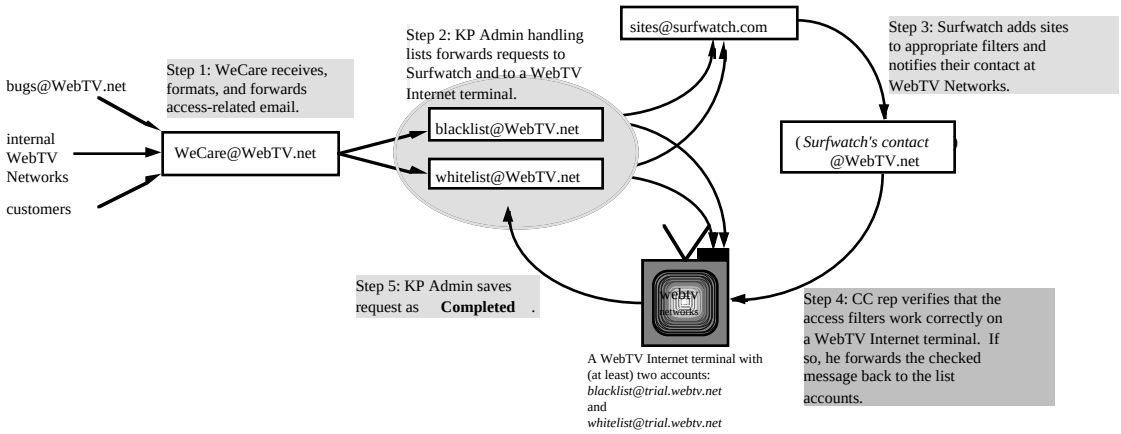

Figure 3-2 Diagram of Customer Care's proposed procedure for handling additions to the "black list" and "white list." Shaded boxes indicate a summary of each step in the process.

Step 1: All Customer Care agents

All requests for additions to the "black list" or "white list" should start in *wecare@webtv.net*. The Customer Care representative that handles the particular piece of email should reformat it, removing any excess text, and send it to either *blacklist@webtv.net* or *whitelist@webtv.net*, as appropriate.

Step 2: KidsProtection Administrators (designated mid-tier agents)

When a new request comes into *whitelist@webtv.net* or *blacklist@webtv.net*, it should be checked to make certain it contains a fully-formed URL that connects to something -- this can be done on Netscape or Internet Explorer. If there is a problem, the message should be saved to the Investigate folder.

Once the URL is determined to be something that needs to be added to a filter, it should be:

3. forwarded to *sites@surfwatch.com*
4. forwarded to either *white@webtv.net* or *black@webtv.net*.
5. saved in the Pending folder.

Step 3: Surfwatch Customer Care Contact (designated mid-tier agents)

After SurfWatch updates its filters, it will send new filters to Engineering. Eventually, these new filters will be installed automatically, with notification going directly to Surfwatch Customer Care Contact. For now, after Engineering installs the filters manually, they will forward notification of filter changes to Customer Care.

Step 4: Any Customer Care agent

Once new filters are installed, they need to be tested. A Customer Care agent should go to the WebTV Internet terminal handling the *whitelist* and *blacklist* accounts, and check each link in the email sent there to make certain the terminal

handles it appropriately. “Completed” URL’s on the *whitelist* account should be accessible, while “completed” URL’s on the *blacklist* account should be inaccessible.

When all of the URL’s in a single message check out, the Administrator testing those links should forward that message back to *whitelist@webtv.net* or *blacklist@webtv.net*, as appropriate. The copy on the WebTV Internet terminal should then be discarded.

Step 5: KidsProtection Administrators (designated mid-tier agents)

When either *whitelist@webtv.net* or *blacklist@webtv.net* receives a message from “Whitelist Approved” or “Blacklist Approved,” that message should be saved to the account’s **Completed** folder, and the version of that message in **Pending**, which should have the same “Subject:” line, should be deleted.

Interactions with problem users

WebTV Networks Customer Care is currently in charge of interacting with and, if necessary, disciplining users who abuse their privileges on the service. This includes, but is not limited to, incidents of denial of service, harassment, posting junk mail, and other violations of the terms of service.

What constitutes “unacceptable behavior” will depend upon the localization of the service and the terms of service offered by that localization. However, WebTV Networks, Inc. strongly recommends considering the response to troublesome users before being faced with an incident.

Some possible responses to problem users include:

- Written warning.
- (For children) Request for action by a responsible parent.
- Refusal of some component of service.
- Refusal of service.
- Legal action.

Chapter 4

Service Operations

This chapter defines the responsibilities and duties of Service Operations, the branch of the Operations group assigned to solve routine problems with the WebTV Network. It also contains reference material on solving some common challenges to the service.

The following topics are covered in this chapter:

- Responsibilities and Work Load
- Monitoring
- Maintenance
- Troubleshooting

Responsibilities and Work Load

The Service Operations Center is chartered with managing and maintaining the quality of the WebTV Network on a day-to-day basis. This group performs proactive monitoring to identify problems before they impact customers, corrects standard problems as quickly as possible, and escalates non-standard problems to those who can resolve them. To fulfill this mission, this group will have the following responsibilities:

Unix System Operations

- Day to Day Monitoring of the Production Unix Environment
- Day to Day System Maintenance of Production Unix Servers
- Standard Problem Resolution on Production Unix Servers

IAP/ISP Operations

- Day to Day Management of the IAP connectivity
- Day to Day Management of the ISP connectivity

WebTV Network Operations

- Content Partner Monitoring
- Customer Care Problem Delegation
- Day to Day Internet Service Monitoring and problem resolution

Monitoring

Monitoring Production Unix Environment

Service Operations uses a number of tools to monitor the load and performance of the production service machines. These include:

Cabletron SPECTRUM

Cabletron SPECTRUM is a network and host management package that will be used by WebTV Networks to monitor the production service and underlying hardware. SPECTRUM consists of a database and a front end, and collects data from production machines and network hardware via *SNMP* and *SNMP traps*.

SNMP (Simple Network Management Protocol) is a query-response protocol useful for trend tracking. SNMP traps are unsolicited events sent by a machine without a triggering SNMP request. Normally, traps are used to send alerts when a machine shuts down or otherwise experiences a failure state.

WebTV Networks chose Cabletron SPECTRUM for its network monitoring agent because, unlike many packages, it can be implemented to monitor both networks and hosts. To allow host monitoring, SPECTRUM must be combined with the Empire Technologies System Management Agent, which runs as a daemon on all monitored host machines. The Systems Management Agent uses SNMP to provide information about the system's configuration, status, performance, users, applications, file systems, processes, and other variables into a frame that SPECTRUM can view and analyze.

system activity reporter

The *system activity reporter* (or *sar*) is a package of UNIX utilities provided with Solaris, and used to monitor and track system performance. It can provide raw, low-level system information; however, this information needs to be filtered and processed to be useful for monitoring purposes. See the man pages on any Solaris machine for more information.

Monitoring ISP Performance and Connectivity

Ping based tools

Forthcoming.

Using WebTV Network Log data to monitor Internet Connectivity

Forthcoming

Duration Information Forthcoming.

Monitoring IAP Performance and Connectivity

Verification of IAP Billing/Accounting Information

At WebTV Networks, Service Operations functions in cooperation with the Finance group to verify that the bills provided by WebTV Networks' IAP partners are accurate. Information in billing invoices is independently checked against information stored in log files and harvested reports.

Using WebTV Network Logs to monitor IAP Performance

(a) Duration Information

(b) phonelogs

(i) Busy

(ii) No Carrier

(iii) Black Hole

(iv) PPP Handshake

(v) Bad password

(c) Radius Logs/Toll free Usage Analysis

Using IAP-provided information

Forthcoming

Using delegations from Customer Care

Forthcoming

Monitoring the WebTV Network

Monitord

`Monitord` is a WebTV Networks proprietary tool developed to monitor the availability of the WebTV Network's specific services on the production machines. Monitord runs at frequent intervals, tests various components of the service and reports its success/failure information via e-mail and pager. See "monitord" on page 85 for more information.

Maintenance

Maintenance of Production Unix Environment

System Backups

Backups on the production service NetApps are done from cron on admin-1 at 2:05 AM. The dump scripts are stored in /operations/backup/. At WebTV Networks, the tape devices are directly connected to the NetApps and are both DLT-4700 stackers. Since the backups are done only to deal with the event of catastrophic failure, not for recovery of individual files, only level zero dumps are done. Each drive has two tape magazines (currently containing six tapes each since the NetApps can't hold more than 100GB.) The magazines are swapped every day, so the previous night's level 0 is always outside of the stacker. The alternate magazine is kept on the shelf beside the stacker.

Currently, these file systems are being backed up:

mailstore-1: /etc /store/mail

otherstore-1: /etc /store/setupd /store/cookied

/store/favorited /store/explored

The log files produced by the dump scripts on admin-1 are named SYSTEM-YYMMDD (no prefix of "backup-") and the SYSTEM is the fully qualified domain name of the system, i.e., mailstore-1.store.alma.webtv.net. The file format, however, is the same as on the other systems.

User Account Management

Unix Configuration Management

Unix Hardware Description

Unix Hardware Field Service

Maintenance of IAP and ISP Connectivity

Maintenance of WebTV Networks Software

Software Distribution

Forthcoming.

PhoneDB Maintenance

To assist in maintaining the PhoneDB, Service Operations uses a set of tools that reside in the *soctools* directory on admin-101. These tools exist to streamline the maintenance of the WebTV Network PhoneDB. There are four main scripts, written in perl and bash shell script.

fetchpops fetchpops can be used to gather IAP-provided .in files from designated locations on the Internet, as specified for each provider in the soctools/fetch/ directory. .in files include a listing of the POP's provided by each partner IAP, which can then be sorted and converted into a normalized format by the makepops tool.

makepops makepops converts the IAP-provided .in files into .pop files, which have a normalized format allowing for easy processing. makepops looks in soctools/conv/ for appropriate conversion routines for each .in file, runs the conversions, and then normalizes the results.

comparepops comparepops prepares a report of changes for each IAP since the last PhoneDB. It does this by comparing the local .pops files to the .pops files that generated the current PhoneDB. The output is presented as formatted text, which can be printed or otherwise analyzed. With the output of comparepops in hand, a Service Operations Specialist can then attempt to identify and correct any problems in the .pops files by hand.

installpops installpops is a script that moves the .pop files to where they can be used to build a new PhoneDB.

POP Balancing

Service Operations, as the maintainers of the PhoneDB, play an important role in determining how to balance which POPs are provide to which subscribers. The controls available to Service Operations and the entire Operations group allow them to determine a number of factors involved in how POPs are provided. See Example 4-1 for a sample POP balancing segment from the configuration file. All of the options in the configuration file for POP balancing are “phase 2" options, which means that they can be changed on a running system without disrupting customer activities.

Example 4-1 Configuration Controls for POP Distribution. Taken from */src/config/arcadia.cf*

```
; If an exchange has more than one local POP from the same provider,
; this will tell the tellyscript generation to alternate between them.
; This is usually (always?) a good thing to do, and will work even
; if the other load-balancing items are off.
;
; (Set to 1 for on, 0 for off.)
;
TellyRotateEquivalent 1

; Suppose we have a flat-rate provider as the primary POP, an hourly
; as the second, and some nasty toll as the third.  The algorithm
; decides that we want to use the hourly POP.  We can choose to do
; one of three things: (1) let it be, (2) use the flat-rate as
; secondary, or (3) use the toll as secondary and ditch the flat-rate.
; These options are expressed as "hold", "secondary-okay", and
; "prefer-toll".
;
TellyFlatPrimaryTenacity hold

; Balance priorities for the different providers.  These are normalized
; out to 256 and then compared against the last byte of the silicon
; serial number for the box.
;
TellyProviderBalance cnc 10
TellyProviderBalance uunet 5
```

Depending upon the IAPs available to the replication of the WebTV Network, the exact options chosen in the configuration file will vary. For an example of POP Balancing testing and the effects of various configurations, see */src/docs/POPBalancing* in the WebTV Network file tree.

Console Resets

If you find a console marked as “down”, such as in Example 4-2, first try to reset it by doing a “Control-e c o”.

Example 4-2 Typical message for a “down” console.

```
jpj@yfandes> console ns-1
Enter password:
console: ns-1 is down
```

If this says [line to host is still down], try doing the following:

1. Lookup the line in *console:/etc/conserver/conserver.cf* for this host:
   `ns-1:!console-ts-4:2006:/var/consoles/ns-1:`
2. Telnet to the console server, which is the second field. In this case, for console-its-4:
   `telnet console-ts-4`
3. Login by typing in the password, then go to enable mode by typing “enable” and entering the console password.
4. Type in:
   `clear line <port number>`

   The port number is the 3rd field in the line from step 1, minus 2000. So, in this case, the port number is 6, and the line you type would be:

   `clear line 6`
5. Type in:
   `quit`
6. Do the “Control-e c o” again and it should work

WebTV Networks Configuration Maintenance

Troubleshooting

Troubleshooting: Production Unix Environment

Wedged servers

Full partitions

Rebooting after crashes

Correcting for overload

Console settings

The serial ports on Sun workstations default to the following settings:

```
9600 baud
8 data bits
no parity
1 stop bit
no handshaking (hardware or software), indicated by'-'
ignore carrier-detect
assert RTS and DTR
```

To see what the current settings are when in multi-user mode, use "*eeprom*". If that fails, look in */usr/sbin*, and then in */usr/platform/`uname -i`/sbin* if that also does not work. There is also a man page in section 1m.

The variables to look for are:

```
ttya-mode
ttya-ignore-cd
ttya-rts-dtr-off
```

and similarly for ttyb. Hence, "*eeprom | grep tty*" is a good thing to use.

Freeing Up Inodes on NetApps

While the NetApps used by the WebTV Network may run out of storage space, a far more frequent problem is a shortage of inodes. Inodes are the file allocation information for the NetApp, primarily a collection of pointers to real files.

The current state of inodes on a NetApp can be determined by:

logging into the NetApp from a console

typing "df -i", which will return the maximum number of inodes, the number of inodes in use, and the percentage of inodes in use.

If the number of inodes in use is near or equal to 100%, the Service Operations Specialist should type:

```
maxfiles (larger number of maximum inodes)
```

This will allot more inodes to the NetApp, clearing up the shortage of inodes. Generally, the maximum number of inodes available should be minimized. As more inodes are allocated, the total amount of free disk space on the NetApp decreases. WebTV Networks' standard production NetApp begins with approximately 2,000,000 free inodes, and needs to be monitored periodically to ensure that its inodes do not run out.

Shutting down the service

In the event of a power outage or a similar problem, the WebTV Network should be able to fall over to UPS power until generator-supplied power comes up. However, in the event of a predicted failure of these backup systems, Service Operations should be prepared to be able to shut down the service gracefully.

Note: In all cases, shutdown instructions are given for a system administrator logged into the consoles of the production machines.

Main Goals: There are two main goals to keep in mind when shutting off the service: 1) Keep the service up as long as possible, and 2) Shut down machines cleanly, so they reboot nicely.

Useful Techniques:

1. Do not depend on name service. Use only numeric IP addresses. This means using the -n option on "netstat", for example.
2. Do not depend on NIS. Use only uids. This means the -n option on "ls", for example.
3. Do not try to use NFS-mounted files, or even cd into NFS-mounted directories.

4. Shut down NFS clients before NFS servers, avoiding timeouts.
5. “gentle shutdowns” mean using “sync;sync;halt”

Overview of Shutdown Procedures: The main phases in the shutdown procedure are outlined below. Based on the power level of the UPS, the reason a shutdown is required, and the judgement of the Service Operations personnel handling shutdown, the sysadmin must decide when to progress from one stage to the next.

In practice, Stage I should generally be performed immediately, and Stage II should follow unless there are extenuating circumstances. Sometimes, UPS’s will fail to perform up to expectations, losing charge much more quickly than anticipated. In these cases, the system administrators should choose to begin Stage III when approximately five real minutes of charge remain in the UPS. By doing this, they will preserve the service as long as possible and attempt to keep customer inconvenience to a minimum.

Stage 1: Nonessential machines on the UPS are shut down.

- Shut down all machines on the service that are not essential to the service.
- Turn off any backup devices that are not active.

Stage 2: Non-production machines on the UPS are shut down.

- Shut down all service machines used by service groups other than paying production customers. This includes alpha- and beta-test service groups.
- Turn off all sequencers that are no longer in use.

Stage 3: Production machines are shut down.

- Shut down all production machines.
- Turn off all routers.
- Turn off the service modem banks and their supporting fan banks.
- Turn off any ANI machines.
- Turn off all sequencers that are no longer in use.
- Shut down any remaining hardware.

Jumpstarting replacement machines

Forthcoming.

Troubleshooting: WebTV Network

Forthcoming.

Troubleshooting: IAP and ISP Connectivity

Forthcoming.

Chapter 5

Network Systems

This chapter describes the activities typically performed by Network Systems, the branch of the Operations group responsible for solving unique problems with the service. The following topics are covered:

- Network Systems Responsibilities
- Contact list
- Escalations to WebTV Networks USA

Network Systems Responsibilities

Database Administrator

WebTV Networks has developed a number of internal scripts, generally written in the perl scripting language, to make the administration of the service databases more efficient. Any database administrator working on the WebTV Network should begin by making themselves familiar with the functions of this library of scripts.

Overview

WebTV Networks runs a variety of databases associated with the service. These include the Customer Master Record, the Remedy database (see "Customer Master Record tool" on page 70, "Service Manipulation Tools" on page 110, and "Customer Care Knowledge Base" on page 70), and others. In a production service, the Customer Master Record and Remedy database are respectively maintained on the custmaster and custservice machines, each of which is protected from storage failures by a RAID device and backed up onto both tapes and a third machine capable of replacing either in an emergency. The backup machine, custrecover, stores an image of the database that is taken approximately once a day. See Figure 5-1 for a schematic of the database machines.

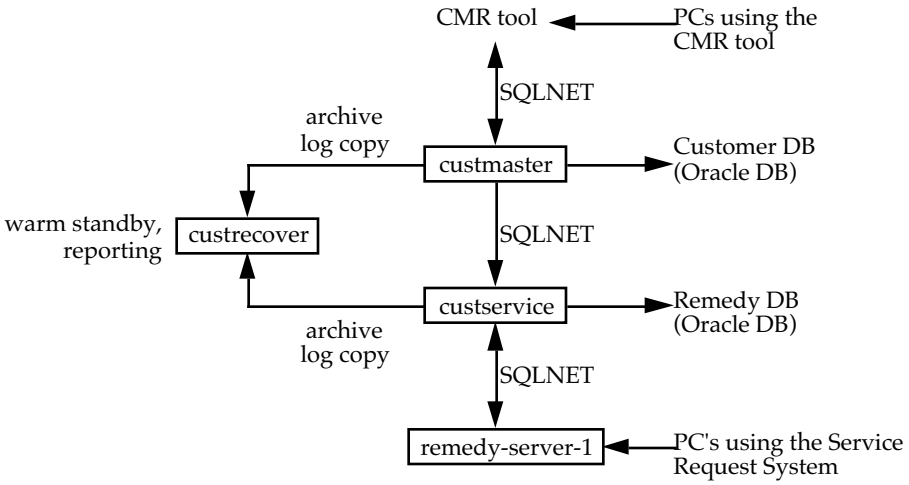

Figure 5-1 Overview of relationships within the customer database and supporting machines.

The hardware configuration of the database system used at WebTV Networks is displayed in Figure 5-2. The main databases are supported on the custmaster-1 and custservice-1 machines, both of which are currently Ultra-1's. Because of the CPU-intensive nature of the database system, WebTV Networks plans to move its databases to Ultra-2's in the near future, once the new platform is properly tested.

The data on custmaster and custservice is backed up into archival logs, as well as to the custrecover-1 machine, which is used as a warm standby and a machine for reporting. The custrecover machine's data is also backed up to a *DLT*, or *digital linear tape*. The backups and redundancy of the entire system are maintained such that the only single point of failure is the CPU of a particular machine. In the event of a CPU failure, WebTV Networks has a "hot spare" standing by to replace a machine with a failing CPU.

Both the customer service production database (CSPRD) and the customer master production database (CMPRD) are in constant recovery mode as archive logs. These archive logs are copied and applied to the databases on custrecover. Once a night the recovery is stopped on custrecover, and a VERITAS snapshot is taken. From that snapshot, the customer master reporting database (CMRPT) is created; all reporting functions are run against that database.

The custrecover system's hardware is identical to that of custmaster and custservice. In the service architecture, it functions as a "warm standby", a database identical to the mother database that receives log files from the mother database. As a mirror of the main databases, it can be used for recovery, and also for the facilitation of reporting. Because it is not used as the actual repository of the customer database, reports can be run against it to generate summaries of activity in the prior day.

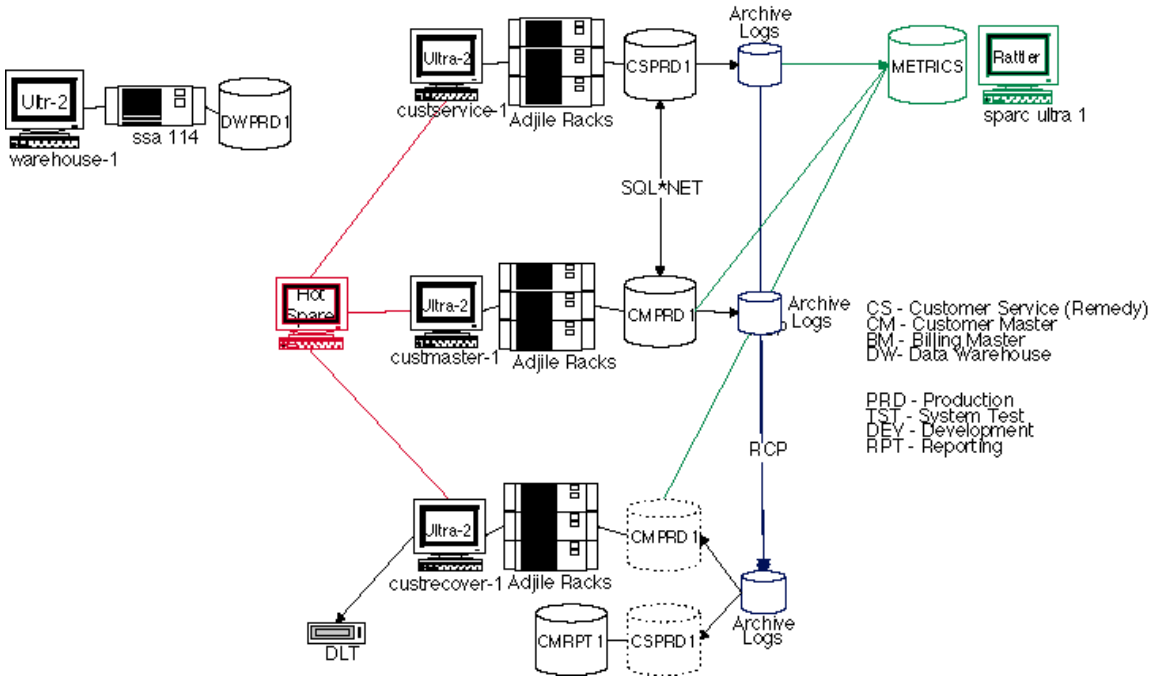

Figure 5-2 Hardware configuration of the WebTV Networks database system.

The controller/disk configuration is listed in Figure 5-3. Each database machine has three SunSwift cards to support the three attached disk arrays. Each raw partition is 250MB, and the total DB size is 12MB. The disk arrays' data is protected by a round-robin 3-way stripe across controllers, plus mirroring across controllers. A failure in one disk controller will not prevent the databases from continuing to operate.

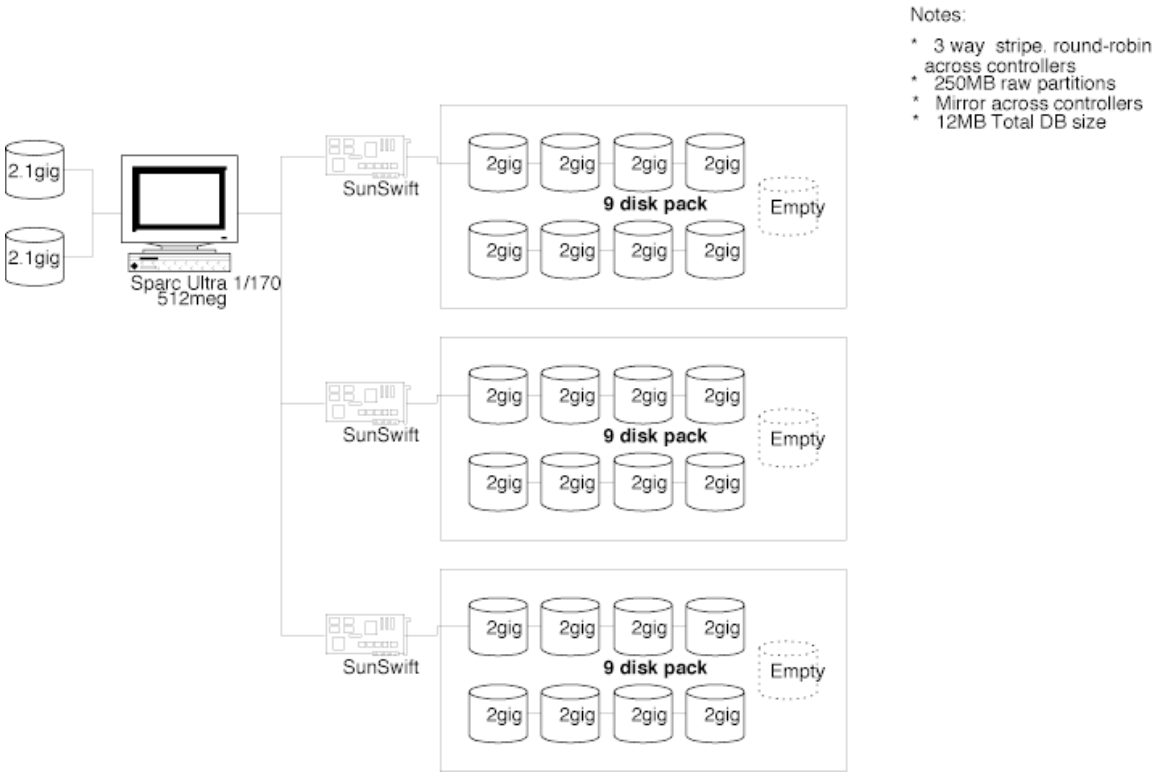

Figure 5-3 Controller/Disk layout on WebTV Networks database machines. WebTV Networks uses a mixture of Sparc Storage Arrays and Adjile External Disk Arrays in the WebTV Network.

Disk management on all database drives, including internal drives on database machines, is maintained using the VERITAS Volume Manager package. The database information itself is protected by RAID 0+1 layout, which includes both mirroring and striping (Figure 5-4). The tablespace layout and volume layout are also described in Figure 5-4.

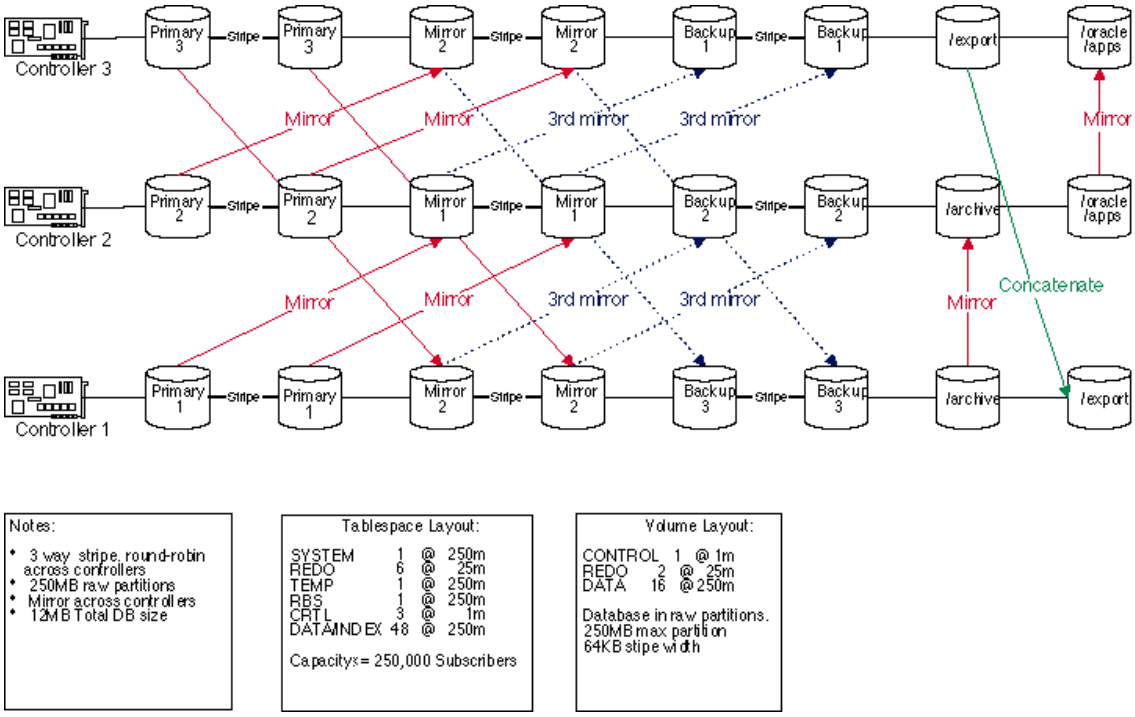

Figure 5-4 RAID 0+1 layout on the WebTV Network database machine disk arrays. Also includes notes on controller/disk configuration, tablespace layout, and volume layout.

Database Structure

WebTV Networks, Inc. uses a modified OFA (Oracle Flexible Architecture), the Oracle recommendation for how to lay out the tree structure of a database, which basically removes the top 3 layers (see Example 5-1). In general, the WebTV Network database structure uses symbolic links to make working with the database easier and more efficient.

Example 5-1 WebTV Networks Oracle directory hierarchy

```
/oracle
  /product    /app
    /7.3.x.x    /oracle (sym link back to top)
        /...
```

Additionally, the WebTV Network databases use symbolic links in the *$ORACLE_HOME/dbs/nit.xxx.ora* directory to refer to the Veritas file structures in */dev/vx/rdsk/* .

WebTV Networks Database Components

WebTV Networks, Inc. will provide INIT.ORA, a Data Dictionary, and all of the source table rows. Additionally, WebTV Networks, Inc. will provide a Transaction Code Table, which will contain options for transaction codes.

Database Backups and Restores

Currently, the backup process for each database begins with a daily disk to disk backup, via a VERITAS snapshot stored on the 3rd mirror disk (see Figure 5-4). That snapshot is then backed up daily to a digital linear tape (DLT) drive. "Hot" backups are made every day; "cold" backups are made once a week.

For "hot" backups, the database is up and running during the length of time it takes to complete the backup. However, the database's contents are placed in 'backup' mode during this time, allowing for completion of the backup without interruption. For "cold" backups, the database is shut down.

These backups are run using a script in */usr/tools/oracle/Backup_oracle.pl*, which can be found on any database server.

Database restores and recoveries are currently done manually, using Oracle's recovery and restore abilities. WebTV Networks is currently developing improved methods for restoring information from databases.

Billing and Payment Verification Issues

Open issues: different currencies, different taxes, different payment options, on-line credit checks? Oracle is set up to be multi-national already. Character set? Is Oracle billing the solution?.

Postmaster

`sendmail` is customized for the WebTV Network, and rolled out by the Postmaster. mail.local.webtv is part of the distribution, and is called by sendmail.

The Postmaster's duties include:

- designing a mail transport facility for a very large mail system (hardware, software, operating procedures).
- evaluating mail transport agents for use in the production system.
- configuring mail transport agents (currently sendmail v8) in the production environment to meet volume and reliability demands.
- tracking new tool releases (e.g. sendmail).
- evaluating/implementing/testing new features.
- monitoring security issues.
- monitoring/handling mail bounces and related problems.
- monitoring and summarizing mail traffic on the production service.
- implementing mailing lists (e.g. using majordomo) and other value–added services for service customers and corporate users.

Service Delivery Engineer

The chief duty of the Service Delivery Engineer is ensuring that changes and improvements to the WebTV Network, its content, and the client are properly installed. This includes retrieving the new files to be installed, delivering them to

the appropriate machines, and then verifying that installation was successful. In the event of a problem, the Service Delivery Engineer is also responsible for ensuring that there is an earlier version of the material being replaced to fall back to.

WebTV Network Delivery

This section describes how a new version of the network services on the production machines should be rolled out. A rollout involves setting aside a subset of all available machines to run a new release of the services and assigning a subset of all users to these.

The Configuration File The WebTV Network includes the configuration file ~production/webtv/config/production.solaris2.cf, which lists all of the servers that a new service needs to be installed on. When the service reads in the configuration file, it looks to ~production/webtv/config/webtv.solaris2.cf, which should be a symbolic link to ~production/webtv/config/production.solaris2.cf.

Service Groups Important to an understanding of the WebTV Network delivery process is the notion of "service groups," subgroups within the WebTV Network using different versions of the WebTV Network or content. Service groups can be created to represent any division of users, be that between testing groups (for example, alpha testers, beta testers, and production), geographic groups (for example, users in the east versus users in the west), or user types (for example, business users versus private users).

After a successful connection, the headwaiter determines which service group a user belongs in, and directs them to the appropriate set of machines and services. Users can be moved from one service group to another using the move-store and setdb tools (see "move-store" on page 89 and "setdb" on page 111 for more details).

Preparing for Delivery Before beginning a service roll-out, the Service Delivery Engineer will have to decide on:

- What version of services should be made available?
- What machines should be used for the new services?
- What set of users should the new services be given to?

For example, to do a rollout of a complete set of services to a small group of testers, the SDE might want to set aside one of every type of machine (storeless, storefull, etc.) and let all users in the "alpha" category try them out. This is usually done well in advance of a production rollout, and to a relatively static set of machines isolated from the main production service.

After the rollout, the SDE should end up with a subset of all users using the new services while the rest of the users get the old services (just like before). After the new services have been tested for a while, the SDE can switch more and more users over to the new set of services until all have been moved. As long as the assigned machines have enough capacity, all the SDE will need to do to move more people over is to change the corresponding Service Group entry in the configuration file to include more or larger groups of users. At a minimum, this should be done on the machine(s) that run headwaiterd and preferably preregistrationd too.

Practically, WebTV Networks deals with testing the rollout of a new service by rolling it out to the alpha and/or beta service group one week, and then delivering it to the remaining production users the next week after resolving any installation issues.

Services are normally installed in `~production/webtv-`*number*. The number will correspond to the build, e.g. webtv-100. While there can be multiple versions of services installed on the same machine (in webtv-100, webtv-101, etc.), there cannot be more than one "active" version at any point in time. That is, all services running on the same machine should be from the same version.

Delivering a New Service

Software distribution should be done in staging areas, through a 10Mb network switched with 100Mb ports. In general, a faster network is better for quick distribution, but by planning ahead for the delivery time, the SDE can roll out the service on any network.

Before the installation of a new service or new content, the SDE should run remote-exec (see "remote-exec" on page 107), an internally-developed application designed to verify that each service machine contains sufficient disk space to store the new files. remote-exec gathers information on all of the machines within a particular service group.

To do the rollout, follow the following step-by-step guide:

1. Verify that there is sufficient disk space on the target machines.
2. Install the new services on the machines that will be running them.When delivering a new service, the Service Delivery Engineer uses InstallBuild, an internally-developed tool that puts the new service software out to all of the machines within a specific service group. Given a list of machine names and the particular service group to be pushed onto, InstallBuild will automatically handle software distribution.

   InstallBuild is based around sdist, a secure version of the UNIX remote distribution tool rdist. Typically, the SDE should begin service delivery a day or two before the actual service roll-over. By running InstallBuild multiple times, software can be developed to multiple service groups. Normally, the SDE will either roll out the service to the alpha service group only, or to all of the production groups including the alpha service group. For more information on InstallBuild and other installation tools, see "Installation Tools" on page 82.
3. Normally, the service groups are very static and will not need edited during a rollout. However, if it is necessary, edit the current `network/src/config/arcadia.cf`, create a new `ServiceGroup` and rearrange the services. For example, to install build 101 for use by all "alpha" users you might want to enter:

```
ServiceGroup Build-101 alpha
```

   Put all the service declarations pertaining to Build-101 immediately after this line. Make sure that the old service entries (still) are preceded by another `ServiceGroup` entry, e.g.

```
ServiceGroup Build-100 *
```

   The "*" (or "all") will make sure that all other "normal" users get this group.

Note: If you only want to test a few -- maybe only one -- new service(s), you can put them in their own group and make it include a “base” group containing the rest of the (old) services. You do this by creating a new group containing all the old services minus the new ones (call it Build-100-Base) and include that group from both the new and the old group, like this:

```
ServiceGroup Build-101 alpha
    IncludeServiceGroup Build-100-Base
...
ServiceGroup Build-100 *
    IncludeServiceGroup Build-100-Base
...
ServiceGroup Build-100-Base
...
```

Note: For example, to test a new proxy Build-100 and Build-101 would contain the proxy services and Build-100-Base all the other services. Note that Build-100-Base has no user group categories following it and will therefore not match any users.

4. Test the new `production.solaris2.cf` file by installing it on a stand-alone machine and running a few passes of `list-services -usergroup` on it. This will print out what set of services a users of the given usergroup will be given when the connect. For example:

```
% list-services -usergroup alpha
```
<all services in Build-101>
```
% list-services -usergroup normal
```
<all services in Build-100>

The Rollover Process To roll over to a new service or new content, use the carbon xterm utility to mirror keyboard input in all of the machines within a particular service group. Then, execute the following scripts in sequence:

1. stopservers: stops all of the services on the particular machine.
2. switchservices: switches the machines to the new service or content. This script functions by creating symlinks in the webtv directory of the production machine, thereby directing queries to the newest version of the service.

```
# switch-services webtv webtv-101
```

3. Verify that the new `production.solaris2.cf` file has been installed on all of the servers.

Note: If the new services require a configuration file based on the new build, repeat the previous two steps above on it and install that on the new servers instead.

4. startservers: looks at the service configuration file and restarts the appropriate services on that server.

Once the service is successfully restarted, the SDE should go to the monitoring machines and begin to monitor the service using check-servers, which spawns a monitord process. Once monitord is restarted, it will look at the configuration file and validate that all of the appropriate processes have restarted successfully and are behaving correctly.

Testing the New Service

Test by logging in as a subscriber of each named category. You can change categories for an existing subscriber by running `setdb`, e.g.

```
% setdb -subscriber 123456 -category alpha
```

You can find out what the subscriber ID is for a given username by running querydb, e.g.

```
% querydb -subscriber bob
```

Issues in New Service Roll-Outs: Since the category is part of the subscriber record and no subscriber normally exists yet at registration time, all new users will still be registering using whatever group is defined in the configuration file.

The first time a user of the new service group (“alpha”) connects, they'll go to the old preregistrationd and headwaiterd and then handed off to the new services. Not until they connect the second time will they be going directly to the new headwaiterd.

Installing new flashrom images

The service doesn't use this directly, so it can be rolled out separately. There are 3 classes of machine that run flashromd: storefull, storeless, mailtod. There is a staging area in ~production's home directory on the internal network, that the SDE can to from the distribution machine. Release Control will put together a client build and will update this directory (flashromimages) when they want, removing old versions and inserting new. Note that the tool genrom is necessary to turn a newly built version of the client into a useful flashrom image that the internet terminal can download and use properly.

The SDE can run an “InstallFlashromimages -production -m list-of-machines -k” and it will do an automatic sdist to the appropriate machines, e.g.,adding version 254 and removing 249 or so. (There are generally 3-5 versions in place.)

The SDE should then install willy.tmpl and flashrom.cf. flashrom.cf tells flashromd what version to give to the dumb client. willy.tmpl is a menu of possible versions that are available to the sophisticated user who knows the password for the wtv-tricks service.

Delivering New Content

Like the delivery of a new service, the delivery of new content requires the use of installcontent. installcontent is an internally-developed script designed to deliver new content trees to all of the appropriate machines within a service group.

Release Control puts together `~production/dynamic-content` directory. `content-yyyymmddhhmm` will be the name of the content directory within it. It contains the homed content, explorerd content, and the kidsafe filter list. This is the content on the home page and in the explore tree; it consists of lots of tmpl files. tmpl is HTML-like language with some extensions that can substitute the user's name. WTVP is the HTTP replacement. There's a lot of other content that's part of the release that doesn't get touched: instructions, help, registration templates, etc.

The actual content installation itself is performed using:

```
InstallContent -build yyyymmddhhmm -m list-of-machines -k
```

This script tars up and gnuzips the whole directory into 3 files so it's not necessary to sdist a bunch of small files (which would take forever). Then it scp's (rcp's with ssh) the 3 files to the -m machines: storefulls and storelesses, and uncompresses and untars it. But this is just out in a temporary place.

To switch over content, which usually happens during downtimes for most changes (though it can happen live), run:

```
InstallContent -switchto -m ... -k
```

This does an ssh to each machine, calling rename-link which points a link in the new direction.

There's no cleanup facility, so the SDE has to make sure there's space before he does the install! He should removes the oldest content from each of the storefulls and storelesses. Hopefully, this should soon be handled automatically by a script.

Changing the WebTV Tricks password

The password for the WebTV Tricks service is specified in the arcadia.cf source file, and can be changed by altering the entry within that file and then building a config file from arcadia.cf. This can be done at any time without disrupting the service.

System Administrator

Jumpstarting New Servers

production account uid When jumpstarting a machine, the system administrator should install a production account with a uid of 100. This shouldn't be changed. Oracle and other applications may want the uid of 100 also, but production should have the highest priority for the uid of 100.

Database connection configuration One of the files that needs to get installed on all the server machines is /var/opt/oracle/tnsnames.ora. It contains information about which database to connect to. Here is an example contents for the Japanese Trial1 service:

```
CMJPN1=
          (DESCRIPTION =
                    (ADDRESS_LIST =
                              (ADDRESS =
                                        (COMMUNITY = tcpcom.world)
                                        (PROTOCOL = TCP)
                                        (Host = lexus.japan.webtv.net)
                                        (Port = 1526)
                              )
                    )
                    (CONNECT_DATA =
                              (SID = CMJPN1)
                              (GLOBAL_NAME = CMJPN1.WORLD)
                    )
          )
```

Replacing Production Machines Without Downtime

Normally, when machines are taken down on the WebTV Network, they trigger long delays in the network while other machines search for the services the victim machines previously offered. By substituting another machine on the same

network and at the same IP address, delays from time-out errors can be avoided. The surrogate machine, even though it does not offer the services of the victim, can directly refuse the rest of the network services, eliminating time-out errors. Then, when the victim machine is ready to return to the network, the surrogate's IP numbers can be reset and the victim can come back and take its place on the net again.

Procedure for substituting a surrogate machine during a victim machine's downtime

This procedure requires a surrogate machine, configured to function correctly on the ethernet, but otherwise requiring no special software.

Victim:

1. Take the victim machine out of service.
2. Disconnect the victim from the networks that the surrogate is to be placed on.

Surrogate:

1. Plug in the surrogate to the same networks as the victim.
2. On the console, check to see what interfaces are in use by the surrogate. Do:
   ```
   /sbin/ifconfig -a
   ```
3. Find a series of open interfaces (not currently in use). From lowest to highest, a typical sequence of interfaces is: le0, le0:1, le0:2, le0:3, etc. Find the highest-numbered virtual interface in use, and, for the next step, increment the interface number by one for each IP address added to the surrogate.
4. On the console, bring up the surrogate on the victim's IP addresses using /sbin/ifconfig. For each of the victim's IP addresses requiring a surrogate presence, do:
   ```
   /sbin/ifconfig [interface] [victim's IP address] up
   ```
5. To get the router's attention, do:
   ```
   ping [router IP address; ends in .254]
   ```

Reversing the above procedure

Surrogate:

1. Halt the surrogate's use of the victim's IP addresses. For each of the victim's IP addresses requiring a surrogate presence, do:
   ```
   /sbin/ifconfig [interface] [victim's IP address] down
   ```
2. Reboot the surrogate. This may be optional.

Victim:

1. After the surrogate goes down, restore any disconnected network connections, and bring the victim back up.
2. On the victim, get the router's attention by pinging the router on that network.
   ```
   ping [router IP address; ends in .254]
   ```

Rebuilding Proxy Cache Disks

Periodically, cache disks on proxy machines will fail. Replacing the dead disk itself is generally simple, but it will also be necessary to rebuild the hierarchy of proxy cache directories on the defunct disk. Example 5-2 outlines the procedure for doing so.

Example 5-2 Rebuilding Proxy Cache Disks

1. Execute “stop-servers” as production.
2. Bring down the machine using “shutdown”, and power both the machine and disk array off.
3. Physically replace the dead disk.
4. Boot the machine into single-user mode.
5. Create a file system on the new disk using “newfs”.
6. Mount the replacement disk on /cache/ as appropriate, and chown it to production.other.
7. Reboot the machine.
8. Bring the machine up in multiuser mode and log in as production.
9. Execute "stop-servers".
10. Run ~/webtv/tool/make-proxy-cache. This will take a few minutes.
11. Execute “start-servers”.

Once the disk has been replaced and the proxy cache hierarchy rebuilt, the proxy machine should function normally.

Escalations to WebTV Networks USA

TBD

Contact list

TBD

Chapter 6

Tools

WebTV Networks, Inc. in Palo Alto uses a number of tools to operate, support, and monitor the WebTV Network. These include both internally-developed and 3rd-party packages. Many of the tools developed in-house have their own help menus available, by typing the command name with the option `-help.`

This chapter describes covers the following topics:

- bulkmail
- cat-event
- clientinfo
- clientpopedit
- computil
- crashlog-filter and associated tools
- CS-Ticket, the Service Request System
- Customer Care Knowledge Base
- Customer Master Record tool
- dpedit
- distill-log
- dumpcrash
- Etherpage/Pager software
- genrom
- Harvester Statistics
- idzify
- Installation Toolsmonitord
- keytool

- move-store
- PhoneDB Tools
- querydb
- Real-Time Service Overview and overviewd
- rename-link
- remote-exec
- Service Logs and Log Analysis
- Service Manipulation Tools
- setdb
- sweeper
- tellygen
- Status Report and statusd
- top
- traceroute
- User-event logs
- userStoreLockAndDo
- userStoreName
- WebTV Internet Terminal Secret Codes
- WebTV Tricks Service

bulkmail

bulkmail is a customized mass mailing system for the WebTV Network.

Usage

```
bulkmail [-verbose -domain domain] mailboxfile < userdata
```

bulkmail is used to “mail merge” e-mail to customers from a list of customer information and a message template.

The -verbose option provides status information as each message is sent.

The -domain option adds an optional domain to the user's address. By default, the domain will be webtv.net.

The user data is in the form of a tab delimited file, where the first line contains the template substitution names and subsequent lines contain the values for each of those variables. See Example 6-1 for a sample user data file.

Example 6-1 Sample userdata file for bulkmail

```
username   first_namelast_namefav_colorzip_code
freddyman  fredericksmithsonmint green94114
bobbysue   franciscorobertsonbaby blue94301
.
.
.
```

Note: Note: every userdata file must contain a column labelled username that contains the user's email address.

The mailboxfile contains a mbox formatted template file containing the message to be sent to the customers. See Example 6-2 for a sample mailboxfile.

Example 6-2 Sample mailboxfile for bulkmail

```
From StevePerlman@webtv.net
From: StevePerlman@webtv.net (Steve Perlman)
To: {username} ({first_name} {last_name})
Subject: Welcome to WebTV
Mime-Version: 1.0
Content-Type: text/html

<h3>Welcome to the WebTV Adventure!</h3>

By now, you're sitting back on your {fav_color} sofa,
drinking your beer in your {zip_code} neighborhood.
.
.
.
```

cat-event

Usage

```
cat-event [-cf configfile] filename
```

cat-event is a tool which takes user-event logs and normalizes them into a more readily machine parseable format (see also "User-event logs" on page 115). Each name: value pair is separated by a tab, the entire contents of a single event-log entry being contained in one line this way. This output of this command is often piped into distill-log (See page 73), which pares down the information, putting it in a nice, easily read format. Example 6-3 is a sample of typical cat-event output from a general production server's user-events log. There is also an option if you wish to log the output to a file [`-logFile filename`].

Note: You should be user `production` for this command to work properly, as by default it looks for certain config files in the current user's home directory. This can be overridden by using the [-cf] option.

Example 6-3 Typical output of cat-event

```
gmt=1997/03/10 10:00:04 local-time=1997/03/10 02:00:04  server-name=statusd
host-name=testimony      pid=19183        ip=207.79.34.4  queue-duration=0.000246
visit-url=wtv-status:/overview  bytes-served=81 bytes-received=35        duration
=0.002437        action=overview
gmt=1997/03/10 10:00:10 local-time=1997/03/10 02:00:10  server-name=statusd
host-name=testimony      pid=19183        ip=207.79.34.4  queue-duration=0.000244
visit-url=wtv-status:/overview  bytes-served=81 bytes-received=35        duration
=0.043827        action=overview
gmt=1997/03/10 10:00:15 local-time=1997/03/10 02:00:15  server-name=statusd
host-name=testimony      pid=19183        ip=207.79.34.4  queue-duration=0.000241
visit-url=wtv-status:/overview  bytes-served=81 bytes-received=35        duration
=0.002369        action=overview
gmt=1997/03/10 10:00:21 local-time=1997/03/10 02:00:21  server-name=statusd
host-name=testimony      pid=19183        ip=207.79.34.4  queue-duration=0.000239
visit-url=wtv-status:/overview  bytes-served=81 bytes-received=35        duration
=0.043751        action=overview
gmt=1997/03/10 10:00:26 local-time=1997/03/10 02:00:26  server-name=registerd
host-name=testimony      user-id=Registration     pid=19274        ip=207.79.34.4
serial-number=TestClientID-1     queue-duration=0.000231 db-function=CustomerDB_N
ewSubscriberFromClientID          db-not-found=true        db-duration=0.055464
visit-url=wtv-register:/register          bytes-served=497          bytes-received=3
94       duration=0.060541        action=register
gmt=1997/03/10 10:00:26 local-time=1997/03/10 02:00:26  server-name=websim
host-name=testimony      pid=19191        ip=207.79.34.4  queue-duration=0.000287
url=/tests/get?Content-Length=300        visit-url=/tests/get?Content-Length=300
bytes-served=828          bytes-received=193       duration=0.003425        action=t
ests/get
gmt=1997/03/10 10:00:26 local-time=1997/03/10 02:00:26  server-name=proxyd
host-name=testimony      user-id=tu-2     pid=19330        ip=207.79.34.4  queue-du
ration=0.000302 visit-url=http://websim/tests/get?Content-Length=300     cache-mi
ss-reason=NotFound       server-addr=207.79.34.4 not-cached-reason=NoExpireInfo
content-type=text/plain content-len=716 bytes-from-server=828   dn-duration=0.0
02334   server-duration=0.009138          bytes-served=769          bytes-received=6
6        duration=0.015488        action=http
gmt=1997/03/10 10:00:26 local-time=1997/03/10 02:00:26  server-name=websim
host-name=testimony      pid=19191        ip=207.79.34.4  queue-duration=0.000247
url=/tests/get?Content-Length=1024       visit-url=/tests/get?Content-Length=1024
          bytes-served=1555        bytes-received=194       duration=0.003251
action=tests/get
gmt=1997/03/10 10:00:26 local-time=1997/03/10 02:00:26  server-name=proxyd
host-name=testimony      user-id=tu-2     pid=19330        ip=207.79.34.4  queue-du
ration=0.000315 visit-url=http://websim/tests/get?Content-Length=1024   cache-mi
ss-reason=NotFound       server-addr=207.79.34.4 not-cached-reason=NoExpireInfo
content-type=text/plain content-len=1442          bytes-from-server=1555  dns-dura
tion=0.002252   server-duration=0.008735          bytes-served=1496        bytes-re
ceived=67        duration=0.014876        action=http
```

clientinfo

Usage

clientinfo [silicon serial number]

Queries the database for information on a client with a particular silicon serial number. This information principally relates to the client's dialing patterns and other communications data. See Example 6-4 for a sample output of the clientinfo command. Note that the clientinfo command will also produce output relating to the current status of the PhoneDB it is run against, including disabled POPs, PhoneDB build date, and load balancing information. See "PhoneDB Tools" on page 92 and Appendix C, "Choosing the Right POP" for more information on the PhoneDB.

Example 6-4 Sample clientinfo output

```
--- Client info for serial '8192b30000b002ca' ---

  ANI ....................... 00 408-245-3551 (SUNNYVALE, CA)
  Shared secret ............. 'z3faxU5TsUc='
  Script locked? ............ no
  Fallback disallowed? ...... no
  Revisit preregistration? .. no
  Call Waiting Threshold .... 0 (not set)
  Last successful connect ... 990-2255

  Most recent script sent to client:
    Hash 0xf9cb1b98, sent Tue Feb 25 16:43:55 1997
    v17 base/-
    v2  __ziplink/9902255
    v2  __cnc/4900610
    v2  __ziplink/9902255
    v2  __cnc/4900610
    v1  __artemis/18006108918
  Previous script sent to client:
    Hash 0x40ad29e9, sent Tue Feb 25 13:47:39 1997
    v17 base/-
    v2  __ziplink/9902255
    v2  __cnc/4900610
    v2  __ziplink/14089902255
    v2  __cnc/4900610
    v1  __artemis/18006108918

  Established dialing patterns:
    ANI 408-245-3551 + POP 408-990-2255 --> mode=7-digit
    ANI 408-245-3551 + POP 408-490-0610 --> mode=7-digit
    ANI 408-245-3551 + POP 888-329-0489 --> mode=11-digit
    ANI 408-733-3266 + POP 888-329-0489 --> mode=11-digit
    ANI 408-733-3266 + POP 408-490-0610 --> mode=7-digit
    ANI 408-733-3266 + POP 408-990-2255 --> mode=7-digit
    ANI 408-733-3266 + POP 408-988-5800 --> mode=7-digit
    ANI 408-733-3266 + POP 408-492-1481 --> mode=7-digit

  Dialing overrides:
    (none)

  Current set of POPs for user's exchange:
    ziplink 408-990-2255 (SUNNYVALE, CA) [cost=240/0.0mi  local]
      (9902255 or 14089902255)
    cnc 408-490-0610 (SAN JOSE W, CA) [cost=240/7.9mi  local]
      (4900610 or 14084900610)

   Next best:
    uunet 408-273-0562 (SAN JOSE N, CA) [cost=240/7.3mi  local]
      (2730562 or 14082730562)
    ziplink 408-490-2255 (SAN JOSE W, CA) [cost=240/7.9mi  local]
      (4902255 or 14084902255)

  Same thing after load balancing, using local cf file:
    ziplink 408-990-2255 (SUNNYVALE, CA) [cost=240/0.0mi  local]
```

```
      (9902255 or 14089902255)
    cnc 408-490-0610 (SAN JOSE W, CA) [cost=240/7.9mi  local]
      (4900610 or 14084900610)

  Next best:
    uunet 408-273-0562 (SAN JOSE N, CA) [cost=240/7.3mi  local]
      (2730562 or 14082730562)
    ziplink 408-490-2255 (SAN JOSE W, CA) [cost=240/7.9mi  local]
      (4902255 or 14084902255)
```

clientpopedit

TBD

computil

computil is a utility for deciphering tellyscripts and other compressed data embedded in log files. It is useful to Service Operations and Network Systems personnel investigating problems in log files.

Overview

This tool is intended to exercise the compression functions available in the server sources. It's not meant to be a replacement for programs like compress(1) or gzip(1).

The input and output files are memory-mapped, so compression of very large files is supported even for functions that expect to be handed all of the data at once in a single buffer.

Usage

```
computil [-d [-a -t]] [-v] [-u] [-m method] infile outfile
```

Valid compression methods are ‘none’, ‘lzss’, ‘lzh’, ‘rle7’, ‘ahuff’, and ‘lzpf.’

[-d] decompress (a/k/a expand) instead of compress

[-a] before decompressing, decode MIME base64 encoding

[-t] before decompressing, strip off the 20-byte tellyscript header

[-v] toggle ‘verbose’ flag

[-u] do the same file 50 times and print rusage() statistics when done. This turns off computation of the CRC to avoid affecting the usage statistics. Useful for doing speed tests on changes to compression code.

[-m method] specify the compression method. The set of compression methods may vary. See src/doc/Compress for a description and comparison of the various algorithms. If no method is specified, “none” is used. This does not have to be specified with “-d”.

infile file to compress or expand

outfile file to write the output to

A 32-bit CRC is included in the compression header to verify data integrity. The program will exit with a nonzero code if the CRC doesn't match during decompression.

Usage: validate method

This shell script will compress, expand, and verify every "plain" file in the current directory, recursively descending into subdirectories. The method must be specified.

Extracting tellyscripts

The procedure for extracting somebody's tellyscript involves pulling their preregistration entry out of the user-event log for the date they got a tellyscript. Finding the appropriate date can be more difficult, though sometimes you get lucky and the "date tellyscript last sent" found by running clientinfo (see page 60) on their box will give you what you want.

One of the entries in the user-event will have a **large** uuencode-like section. This is a MIME base64-encoded copy of the tokenized and compressed tellyscript. You have to pull that line out, trim off the start and end so you're left with nothing but uuencoded material, then use computil to unencode and uncompress it.

Here's an example:

```
% ssh prereg-1.alma.webtv.net
% cd ~production/webtv/log/event_log/
% less user-events.log
```

Page down a few lines until you see something like the text of Example 6-5.

Example 6-5 Sample MIME base64-encoded copy of a tokenized and compressed tellyscript

```
preregister                02/03/97 13:27:33 -0800
          pid=9641                    ip=10.2.1.5
          serial-number=8168b30000b00277
          queue-duration=0.000696 db-function=CustomerDB_NewClientFromID
          db-duration=0.017374     db-function=CustomerDB_NewClientFromID
          db-duration=0.020931     db-function=CustomerDB_NewClientFromID
          db-duration=0.005399     caller-has-local-pop=true
          tellyscript-id=0x3143393c
         tellyscript=QU5EWQAAAAAAAAABMUM5PDL2WEUAAA+RAAAulDz2BPkBAf//+X8ADwMLbElkdGW/
cmF0ZQA7CwFj/gsHcHJvdG9jb/1sGAJjb21wcmXfc3Npb24PBG5un2VjdGVkNgBUCEn/V2lua0RUUgD/KCl/
e39Jc2X/dGR0cgAoQwD/KTt/SWRlbGFNeQwAMT4OAhwGMR0Pr39/ckOQAH1UA1P/
dHJMZW4AKGPnKklzTgBaAWxJbLoRAD0kAX93KBcDUHspfxUCUDt/
ch4CzkAJbkNwewBBAGRzbXRPAAAsTAFyYwoCjt4BdW50WAJMAhcDKd97f2koIRsERClvf2I7fz4GUD0+Bixu
AKcAaShEBD7sACcH+KEDzAjQA2V0UHJv6WdQEdUCbVkQYWdlPokBcGVyY2WLBD8R/
IUQJwJ0ZXh0ACggKQZAFB0FNQRDAShCByMLD2RpcnR+Em4RkQy9FP9Jbml0aWFsafF6RgBsEOQQZmlnQ19tZ
EJ1ZjASZBwA+A4E3xGPEWV0dGluv2dzAD1JZ6wAaJdvbmUSBijWEEUBYe9sbFdhWABuZ1D/
cmVmaXgAPUDeNwdbQwBdQiFjSf91c2VQdWxzZc1EgQFuZ1EAJAg2MP4mAWNJYXVkaWLpbCMPIwIxIwNkaXOl
YSMAQ3cHSwwycgV3npYARm9yVLUAbgw1eJUClAJHCEhhY2uXDH03cwVhbFNwZQ8w+rYLOLYDY3dTZW5Nc1wQ
dmmDENkLOQATvioSd1ZhbHWLAFOTMTRYN/QhY70hkBx//1NBVCtNUz0x+zEsAQA0NDAwU/czOD0EADA9MTj/
MFM5NT0zNlP/MTE9NjAmRDL/
VjFFMEwzJlEfNSZLMw1hMlcI1xgNLFQAVjAgA9MgbxxhMFKMAWGKITQKTcQyZR4P4B4EUQdzGU4PgwRTNj3P
MTBYNFMPIws0WN0zqQZ2ZXKdQigpJ0dDN7szvR9rGASPG7VFgkF/
fRkxMw5QZRYoD0MyKA02KA9RApgAhlEJODgnD3kC4wB5CTl5MicBEQ59f31l0Q/00QQtD3+8UGludGb/
```

AChTVGVsbHn/U2NyaXB0OiD+IxE9JWQsIGN3Ey1oyyALBXOcIBcCzjT/
IFMxMCB0byDTJXNlIFkYLH8PawCBLN0sDwHGI+VSux/
fETGDMD0eD2lFQAlZUl03RfBKYFhhNg9VAjE9MjCCvCJ9axE2CWRgNw83CDEBMTcPbgHwEDcPbwiYQDYPAP4
SNg9uCTUCYzP4TRU/
mTQTRFAQPx4JVDAYAl5VD7BsFZ9eHULRdWVuTFFtz29kZW3RUtVyZmxbb3cCgHRyHoAo6yEyz3NinlDtcD4x
hBO+Re9ldG5h4mBlcnb7aWOgYEM8PzQ8/zs0ODMALEM8fz40Mzk4MDVzBv4zgXdpbmRvd3OQhmLLMUF/
qWJNkwOoZnX8xoCfgENvbW1hbv1kJoRyZXRyaWX5c6aBjoFzdGF0daZ0cEM2dIKIelOeYHAFYTgwZ/
shFhGgAKYi14nYyIBTBX6CdyhjBlBMvvcUZmx1c2gtFm7QcoAEkKUPpQI7FFVmb64pkFNPSxUQMhkQN0c4AC
lDIck/yTFDSwQHZWQgmhNCkJ+AJUExD/
77MlRJTUVPVVQRIPNh4jFuACBsAHqR3AmIaUDMgjcVPsokVA+GA3U/bGRuJ3Qg/HBNAJ8gZnJvbZAH/
5ty6BNQ0pRCIm/ogGhvb81rAaV/
ciejlYxJUP9hcnNlUmVzdSFsAJDQEQkC3RZjmSAIozaxgm1tPZE9U5ehxgKANwI2QZChHwf7AcoTQAQx4G+i
olEzBsNRNApOTyB/
Q0FSUklFUjsL8Tw7D18gOw5ESUFML1RPTkV4CzU8D79gXq0LQlVTWa4LNzUPPDAgrg5BTlNXrQ2yYULpDDHX
YR8aYCDgsyEmEQIjxT3bM8ywAMc6H1BxOx4MUA9QAzE8slFRD4weogQ+osU9QzI1ODgBtB1stRQnCD4+Jw9D
M3lxbk8KMzg0Jw9DM5tAbncLNDE6Jw9DM/
EUwicIO9gCVC3cFMcHNTSw3aAnD84UJwg9PCcPQ7szORcdNjcyJw9DGzM6Px03MD8fkIAKJwDx1pAfuYEeNy
gIrYNRC0EUDFELJw80PPXDeOYkDy+Q+DeAJAybD0M0PgB8SMc4WAT6CDkSL4uQPzUJ2TQUoXAqODMgL0VDMT
UtsLYMBB9DNU0E3AiyDlNl8gNdXzog+Ugg/
y0tICVkIHVuH2tub3duc5AGVcNK4Tkz804PTg9OAiVzIAkoQlQWoClWB/
WQuUVqAQBkUzbhcVY97yVwltGVdy3ifIx0bvFudW1iZWzQPLzRdPFpc0xvfIBK7/
hK70rkf+FPdXRzacNkZUrvSuD3ICXhKkkXYWNj8eBOcAJu7MIQsEnvSe/
V1m8ETGkj7jP+SANicm9rZW5Qy0JYsew6iwQ08UxEDK8P+uA4PPzhCRB5GDJg/
XV2EmlzVG9sbBdGcmVsEWPTEEABifAZZtIA6gZJYhAFIfOqgJ9TdHJuQy7iGwQsjnkcQzE/
9nI6Bg8AXRxrMnySb3JrhhCsE0QHODHjkwicMmkoKNYbTACLAF28QiYYDXXwFtAwOBgPRxAYAil8f00PTQ8A
TQkYD00EPJFuMLEMKUMu7vpFACEyK1tDAF0mgNkYTuLBLOPQVMNtP2vRQrAWJVTRWdCGoHBwW9FsR2VhZGPR
MfIUO1B/
lIAmQe0anxS3DubjYSq6AiHYXDegDwzkVXPp0UxEISBZI47WOiJtKizDK7faBigLUywN05YGPTkR1WAPAGAD
2TtlD2UNFfdpD2kPaQ8AaQ9pD2kGmsQ/xpbCRy9WTaCgE1suHOF7y4dEILlDIIB+NKswtnBf5BBIPw8/BVd/
ZWJUVi4uLjoIYgYxcwMlFEZxBiVz5vEAJh2sWireWkafBXnDkg+SBlAqCN+yOU9yME50ciYZD5AZAJNQD98P
1lNl0LjRJwclcycEB8dhhWrlD+UMAK3XvWkZC4laNAs1HY/fsQ/
ApwKJD1QPVAYm81bkUzsRDcTSc+i79mlCk1q6suIVIW24Jn5gPA/xH9J7YdCAchVwq/
DoGD85YTA2BJbgrLBpq0H/
Ia1jX9HpDwh573nmtOIgxIgq6uwhRMGuGoByQzT+YGU94Dj03IQA5D0Y60lhdG8Baa5oV/
PgxypUVgQZYawrHVcW9HRvIIjEh/
HlMg5rGEFURG0ThAWhomEzuOvpCR+18mdvdKEjYf90IG9kZCB0aYNtZfFwYnflBYP19OJOKFwSIh/
41jzDEj52oNgBACkVq5E08nTyUhQiAshK+dD63T1TREIgYW5zdwVlxwBnCRAt0ZgE2w/
vYgwi5EegLCAytAsCzbUYAgBUyHCgnjJZ5GNg2NSkMSO0AAyRhccmJCfkl8CMUYxBQw/
gQw9DB1Igvay3oklBUHy1oobgZmlnQ22zo3zDogsEaWFwTmGwEfjLpiewxqcsbCpJcPiOJOqxDQc9SUluaQF
0GLRgCEFiNAfnEe8gPMFcZB+zc0NhbLNUL7RSAJIGkAWmkXgJkbjLB70GgrfgzDK/
CYAP5ROmZGZhaR9sdXJlLMI0zBL+CAxODFXgAHwOCVPwqSIo0oDbYDkYwSKolKkeasfFg2XnZENotnE+wHVz
ZfFyxDFqwgwBUmVwbD15eMJwYXNzGgYMAXQaA4/RbDcGTGVuZ7ApbCwGDwN/
Hwo9XLEoAQEobgftMjMIIwh3BjyxUB/+0RNPTk5FQ1Qv5XPPBCwmWs5iKGxlQC0xntCNByfAuAm/
E0lhUXhdUJ4JJwssQzc9ruDgA03cMgNPA0Y4YCxDNPk7xw/
aS2xvZ2luBvAxbXDxoeBrkhdnexcLAFwPS7ESHxIW3Bi6Gp2TEBcMJg8QFDQ7EBR6D7JVPyECFtBk2g+7c1I
oKIsXCl8PdCbBUKByFXJQUFBfE0hlqScwmB5D/0SXUjWOcnKcwCHwoEkAcHBwLPC4zg/
0tWUAIG5lh2Bp4WFBUccyFoCBfnJDOxBudEcP3cEO8GsodT8DHrsg3MO4D4WCZHEUpTZYYpJSAC2fQ48JLQA
ph6dDD0MPQwoyzTl9Pw8w02NocIDFhhM9IMBhrfNwfaN5EcEUckkZOcRPnpNQQVC+Ss5dYUJhc2FjktBlZCl
QeSRQYmPjQm6CYTTwjzAAXTsSAJslEgcqChugcn7AX8L0zZISA9T1AGcjEsZWBlRDhgdJDPHDmAsInINt2LE
FLMENDbSJxyoH/
VNKFG1wdXRlZoFjYMF5DDjDmB9DWk8QLODdJ1M3Ng8A+IQQIHdpC3RoOsIvrEgRFGAWnDVAhBY4GMGkjBYHG
BoGcPOhABo15j8tP+Y/5j8UrrmwA0A+96QoKUVDM5m/
DUf++wBhdXRoZW50IWn+4BBF2HADQzoDT0tPAEtPPU89Tz1PPU8PHz1P0Z+O/
MVtYWmW8TgxEjF0+t6xc9vDcGhvbmWwEgYz0SWApPo9QDIHWxNDNGw0G6dbuEGAMiGlohEFbIhTRNBTEGtdA
2w+yrUsSWVyct+SBwEEF6O48Xpq0Xmod6bpszQBAArQPAVrI2j0VSojlTzAzgoIQvHTkDrWQesCUOJ3DyOjV
A4aodBledp0crBuPw+gtwmtIuHVgi//ZXV2EixNIDmAJWQ0gKvgZNqg638oPxYtURQpL0ODMzxeIzejVA/

pVJ2kIAEjjMFYDIoWYQ9hBwx8Sdl6yVZJeqFfY25jZCP+ntF3aW5kb3dz4s4SQzvijtDmV1Mlcz8hd2VidHZ
a4Mj1HkoIdXVuZbMxTA9MBc5LCVdUVgPCSQ90X05SgGVtadkhSw8oEUE80wGWB3d0dl9KD6FigOgZHN+kJUi
yUeTiI2PAZaN4dEw0Na+SdWLtYC5/
dHNmIHZlchThQyAx0DP84BEEzYBHFPDwQCFRCNNiQERyZXRyAXmGcthiFiIqCe3UT/
CSCIFFtmWBUifTAeLNLwnhU/
7mESxTOTM4NDR7NjlGsDEALECWM0CK8k30VQTbMx4orvEhxTMBe6xECnE6QQdAsLJLBMCBgE8JCVAoD0t1EE
WxgG0YUIA60e0NRwPuD+4P7gKKIyx/UzIyMzk2NewAAPAP8A/EKvIP8g/yD/IP8g+I8g/yD+ARMvIP8g/
hHTEHMjA25R/0D+Uf8g/yDwDyD/IP8g/yD/IP0yPrc/IPAPIP5Rr0Aekf9A/pH+kf9g8A9g/2D/YP9g/
2D8o1BID2D/D2D8s1HVXtEDgwMDY/MTA4OTE4/
UH3D2D3D1xZV0GmBrIBQz0iaIBkejtfaHV5jUxfGtEQXiaAAV8BUyeAhgIQHxAf7TQtAvqDLfV9Cy+wo3euq
xgVPxgCccoV/8D//w==

```
         ANI=00206938xxxx          old-ANI=0
         db-function=CustomerDB_UpdateClient
         db-duration=0.037481     bytes-served=4695   bytes-received=338
         duration=0.422553
```

You can select and copy the encoded tellyscript out carefully. Experience shows that it works best to paste into a "vi" session. If you try to paste into "cat > foo64", X will sometimes drop characters. In the log it's all one long line, but the base64 decoder is smart enough to deal with line breaks.

Once the above is in a file, remove all the text above and below the large encoded section, and trim the first line so the the file looks like this:

QU5EWQAAAAAAAAABMUM5PDL2WEUAAA+RAAAulDz2BPkBAf//+X8ADwMLbElk dGW/
cmF0ZQA7CwFj/gsHcHJvdG9jb/1sGAJjb21wcmXfc3Npb24PBG5un2VjdGVkNgBUCEn/
V2lua0RU

[... deleted for brevity ...]

UH3D2D3D1xZV0GmBrIBQz0iaIBkejtfaHV5jUxfGtEQXiaAAV8BUyeAhgIQHxAf7TQtAvq
DLfV9Cy+w o3euqxgVPxgCccoV/8D//w==

Save that to a file called "foo64". Now run:

```
% computil -d -a -t foo64 out
```

Which gives you the following as output:

```
Base64 decoded 5433 bytes into 4021
Skipping 20-byte tellyscript header
WARNING: compLength=3985, compFileLength=5433, sizeof=16 (fixing)
Expanding 'foo64' to 'out' (lzss)...
CRC matches
```

Ignore the warning about file lengths. So long as you see "CRC matches", you know you have good data.

The file called "out" has the tokenized tellyscript in it. The output file will be in a format similar to a binary file, but if you look carefully down near the bottom you will be able to see phone numbers and text strings. The decoded example above (a random sample taken from the user-event logs), when viewed with "less", looks like the text of Example 6-6.

Example 6-6 Decoded version of a sample MIME base64-encoded copy of a tokenized and compressed tellyscript

```
[ ...header deleted for brevity... ]
,S2239651^@,C1^@,@Ierr^@);^?i(Istatus^@EC^@){^?Ierr^@=IChat_uunet^@();^?i(!Ierr
^@){^?rC^@;^?}e{^?Idialerror^@(Ierr^@);^?}^?}ei(Istatus^@EC1^@){^?Idialerror^@(I
err^@);^?}e{^?rIerr^@;^?}^?InextNumber^@P;^?}^?i(InextNumber^@EC2^@){^?Istatus^@
=IDialIAP^@(IconfigCmd^@,IdialCmd^@,Scnc^@,S12069384469^@,C1^@,@Ierr^@);^?i(Ista
tus^@EC^@){^?Ierr^@=IChat_cnc^@();^?i(!Ierr^@){^?rC^@;^?}e{^?Idialerror^@(Ierr^@
);^?}^?}ei(Istatus^@EC1^@){^?Idialerror^@(Ierr^@);^?}e{^?rIerr^@;^?}^?InextNumbe
r^@P;^?}^?i(InextNumber^@EC3^@){^?Istatus^@=IDialIAP^@(IconfigCmd^@,IdialCmd^@,S
uunet^@,S12062239651^@,C1^@,@Ierr^@);^?i(Istatus^@EC^@){^?Ierr^@=IChat_uunet^@()
;^?i(!Ierr^@){^?rC^@;^?}e{^?Idialerror^@(Ierr^@);^?}^?}ei(Istatus^@EC1^@){^?Idia
lerror^@(Ierr^@);^?}e{^?rIerr^@;^?}^?InextNumber^@P;^?}^?i(InextNumber^@EC4^@){
^?Istatus^@=IDialIAP^@(IconfigCmd^@,IdialCmd^@,Sartemis^@,S18006108918^@,C^@,@Ie
rr^@);^?i(Istatus^@EC^@){^?Ierr^@=IChat_artemis^@();^?}^?rIerr^@;^?}e{^?rC=^@;^?
}^?}^?^?^?^?^?IAccessDial^@(c*IconfigCmd^@,c*IdialCmd^@,c*IaccessNumber^@)^?{^?l
Istatus^@,Ierr^@;^?^?i(Iversion^@()GC8^@&Igetconnectretrycount^@())^?rC=^@;^?^?I
status^@=IDialIAP^@(IconfigCmd^@,IdialCmd^@,S-access-^@,IaccessNumber^@,C1^@,@Ie
rr^@);^?i(Istatus^@){^?Isetdtr^@(C^@);^?rIerr^@;^?}^?Ierr^@=IChat_cnc^@();^?^?rI
err^@;^?}^?^?^?^?<FF><FF>
```

Most of it's control characters shown in inverse text, but if you look carefully you can see the phone numbers (e.g. 12069384469). The ANI shown iin the log entry was 00206938xxxx, so the choice of POPs makes sense.

If this user had gotten a "this may be a toll call" message, you would have seen it embedded in the message above. The finer points of interpreting tokenized tellyscripts are beyond the scope of this document.

crashlog-filter and associated tools

TBD

CS-Ticket, the Service Request System

The WebTV Networks Operations group uses CS-Ticket, a service request system implemented using the Remedy toolkit, to track calls, problems, and customer metrics. This SRS interfaces with Customer Care's e-mail alias, the customer database, and the developers' bug database to provide the Operations group with a powerful tool for storing and retrieving information on problem status.

The service request system is integrated with the Customer Master Record tool (see "Customer Master Record tool" on page 70) and the customer database (see "Database Administrator" on page 41 for more information on the customer databases), allowing unified problem tracking across applications.

Overview

In WebTV Networks' implementation of the service request system using Remedy, there is one central Remedy system that is divided into a number of subgroups, each of which can see some component of the whole data in the Remedy database. This centralized organization allows one group to develop Remedy applications for a number of separate departments or groups. The service request system itself is organized into three main pieces: the clients, the

server, and the underlying Oracle database (see Figure 6-1). In the implementation of the Remedy system at WebTV Networks, the Customer Master Record is also part of the service request system because it is queried to populate data fields in the Remedy database.

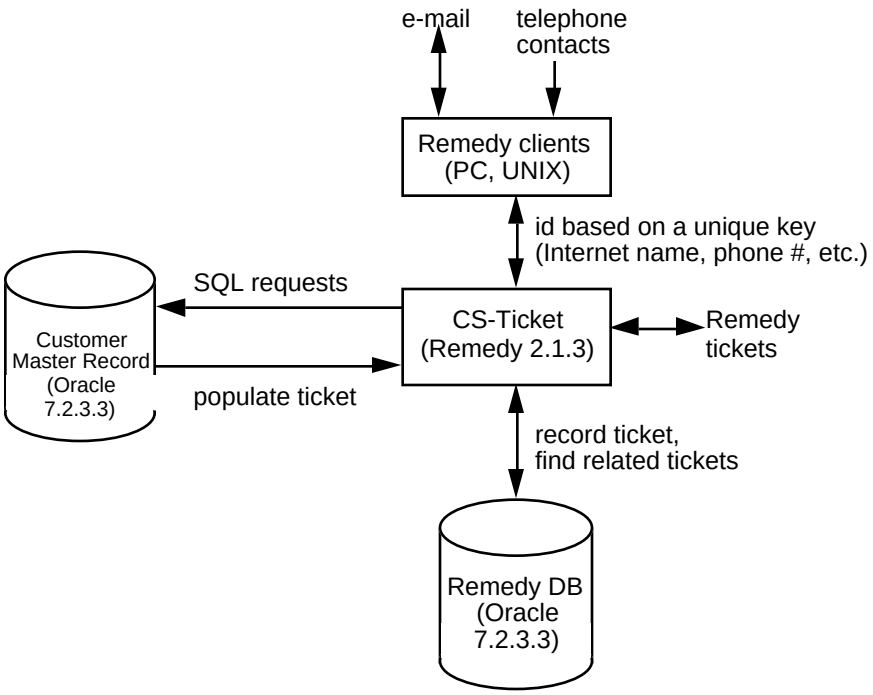

Figure 6-1 Schematic diagram of the relationships between CS-Ticket, the Customer Master Record, the Remedy DB, and the Remedy client.

Customer Care uses CS-Ticket to record all customer contacts and resolutions of those contacts. These contacts come in as both e-mail and as phone contacts. E-mail and phone contacts are two separate sets of work, handled in different work queues. However, all information from both sorts of contacts is stored in CS-Ticket, the section of the database for recording customer contacts. CS-Ticket serves as the single focal point for the Service Request System; it stores a history of the contacts, allowing a user to assess the total volume of work as well as to break down that work by type.

Workflow in the Service Request System

For phone support, the workflow in the service request system is simple. Calls come in, are entered into CS-Ticket, and then either resolved and recorded as done, put on hold until they can be resolved, or forwarded to another component of the service request system (for example, requests for sales support are redirected to a separate component of the SRS where they can be dealt with appropriately).

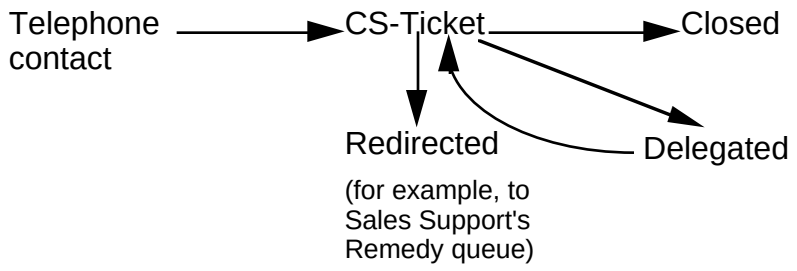

Figure 6-2 Service request system-directed workflow for incoming telephone contacts from customers.

In comparison, the workflow for e-mail is somewhat more complex, because an entry in CS-Ticket is recorded for each separate question in the incoming email, and for the answer to each question, as sent in an individual piece of outgoing email for each question. WebTV Networks is currently developing an alternative to this system to reduce the amount of time lost in workflow.

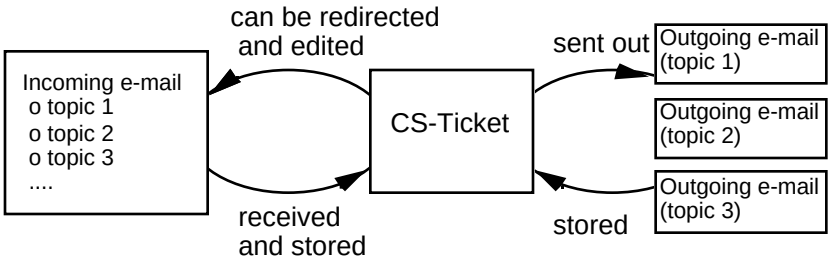

Figure 6-3 Service request system-directed workflow for incoming and outgoing e-mail contacts from customers.

In cases where there are many queries about a single problem, a Master Problem can be generated. Each ticket with a question about the subject of the Master Problem can be assigned to the Master Problem, and then these tickets can be answered, resolved, and otherwise altered simultaneously and efficiently.

When a contact comes into Customer Care, a field for identifying the customer is entered into CS-Ticket. In the United States, this is the customer's Internet name, although different indices can be used. CS-Ticket populates the other fields of the user's interface with additional information on the customer, and allows the user to access the customer's entire history of contacts, as stored in CS-Ticket. To populate fields, CS-Ticket (and the Service Request System) need to be able to make SQL queries of the customer database, which returns the desired information.

WebTV Networks' goal is that 90% of customer contacts are resolved by the same person who opens them. Currently, approximately 80% of requests are closed in this fashion. Partly, this is because of the diverse nature of customer contacts. While the above descriptions of workflow apply for the majority of Customer Care contacts, there will always be exceptions requiring different workflow plans.

For example, requests for billing credit may need to be authorized by a number of groups other than Customer Care to be approved. By considering problems as they arise and adapting the Service Request System to address them, workflow can be made more procedural and efficient.

Escalations

Remedy applications can be configured to automatically escalate tickets to a more prominent urgency. The triggering events can be based on either qualities within recorded tickets (for example, billing requests above $1,000), or on time limits (for example, tickets that remain open for more than 14 days). These triggers can be customized by the Remedy application designer to best suit the needs of the system's users.

Designing Applications of the Remedy Toolkit

An experienced Remedy programmer is suggested as a resource for modifying and developing Remedy applications. However, the basics of the Remedy system may be useful to Network Systems agents if the Service Request System undergoes problems.

For more information on programming Remedy and service request systems, consult the *Action Request System™ 2.1 Administrator's Guide for OSF/Motif™*, and the *Action Request System™ 2.1 User's Guide for OSF/Motif™*.

Objects

Remedy uses five classes of objects to produce applications. Those objects are:

Fields: Fields store information, such as integers, date and time, character strings. They can be configured with different lengths and properties.

The next three types of objects are all *events*. Events in Remedy are actions that can cause data to change or to move between schemas, trigger alerts, change ticket status, or otherwise cause actions to the Remedy database and associated data. Each of the three types of event objects is triggered by a different type of action:

Active Links: Active links are events triggered by a user action, tied to a list of tasks that are then performed. For example, a command to show active tickets would be designed using an active link. Unlike *Macros* (see page 69), they are server-side operations.

Escalations: Escalations are events triggered by the passage of time. They are server-side operations like Active Links (see page 69) and Filters (see page 69).

Filters: Filters are events triggered by data events. For example, when a ticket is changed to closed, some portion of its fields may be logged for future reference. Filters are server-side operations.

Macros: Macros are client-side action sequences, triggered by the user to perform a series of operations quickly and efficiently. Macros can be recorded by any user of the Remedy application.

Schemas

Remedy applications are also known as *schemas*. Schemas are all of the information presented on one screen of a Remedy application, including all of the objects and databases associated with that screen. Each schema uses non-relational names for its fields -- that is, a field with name *foo* in one schema will be

represented by a different internal field name than the field with name *foo* in a second schema. This makes it challenging to get data from a Remedy application into an outside database.

Customer Care Knowledge Base

At WebTV Networks, Customer Care uses a set of internally-developed web pages as a knowledge base. These pages include a wide range of details on various aspects of the service, including connectivity issues, troubleshooting, and supporting cgi scripts. Much of the information contained within the knowledge base has been reproduced within this document; by placing the HTML version of the WebTV Network Operations Reference Guide on an internal web server and providing access to it to Customer Care agents, the Customer Care Knowledge Base can be largely reproduced.

Customer Master Record tool

The Customer Master Record tool, or CMR tool, allows the Operations group to access and modify the customer database. This allows agents to cancel accounts, uncancel accounts, change dialing information and passwords, and even swap accounts from one WebTV Internet terminal to another.

The CMR tool is an internally-developed Oracle forms-based application that provides access to, and manipulation of, the customer database. The degree to which one can access and manipulate the database is determined by the Director of Customer Care, and implemented by the Database Administrator.

dpedit

Also referred to as the Dial Pattern Editor, this tool allows you to add, delete, and edit the dial patterns associated with a subscriber, as well as change the recorded value for the subscriber's phone number (ANI). Before using this tool, it is advisable to read the appendix titled "Choosing the Right POP", which provides a wealth of background on dialing patterns.

Overview

The server keeps track of the dialing patterns that work. If, calling from Palo Alto, you were able to connect to the server by sending "2330570", you will dial that from now on. If instead you had to dial "14152330570", you would use that instead. The pattern chosen depends on your ANI, so that if you move somewhere else the server won't try to use the patterns established in a different area. The service will remember up to 8 patterns, and will drop the least-recently-used entry to make room for a new one.

The patterns are recorded after successes, not failures. A customer might never have a successful connection, perhaps because they need to dial 7 digits but the server will only try 10 or 11. This is a common situation when the service mistakenly thinks a local call is toll. The terminal could automatically try 7-digit dialing, but that increases the possibility of calling a MOM instead of a POP, so it's not attempted. Since the client never tries 7, it never realizes that 7 works, and the user is out of luck. This tool is intended to solve this class of problem, by allowing customer care to manually seed the set of successful dialing patterns.

The usual causes of these problems are errors in CCMI (e.g. it says a call is toll when it's really local), errors in the interpretation of CCMI data, or telco error (e.g. the area code hasn't split but the ANI information is being returned as if it had). This tool is **not** expected to resolve problems caused by area code splits or user error. If the customer's phone number has changed, either because the area code split or the Internet terminal was physically moved, it has to come back in through preregistration via the tollfree number so the database can get the current version of the customer's phone number. Methods of accomplishing this that don't require customer involvement are beyond the scope of this tool.

This is also not intended to deal with bizarre dialing requirements or customers wanting a specific POP, as opposed to the one that they would normally be assigned first (see "clientpopedit" on page 62 for help on these cases).

Usage and Options

```
% ./dpedit [-D] -c silicon-serial -- [server-options ...]
```

The arguments are:

`-c silicon-serial`

Right now the only way to specify the Client record is by silicon serial number. In the future other ways may be provided, so save yourself some re-education time and get in the habit of typing "-c".

`-D`

Turns on "debug" mode, which just makes sure that the database doesn't actually get written to.

`-B`

An undocumented option that turns off the boldface stuff.

`--`

Marks where the server framework options can begin. Invoke the program as `dpedit -c foo -- bar` to see a list of options.

If you wanted to invoke it with an alternate PhoneDB file, you might use:

```
dpedit -c happy-serial -- -PhoneDBFile ~/PhoneDB
```

Unlike most other pieces of the service, dpedit will accept it if it can't map the PhoneDB file. If it can't, it will print a warning message and proceed onward, but certain features (like automatic verification of POP numbers) will be disabled.

Command Overview

After dpedit is executed and finds the specified subscriber, it prints something that looks like this:

Example 6-7 dpedit printout

```
The dial patterns for '01fad82501b002ba' (ANI=004154631671) are:
S  # ANI            POP            Mode
+  0 415-614-5539 415-233-0570 7-digit
+  1 415-614-5539 415-322-0489 11-digit
+  2 415-463-1671 415-233-0570 7-digit
I  3 415-463-1671 415-666-9999 7-digit
+  4 415-463-1660 415-322-0489 11-digit
+  5 415-463-1660 415-233-0570 7-digit
N  6 415-463-1660 510-742-0207 11-digit
-  7 <empty>
Available commands are:
A)dd   I)nsert <entry>   E)dit <entry>   D)elete <range>
C)hange ANI   P)rint   W)rite   Q)uit
[modified]>
```

The 'S' column has a quick piece of status. '-' means the slot is empty, '+' means there's something unremarkable there, 'I' means the POP for that slot is invalid, and 'N' means that the entry has been modified. A POP is marked as invalid if it can't be found in the set of POPs in the PhoneDB, so if the PhoneDB couldn't be mapped there will be no 'I' flags.

The '#' column has the entry number. There are always 8 in the current release of the server, though this is an arbitrary number (bounded in part by the size of the field in the Oracle database).

The next three columns should be read from left to right. Entry #0 in the example above could be described as, "When the caller's ANI is 415-614-5539, and they're trying to call the POP at 415-233-0570, use 7-digit dialing".

Below the pattern listing is a list of commands, followed by the command prompt ('> '). The prompt will be '> ' when the program is first launched or changes are written, and '[modified]> ' when there are changes that need to be written to the database.

Commands are entered as a single character, possibly followed by one or more numbers. For example, to add a new entry you would type "a <return>". (Don't type '<' 'r' 'e' 't' 'u' 'r' 'n' '>'. Hit the letter 'a' and then the "return" key.) To edit an entry, use "e 5 <return>". To delete entry #2 and #4 through #6, you could use "d 2,4,5,6" or "d 2,4-6".

You can bail out of the program at almost any time with Ctrl-D. Ctrl-C and Ctrl-Z don't work because of some things the Oracle libraries do, but Ctrl-\ will work (and give you a core dump).

Bear in mind that the combination of ANI and POP must be unique. Specifying two different patterns for the same POP called from the same place wouldn't make much sense. Thus, you can’t create duplicates in the list.

Commands in Detail

A)dd - shortcut for "i 0". Inserts a new entry at the top.

I)nsert <entry> - add a new entry before the entry specified. If the list is full, the last entry (#7) will be thrown away. You will be prompted to enter the ANI, POP, and Mode values, with a default value provided. If you want the default, just hit <return>. The default value for the ANI will be the subscriber's current ANI setting. The default value for the POP will be the first of the caller's POPs that doesn't already appear in the list. If the PhoneDB is mapped, and the POP you enter doesn't appear in the set of known POPs, a warning will be printed.

E)dit <entry> - edits an existing entry. You get the same set of prompts as for Insert, but the defaults come from the current entry rather than the current ANI field or the list of POPs in the PhoneDB file.

D)elete <range> - delete an entry or set of entries. The entries are removed and then the list is compacted. Typing "d 0" several times in a row will delete all of the entries in the list, as will "d 0-7".

C)hange ANI - change the subscriber ANI value. Just enter the 10-digit phone number. The ANI code 96 (which identifies an ANI value that was manually entered by Customer Care) will be prepended automatically.

P)rint - the list is automatically re-printed whenever something changes. If you're error-prone or the list has scrolled off the top of the screen, you can show it again by hitting "p <return>".

W)rite - write the changes to the database. Until this is invoked, none of the changes will be permament. If the program was launched with the "-D" flag, some internal updating takes place, but the database write doesn't.

Q)uit - quit the program (Ctrl-D also works). If you have made changes, you will be offered a chance to write them before exiting.

distill-log

Usage:

`distill-log [-wide] [-width] [-cf` *`config filename`* `]`

distill-log is a tool used usually with cat-event (See page 58), though it can be used in other ways. It takes normalized event logs, as produced by cat-event, and pares them down, presenting them in a more readable format. The first line is the process name, process id, and machine name. Subsequent lines are digests of actions that process took, each of which would be recorded as an individual event in the complete event log. Each of these lines is truncated at 88 characters by default. So, for each series of associated actions a process takes, there is a single line describing the process name, id, and machine name, followed by single line digests of each individual action it took. A blank line after the last of these actions separates the digests from the next process listed. The options [-wide] and [-width] change the point at which truncation occurs; in the former case, there is no truncation, while in the latter case the argument for [-width] is the number of characters across that will be displayed by distill-log. Below is a typical example of output from distill-log.

Note: You should be user `production` for this command to work properly, as by default it looks for certain config files in the current user's home directory. This can be overridden by using the [-cf] option.

dumpcrash

Usage:

```
dumpcrash [-f] [-d symbol-table-dir] [-s symbol-table-
override] [crashrecord.log] ...
```

dumpcrash is a tool that manipulates crash log data. Its first job is to try to find an appropriate symbol table. Symbols are appended to the end of some client ROM builds. If the ROM doesn't happen to end on an exact 1MB boundary, dumpcrash assumes that the symbols are appended at the end. This is verified by an internal comparison to information in the ROM file itself. When a crash record for a particular version of the client first appears, dumpcrash looks in the current directory for a ROM image for that version. If it finds an image with valid symbols, the symbol table is read and held for the lifetime of the dumpcrash process. If the symbol table isn't found, dumpcrash will look in the same directory every time that version is requested to see if an appropriate symbol table has been added in the interim.

dumpcrash understands three kinds of ROM files:

- Files < 1MB are assumed to contain nothing but symbols.
- Files > 1MB and < 2MB are assumed to be "approm" code-only images; dumpcrash looks for symbols one megabyte into the file.
- Files > 3MB and < 4MB are assumed to be "unirom" code/romfs images; dumpcrash looks for symbols three megabytes into the file.

The command syntax for using dumpcrash is:

```
% dumpcrash [-f] [-d symbol-table-dir] [-s symbol-table-
override] [crashrecord.log] ...
```

Executing `"dumpcrash -\?"` (which is "-?" with the question mark quoted) will give the usage information as well as the default symbol-table-dir and the ROM image file name search order.

The `[-f]` option causes dumpcrash to work similarly to `tail -f`. It processes the entire file, then checks every second for new input. If the log file is renamed and restarted—something that currently happens every night at midnight—dumpcrash will switch to the new file within 30 seconds after it appears (value is `kTailMaxCount`, in `utility/TailStdio.h`). This option is ignored if the input is from *stdin*.

The `[-d symbol-table-dir]` option tells dumpcrash to look for ROM images in the specified directory instead of the default directory (which is currently "."). The recommended way to see the symbols on a "private" (wtv-system-version: 0) build is to execute `ln -s approm approm.0` to establish an image for version 0 and use `-d` to tell dumpcrash to point into the directory where the ROM image is located.

The `[-s symbol-table-override]` option loads the specified file as the symbol table file for *every* ROM version. This option is handy if you want to use your own ROM image instead of the default; you need not rename the image to `xxx.version` and then pick it up with `-d`.

You can specify one or more crash records on the command line. dumpcrash will dump the information from each in turn. The file name "-" corresponds to *stdin*; if file names aren't specified, then dumpcrash assumes *stdin*.

Etherpage/Pager software

WebTV Networks, Inc. in Palo Alto uses Etherpage, a commercially-available software package licensed from Personal Productivity Tools, Inc., to provide the Operations group with a method for receiving alphanumeric pages from monitord and admind. Combined with internally-developed additions to the Etherpage application, this product allows the Operations group to receive nearly instant notification of problems detected by the WebTV Network.

genrom

Overview

Flash ROM images need to be split into pieces and compressed before being sent. This is accomplished with a tool called genrom. genrom can split a ROM image into pieces and regenerate the original image from the pieces, the latter being useful for verifying that the generation process succeeded.

Sending the entire flash image at once is impractical, because there may be more flash ROM than the Internet terminal has RAM. Since the Internet terminal needs to perform the download in discrete pieces anyway, the only disadvantages to sending the ROM image down in several parts are a slight increase in overhead and some additional complexity.

One major advantage of this approach is the ability to download several blocks that aren't adjacent, because address and size information can be embedded in each block of data.

The current flash ROM header looks like this:

Example 6-8 Flash ROM header

```
typedef struct FlashBlockHeader {
    /* minimum needed */
    ulong               magic;          /* value that ids this as flash */
    CompressHeader      compressHeader; /* generic compression info */
    ulong               flashAddress;   /* address at which to start */
    ushort              headerVersion;  /* what's in the rest of this */
    ushort              dataOffset;     /* offset from top of struct to data */

    /* required */
    ushort              blockNum;       /* which block # in the sequence */
    ushort              pad;
    ulong               totalCompLength;    /* useful for progress bar */

    /* optional */
    uchar               message[kFlashMessageSize]; /* null-terminated str */
} FlashBlockHeader;
```

The compression header, which is intended to be generic, looks like the following:

```
typedef struct CompressHeader {
    ulong       compressedSize;         /* how many bytes to read */
    ulong       expandedSize;           /* how big it's going to get */
    ulong       crc32;                  /* CRC-32 on *uncompressed* data */
    uchar       type;                   /* compression method used */
    uchar       flags;                  /* flag bits, see below */
    uchar       pad[2];                 /* (explicit 32-bit alignment) */
```

```
} CompressHeader;
```

Different kinds of compression are supported, but it's important to use one that's implemented in the boot ROM on the internet terminal ("none" and "LZSS" should work).

The "totalCompLength" field holds the total size of the ROM after it's been compressed, and appears in every header. For obvious reasons, this is filled in after the compression completes.

There are different kinds of flash parts in use by the Internet terminals. The one used on development boards has a single contiguous 4MB address range, with the 256K boot ROM at 0xbfc00000, the 2MB ROM filesystem starting at 0xbfc00000, and the 1MB code section starting at 0xbfe00000. The final 1MB isn't used.

One manufacturer uses a different part, which is similar to the above except that the code section (i.e. the "app" ROM) starts at 0xbf000000, and the final piece of the ROM isn't a solid 128K piece. The internet terminal currently deals with this odd final piece itself, but the change in address has to be dealt with by genrom.

The former ROM is known as a "bfeapp" ROM, and the latter is known as a "bf0app" ROM, because of where the code section starts. In addition to the individual ROM files (which must be called `partNNN.rom` for the flashromd to find them), genrom creates a file called `rominfo.txt`.

The contents of the file look like:

```
parts=8
crc=0x60bc0e35
```

This tells flashromd how many parts there are, and what the 32-bit CRC of the entire image (before compression) is. The CRC-32 is a 32 bit checksum used for error detection and checksum verification of files. This file is generated at the same time as the ROM images, and should be copied with them. Having this file ensures that flashromd can detect if parts of the image have been inadvertently removed, and that it won't go looking for extra parts if somebody overwrites a 9-part set with an 8-part set (leaving an extra part that shouldn’t be downloaded).

At present, the file format is **NOT** flexible, and the `rominfo.txt` file should **NOT** be edited. The file **MUST** be present for the flashromd to work correctly.

The Control Script

The ROM image was once a single 1MB file that got cut into small contiguous pieces. Currently, it is a 3MB file that begins with a boot ROM image that requires special treatment, or there are two input files that don't have contiguous start addresses. Different sections of the image may require different block sizes, and some parts of the ROM (e.g. the "nvram" in the top 8K of the approm) shouldn't be downloaded.

To deal with this, the current incarnation of genrom uses a "control script" to control the processing of the files. A simple script for creating a bfeapp ROM from a "unirom" file looks like this:

Example 6-9 Sample script for creating a bfeapp ROM

```
oldheader=true
romtype=bfeapp

file=unirom
  # boot ROM, runs from 0xbfc00000 to 0xbfc3ffff
  addr=0xbfc00000
  blocksize=256k
  len=256k
    DoMakeBootRom

  # romfs and approm, runs from 0xbfc40000 to 0xbfeffff
  len=2816k
  blocksize=128k
    DoMakeRomParts

DoFinish
```

See “Control Script Definitions” on page 79 for a complete description of the control script language.

To avoid having to carry a lot of control scripts around, common control scripts are compiled directly into genrom, and selected by using a control script name that begins with a '+'. The list of scripts can be viewed by running genrom with no arguments.

Compressing ROM Images

The command for converting a ROM image into pieces is:

Usage: `genrom -s scriptName [-m messageFile] [-n] [-v]`

`-s scriptName` Specifies the file with the control script in it. If the name starts with a '+', genrom looks in its set of cached scripts for a match.

`[-m messageFile]` ROM images generated with "`oldheaders=false`" can hold a short message in each part. This message is displayed while the image is downloaded. See “Message Files” on page 78 section for more details.

`[-n]` Do nothing (just like cvs -n or make -n). This pre-tests the compression, making sure that all of the necessary files are there and that none of the commands does anything obviously wrong. It also dumps the control script after it has been processed, which can help to find bugs in the script, and provides a way to see the '+' scripts without looking in the source code.

`[-v]` Toggle the "verbosity" flag (default is on).

In most cases, you should be able to do everything you need just be specifying the name of the script. For example, a bfeapp "unirom" would be converted with:

```
% genrom -s +bfeapp-full
```

and a bf0app "approm.split" and "bootrom.split" combo would be converted with:

```
% genrom -s +bf0app-full
```

Verifying ROM Images

The command for converting the individual parts back into the original image is:

Usage: `genrom -d -s scriptName [ -n] [ -v]`

The "-d" flag tells it to decompress instead of compress (just like gzip -d or compress -d); all other arguments are the same. The script must be the same as the original or the conversion will fail. For convenience, the name of the script used can be found in the header of the `rominfo.txt` file.

If all goes as it should, you will have duplicates of the originals, only with "GEN" prefixed to the name. So if you created a bfeapp ROM from "unirom", genrom -d will create "GENunirom".

Genrom computes a CRC-32 on the entire uncompressed input, boot ROM and all, and stores it in the `rominfo.txt` file. If the CRC doesn't match, an error message will be displayed.

If you don't have faith in the CRC-32, you can then compare the binary images directly with something like:

```
% cmp unirom GENunirom
```

If the image didn't generate a boot ROM, then the first 256K of the romfs file will be zeros. You can still do a compare by skipping past the first 256K of both files, which you can do with /usr/bin/cmp on bsdi or solaris (but not GNU cmp):

```
% /usr/bin/cmp unirom GENunirom 262144 262144
```

Displaying Headers

Usage: `genrom -l partName [ partName...] [ -v]`

This causes genrom to read the "part" files and print out the contents of the header, making it possible to verify that the addresses used when generating the ROM image are correct. You can quickly verify an entire set with something like:

```
% genrom -l bfeapp-part*
```

Each new-style flash part header has a 32-byte "message" field that gets displayed as the blocks are sent down. If the field is left blank, a default message is displayed.

Message Files

The message file has a rather crude format. Lines with leading '#' signs are ignored, but blank lines count as empty messages (careful though: a line that looked blank but really had a space in it wouldn't count as empty). Strings may make use of some special format characters that are specified with '%'.

The currently supported format characters are:

`%p` - Insert the current part # as a decimal number.

`%P` - Insert the total number of parts as a decimal number.

`%v` - Insert the ROM version [not implemented].

`%%` - Insert a '%' sign.

`%#` - Insert a '#' sign (useful at the start of a line).

Other text is copied. Lines are truncated after 31 characters and null-terminated; long lines cause a warning, as does running out of message lines while there are still parts to go. If genrom runs out of message lines, it puts in empty strings.

Other Features

One feature of the flash ROM downloader on the Internet terminal that saves download time and allows updates of partial regions is that areas filled with 'joeb' at the start and end of a block can be ignored. This is most important when dealing with the "non-volatile storage" section at the end of the last flash block. By filling the end of the last flash block with 'joeb's in the source ROM image, genrom knows not to send down that part of the block, and so that section doesn't get overwritten.

This is somewhat unsupported, since it's difficult to recreate the original image successfully if the final block of a section is completely removed. There are different ways of solving this, but since the client really doesn't need this feature (and in fact may not support it), and the final ROM images should be filled to the brim anyway, there's not much point in improving this.

Control Script Definitions

There are two kinds of commands, variable assignments and actions. Variable assignments are "name=value", actions are "Action", comments being with '#' and cause the rest of the line to be ignored, and all whitespace is removed before processing begins. Case is insignificant.

There are five kinds of variables:

unsigned long

what it says

signed long

also what it says

size

unsigned long, but can be specified as "256K" or "3M"

boolean

may be "true" or "false"

string

character string, may not have spaces

compressType

values may be "none", "lzss", or "lzh"

Note: Variables without default values MUST be initialized to something.

The recognized tokens are:

romtype (string) (no default)

What kind of ROM this is, usually "bfeapp" or "bf0app". This is used when naming files and not much else.

blocksize (size) (128K)

How much uncompressed data to read at a time. This should be equal to the size of a flash block on the box.

skipgaps (boolean) (false)

Just leave it off.

oldheader (boolean) (false)

If set to "false" old-style version 0 headers are used. If set to "true", new-style version 1 headers are used. The advantage of the new headers is that they're more flexible and can include messages.

compression (compressType) (lzss)

Set the compression type. The boot ROM is never compressed; if you set it to something other than "none" it informs you of the error and does what it knows is right.

docrc (boolean) (true)

If set, include this section in the global CRC. If not, don't. It's useful to turn this off for "adjust" sections, but that's about it.

"file" (string) (no default)

File to read input from while compressing or write output to while expanding. While expanding the filename will have "GEN" prepended.

"addr" (unsigned long) (no default)

Flash address where the block will eventually appear. These MUST be on flash block boundaries.

"len" (size) (no default)

The number of bytes to compress (or expand into) in the current section.

"flags" (unsigned long) (0)

Sets the "flags" field in the flash block header. Does not affect the "flags" field in the compress header. Currently defined flags are:

- 0x0001 - after receiving a boot ROM, reboot and get "/current".
- 0x0002 - after doing download, don't put up button, just reboot.
- 0x0004 - reboot and log into service after flash d/l (needs 0x0002)

"DoMakeBootRom" (command)

Generate a boot ROM file. Creation of boot ROMs isn't something that will be useful for very long, since boot ROMs are going to be in mask ROM in the final product anyway, so some things just aren't checked. For example, the entire boot ROM image must fit in one piece.

"DoMakeRomParts" (command)

This tells genrom to grab "len" bytes of data from the "file", chop them into "blocksize" pieces, compress them with "compression", give them addresses starting at "addr" and increasing upward, with a file name of '"romtype"-partXXX.rom'.

"DoSkip" (command)

Used for skipping over a piece of the input file. Not the same as a seek, because the skipped-over sections are considered part of the flash address space. This is really meant for skipping over the boot ROM section of the romfs if you don't feel like generating a boot ROM.

"DoFinish" (command)

Tell genrom that you're finished. If you're compressing the parts, this causes the `rominfo.txt` file to be written, and then genrom goes back and writes the "total compressed length" field in the header of all the parts. When expanding, this causes genrom to compare the CRC from `rominfo.txt` with the one it just created.

The cached scripts provide examples of how to use these.

Harvester Statistics

TBD

idzify

Overview

CCMI and other telco database vendors distribute their data in a format that is easily parseable by advanced languages like COBOL, because the transition to a more efficient language would cause most of the CPUs in use by the telco service industries to fail dramatically.

Because the software from LAA expects the input to be provided in the ASCII format, translating the databases into a more compact binary form is inefficient because each record would have to be translated back before printing.

However, the databases are large and don't fit entirely in memory, making access slow. To speed this up, the files are converted into a compressed form with an index, called an "idz" (indexed/compressed) file. This trades a slight increase in CPU load (assuming RLE7 compression) for a possibly tremendous reduction in I/O thrashing.

For idzify to work, the input file must:

- be composed of fixed-length records
- have a fixed-width "key" field at the start of the line
- have records sorted in ascending order
- have records that end in CRLF (that's easy enough to turn off though)

In other words, it should be a phone database supplied by CCMI. When the LAA software stops being used, idzify will become obsolete.

Using idzify

Usage:

```
idzify [-v] [-m method] infile.txt outfile.idz
   or: idzify [-v] -d infile.idz outfile.txt
   or: idzify [-v] -k infile.idz
```

The first form converts "infile.txt" into an Idz file called "outfile.idz". The optional [-m method] argument lets you try different compression methods. The default method, "rle7", works about as well as anything else. "lzh" may offer a small improvement at the expense of a great deal of time.

The second (-d) form converts "infile.idz" back into "outfile.txt". This is handy if you want to make sure that idzify didn't mangle the input.

The third (-k) form does a search for the key specified, and prints out the matching record.

The "-v" flag toggles verbose output mode.

Installation Tools

These tools are used to install a freshly built service or content release to the test network or to the production network.

Scripts Overview

Primary Scripts

InstallBuild Installs a specified build to a set of machines.

InstallContent Installs specified content build to a set of machines.

remote-exec General purpose command for executing files on sets of machines.

list-hosts Lists machines specified by command arguments.

available Shows content and builds available for installation.

Primitives (InstallBuild)

PostInstallSl Sets symlinks after a build is installed.

PreInstallCheck Checks destination machine for build install suitability.

VerifyCopy Verifies installation against a set of mtree boms.

Primitives (InstallContent)

PreContentInstallCheck Checks destination machine for content install suitability.

SwitchContent Switches content on a destination machine to the specified version.

UnpackAndVerifyContent Unpacks content tar archives and checks them against mtree boms.

copy-content Run locally to copy a given content release to a single machine

Libraries and Classes

CPC.pm General purpose perl library.

ConfigFile.pm Perl5 class that uses service

configuration files and list-services to yield simple hostname queries.

MachineManager.pm Perl5 subclass of ConfigFile. Provides easy support of machine list related arguments like "-production" for scripts.

ReleaseCopier.pm Perl5 subclass of MachineManager. Implements service installation (currently based on rdist) and verification.

ContentCopier.pm Perl5 subclass of ReleaseCopier. Extends ReleaseCopier to support content build installations.

Host specification:

All the primary scripts utilize MachineManager.pm and its superclass to implement a standard mechanism for specifying target hosts. The commands are currently grouped in two catagories. See Example 6-10 and Example 6-11.

Example 6-10 #1 Machine Selection Defaults

```
    -test                              # Test Network
    -thrash                           # Thrash network (load testing)
    -trial                             # Thrash network (trial testing)
    -demo                              # Thrash network (demo)
    -production                       # Production network (production)
```

Example 6-11 #2 Selection Commands

```
    -build version                  # Specify build version
    -all                              # Add all hosts from config file
    -except host[:host...]        # Remove host(s) from working list
    -machine host[:host...]       # Add host(s) from working list
    -config file                    # Specify config file
    -wns                              # Add hosts with NoStart
    -cat category[:category...]  # Add hosts for this category
    -serv name[:name...]          # Add hosts implementing service
name(s)
    -bin name[:name...]           # Add hosts running binaries
    -ug usergroup[:usergroup...] # Add hosts for group(s)
    -wons                             # Add hosts without NoStart
   -exceptwons                          # Remove hosts without NoStart
    -exceptwns                             # Remove hosts with NoStart
    -exceptcat category[:category...]   # Remove host(s) in this
category
    -exceptserv service[:service...]    # Remove host(s) implementing
name(s)
    -exceptbin binary[:binary...]        # Remove host(s) running
binary
    -exceptug usergroup[:usergroup...]  # Remove host(s) in this
usergroup
```

These arguments can be used in any combination and the tools will enforce required arguments. For example, any arguments that end up running list-services will force you to specify a build version:

```
-test requires -build
-all  requires -build (and a config file for that matter)
```

The queries are just a standardized interface to various arguments supported by list-services. Precedence is as you would expect, all "additive" queries are processed first (e.g. -bin mailtod). Next all "exception" queries (e.g. -exceptwns) are processed. The queries are just a standardized interface to various arguments supported by list-services. Finally explicit additions followed by explicit exceptions. List-hosts is a good tool to test out potential host selection arguments to make sure you're getting what you want.

The "Defaults" like -test are more or less equivalent to specifying an explicit config file (-test uses test.solaris2.cf in the specified build's config directory) and adding -all. You can except individual hosts or sets of hosts matching a criteria using -except...

Note that although fully qualified domain names like isp.bryant.webtv.net appear in .cf files, ConfigFile.pm will automatically map "isp" and "iap" to "admin". Certain names (currently only www.excite.com) are also excluded.

Other Notes

Mtree

Mtree has been enhanced by adding the notion of exceptions to its specification creation and checking abilities. This makes it possible to have mtree ignore specified directories entirely when doing its checks.

InstallBuild

InstallBuild currently utilizes sdist as the primary copy engine for getting things moved over the wire. This is not a good thing because rdist is remarkably stupid in not catching some problems and gleefully returning a 0 exit status. Eventually this will be changed.

Options on InstallBuild

-k: no prompt for password. Requires SSH to do public/private key encryption between two or more machines.

-m: specify a list of machines. The default is all machines. The dot files for InstallBuild include a number of standard aliases. New versions of InstallBuild will eliminate the need to itemize lists of machines.

-s: select a directory which holds a particular version of the software.

InstallContent

InstallContent follows a different installation model than InstallBuild. It first creates gzip'd gnutar archives of content, and then copies these files using scp in parallel to the destination hosts.

Misc

All the scripts check return codes nearly every operation, and for the most part take appropriate action like stopping and complaining loudly if an error is encountered. When copies are complete, the copied files are checked against an mtree specification.

keytool

Usage: keytool <randfile> <keyfile>

keytool deals with the creation of session keys, used by the service as encryption keys for the wtv-tickets, one of the layers of system security. It generates a file with the new session key and the previous session key. At present, it is run periodically by hand, but will likely be run automatically every 24 hours in a future version of the service (Likely version 1.5).

monitord

Overview

`monitord` is a WebTV Networks proprietary tool developed to monitor the availability of the WebTV Network-specific services on the production machines. Monitord runs at frequent intervals, tests various components of the service and reports is success/failure information via email and pager.

The `monitord` service wakes up periodically and executes a series of tests designed to verify that the service is working correctly. The tests executed by monitord are a subset of the individual service tests (e.g. when monitord tests flashromd it doesn't execute all of the tests that "testFlashRom" does).

Only one monitord is needed per service, with the caveat that different monitords should be used for different service groups. If the "alpha" users have access to the latest flashromd, then the latest monitord should be used to monitor that service. Otherwise monitord may attempt to execute older tests on newer services, and start reporting failures when everything is working properly. monitord typically runs on an "admin" host.

There is no need for terminals to contact monitord. The server doesn't take requests. Results are sent out through e-mail.

The configuration for monitord is the list of services in the config file. All services that accept requests are monitored. Services without specific tests are simply issued a "help" request to verify that they're alive. The Radius servers have special configuration statements, and are tested differently from the other services.

Other configuration options allow control over the interval between scans and the e-mail addresses were problem reports are sent.

The monitord log file shows progress information, and has some level of detail about the nature of failures encountered.

For added robustification, monitord will time out and report a general failure if the total test time exceeds a certain limit. If monitord doesn't send out an "all's well" message every few minutes, a "parole daemon" will assume that monitord is dead and start sending complaints.

monitord Configuration

`monitord` has a configuration section within webtv.solaris2.cf, the service configuration file. It is relatively brief, and largely self-explanatory. See Example 6-12 for a typical monitord configuration.

Example 6-12 Sample monitord configuration from the WebTV Network webtv.solaris.cf config file.

```
<monitord>
     UserRoot /usr/home/production/webtv/userstore/shared/mail

     RequireNFSUserRoot true

     LogFile /usr/home/production/webtv/log/monitord.log
     TestTimeout 300
     TestFrequency 300
     SocketTimeout 20

     ; max length of messages sent to a pager address
     MaxPagerLength 80

     ;ReminderFrequency 3600

     WarnAddress monitorfolk@corp.webtv.net
     WarnPagerAddress production-warn-pager@corp.webtv.net
     FailAddress monitorfolk@corp.webtv.net
     FailAddress monitorfolk-fail@corp.webtv.net
     FailPagerAddress production-fail-pager@corp.webtv.net
     ParoleAddress production-monitord@parole.webtv.net

     ; monitord clauses below limit testing on a given machine to the
services
     ; used by a specific user category
</monitord>
```

TestTimeout represents the maximum duration of the complete set of tests, in seconds, before monitord gives up and records an error.

TestFrequency represents how frequently, in seconds, monitord runs its battery of tests.

SocketTimeout represents the maximum duration of any one test, in second, before monitord gives up and records an error.

MaxPagerLength is the number of characters in the longest message monitord will send to attached pagers.

The different Address parameters determine where monitord sends messages. ParoleAddress is a special case. With a frequency of TestTimeout, monitord will send a report to ParoleAddress. If the daemon that receives mail at the ParoleAddress does not get a report, it will record an error and send a message of its own out reporting that monitord may be failing.

Monitord Message Analysis

Monitord messages can indicate both transitory errors and serious problems. In general, problems that affect multiple machines or multiple services tend to be serious, requiring immediate attention. Errors in single services on a machine running a number of different services tend to suggest transient problems. If a failure message persists through several rounds of monitoring, it should be investigated, even if it seems to be a relatively benign failure.

Where Mail Goes

There are two kinds of messages sent out by monitord, warnings and failures.

Messages from monitord Tests

`monitord`, as implemented at WebTV Networks, Inc. in Palo Alto, produces two types of notification of problems. It sends a very brief email notification into the Etherpage system, which is then forwarded to the alphanumeric pagers of the appropriate members of the Operations group. It also sends a longer email message to a designated email address. In both cases, messages produced by monitord contain three pieces of information: the machine producing a monitord message, the port of the service reporting a message, and the type of message.

In all cases, the port numbers listed in messages can be correlated with service names by looking in the webtv.solaris2.cf configuration file in ~production/webtv/config directory on any production machine. Port 1666 is a special case; warnings or failures on port 1666 can represent a number of things. Further information on the meaning of a warning or failure on port 1666 can be used by querying statusd on the troubled machine. See “Status Report and statusd” on page 112 for more information on statusd.

For monitoring purposes, warnings are defined as new errors during monitord testing, while failures are warnings that persist over two or more sessions of monitoring. Recoveries are recorded when a test that previously encountered an error succeeds. At WebTV Networks, Inc. in Palo Alto, monitord is currently configured to send page notification for all failures and recoveries, while email notification is sent for all warnings, failures, and recoveries.

Typical Alphanumeric Page Message

Example 6-13 is a typical example of the alphanumeric pager output of a monitord test failure. It includes the name of the machine triggering the message, the port number of the service producing that message, and whether the message is a warning (‘W’), failure (‘F’), or recovery (‘R’).

Example 6-13 Alphanumeric page notification of a problem discovered by monitord.

```
03:F: arnold;
postoffice-1:1666:F,
postoffice-2:1666:F
<1>
```

In this example, the simultaneous warnings across the same machine class suggest that something is troubling a specific part of the service. During the time when this example was recorded, WebTV Networks, Inc. was pushing out a bulk mailing to several thousand customers through the postoffice machines, resulting in load warnings.

Typical Email Message

Example 6-14 is a typical example of the email output of a monitord test failure. It begins with a notification of the completion of some number of tests, followed by the testing duration and a summary of messages. Each notification contains information on the service affected, the machine it runs on and the port it uses, as well as some information on the particular error.

Example 6-14 Email notification of a problem discovered by monitord.

```
The monitord on storefull-199 (pid=27273) completed 69 tests.
Testing took 33 seconds, finishing at Mon Feb  3 19:36:45 1997.
There were 0 repeat failures, 8 new failures, and 0 recoveries.

--- failures (0):

--- warnings (8):
  wtv-explore on storefull-199.iap.bryant.webtv.net:1624 (error=Test failed)
  wtv-explore on storefull-198.iap.bryant.webtv.net:1624 (error=Test failed)
  wtv-home on storeless-199.iap.bryant.webtv.net:1612 (error=Test failed)
  wtv-channel on storeless-199.iap.bryant.webtv.net:1626 (error=Test failed)
  wtv-aroundtown on storeless-199.iap.bryant.webtv.net:1628 (error=Test
failed)
  wtv-home on storeless-198.iap.bryant.webtv.net:1612 (error=Test failed)
  wtv-channel on storeless-198.iap.bryant.webtv.net:1626 (error=Test failed)
  wtv-aroundtown on storeless-198.iap.bryant.webtv.net:1628 (error=Test
failed)

--- recoveries (0):
```

In this particular example, the simultaneous warnings across a number of different services and machines indicate a serious problem requiring investigation. In this instance, the content tree disappeared, triggering warnings on all of the services dependent upon it.

Log Messages from monitord Tests

TBD

Log Messages from Specific Tests

TBD

Monitord and External Content Partners

`monitord` can also be configured to monitor external content partners, and send messages if their sites fail the monitoring tests. Currently, all monitord messages are sent to the same set of addresses. However, WebTV Networks is currently developing the capability to send different types of monitord messages to different mail aliases.

move-store

Overview

move-store is a new tool for moving users between categories. It includes a wide range of options to make it a flexible tool for a variety of tasks. It makes it easy to move a single subscriber, and more efficient to move a large number.

Furthermore, if any type of error occurs, move-store tells you what it did and what it didn't do so that you can go in, if you need to, and fix things by hand.

The following are guidelines on its use:

- You can only use move-store to move a user (all users for a subscriber, actually) from one production category to another, or one daily category to another, not from daily to production or vice versa.
- You only need to use move-store when you're moving users between user categories that are running on service groups that **don't** share a user store.

To move all of the users for a production subscriber from one service to another, you don't need to use move-store if their store is already in the same device. You only need to use setdb (See page 111). But to move all users for a regular production subscriber from normal to alpha, you do need to use move-store.

Normal Use

```
move-store -from from_userstore_directory
             -to to_userstore_directory
             -subscriber subscriber_id
             -category new_category
```

will:

- set a lock in the database to prevent all of a subscriber's users from logging in.
- send a notification that powers off their box.
- move their storage from `from_userstore` directory to `to_userstore` directory.
- send notifications to cancel the poweroff notifications.
- unset the lock in the database.

Options

The important things that "move-store -help" tells you are:

-from: Source userstore base dir (contains mail dir, ...)

-to: Destination userstore base dir (contains mail dir, ...)

-subscriber: data for all users associated with the subscriber are moved

-category: New user category for the subscriber

-noCategoryChange: Don't change the category the subscriber is in.

-ignore: Ignore extra files and directories in the from directory.

-quiet: Don't print a list of the files copied

-noDelete: Don't delete files from the from directory after they are copied

-deleteOnly: Delete files from the from directory without copying them to the to directory; this is useful after running the command with the noDelete option and checking that everything went well. This option also sets the -noLock, -noCategoryChange, and -noNotify flags.

-noLock: Don't prevent the subscriber from logging in while this command is executing. This should probably only be used if the locking is being done externally (e.g. all alpha users are already locked) or if services are not running.

-delay: delay in seconds after the last poweroff notice delivery (default 10)

-noNotify: Don't poweroff the subcriber's box before moving the subscriber's storage, and don't cancel the poweroff notice after moving the storage. This should probably only be used if you are sure that the subscriber is not using the service and will not reconnect while the command is active, or if the poweroff notice is sent before invoking this command and the cancellation notice is sent after the command completes.

Warnings

1. `aliasd` doesn't yet correctly deliver mail that comes in while the move is happening. you must either
   - not care that mail that arrives during the short move window will not be correctly delivered
   - have someone from Network Systems stop sendmails before the move and restart them after the move
   - have someone from Network Systems check the sendmail queues after the move and if a message was queued, direct it to the correct place.
2. If a user is using his box when he is being moved, the experience is unpleasant: the Internet terminal powers off, and if it manages to reconnect before the move finishes (which is unlikely) the user will be told to call customer service.

   Fixes to both of these are under development for 1.4.
3. The database sometimes fails to update a field without reporting an error; this will currently happen if you specify an invalid value for category. For now, be sure that the category you specify is correct, and then use querydb (see “querydb” on page 105) to check that the category was correctly updated.

   move-store revision 1.4 or greater works around this problem. Use ident `which move-store` to see the revision number; if no id string is found, it's 1.3 or older.

Retail subscribers have a special credit card number. If a subscriber in the "retail" user category has a non-retail credit card number, they do not get retail behavior, and if a subscriber in any category has a retail credit card number, they get retail behavior. The retail category should be considered a sign that the subscriber probably sees retailer-specific behavior, but the real decision is based on the credit card number. So: if you use move-store (or setdb) to move someone into or out of the retail user category, you will also have to ask a Database Administrator (part of Network Systems) to fix up their credit card number to match.

Example Use

Some links to the userstores on one production and one daily machine make the use of this command less error prone. See Example 6-15 and Example 6-16 for more details. These examples are specifically related to the service configuration at WebTV Networks, Inc., but should be easily adaptable to any other service configuration. They show all of the cases in which you should currently need to use move-store; all other category changes (alpha to beta on production, for example) can be done using setdb (see “setdb” on page 111).

It’s advisable to run "script" before you run move-store so that if anything goes wrong, you have a complete record of what was printed to your screen so that you can go back and repair any damage.

Example 6-15 Moving users from one service group to another on daily

logon as `production` andget the subscriber id and current category of the user you would like to move, using querydb (see “querydb” on page 105)

```
querydb -all username
cd ~production/sullivan/userstores
```

Moving all of a subscriber's users from normal to alpha

```
move-store -from normal-daily -to alpha-daily -subscriber sub_id -category alpha
```

OR

Moving all of a subscriber's users from alpha to normal

```
move-store -from alpha-daily -to normal-daily -subscriber sub_id -category normal
```

Example 6-16 Moving users from one service group to another on production.

logon to admin-101 as production, and get the subscriber id and current category of the user you would like to move.

```
querydb -all username
cd /usr/home/sullivan/userstores
```

Moving all of a subscriber's users from normal, retail, or pionr to bryant, alpha, beta, or gamma

```
move-store -from alma -to bryant -subscriber sub_id -category bryant
(or alpha, beta, or gamma)
```

OR

Moving all of a subscriber's users from bryant, alpha, beta, or gamma to normal, retail, or pionr

```
move-store -from bryant -to alma -subscriber sub_id -category normal
(or retail or pionr)
```

PhoneDB Tools

The first group of PhoneDB tools are intended to be used on built "PhoneDB" databases to extract information from them. The second group of these tools work on the raw CCMI database itself, and includes the tools necessary to transform raw CCMI data into a PhoneDB that the WebTV Network can utilize. For more information on the tools necessary to manipulate the data used to build the service's PhoneDB in the first place, see "PhoneDB Maintenance" on page 36.

Index

PhoneDB Overview

Useful Terms and Definitions

Tools - Summary and Syntax

- lookuppop and showpop
- popcoverage
- popusage
- listlocal
- dumpnpas
- exportnpas
- dumppops
- changepop
- eainfo
- distcompare
- localreport
- testlookup
- balancereport
- Special dumpnpas Fields

Raw PhoneDB Tools (those that work on the CCMI database directly, not our PhoneDB)

- tolinq
- rgenphonedb
- rcallcost
- rdumpnpas

PhoneDB Overview

The PhoneDB is a large file that is mapped into the address space of many different processes. Because this is done with mmap() as a shared, read-only resource, only one copy of the data appears in physical memory. Because access to the data is controlled by the virtual memory system, the PhoneDB is demand-paged, keeping its memory footprint to a minimum.

For details about the contents of the PhoneDB file, see the PhoneDB.h header file. Most programs shouldn't need to look at fields in the database directly. Look at the programs in this directory for ideas, or at preregistrationd and headwaiterd.

Useful Terms and Definitions

npa

area code (the first 3 digits of a 10-digit number in the USA).

nxx

prefix, a.k.a. exchange, the first 3 digits of a 7-digit number in the USA.

POP

Point Of Presence, a modem bank connected to a service provider. The client calls a POP to connect to the Internet.

Rate Center

When you place a call, the cost of the call is based on the distance between your rate center and the rate center of the place you're calling.

RBOC

Regional Bell Operating Company, e.g. the local PacBell office.

Telco

telephone company

V&H

Vertical & Horizontal coordinates of the rate center, used for distance calculations. It's possible, but complicated, to transform these into latitude & longitude.

Tools - Summary and Syntax

All of the tools are actually just one program that checks its argv[0] to see what it's supposed to do. They all take one standard argument:

`[-i phonedb] the name of the PhoneDB file` (default is "./PhoneDB")

Most people will probably want to specify the PhoneDB file sitting under "webtv/config" in their home directory. As an example, to look up the POP assignment for a particular number, you might do:

```
% cd ~/webtv/bin.solaris2
% ln -s phonetool lookuppop
% lookuppop -i ../config/PhoneDB 415-614-5539
```

lookuppop and showpop

Usage: `lookuppop [ -i phonedb] phone-number`

Usage: `showpop [ -i phonedb] phone-number`

Look up the POP assignment for a particular phone number. This tells you what POP a client will dial when it calls in. It's not necessary to use parentheses, hyphens, or other punctuation in the number. All characters other than digits are ignored.

The "showpop" command does essentially the same thing as "lookuppop", but in a safe and friendly manner that could be shown to customers. Pop-o-Rama is based on this tool, with a web interface suitable for Customer Care

Using a short form of the phone number (npa-nxx) as an argument is also allowed.

popcoverage

Usage: `popcoverage [ -i phonedb] max-cost`

Determine the percentage of npa/nxx's for whom the cost of dialing the primary POP is less than or equal to max-cost. This can be used to evaluate a set of POPs.

For CCMI data, specify the cost in 1/10 cent for a 30-minute call, i.e. to evaluate POPs that cost less than $1.50 for half an hour use 1500. For V&H this isn't going to be useful.

popusage

Usage: `popusage [ -i phonedb] pop-number`

This will show all of the exchanges that use the specified number as the primary or secondary POP. This is useful for figuring out whether a POP is actually being used. A POP will be included in the "primary" list if it's in the third or later slot but could be used because of provider rotation.

Keep in mind, that this does **NOT** tell you which areas the POP could support, only the areas that it does support. Regenerating the PhoneDB with a different set of options could result in completely different output.

listlocal

Usage: `listlocal [ -i phonedb] [ inv|flat]`

Print a nicely formatted list of all exchanges for which there is a local POP. This is meant for distribution by the licensees to stores where the WebTV Internet terminals are sold. It may be distributed in some sort of book form at some point, but currently this purpose is served by Pop-O-Rama (see "lookuppop and showpop" on page 94), which is accessible online to prospective customers as well as the vendors.

If "inv" is added, the output is the inverse of the usual report, i.e. the list of areas that don't have coverage.

If "flat" is added, the report is one long column of "npa-nxx" entries, meant for diffing.

dumpnpas

Usage: `dumpnpas [ -i phonedb]`

Dump a summary of npa/nxx info for all known exchanges. See end of this section for more details on how this information is presented.

exportnpas

Usage: `exportnpas [ -i phonedb]`

Dump a summary of npa/nxx info for all known exchanges, in comma-separated fields that can be easily loaded into Excel or other programs.

dumppops

Usage: `dumppops [ -i phonedb]`

Dump the set of known POPs, including the global default POP. Because overrides are processed at PhoneDB generation time, they aren't shown.

changepop

Usage: `changepop [ -i phonedb]`

Change a POP assignment in the PhoneDB database. This works the same way an npa/nxx override does. The effects are immediate (i.e. currently executing programs will get them) and permanent. This program is dangerous, and should only be used on a **copy** of the PhoneDB file except in dire emergencies.

eainfo

Usage: `eainfo [ -i phonedb] phone-number`

Print some information about the exchange area that the phone number is a part of. This includes the EAA (which is used by CCMI in various calculations), the location name, and a list of the other npa/nxx combinations that are in the same exchange area.

Exchange areas are interesting because all numbers within one are equivalent for billing, and therefore have the same POP assignment. The PhoneDB does NOT assume that they're all in the same area code, so they may have different dialing patterns.

Using the short form of the phone number (npa/nxx) as an argument is allowed.

distcompare

Usage: `distcompare [ -i phonedb]`

Runs through the PhoneDB, looking for exchanges for which a more expensive POP was chosen in favor of a cheaper one. Ignores exchanges for which cost information isn't available. (This feature is now built into the PhoneDB generation process.)

localreport

Usage: `localreport [ -i phonedb]`

When fed a long list of phone numbers like this

ANI: 062013631234

1-201-427-5678

(201)661-1234

1 2 3 201-691-5678 4 5 6

It outputs some nicely formatted information like this:

06,201-363-1234,LEONIA ,NJ,2

--,201-427-5678,HAWTHORNE ,NJ,1

--,201-661-1234,NUTLEY ,NJ,2

--,201-691-5678,NETCONG ,NJ,0

The first column is the ANI category, the second is the phone number, the third and fourth are city and state, and the fifth is the number of local POPs available to a customer caling from that number. The numbers are read from *stdin*, so it can be fed the output of a *grep* on a logfile or a query on a database. Errors are sent to *stderr*. See comments in `PhoneTool.c` for details on how it figures out where the phone number starts and ends.

testlookup

Usage: `testlookup [ -i phonedb]`

When given a list of phone numbers, outputs the POP assignments for those areas in a single-line format suitable for diffing. This is meant to be used as part of a regression test.

balancereport

Usage: `balancereport [ -i phonedb]`

Report the effects of the current load-balancing parameters on a list of phone numbers. Input should be two columns of numbers, with silicon serial number on the left and phone number on the right.

(The "balancereport" command may not be available. It's only built if certain things are defined, because it looks for the Oracle libraries even though it doesn't always use them.)

Special dumpnpas Fields

The dumpnpas output includes several abbreviated fields that can be useful to know about when investigating a problem. A normal entry looks like this:

```
+201-200 B0 JERSEYCITY,NJ EST + (05006,01409)(05006,01409) 0 07307 C
```

The fields from left to right are:

"+" - this exchange is active. Inactive exchanges, such as those for countries out in the carribbean, are marked with "-". The service doesn't assign POPs or send tellyscripts to people calling from inactive exchanges.

"201-200" - NPA/NXX for this exchange.

"B" - exchange is handled by a Bell Operating Company (BOC). Other values are "I" for Independent Operating Company (IOC), "U" for unsupported operating company, and "?" for bad CCMI entries.

"0" - this exchange is primarily used for voice. Other values are:

1 - cellular

2 - paging

3 - centroid

4 - miscellaneous (PCS, Mobile, etc)

? - bad data in CCMI

"JERSEYCITY" - city name, 10 chars, all uppercase.

"NJ" - state, with extended codes like "PR" for Puerto Rico.

"EST" - time zone is Eastern Standard Time. (Daylight savings time is factored in by the services, not the PhoneDB.)

"+" - this exchange supports daylight savings time. Some places out in Arizona have a "-" here.

"(05006,01409)" - major V&H coordinates, sometimes known as the "over 40" coordinates. Used to price calls outside 40 miles.

"(05006,01409)" - minor V&H coordinates, sometimes known as the "under 40" coordinates. Used to price calls closer than 40 miles. Almost always the same as the major coordinates, except in major metropolitan areas like Los Angeles.

"0" - rating problem code. (See below for details.)

"07307" - zip code of something nearby. Not terribly accurate, but it'll give you something close.

"C" - dialing pattern code. An index into a table of dialing patterns. Often accurate, occasionally not. (See below for details.)

Some places have trouble when the service tries to price calls, perhaps because the local calling area for the exchange is unsupported. These show up in the "rating problem code" column. For example:

```
+204-200 ?3 LANGRUTH   ,MB CST + (05006,05713)(05006,05713) 1 00000 E
+205-927 U0 CENTRE     ,AL CST + (07298,02332)(07298,02332) 5 35960 B
+207-862 U0 HAMPDEN    ,ME EST + (03794,01322)(03794,01322) 6 04444 A
```

These are visible when doing a "lookuppop" on the exchange as well. The codes used are:

0 - No problem.

1 - Bad origin (result code 1 from LAA).

2 - Not in DB (result code 7 from LAA).

3 - Error in DB (result code 3 from LAA).

4 - BIG error in DB (remaining nonzero LAA result codes, e.g. 2)

5 - LCA not supported.

6 - Local call not supported.

7 - No residential rates available; rates used are commercial.

8 - Local plans were listed but didn't actually work for this exchange.

The dialing pattern table tells you how many digits should be dialed. Calls are divided into four categories, HL (Home area code/Local), HT (Home area code/Toll), FL (Foreign area code/Local), and FT (Foreign area code/Toll). Some places also have "permissive" dialing patterns in addition to the regular ones. As of Oct 1996, the table looks like the following example:

Example 6-17 Dialing pattern table.

#	HL	HT	FL	FT	PHL	PHT	PFL	PFT
A	7	1+10	1+10	1+10	n/a	n/a	n/a	(always
B	7	1+10	7	1+10	n/a	n/a	n/a	matches
C	7	7	1+10	1+10	1+10	1+10	n/a	FT)
D	7	---	10	1+10	10/1+10	n/a	1+10	
E	7	1+10	---	1+10	n/a	n/a	n/a	
F	7	1+10	1+10	1+10	1+10	n/a	n/a	
G	7	7	1+10	1+10	n/a	n/a	n/a	
H	7	1+10	7	1+10	1+10	n/a	1+10	
I	7	1+10	10	1+10	10/1+10	n/a	1+10	
J	7	---	1+10	1+10	1+10	1+10	n/a	
K	7	1+10	7	1+10	10/1+10	n/a	10/1+10	
L	7	1+10	10	1+10	n/a	n/a	n/a	
M	10	1+10	10	1+10	n/a	n/a	n/a	
N	7	1+10	7	1+10	1+10	n/a	n/a	
O	7	---	1+10	1+10	n/a	n/a	n/a	
P	7	7	1+10	1+10	n/a	1+10	n/a	
Q	7	---	10	1+10	10	n/a	10	

The client tries two different dialing patterns for most POPs. For cases where there's no "permissive" pattern, the service tries to do something reasonable, e.g. for pattern "B" FL calls the client will try an 11-digit pattern even though it doesn't say that it can. (To avoid calling MOMs, the only places where the client tries 7-digit calling first for FL calls are patterns 'B' and 'N'. The service will try it if 11-digit fails for patterns 'H' and 'K', since PFL says 11-digit should work.)

The reality is that the patterns don't always behave as expected, which is why the client tries more than one pattern and why Network Systems has "dpedit" (see “dpedit” on page 70) to clean up the rest. "---" in the above table means "can't happen", though there has been at least one case where an area code split made an 'E' FL call possible.

Raw PhoneDB Tools

Introduction

These tools operate on raw phone databases supplied by Bellcore or CCMI. The primary purpose is to convert a mountain of raw data into a compact form that can be used by our servers. Problems with these tools should be referred back to WebTV Networks Inc. in Palo Alto, specifically directed at fadden@corp.webtv.net

Index

V&H Database Overview

CCMI Database Overview

POP Files

Tools

rgenphonedb Error Messages

V&H Database Overview

The V&H database supplied by Bellcore (referred to as the "vnh" database throughout the source code, because it sounds the same when you say it fast) supplies basic information about each exchange, like location, 10-character city name, and time zone.

It does not provide any information about the cost of a phone call, though you can do a reasonable job of selecting POPs by just picking the ones that are closest.

Information about the raw database can be found on a few photocopied pages that Bellcore sent us.

In the 96/06/15 and 96/07/15 minorvh.dat files, the minor V&H coordinates for places in Alaska are thousands of miles off. **DO NOT** use the V&H coordinates for real customers.

CCMI Database Overview

The URATF and URCF databases supplied by CCMI provide everything that the Bellcore database provides, as well as all the data necessary to determine the cost of a call between any two points in the United States, Canada, or Mexico. It's also several times larger.

Information about the raw CCMI data can be found if necessary. Contact WebTV Networks, Inc. in Palo Alto if this is absolutely necessary. The actual processing of the data is handled by software licensed from LAA.

POP Files

The most important information provided by the PhoneDB database is what POP number to use from every npa/nxx in the supported region. Normally, each IAP used by the service will provide a POP file, which will need to be formatted and then built into a PhoneDB. See “PhoneDB Maintenance” on page 36 for more details on turning POP files into PhoneDB-ready formats. The PhoneDB format supports multiple POPs per npa/nxx (currently set to 2, so you get one primary and one alternate), as well as some special overrides.

A simple POP file would look like this:

```
---
#
# Artemis local POPs, 96/06/23
#
Provider: @ artemis 100 hourly "Artemis Research"
415-329-0511    # main number
510-644-0560    # Berkeley POP
---
```

Comments start with a '#', and cause the rest of the line to be ignored. Blank lines don't count.

A line starting with "Provider:" describes the provider of the POPs. A provider line must come before any POPs in the file, and the provider is associated with all POPs that come after it in the file. It's valid to have more than one provider per file, and you can have POPs from the same provider in different files so long as the provider line appears in each file. (If the fields don't quite match up, the PhoneDB generator program will yell at you.)

The first item on the provider line is a single-character symbol output by **exportnpas** (see “exportnpas” on page 95). It should be unique among the providers, or the report isn't going to be as useful as it could be.

The second item is the provider abbreviation. This tag is used to identify the provider. Two provider lines with different abbreviation fields are considered different providers. It's also used when looking for the appropriate tellyscript fragment to send down to the client. The suffix ".tsf" is added to the abbreviation, and a file with that name gets included in the tellyscript.

Warning: Don't add a new provider without adding a matching tellyscript fragment to `src/lib/tellyscript/scripts`.

The third item is the provider cost. This is a relative cost that doesn't mean much by itself. If two POPs will cost the customer the same amount, the PhoneDB generator will choose the one that costs us the least amount. The exception to this rule is that POPs more than 40 miles from the customer ignore the provider cost (this was done because some rather distant POPs have the same cost as not-so-distant POPs, and customers were concerned because their terminals were preferring the farther CNC one over the nearer UUNET one).

The fourth item is the billing method, which can be "flat" for flat-rate per user, "hourly" for hourly billing, or "per-port" for providers from whom the company purchases a fixed number of ports. Flat-rate providers are treated differently from the rest (they're never given out as a secondary POP).

The fifth item is the full name of the provider, used strictly for display purposes.

Following the provider line are the POP numbers, one per line. These must be ten digit numbers, though leading 1s are automatically stripped, as are dashes, parenthesis, and other garbage. The generator will give an error message if you have duplicate POP numbers, or if the POP number can't be found in the npa/nxx listing. The latter case happens most often when an area code splits into two; you can usually resolve it by calling the POP by hand and listening to the "you now need to dial xxx" recording.

There are a few special items found in POP files:

```
---
800-613-8199     GlobalDefault
800-555-4758     Special
415-614-5539     ComingSoon
ComingSoonPop: 800-555-4758
Override: 415-614 415-329-0511 415-233-0570
Override: 415-615 415-329-0511 0
Override: 212-*    800-555-4758 0
IfCostMoreThan:  15876    800-613-8199 0
---
```

The "GlobalDefault" is a tag attached to a POP number. It should be placed on exactly one POP (i.e. you need to have one, but it doesn't make sense to have more than one). This is the POP that will be assigned to customers calling from an npa/nxx that, as far as the database knows, doesn't exist. (This should only be caused by errors in the raw phone databases or out-of-date data.)

The "Special" tag should be used for POPs that aren't supposed to be part of the standard "find the closest" POP calculation, either because they're reserved for a special purpose (like a demo) or there's no distance information associated with them (like a 1-800 number). These POPs will only be used by overrides, and the cost to call them will always be zero.

The "ComingSoon" tag on a POP means that the POP isn't in service yet, but anybody who is local to that POP *and* isn't local to any other POP would get access via a special number. The number is defined by the "ComingSoonPop" line for the POP's provider. The POP in the ComingSoonPop line is tollfree (usually an 800 or 888 number), and can be from a different provider. Only one ComingSoonPop line may be specified for a provider, and it MUST be listed before any ComingSoon POPs.

The "Override" line allows you to explicitly override the POP assignment for a particular npa/nxx. This is factored in when the PhoneDB file is generated. The POP phone numbers have to exist, but it isn't necessary that the POP phone numbers appear in the same file as the overrides. You may specify one or two POPs. If you only want to specify one, put in "0" as the second.

If the area code is replaced with "*", all exchanges in that area code will have their POPs overridden.

You are allowed one "IfCostMoreThan" line, and may omit it entirely. It causes the listed POP to be used instead of the normal one if the cost of the call exceeds the cost value on the line. [What's a cost value? This isn't altogether clear. For V&H it's distance in V&H coordinates squared] This is useful for redirecting people in the middle of nowhere to a 1-800 number, which will end up being cheaper than having them call the local access number.

Tools

Usage: `tolinq`

Unlike all the other rawphoneDB tools, it requires a different syntax and different information. Its purpose is to determine the approximate cost per minute of a call. It also uses the raw CCMI database, instead of PhoneDB, and is useful for answering such questions as: "is it **really** cheaper to call point B from point A than point C from point A?"

You will be prompted for a variety of information, including the area code and exchange for both the origin and destination. You will be prompted for a variety of other information, most of which you will use the default values for with the following exceptions:

TOD Class -- Use **2**

Duration -- Use 30.0 as a benchmark

Carrier Code -- Use ATT as a benchmark

After hitting enter a bunch of times to allow the default settings to be used, you will get output telling you the expected cost of the call per minute.

Caution:This information should almost never be given out to customers directly. The telephone company will always have more accurate data, and should be the ones answering these questions for certain.

Our data from this tool is only meant as a guideline, and is primarily for use in determining if the customer does have a significantly cheaper POP available that our software overlooked for some reason.

rgenphonedb

Usage: `rgenphonedb -cf phonedb.cf`

This is the most important tool of this group. This is what generates the PhoneDB file from the raw database. It uses standard config file arguments, but you have to specify the file by hand. (At some point this should probably default to `phonedb.cf.`)

The different options are explained in the sample `phonedb.cf` file.

If V&H data is used, the files `nxxmstr.dat` and `minorvh.dat` are mapped from the `phonesrcdir` directory. For CCMI, `urat.idz` (rating data) and `urcf.idz` (rate center data) are used. The *idzify* tool (see "idzify" on page 81) is used to create ".idz" files from the text data that CCMI distributes.

The rest of the tools take two standard arguments:

`[-i phonesrc]` the value of "phonesrc" can be "vnh" or "ccmi", and determines which type of raw data the program looks for. The default is "ccmi".

`[-d phonesrcdir]` which directory to look in for the raw phone db files. The current directory is used as the default.

Usage: `rcallcost [-i phonesrc] [-d phonesrcdir] npa-nxx npa-nxx`

Determine the cost of a call between two points, using the data in the raw databases. This may not execute very quickly. The command should look like: `rcallcost 408-777 415-614`

Usage: `rdumpnpas [-i phonesrc] [-d phonesrcdir]`

Dump a summary of npa/nxx info. Much like "dumpnpas", but uses the raw data instead of the processed PhoneDB data. Occasionally useful for cross-checking CCMI against V&H.

rgenphonedb Error Messages

When generating the PhoneDB, you will occasionally see messages indicating a problem. Some messages you might see are:

Found Cellular Point 'Y' in 418-552, using default (0)

This field used to be 'Y' or 'N', but now it's a number from 0 to 4. CCMI should fix these entries.

Found time zone 0 in 860-270, using default (4)

They didn't set the time zone entry, so it defaulted to Pacific.

WARNING: 203-834 not found by LookupByNpaNxx

ERROR: unable to find npa/nxx for POP '203-834-6730' (203-834)

The first message is a generic warning from the "lookup" function. The second tells you that the exchange for the POP doesn't exist. Most often this happens because a POP number is bad. Calling the number will usually reveal the problem. (In this particular case, CCMI had erroneously moved an exchange from the 203 area code to the 860 area code as part of an area code split.)

WARNING: good com, bad res 215-693 to 215-693 (7)

Was able to do a "price this call using the default commercial plan", but asking for a specific residential caused an error. It turns out these places shouldn't have local calling plans (they're UOC or paging/cellular exchanges), so treating them as AT&T calls is probably correct.

NOTE: using pessimistic plan 5 instead of 4 for 216-350 to 216-573.

The "UseWorstLca" option was turned on, and the calculation found a plan that was more restrictive than the default. Plans 1-6 are commercial, and plans 7-9 are residential.

WARNING: worked for plan 1 but not 3 (result=3) 618-266 to 618-242

The service was able to price a call with local calling plan 1 but not with local calling plan 3. This is an error in the CCMI database.

NOTE: could not price 702-535 to 702-535 (result=7)

The attempt to price the call failed. This usually happens in unsupported LCAs or other screwed-up areas.

NOTE: unable to price call originating from npa 204

Not surprising, since this is in Canada. A summary of area codes is printed at the end of the run.

WARNING: inconsistency in EAA 0642103549 (309-244)

A consistency check failed. For example, there might be a very expensive local call in front of a very cheap toll call. It's a good idea to use "`lookuppop -i PhoneDB.new`" (use the name of the PhoneDB you just generated in place of "`PhoneDB.new`") and see if there's anything strange about the POP assignment in that exchange.

NOTE: free toll calling in EAA 0726301424 (912-728)

The assigned POP is a zero-cost toll call. Believe it or not, there are such things because of PUC-enforced tariffs. Expect to see these on the EAs for 912-728, 912-756, 912-986, 912-935, and 912-698.

WARNING: gave 7-digit dialing to cross-npa PseudoLocal (360-444 to 206-261)

This is a warning that the calculation converted a call from being toll to pseudo- local, and as a result gave a 7-digit dial pattern that crosses an area code. Some places require 7-digit dialing for local calls across area code boundaries, so this probably does the customer a favor, but it doesn't hurt to check occasionally. If the POP associated with the above complaint were 206-261-1234, you should try calling 360-261-1234. If you get something other than a recording, it's time to panic, since everybody in the EA for 360-444 is going to call that number. Setting the UnsupLCADistOnlyRadius parameter to -1 will prevent this.

Most of the remaining error messages are complaints about duplicate POPs in POP lists or other relatively mundane and self-explanatory errors.

Some of the messages contain result codes from the LAA call pricing software. Two examples:

WARNING: worked for plan 1 but not 3 (result=3) 618-266 to 618-242

NOTE: could not price 702-535 to 702-535 (result=7)

The possible results are (from the LAA documentation):

0. Successful.

1. Bad origin NPA & NXX. The origin NPA & NXX were not found in the URCF (or MRCF) file.

2. Bad dest NPA & NXX. The dest NPA & NXX were not found in the URCF (or MRCF) file.

3. Error in RATF file. An error was found in the RATF file and the call could not be rated. [these should be reported to CCMI]

4. Bad start time. Start time was not between 00:00:00 and 24:00:00.

5. Bad call date. Call date was not found in the RATF Calendar Table.

6. Service Type not available to the destination.

7. Service for the origin, destination, and carrier not provided in your Q-Tel 9000 Database. [usually associated with unknown LCAs]

8. Not in use.

9. Bad Service Type.

10. Bad Local Calling Plan.

11. Bad Time-of-Day Class.

12. Carrier not available at the origin point.

13. Used in Canadian LAA Toll Call Pricer.

14. Carrier does not support service requested.

15. Invalid V or H coordinate found in Q-Tel 9000 database.

querydb

Usage:

querydb [-all] [ID]

Queries the database about a particular user and returns a range of information about that user, where ID is either the clientid, subscriberid, userid, or username. See Example 6-18 for an example of the output of "querydb -all".

Other useful options include:

[-dict] which alters the output to be more easily parsed by automated tools. It prints out a series of field: value pairs instead of the more easy to read default format. Since this format will be basically invariant, any scripts utilizing querydb should use the [-dict] option.

[-includeCancelled] which includes cancelled users or subscribers in the search.

Example 6-18 Sample output of "querydb -all entropy"

```
[ClientID 8192b30000b002ca]
  ANI: 004082453551
  Dial Patterns: 8~408-245-3551|408-990-2255|1~408-245-3551|408-490-
0610|1~408-245-3551|888-329-0489|3~408-733-3266|888-329-0489|3~408-
733-3266|408-490-0610|1~408-733-3266|408-990-2255|1~408-733-3266|408-
988-5800|1~408-733-3266|408-492-1481|1~
  Last Dial Success: 9902255
  Last Script: 0x3313874b-0xf9cb1b98-base:17:-
|__ziplink:2:9902255|__cnc:2:4900610|__ziplink:2:9902255|__cnc:2:49006
10|__artemis:1:18006108918
  Prev Script: 0x33135dfb-0x40ad29e9-base:17:-
|__ziplink:2:9902255|__cnc:2:4900610|__ziplink:2:14089902255|__cnc:2:4
900610|__artemis:1:18006108918
  Mail Check Time: off
[SubscriberID 2644903]
  Name: Karl Larson {alpha}
  Address: 201 W Califoria Ave #324; Sunnyvale, CA 94086
  Send address to mfr: nope
  Phone Number: 408-245-3551
  Credit Card: VI 4128********8995, exp 0797
  Product ID: sony-webtv-000
  Billing Status: ok
  UserStore locked: n
[UserID 2644912]
  User Name: karll (Karl Larson) {Subscriber}
  No password.
[UserID 2645496]
  User Name: war (Stuart Lin)
  No password.
[UserID 2645843]
  User Name: famine (Dan Snyder)
  No password.
[UserID 2678225]
  User Name: entropy (Neil Laughlin)
  Password protected (length 8).
[UserID 3738434]
  User Name: Pollution (Jonathan Tourtellot)
  No password.
```

Real-Time Service Overview and overviewd

`overviewd` is an administrative service on the WebTV Network that monitors each service machine and outputs information to the real-time service overview HTML pages. The real-time service overview includes information on the load on each machine, as well as information on many of the services running on individual servers. For each service, overviewd outputs information on the number of processes of that service out of the total allowed, the number of connections to it out of the total allowed, and the port it operates on.

The HTML output of `overviewd` is updated automatically every ten seconds, and includes a breakdown of connection allocation. An explanation of the different categories of connections can be reached though a dynamic link attached to each machine name.

rename-link

`rename-link` is a tool which renames symbolic links in UNIX. It is used in the content installation tool (see "InstallContent" on page 84).

remote-exec

Forthcoming

Service Logs and Log Analysis

"Ideal" log files for certain services (e.g. `mailtod`, `crashrecord`) are empty, having had all the "uninteresting" messages filtered out of them (i.e., no new errors that haven't been dealt with routinely in the past). In reality, this will never happen, or if it does, it'll likely be because the mechanism for recording the log failed. Other log files maintain records of normal usage (e.g. logserverd, preregistrationd), and each entry need not indicate an error.

The WebTV Network records a variety of logs, including the service logs in the directory:

`~production/webtv/log`

Additionally, on each production machine there is a file called **user-events.log** in:

`~production/webtv/log/event_log`

which details all of the various transactions a given machine has in the course of providing the service. Each entry is labelled to indicate the time, the process involved, and records data on the transaction.

Briefly described below are the "major" service logs recorded by the WebTV Network.

1. **mailtod.log** - Found on the mailtod machines, it records the events pertaining to mail, including how it's handling MIME attachments, HTML based messages, and the like.

2. **logserverd.log** - Found on the storeless machines, it records the phone logs from the client, including data on any disconnections or failed connections since the last time a phone log was recorded for a given client, as well as data on successful connections.

3. **proxyd.log** - Found on the proxy machines, it records the events associated with serving http requests, transcoding pages, and page caching.

4. **preregistrationd.log** - Found on the preregistration machines, it records the events associated with a client being run through the preregistration process. This includes handing out tellyscripts, and recording client ANI information from the 800 service.

5. **headwaiterd.log** - Found on the storefull machines, it records the comparisons between tellyscripts the client currently has and what the current tellyscript should be and records when a new tellyscript is given to the client. It also records giving services to the clients as they connect to the WebTV Network.

Logs are also kept for the smaller services, and include the following, (many of these are logs that are recorded on more than one and frequently all production machines)

storefull machines - cookied.log, explored.log, favorited.log, flashromd.log, harvestd.log, registerd.log, servicelauncher.log, setupd.log, statusd.log

storeless machines - aroundtownd.log, channeld.log, contentd.log, crashrecord.log, flashromd.log, guided.log, harvestd.log, homed.log, searchd.log, servicelauncher.log, statusd.log, tricksd.log

prereg machines - anid.log, harvestd.log, servicelauncher.log, statusd.log

mailtod machines - aliasd.log, flashromd.log, harvestd.log, servicelauncher.log, statusd.log

mailsorter machines - aliasd.log, harvestd.log, servicelauncher.log, statusd.log

postoffice machines - aliasd.log, harvestd.log, mail.local.webtv.log, servicelauncher.log, statusd.log

Most of the WebTV Network logs share a common three line format, pictured in Example 6-19 and in Example 6-20.

Example 6-19 Typical three-line output.

```
<error-message>: <error-type>
<reporting-function>() <client-id>/<subscriber-id>
<reporting-file>:<line-number> <service>[<pid>] <date> <time><flag>
```

For Example:

Example 6-20 Example of three-line output.

```
Error reading headers from client: Unexpected EOF HandleCommands()
-/-
Protocol.c:203 mailtod[235] 10/03 12:46:22*
```

Note: It's fairly typical to have more than one log entry for a given error. This happens as the error codes percolate up the stack.

How to Interpret a Typical Log Entry

The error message

The error message is an approximate description of what went wrong. It should always appear first in a log entry.

The error type

The error type is a refinement of what went wrong, when the system has a specific idea of where it failed. Unexpected EOF and broken pipe are common sorts of messages to see here. Error type does not appear in all log entries.

The reporting function

The reporting function is the section of the mail service which found the error. It should always appear in a log entry.

The client id

The client id is a seven digit number which refers to the box itself specifically and uniquely. -/- indicates that this information is still unknown. This could be simply because the box hasn't been registered yet, or more likely, at this point in the connection process, the service hasn't received this information from the client.

There is also a user id, which relates specifically to one of the (potentially multiple) users on a box, but this information does not show up in the log.

The subscriber id

The subscriber id is a hexadecimal string linked to a single name and billing address. -/- indicates that this information is still unknown, for the same reasons the client id wouldn't be known.

The reporting file and line number

The reporting file is the specific file that found the error and reported it, and the number that follows refers to the line of code which encountered the error.

The service

The service refers in this case to mailtod, and should be the only service referred to in the mailtod.log files.

The pid (or process id)

The process id is the number that refers to the process that encountered the error. In conjunction with the time stamp, it is possible to determine which of multiple log entries refer to the same event.

Date and time

The date and time are a stamp to isolate when the event actually happened. In combination with the process id, you can isolate which of multiple entries refer to the same problem.

The flag

The flag is an asterisk that can appear after the date and time. Currently the system tries to delineate between serious problems (errors) and warnings or irregularities. The presence of the flag does not guarantee that the problem is serious, but indicates it should be looked at further. The lack of a flag does **not** indicate that the problem isn't serious, only that the system hasn't been able to interpret it as such.

Usage of the log files

The WebTV Network logs are most useful for analyzing certain types of problems, frequently related to customer connection problems. Armed with the silicon serial number, one can parse the logs for occurrences of a given serial number, and find entries related to that specific client. Using the logs it's possible to trace the entire connection process of a client, and determine where problems are occurring.

The most useful logs for tracing connection problems are logserverd.log (see "Logserverd.log" on page 155) and preregistrationd.log (see "Preregistrationd.log" on page 174). As one of the first troubleshooting tactics for connection problems is to force the client back through the preregistration cycle (by unplugging the Internet terminal and selecting "New Number" when powering it up again, or using an appropriate secret code), the preregistrationd.log will have an entry for every successful connection to the preregistration server. In the log will be a record of the ANI (automatic number identification) received by the server and the details of the tellyscript subsequently sent down to the client (if a new one is issued).

The more powerful tool for connection problem diagnosis is often logserverd.log. After a successful connection from a local POP, the client uploads its entire phone log, which is recorded. The information there includes the nature of any dial-in failure, as well as the parameters the client used in dialing (call waiting prefix, dialing prefix, dialing speed, etc.). Based on the various failure codes, it is often possible to discern what has been causing the connection problem. (See appendix for more details on the precise contents of the major log files)

When there is an entry in the preregistrationd.log and no corresponding entries in the logserverd.log, it demonstrates that the client is basically functional, but that there is a breakdown between either the client and the local POP, or between the local POP and our service. This may be the result of a bad dialing pattern, the necessity of an access code for possible toll calls, or a variety of other causes.

Service Manipulation Tools

There are a number of scripts that deal with starting, stopping, and testing the servers and the various services that run on them, as well as making certain that crashed services are properly restarted.

Note: All of these tools need to be run as userid `production` to affect the production service, as by default they look for a version of the service in the current user's home directory.

---

check-servers

check-servers is a script which is similar in function to part of monitord (see "monitord" on page 85), in that it checks all the machines that are part of the production service (assuming userid `production`).

list-services

list-services is a script which is invoked by other scripts, which checks the machine to see which services should be running on it.

servicelauncher

start-personal-servers

Similar to start-servers (below), start-personal-servers starts all the services on the current machine, and is run if you are a userid other than `production`.

start-servers

start-servers starts all the production services on the current machine, doing a stop-servers if necessary to make certain that duplicate processes aren't launched.

stop-one-server

Usage: stop-one-server daemonName

stop-one-server stops the named service on the current machine, for the current user.

stop-servers

stop-servers stops all the services running on the current machine, for the current user.

switch-services

Usage: `switch-services webtv webtv-<num>`

switch-services is a shell script which changes certain symbolic links to change which version of the service is running on a given machine. On any machine, it is common to have two different builds of the service, each with a different number, and this script enables you to change which one is the active service as needed. The active service is, by definition, the one in `~production/webtv`,

This is used when rolling a new version of the service, as it significantly reduces downtime to let the old version of the service run until the new version is completely loaded onto the production machines. Just remember to run a stop-servers first, and a start-servers after you're done switching services.

test-services

test-services runs a series of tests on the service, to make certain that all of its parts are functioning properly.

setdb

Usage:

setdb [-client <client-id>] [-subscriber <subscriber-id>] [-user <user-name> | <user-id>] [-billingStatus <number>] [-category <category>] [-lockUserstore] [-unlockUserstore]

setdb is a tool that allows the change of certain fields of customer information. This command must be runs as user production, or else the system will assume you are referring to a personal service, rather than a production service. Most commonly this will be used to change the category of a user [-category]. For example, you may wish to change a particular user from the normal production service to an advanced service group (e.g. beta tester). This would be done by the command:

```
setdb -client username -category beta
```

which will actually change the user's category from whatever it was to whatever "beta" is defined as.

The billing status of a customer [-billingStatus] may also be changed using this tool, although the CMR tool (See "Customer Master Record tool" on page 70) is a much better to execute this particular alteration. This tool can also be used in conjunction with move-store (See page 89), in that the user's "userstore" can be locked and unlocked, in preparation for moving to a different userstore.

Status Report and statusd

`statusd` runs on each production machine. It periodically is contacted by `monitord` to monitor various system level resources, such as CPU load and available disk space.

statusd's reports are provided in HTML format, and can be reached by directing a web browser at:

```
http://machinename.domain.type:1666/machine
```

Example 6-21 shows the typical output of a statusd query.

Example 6-21 Typical output from a query of statusd

```
--- system load (max allowed 2.5) ---
   load averages (1 min, 5 min, 15 min): 0.02, 0.02, 0.03

--- disk usage, columns are % used and K free ---
 19% 396923K free /usr/home/production/webtv/log/statusd.log
 10% 877441K free /usr/home/production/webtv/log/event_log/user-
events.log
   40000K wanted, 636576K avail on /tmp
```

sweeper

`sweeper` is a program invoked by a cron job, or the script run-sweep (located in ~production/webtv/tool), which visits every user's mailbox area, generating statistics for each given mailbox, optionally deleting old messages.

tellygen

Usage

```
tellygen [-v] [-r] ani siliconSerial
```

`tellygen` generates a tellyscript given ANI and a `clientid`. This is usually only used for testing purposes during development. The header it returns is a listing of all the currently disabled POP's. The second section details which PhoneDB the information is generated from, including the build date of the database. The third section gives the POP data in order or preference before load balancing is taken into account. The final section gives the POP data in order of preference after load balancing has been accounted for. Note that the clientid is only necessary for this last part, as the final digit in the string is used for load balancing purposes, to split the client pool into random segments. Below is a typical example of the output of this tool, for the ANI 004156147814 and a null clientid (all zeroes):

Example 6-22 tellygen sample output

```
TS: disabled POP ziplink/415-226-2255
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP ziplink/312-442-2255
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP ziplink/312-584-2255
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP ziplink/312-598-2255
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP ziplink/312-632-2255
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP ziplink/213-621-2768
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP cnc/708-300-0540
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP ziplink/617-824-4949
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

TS: disabled POP cnc/352-331-7789
     AddDisablePOP()             -/-
     ScriptGen.c:2123            (unknown)[1529]                        03/13 16:07:35

Successfully mapped PhoneDB (ver=14)
            Built Wed Jan  8 19:50:06 1997 from CCMI by uid=1057
     PhoneDB_Map()               -/-
     PhoneDB.c:250               (unknown)[1529]                        03/13 16:07:36

TS: WARNING: using 0x00 as silicon serial CRC
     TellyScript_New()           -/-
     ScriptGen.c:238             (unknown)[1529]                        03/13 16:07:36

For ANI '004156147814', before:

For 415-614-7814 from PALO ALTO, CA  (base cost=240):
  In or near PALO ALTO, CA  (cost=240/0 [0.0mi])  LOCAL
     ziplink 3279100
     ziplink 14153279100
  In or near PALO ALTO, CA  (cost=240/0 [0.0mi])  LOCAL
     cnc 6870610
     cnc 14156870610
```

```
 Next-best POPs:
  In or near PALO ALTO, CA  (cost=240/0 [0.0mi])  LOCAL
    uunet 3220489
    uunet 14153220489
  In or near REDWOOD CY, CA (cost=240/232 [4.7mi])  LOCAL
    cnc 4230025
    cnc 14154230025

TS: provider load balance:
         cnc         (pr#3 ) balance=60  norm=154
         ziplink    (pr#5 ) balance=40  norm=102
    NormalizeLoadBalance()     -/-
    ScriptGen.c:2599           (unknown)[1529]                      03/13 16:07:36

After:

For 415-614-7814 from PALO ALTO, CA  (base cost=240):
  In or near PALO ALTO, CA  (cost=240/0 [0.0mi])  LOCAL
    ziplink 3279100
    ziplink 14153279100
  In or near PALO ALTO, CA  (cost=240/0 [0.0mi])  LOCAL
    cnc 6870610
    cnc 14156870610

 Next-best POPs:
  In or near PALO ALTO, CA  (cost=240/0 [0.0mi])  LOCAL
    uunet 3220489
    uunet 14153220489
  In or near REDWOOD CY, CA (cost=240/232 [4.7mi])  LOCAL
    cnc 4230025
    cnc 14154230025
```

top

The top tool is used to monitor process behavior and CPU load on machines in the service. The jumpstart configuration for all service machines should include top and its associated man pages.

traceroute

The `traceroute` tool is used to monitor routing and packet flow on the service IAPs and ISPs. The jumpstart configuration for all service machines should include traceroute and its associated man pages.

User-event logs

The user-event logs are very long binary files which for each machine detail all the transactions that machine has undertaken in the course of the providing service. These files provide a significant amount of detail which varies with the process responsible for the entry, e.g., a proxyd event is necessarily going to look different from a notifyd event entry.

There are a number of tools associated with parsing and using the event-logs, as they tend to be the only long term records kept of all the transactions for the service, as the service logs (see "Service Logs and Log Analysis" on page 107) are only kept for a few days and then deleted. Among them are cat-event (see "cat-event" on page 58), which takes the binary user-event log file and transforms it, normalizing it into an all text format in which each event is a single line, and distill-log (see "distill-log" on page 73) which takes normalized event logs and pares them down, making them more readable and digesting the information contained within the event-log entry.

In addition to being a vast source of information for troubleshooting purposes, the user-event logs are also the source of information that the Harvester system uses to compile usage statistics for the service (see "Harvester Statistics" on page 81 for more on this), thus these logs serve two separate functions.

userStoreLockAndDo

Usage:

```
userStoreLockAndDo UserStoreBaseDir UserID filename command [args]
```

userStoreLockAndDo looks at a specified file in the userstore and then performs a command on that file. For example, if you want to edit the file:

```
/somewhere/a/a/a/aaauser/afile
```

with emacs in your current window, use:

```
userStoreLockAndDo /somewhere aaauser afile emacs -nw
```

The command operates on a temporary copy of the file. If it returns a non-zero value, the file will be left unchanged. Otherwise, the temporary file will replace the existing file. This tool is not used frequently at all, only on those rare occasions when a cookie might be causing problems and needs to be deleted manually, or similar circumstances.

userStoreName

Usage:

```
userStoreName UserID or UserName
```

`userStoreName` takes a UserID or UserName and outputs a path component where files for that user are stored in the userstore. For instance, one could need to access the mailbox for a given user for whatever reason. Going into the mail directory on the correct userstore and following the path given will lead you to the appropriate directory.

WebTV Internet Terminal Secret Codes

The WebTV Internet terminal client includes a number of shortcuts that can be used to affect box function. All of these secret codes can be inputted using a remote control or a keyboard for code entry while the box is "powered off" (the green light is off), but still plugged in. To use these codes, turn the box off, wait at least five seconds, and then type the codes into the keyboard or remote control. The lights on the front of the box should cascade, indicating that the code was

received and acted upon. When entering codes, the “Options” button clears any pending input, so it's a good idea to press the “Options” button before entering a secret code.

Table 6-1 Codes in all builds.

Numeric Code	Alternate Code	Function
217	up, down, right, go	**Go to dialing options** Dial 9+ for outside line? Audible dialing on? Access number, call waiting, or other dialing options?
32768	home, back, up, down, go	**Clear flash NV RAM** This resets the box to behave if it had just come from the factory. It'll dial the toll free number, all dialing options are reset.
080813	home, back, down, up, go	**Clear serial NV RAM (emergency 800 number)** Clears the backup toll-free number downloaded from the service. This number is only downloaded if the regular toll-free number doesn't work for a particular box or region. Clearing the backup number restores the original 800 backup number.
07081969	home, right, left, back, go	**Force flash download** Standard techniques for flash-downloading the latest software aren't working due to a severe problem (usually corrupted existing software in the box)
024995	*-none-*	**Boot into diagnostics** You want to run the hardware diagnostics for the box
7264	scrolldown, back, home, scrollup	**Clears Just tellyscript (not access number)**
77437	back, back, up, up, go	**"Spooky Dialing Options"** Need to dial a prefix before long distance calls? Special number to get toll-free calls? Phone system doesn't require a 1 before long distance or toll-free calls?

Table 6-2 Codes available in Client 1.1 or later Production Builds only

Numeric Code	Alternate Code	Function
411	home, back, recent, go	**Shows technical information for the box**
18004693288	up, down, left, right, up, down, left, right, go	**Force signup again** Use to transfer ownership of an Internet terminal from one subscriber to another.

Table 6-3 Codes in non-production builds only

Numeric Code	Alternate Code	Function
93288	left, right, left, right, go	**Boot into connect mode** Allows the client to connect to a variety of service groups other than production. (e.g. daily, thrashnet, export, personal services)
90210	*-none-*	Boot into tourist mode
031158	home, back, left, right, go	Starts up the Kiosk Demo mode.

WebTV Tricks Service

The WebTV Tricks service consolidates a number of programmer utilities and "tricks" in a single menu accessible through "goto". The WebTV Tricks Service can be reached using "goto" to access "wtv-tricks:tricks". In the implementation of the WebTV Network at WebTV Networks, Inc. in Palo Alto, there is an intermediate screen that requests a password from the user before permitting access to the WebTV Tricks page. This password can be configured by Network Systems. See "Changing the WebTV Tricks password" on page 51.

The following utilities are available from the WebTV Tricks service:

- **Info:** Provides information on the box, server, and current user.
- **Visit Big Willie's!:** Download from a list of possible Flash ROM's.
- **Unregister This Box:** On selection, immediately unregisters the box in question. **Caution! No prompt for verification!**
- **Run a Tour:** Run a tour from a Web page, following all links from the top page down.
- **Visit Vend-A-Telly!:** Download from a list of possible tellyscripts. Also allows POP tours to be run.
- **Blast NVRAM:**Clears the box of its tellyscript, setting it up to enter the pre-registration process again.

At present, in the tricks menu there are also the following functionalities:

- **Client 1.1 Info:** A short tour of the new features available with Client 1.1
- **Guided Tour:** A short introduction to the service and the Internet in general.
- **Try Net News:** A link to the news service, currently in testing as of this writing (1/24/97)

Appendix A

M-Codes

During the course of normal use, a number of messages in grey boxes may appear on the customer's screen, for a variety of reasons. These messages represent a number of standard errors that may occur.

For each of these error messages, there is a corresponding "M-number". These M-numbers map each error message to a particular type of problem. By asking the customer to hold down the **OPTIONS** key when a grey error window comes up on screen, in the bottom left corner of that grey window, the text "M-(some number)" will appear.

Caution: M-7 errors are a special case: be certain to ask for the text of the error, as there are a number of proxy messages that the **OPTIONS** key describes as M-7.

Note: New error messages can be added during development, or removed as the problems causing the error message are eliminated.

This appendix lists the error codes and messages, and discusses troubleshooting guidelines. The following topics are covered:

- Index of Error Messages (By M-Number)
- Index of Error Messages (By Error Text)
- Index of Error Messages (By Category of Error)
- Error Codes

Index of Error Messages (By M-Number)

Table A-1 Error Messages by M-Number.

M0	WebTV could not be contacted, perhaps because of regular maintenance. Try again in a few minutes.
M-2	The item chosen contains so much information that it can't be used.
M-4	The publisher of that page can't be reached.
M-7	A problem came up while communicating with the publisher of the requested page.
M-7	Access to the requested page is blocked by Surfwatch.
M-7	The addressing scheme <*scheme*> is unsupported.
M-7	The item chosen cannot be used just now. Try again in a while
M-7	The publisher <*publisher*> couldn't be reached.
M-7	The publisher <*publisher*> couldn't be reached or is too busy to use right now.
M-7	The publisher <*publisher*> couldn't find the requested page.
M-7	The publisher <*publisher*> is too busy right now.
M-7	The publisher <*publisher*> is unknown.
M-7	The publisher <*publisher*> responded in an odd way.
M-7	The publisher refuses to give you access to the requested page.
M-7	The requested address is missing a host name.
M-7	WebTV ran into a technical problem. Please try again.
M-7	WebTV technical problem: the proxy <*proxy*> couldn't be reached.
M-8	The item chosen contains so much information that it can't be used.
M-9	The item chosen contains a kind of information that WebTV can't use.
M-11	The requested item is not available now. Try again in a few moments.
M-12	The publisher can't find the page requested.
M-14	The publisher can't find the page requested.
M-15	The publisher can't find the page requested.
M-20	The publisher of that page just kept you waiting and waiting.
M-21	The item chosen contains a kind of information that WebTV can't use.
M-22	The connection to WebTV has been interrupted.
M-23	The publisher of that page has not authorized you to use it.
M-25	WebTV has finished using the phone line.
M-26	Your Internet Terminal is having trouble connecting to WebTV. Please try connecting again.
M-27	WebTV could not be contacted. If the problem continues, please call customer service.
M-28	WebTV could not be contacted, perhaps because of regular maintenance. Try again in a few minutes.
M-29	The phone line seems to be in use or disconnected. Check the phone line and try again.
M-30	The line to WebTV is busy. Try again in a few minutes.
M-31	If you have moved your Internet terminal to a different phone number choose **New Number**
M-32	Choose Power Off to Switch Off WebTV.

M-33 Your Internet Terminal needs to be switched off and then on again.

M-34 The publisher can't find the page requested.

M-36 WebTV has disconnected, freeing your phone line for another call.

M-37 The item chosen doesn't contain any information.

M-38 Oops! We crashed.

M-39 [***M-39 message***]

M-40 Your Internet terminal was unable to check for new mail automatically

M-41 WebTV Encountered a communication error and could not play the RealAudio sound.

M-42 This is a type of audio information that WebTV cannot play.

M-43 WebTV disconnects automatically after 10 minutes. Choose **Stay Connected** to stay connected for another 10 minutes.

M-44 [***M-44 message***]

M-45 The line to WebTV has disconnected. Call 1-800-GO-WEBTV if this continues to be a problem

M-?a The provider of that page responded in an odd way.

M-?b The publisher of that page is too busy to use right now.

M-?c There was a problem displaying the page on the screen.

Index of Error Messages (By Error Text)

Table A-2 Error Messages by Error Text.

M-7	A problem came up while communicating with the publisher of the requested page.
M-7	Access to the requested page is blocked by Surfwatch.
M-32	Choose Power Off to Switch Off WebTV.
M-31	If you have moved your Internet terminal to a different phone number choose **New Number**
M-7	The addressing scheme <*scheme*> is unsupported.
M-22	The connection to WebTV has been interrupted.
M-7	The item chosen cannot be used just now. Try again in a while
M-9	The item chosen contains a kind of information that WebTV can't use.
M-21	The item chosen contains a kind of information that WebTV can't use.
M-2	The item chosen contains so much information that it can't be used.
M-8	The item chosen contains so much information that it can't be used.
M-37	The item chosen doesn't contain any information.
M-45	The line to WebTV has disconnected. Call 1-800-GO-WEBTV if this continues to be a problem
M-30	The line to WebTV is busy. Try again in a few minutes.
M-29	The phone line seems to be in use or disconnected. Check the phone line and try again.
M-?a	The provider of that page responded in an odd way.
M-7	The publisher <*publisher*> couldn't be reached or is too busy to use right now.
M-7	The publisher <*publisher*> couldn't be reached.
M-7	The publisher <*publisher*> couldn't find the requested page.
M-7	The publisher <*publisher*> is too busy right now.
M-7	The publisher <*publisher*> is unknown.
M-7	The publisher <*publisher*> responded in an odd way.
M-12	The publisher can't find the page requested.
M-14	The publisher can't find the page requested.
M-15	The publisher can't find the page requested.
M-34	The publisher can't find the page requested.
M-4	The publisher of that page can't be reached.
M-23	The publisher of that page has not authorized you to use it.
M-?b	The publisher of that page is too busy to use right now.
M-20	The publisher of that page just kept you waiting and waiting.
M-7	The publisher refuses to give you access to the requested page.
M-7	The requested address is missing a host name.
M-11	The requested item is not available now. Try again in a few moments.
M-?c	There was a problem displaying the page on the screen.
M-42	This is a type of audio information that WebTV cannot play.
M-28	WebTV could not be contacted, perhaps because of regular maintenance. Try again in a few minutes.

M0 WebTV could not be contacted, perhaps because of regular maintenance. Try again in a few minutes.

M-27 WebTV could not be contacted. If the problem continues, please call customer service.

M-43 WebTV disconnects automatically after 10 minutes. Choose **Stay Connected** to stay connected for another 10 minutes.

M-41 WebTV Encountered a communication error and could not play the RealAudio sound.

M-36 WebTV has disconnected, freeing your phone line for another call.

M-25 WebTV has finished using the phone line.

M-7 WebTV ran into a technical problem. Please try again.

M-7 WebTV technical problem: the proxy <*proxy*> couldn't be reached.

M-26 Your Internet Terminal is having trouble connecting to WebTV. Please try connecting again.

M-33 Your Internet Terminal needs to be switched off and then on again.

M-40 Your Internet terminal was unable to check for new mail automatically

M-38 Oops! We crashed.

M-39 [***M-39 message***]

M-44 [***M-44 message***]

Index of Error Messages (By Category of Error)

Normal Conditions

M-25: WebTV has finished using the phone line.

M-32: Choose Power Off to Switch Off WebTV.

M-36: WebTV has disconnected, freeing your phone line for another call.

M-43: WebTV disconnects automatically after 10 minutes. Choose **Stay Connected** to stay connected for another 10 minutes.

MessageWatch Errors

M-40: Your Internet terminal was unable to check for new mail automatically

Physical Problems that the Customer Can Address

M-29: The phone line seems to be in use or disconnected. Check the phone line and try again.

Sound Errors

M-41: WebTV Encountered a communication error and could not play the RealAudio sound.

M-42: This is a type of audio information that WebTV cannot play.

The Publisher Didn't Serve a Page at All

M-4: The publisher of that page can't be reached.

M-7: A problem came up while communicating with the publisher of the requested page.

M-7: The item chosen cannot be used just now. Try again in a while

M-7: The publisher <*publisher*> couldn't be reached.

M-7: The publisher <*publisher*> couldn't be reached or is too busy to use right now.

M-7: The publisher <*publisher*> is too busy right now.

M-7: The publisher <*publisher*> is unknown.

M-7: The publisher <*publisher*> responded in an odd way.

M-7: The publisher <*publisher*> couldn't find the requested page.

M-7: WebTV technical problem: the proxy <*proxy*> couldn't be reached.

M-12 The publisher can't find the page requested.

M-14: The publisher can't find the page requested.

M-15: The publisher can't find the page requested.

M-34: The publisher can't find the page requested.

M-37: The item chosen doesn't contain any information.

M-?b: The publisher of that page is too busy to use right now.

The Publisher Didn't Serve the Page Normally

M-2: The item chosen contains so much information that it can't be used.

M-7: Access to the requested page is blocked by Surfwatch.

M-7: The addressing scheme <*scheme*> is unsupported.

M-7: The publisher refuses to give you access to the requested page.

M-7: The requested address is missing a host name.

M-8: The item chosen contains so much information that it can't be used.

M-11: The requested item is not available now. Try again in a few moments.

M-?a: The provider of that page responded in an odd way.

M-?c: There was a problem displaying the page on the screen.

There's More Information About a Page the User Asked For

M-9: The item chosen contains a kind of information that WebTV can't use.

M-20: The publisher of that page just kept you waiting and waiting.

M-21: The item chosen contains a kind of information that WebTV can't use.

M-23: The publisher of that page has not authorized you to use it.

Unexpected Conditions that Need to Be Escalated to WebTV Networks, Inc.

M-38: Oops! We crashed.

M-39: [***M-39 message***]

Unexpected Conditions that the Customer Needs to Address

M0: WebTV could not be contacted, perhaps because of regular maintenance. Try again in a few minutes.

M-7: WebTV ran into a technical problem. Please try again.

M-22: The connection to WebTV has been interrupted.

M-26: Your Internet Terminal is having trouble connecting to WebTV. Please try connecting again.

M-27: WebTV could not be contacted. If the problem continues, please call customer service.

M-28: WebTV could not be contacted, perhaps because of regular maintenance. Try again in a few minutes.

M-30: The line to WebTV is busy. Try again in a few minutes.

M-31: If you have moved your Internet terminal to a different phone number choose **New Number**

M-33: Your Internet Terminal needs to be switched off and then on again.

M-44: [***M-44 message***]

M-45: The line to WebTV has disconnected. Call 1-800-GO-WEBTV if this continues to be a problem

Error Codes

The M0 Errors

```
Message: WebTV could not be contacted, perhaps because of regular
maintenance. Try again in a few minutes. (Disconnected Network)
```

If this message appears, it means that the customer's box is still connected, but the WebTV Network isn't operating at the other end of the phone line.

The customer should try again in about 15 minutes, by which time the problem has hopefully been resolved.

```
Message: <a variety of other messages>
```

There are various alerts that are not considered errors, but are part of the user interface. They currently share the same M-code as the Disconnected Network Error, although changing this is on the table for future updates of the software.

The M-2 Error -- Low Memory

```
Message: The item chosen contains so much information that it can't be
used.
```

This sort of message is an unfortunate consequence of limited memory within the box. While it is possible that reconnecting to the service with an empty cache in the box may alleviate this problem, some pages may have elements that are just too large to display. It may be worthwhile to attempt to access the page with a Customer Care box to determine if a WebTV Internet terminal is capable of loading the page at all.

The M-4 Error -- Host Not Found

`Message: The publisher of that page can't be reached.`

The server specified wasn't found by the DNR (Domain Name Recognition) resolver, which is supposed to turn host names into unique addresses. This will generally not appear, since the proxy should supply a message in this case instead.

Usually this is the result of a mistyped URL, either in the **Go To** window or in an author's web page. Have the customer make certain that they typed the URL in correctly. They may wish to notify the publisher that a bad link is present in their web page if retyping the URL fails to alleviate the problem.

The M-7 Errors -- Response Errors

`Message: <The message varies and comes from the WebTV Network>`

In general, these are all errors that have come from one of the proxy servers. Currently, they all have the same error code, M-7. Below is a list of the common messages, and what can be done about them.

A problem came up while communicating with the publisher of the requested page.

Access to the requested page is blocked by Surfwatch.

The addressing scheme <*scheme*> is unsupported.

The item chosen cannot be used just now. Try again in a while

The publisher <*publisher*> couldn't be reached.

The publisher <*publisher*> couldn't be reached or is too busy to use right now.

The publisher <*publisher*> couldn't find the requested page.

The publisher <*publisher*> is too busy right now.

The publisher <*publisher*> is unknown.

The publisher <*publisher*> responded in an odd way.

The publisher refuses to give you access to the requested page.

The requested address is missing a host name.

WebTV ran into a technical problem. Please try again.

WebTV technical problem: the proxy <*proxy*> couldn't be reached.

```
Message: A Problem Came Up While Communicating with the Publisher of
the Requested Page.
```

This is a generic error that the proxy couldn't load the requested URL from the publisher. This is the sort of problem that should be resolved eventually by a good publisher, but retrying is likely to result in the same message. The customer may wish to notify the publisher to inform them of the problem.

`Message: Access to the Requested Page Is Blocked By Surfwatch`

The customer has Surfwatch turned on and has tried to go to a page that is restricted by Surfwatch.

The customer should either turn off Surfwatch or limit viewing to approved sites.

`Message: The Addressing Scheme <scheme> Is Unsupported.`

The user tried to go to a page that uses an addressing scheme like *https*, a protocol that the service does not support.

The WebTV Network cannot go to pages that use *https:*. Often, publishers will have an alternate, *http:* page that mimics or duplicates the information on the *https:* page.

```
Message: The item chosen cannot be used just now. Try again in a
while.
```

This message was seen at a site which requested user/password authentication via standard authentication protocols. Instead of prompting the user for the username and password, the proxy just chokes on it, displaying the above error message. Trying again in a while does not seem to change the problem in this case.

The proxy seems to be unable to prompt the user correctly for the username and password combination. Likely cause is either a bug in the publisher's security protocols, or a bug in the service's software. It is probably worth emailing the URL of the site in question to bugs@corp.webtv.net with a suitable explanation of what went wrong, to verify that it isn't a problem with the service.

```
Message: The Publisher <publisher> Couldn't Be Reached
```

The host does have an entry in the name server, but it can't be contacted.

Most commonly, the publisher has stopped accepting connections. Often this indicates that the publisher has stopped offering a Web service. It also may be that their machines or connections are down. Trying again later may work, but the publisher may be permanently unreachable.

```
Message: The Publisher <publisher> Couldn't Be Reached or Is Too Busy
to Use Right Now.
```

The publisher didn't accept the connection.

Less well-run publishers are sometimes too busy, and don't return a specific explanation for rejecting the connection. Often the customer will have greater success later, although this is by no means assured.

```
Message: The Publisher <publisher> Couldn't Find the Requested Page.
```

The publisher was reached, but the URL specified a page that the publisher doesn't have. Often this is because the publisher has updated its pages, but there are links to old pages.

Retrying will not help this problem, although the customer should check the spelling if they typed the URL. If this appears after choosing a link on a web page, the customer may wish to consider contacting that page's publisher and informing them of the bad link.

```
Message: The Publisher <publisher> Is Too Busy to Use Right Now.
```

The publisher refuses the connection explicitly because it is too busy.

Well-run publishers will provide this information at peak times. A customer is likely to have better luck during non-peak hours later in the day.

```
Message: The Publisher <publisher> Is Unknown
```

The host asked for doesn't have a listed address in the domain. If you ask for an page from a non-existent server, you'll get this message.

Most often, the customer mistyped the URL, and should carefully check the spelling. This message can also appear if a page's author has misnamed a publisher. In this case, the customer may wish to notify the page's publisher of the bad link.

```
Message: The Publisher <publisher> Responded in an Odd Way.
```

The publisher didn't respond to the most recent request as expected.

Trying again later is unlikely to help this problem, as only rarely do most publishers fix these sorts of problems.

```
Message: The Publisher Refuses to Give You Access to the Requested
Page.
```

The user tried to get to a page that requires authorization, but the authorization failed.

If the customer is supposed to have access to this page, is it likely that they mistyped their password. Some customers will see this and not understand that the page they tried to visit requires them to register and enter a password.

```
Message: The Requested Address Is Missing a Host Name.
```

The address didn't have a publisher listed.

Usually a customer can solve this problem by checking the spelling of a URL they typed in. If this message is obtained from an existing link, the customer may wish to contact the publisher to make them aware of the bad link.

```
Message: WebTV Ran Into a Technical Problem. Please Try Again.
```

The proxy was somehow broken.

Trying again usually solves this problem, and it is probably a good idea to notify Service Operations when this message comes up.

```
Message: WebTV Technical Problem: The Proxy <proxy> Couldn't Be
Reached.
```

The proxy is, for some reason, unavailable.

This is a problem with the WebTV Networks, which Service Operations and Network Systems should have more information on. Retrying in a few minutes will usually solve this problem. Reports of this type should be brought to the attention of Service Operations.

The M-8 Error -- Cache Full

```
Message: The item chosen contains so much information that it can't be
used.
```

Similar to the M-2 Low Memory Error, it indicates that the box does not have sufficient memory to display or load the requested URL. While it may help to have the customer try visiting this page after reconnecting to the service, there are no guarantees this will work. This message may always appear for certain pages.

The M-9 Error -- Cannot Parse

```
Message: The item chosen contains a kind of information that WebTV
can't use.
```

Similar to M-21, Unknown Data Type, the box couldn't parse a data type. A non-binary content type was likely specified.

Publishers often provide alternate versions of pages that contain special data types. The customer should look at the page and see if such an alternate page is available. Otherwise, the customer might consider contacting the publisher and asking them to provide an alternate page.

The M-11 Error -- Unknown Service

`Message: The requested item is not available now. Try again in a few moments.`

This can also occur if a service like news or chat is temporarily unavailable. The obvious solution is to retry in a few minutes or so. Perhaps it's a conflict and the proxy was unable temporarily to load the page or some similar problem.

The M-12 Error -- File Not Found

`Message: The publisher can't find the page requested.`

Similar to errors M-14 (Resource Not Found), M-15 (Page Not Found), and M-34, (Too Many Redirects) the server providing that page returned error 404, which means the resource couldn't be found. This ordinarily shouldn't appear, as the proxy's message including the publisher ought to appear instead.

Retrying will not help with this problem. If the customer typed the address, the customer should check the spelling. If this appears after choosing a link on a web page, the customer may wish to consider contacting that page's publisher and informing them that the link is bad.

The M-14 Error -- Resource Not Found

`Message: The publisher can't find the page requested.`

Similar to errors M-12 (File Not Found), M-15 (Page Not Found), and M-34 (Too Many Redirects), the server providing that page returned error 404, which means the resource couldn't be found. This ordinarily shouldn't appear, as the proxy's message including the publisher ought to appear instead.

Retrying will not help with this problem. If the customer typed the address, the customer should check the spelling. If this appears after choosing a link on a web page, the customer may wish to consider contacting that page's publisher and informing them that the link is bad.

The M-15 Error -- Page Not Found

`Message: The publisher can't find the page requested.`

Similar to errors M-12 (File Not Found), M-14 (Resource Not Found), and M-34 (Too Many Redirects), the server providing that page returned error 404, which means the resource couldn't be found. This shouldn't ordinarily appear, as the proxy's message including the publisher ought to appear instead.

Retrying will not help with this problem. If the customer typed the URL, they should then check the spelling. If this appears after choosing a link on a web page, the customer may wish to consider contacting that page's publisher and informing them that there's a bad link.

The M-20 Error -- Timed Out

`Message: The publisher of that page just kept you waiting and waiting.`

The customer tried to get a page, picture, etc., and it accepted the connection but never finished serving the request.

Often trying again in a few minutes will alleviate this problem, although sometimes publishers simply will never respond. You can try to load up the page on a Customer Care box to determine if the latter is the case.

The M-21 Error -- Unknown Data Type

`Message: The item chosen contains a kind of information that WebTV can't use.`

Similar to error M-9 (Cannot Parse), the Internet Terminal couldn't parse a data type, most likely because a non-binary content type was specified.

Publishers often provide alternate versions of pages that contain special data types. The customer should look at the page and see if such an alternate page is available. Otherwise the customer may wish to consider contacting the publisher and asking them to provide an alternate page.

The M-22 Error -- Lost Connection

`Message: The connection to WebTV has been interrupted.`

Usually, this message appears because someone picked up an extension on the same line, disconnected the cord, sabotaged the local phone switch, or otherwise severed the telephone connection from the box to the service.

Try connecting again. A common reason for this message is that someone picked up an extension on the same phone line. It could also happen because of a problem at the phone company or the service, however.

If the failure mode is repeated, try testing the local POP through Vend-a-Telly (wtv-tricks menu). If this succeeds, it is likely that there is a problem between the client's box and the local POP, possibly with their phone line. If the user's line is bad, then Service Operations may be able to check through the logs for connect speeds to see if that customer consistently connects at a low baud rate.

The M-23 Error -- Authorization Error

`Message: The publisher of that page has not authorized you to use it.`

The server expected a particular secret key or password, and the box failed to send the expected code.

If the customer is supposed to have access to this page, it is likely that they mistyped their password. Some customers will see this and might not understand that the page they tried to visit requires some sort of special privileges to access.

The M-25 Error -- Inactive Connection

`Message: WebTV has finished using the phone line.`

The terminal may have given up its use of the phone line in response to detecting another incoming call on call-waiting. Try powering the box off and reconnecting.

This can alternatively occur due to line noise knocking the connection off, or potentially be the result of a problem with the local POP. Check the user's call waiting settings (dialing options, secret code 217). It may also be worth using Vend-a-Telly to check if the user's POP is accepting connections normally.

If this persists, have Service Operations check through the log files to see if there is a reason for the disconnections noted (too low a connect speed, for example).

The M-26 Error -- Generic Phone Error

```
Message: Your Internet terminal is having trouble connecting to WebTV.
Please try connecting again.
```

A peculiar tellyscript result, not something anyone expects, came up.

Try connecting again. This is quite an unusual result, but connecting again should solve the problem. If not, it's worth notifying Service Operations of the problem so they can trace it and determine the cause. Additionally, sending the client to preregistration again, either by selecting **New Number** after powering the box off, or using the 7264 secret code, seems to frequently cure this problem. See "WebTV Internet Terminal Secret Codes" on page 116 and "WebTV Tricks Service" on page 118 for more details on secret codes.

The M-27 Error -- Phone Handshake Failure

```
Message: WebTV could not be contacted. If the problem continues,
please call customer service.
```

For some reason the box failed to make it through the handshake procedure.

The first thing, as with all connection problems, is to turn on audible dialing (secret code 217, or turn off the box, unplug the phone line), check the call waiting settings, verify any access numbers or dialing prefixes, and make certain that the box is connected to an analog phone line that **isn't using** a wireless phone jack. Also, sometimes the phone cord is bad and another one should be tried. Too many extensions on the same line can lead to sufficient signal degradation cause frequent handshake failures.

It may be worth using a different phone cord, and plugging straight into the wall socket instead of the splitter, for diagnostic reasons.

Once the above is done, have the customer try again, and document precisely what happens. The following should be typical questions to ask on a follow-up call:

- Did the customer ever hear modem initialization sounds?
- Has the terminal ever successfully connected to pre-registration, i.e., have they input account information and had it go through?
- If connecting to pre-registration now (either through first usage, or by 7264 secret code, or unplugging box and selecting "New Number"), is it connecting to the 800 number (i.e., does the customer see the words "starting communication")?
- If so, does it dial the local number successfully? Does it dial the local number once and then die, or does it roll over to the second local POP?
- Does it ever roll over to the fallover POP?

Thing to check on at this end:

- Check their local POPs to see if they're working properly.

It's possible that this is a bad modem chip, although handshake failure is a relatively frequent occurrence with POPs. A history of this problem should be examined before concluding that there is a bad modem chip. In any event, repeated attempts and analysis of the results are necessary to diagnose the source of this problem. Service Operations should be notified at some point, as the logs may shed a little more light on the subject.

The M-28 Error -- Phone No Answer

```
Message: WebTV could not be contacted, perhaps because of regular
maintenance. Try again in a few minutes.
```

No modem answered at the POP or at the service's preregistration number.

As with all connection problems, set it to audible dialing, make sure call waiting is set correctly, and that any dialing prefixes or access numbers have been entered correctly. Similarly, check to verify that the line is analog, and that they **aren't** using a wireless phone jack.

There is a bug currently under investigation, which points the box at the fallover 800 number. During rush times, when local POPs are busy, the fallover POP is for some reason not answering properly, causing endless cycles of M-28 errors.

This error seems to be coming up much more often since the 1.1 client upgrade became available. It has been suggested that the customer try powering off the box and turning it back on again, instead of clicking on the **Try Again** button. This should force the box to go back to the beginning of the POP list.

Current wisdom is still out on this particular solution, however, it is confirmed that if the above doesn't work, having the customer unplug the box and choose **New Number** will generally alleviate the problem (sending them back through pre-registration 800). Failing this, using the 32768 secret code to clear out NVRAM cures the problem. See"WebTV Internet Terminal Secret Codes" on page 116 for more information on using secret codes.

The M-29 Error -- Phone No Dialtone

```
Message: The phone line seems to be in use or disconnected. Check the
line and try again.
```

The box couldn't get a dialtone on the line.

This means that the phone line is either in use or disconnected.

A number of things can cause this sort of problem.

- Picking up another phone on the same extension when the terminal is trying to initiate a call.
- Having any extension on the same line off the hook.
- Bad splitter plugged into the jack (Try plugging in directly to the wall socket).
- Bad phone cord (Try replacing the phone cord).
- Various and sundry reasons for a "stutter" dialtone (some voice mail systems, for example)
- PBX's which require a prefix to get an outside line, and no prefix has been entered.
- Use of a wireless phone jack.
- Too many extensions on the same line can degrade service to the point of undetectable dialtone, but it shouldn't usually be the case.

First things first. Get them on audible dialing (in the Dialing Options), so they get to listen in on what's happening when the modem tries to dial.

Generally, after having checked the mechanical problems of phones off the hook, bad phone cord, wireless phone jack (always a no-no), bad splitter, or too many extensions, the customer will either require a dialing prefix to get an outside line, or have a stutter dialtone, which the box can't interpret properly.

If the customer has a voice mail service that produces a stutter (dat-dat-dat-dat) dialtone, turning off "Wait for Dialtone" in the Dialing Options (secret code 217) will alleviate the problem (in some cases, changing dialing speed to slow may also be necessary). Alternatively, putting 2-3 commas in the dialing prefix will often cause the box to pause long enough to get past the stutter part of the dialtone, and all should be well.

If the customer needs to input a prefix to get an outside line, they'll have to input a dialing prefix (again, from the Dialing Options menu), usually 9 followed by a comma (the comma provides a pause, which isn't always necessary, but is often a good idea).

If all else fails, send it up the line to Service Operations, but there's a lot of easily fixable causes here to work from before taking this step.

The M-30 Error -- Phone Busy

`Message: The line to WebTV is busy. Try again in a few minutes.`

This message indicates that somewhere along the line, the box ran into what it perceived as a busy signal. It can happen most often when either their local POPs or pre-registration are heavily loaded, or alternatively, if they are receiving the message "All circuits are busy" type message from their local carriers.

Things to do about it:

This particular problem is more prevalent in certain geographic areas where there is a high volume of calls and less than optimal capacity. 800 numbers and fallover numbers are particularly likely to become busy and produce these sorts of errors.

The M-31 Error -- Phone Unplugged

`Message: If you have moved your Internet terminal to a different phone number, choose New Number.`

The user unplugged their box, somehow switched it off, moved it to a different outlet, or suffered a power interruption.

Ordinarily, just choose continue. This message appears so that a user can tell the client if they have moved their terminal to a new location. That way, it can get another tellyscript and a new local POP for the user.

The M-32 Error -- Power Off

`Message: Choose POWER OFF to switch off WebTV.`

The user pressed the POWER key on the IR remote keyboard. They need to press another button to confirm that they really want to switch the box off.

Having this verification message around allows the user to change their mind, in case the POWER button was pressed accidentally.

The M-33 Error -- Endless Loop

`Message: Your Internet Terminal Needs to Be Reset By Powering Off and Then On Again`

Boxes with remote access numbers set in the dialing options dialog can get into the M-33 Endless Loop because of a bug that will be fixed in version 1.1 of the client software. The problem is essentially that the M-33 dialog does not always properly reset the box when you press continue to acknowledge the dialog (Again, this is only for boxes with access numbers.).

If a customer becomes stuck in an M-33 loop, use the following procedure to free them:

1. Power off the box and wait 5 seconds.
2. Use secret code 217 to get to the "Dialing Options" panel (See "WebTV Internet Terminal Secret Codes" on page 116 for more information on secret codes). The LED's on the box should start flashing. Make sure this is happening.
3. Choose **Access** from the Dialing Options dialog.
4. Enter the appropriate phone number to send the box to pre-registration.
5. Press **Continue** to dismiss the "Dialing Access Number" page.
6. If the customer is calling through a PBX, then:
   - Choose **Basic** from the Dialing Options dialog.
   - Enter the correct dialing prefix for the PBX (probably a "9").
   - Choose **Continue** to dismiss the "Basic Dialing Options" page.
7. Press **Connect** to dismiss the Dialing Options page
8. The box will go through a connect sequence ending in a dialog that says: `Your Internet terminal needs to be reset by powering off and then on again.`
9. Press **Continue** to acknowledge the dialog.
10. The box will automatically power-off and then back on again.
11. **IMPORTANT:** When the progress bar says "Waiting to Connect" or "Connecting", (Client 1.1 returns "Waiting to Connect" after it's picked up the new tellyscript. Client 1.0 returns "Preparing to Call Again" and "Connecting" after the new tellyscript has been downloaded.) power off the box. Wait 5 seconds.
12. Use secret code 217 to get to the "Dialing Options" panel (See "WebTV Internet Terminal Secret Codes" on page 116 for more information on secret codes), verifying that the LED's on the terminal flash.
13. Choose **Access** from the Dialing Options dialog.
14. Remove the 800 number you entered in step 4.
15. Press **Continue** to dismiss the "Dialing Access Number" page.
16. Press **Connect** to dismiss the "Dialing Options" page.
17. The box will go through a connect sequence ending in a connection to the WebTV Network (Home page, Login page, or Registration)

If you find that these steps do not work as described, please escalate the problem to WebTV Networks, Inc.

The M-34 Error -- Too Many Redirects

`Message: The publisher can't find the page requested.`

Similar to errors M-12 (File Not Found), M-14 (Resource Not Found), and M-15 (Page Not Found), the server providing that page returned error 404, which means the resource couldn't be found. This ordinarily shouldn't appear, as the proxy's message including the publisher ought to appear instead.

Retrying will not help with this problem. If the customer typed the address, the customer should check the spelling. If this appears after choosing a link on a web page, the customer may wish to consider contacting that page's publisher and informing them that the link is bad.

The M-36 Error -- Call Waiting Disconnect

`Message: WebTV has disconnected, freeing your phone line for another call.`

The box has detected an incoming call, and has released the phone line so the customer can pick up the call.

The mechanism for detecting the incoming calls may be too sensitive, resulting in a 'false positive' such that the box disconnects when there is no incoming call. If this sort of problem persists, it may be advisable to alter the setting of Call Waiting in the Dialing Options section (input code 217 or have the customer turn the box off, unplug the phone line, and turn the box on again to get to the Dialing Options).

M-37 Error -- [***M-37 descriptor***]

`Message: The item chosen doesn't contain any information.`

The document selected is empty. The customer may wish to email the publisher to inform them of the problem.

M-38 Error -- [***M-38 descriptor***]

`Message: Oops! We crashed.`

Crashed box. This should never appear to a customer, since the box automatically reboots after a crash. If it does, be sure to let someone in Software know about it.

M-39 Error -- Compression Error

`Message: [***M-39 message***]`

An error occurred in the compression algorithm used to compress HTML for transmission from the proxy to the client. This is really a programming error, and the customers should not see this message. If customers observe this error, please escalate the problem to WebTV Networks, Inc.

M-40 Error -- MessageWatch Failed

`Message: Your Internet terminal was unable to check for new mail automatically.`

The customer has set the option to automatically check for mail, but the last time checking was supposed to occur, the Internet terminal tried to check and was unable to. It should work again the next time that hour rolls around. If the customer want to check for mail now, they should simply connect to the service.

This might happen if the phone line was in use when the Internet terminal was due to check for mail -- the box doesn't check precisely on the hour the customer sets, rather, the times are staggered by ten minute intervals to balance load.

M-41 Error -- RealAudio Error

```
Message: WebTV Encountered a communication error and could not play
the RealAudio sound
```

Occasionally, this will happen when the publisher of a "constant-stream" RealAudio segment is overloaded and can't open another stream. Trying again can alleviate this problem.

However, some publishers are simply sending the information incorrectly, at which point there is little that can be done. For troubleshooting purposes, it's worth trying the same link using either Netscape or IE, which by its failure will verify that the publisher is at fault.

M-42 Error -- [***M-42 descriptor***]

```
Message: This is a type of audio information that WebTV cannot play.
```

This message appears when the customer tries to play a type of sound that the service doesn't currently support. At present, this should only happen with RealAudio 3.0 streams.

Hitting this link again will not fix the problem, although many (most, even) sites do provide multiple versions of their sound clips. Trying a RealAudio version 1.0 or 2.0, MIDI, MPEG 2, .VMF, MOD, .AU, GSM, .AIFF or .WAV formats should alleviate the problem if they're available.

M-43 Error -- [***M-43 descriptor***]

```
Message: WebTV disconnects automatically after 10 minutes. Choose Stay
Connected to stay connected for another 10 minutes.
```

This appears if the customer hasn't touched the remote control for 8 minutes, but a sound is still playing (background music doesn't count here). It lets the customer tell the service that they're still there and listening, and that they don't want to be disconnected.

This is just a status message and nothing to worry about. Just have them select **Stay Connected** to continue listening to the audio feed.

M-44 Error -- Inactivity Timeout Before Login Complete

```
Message: [***M-44 message***]
```

This error should appear when the connection to the service times out before getting to the home page. This could happen if, for example, the user were waiting for a number of minutes for the home page. This error is essentially the same as the M-25 (Inactive Connection) error, except that the inactivity timeout occurs before the login has completed. This usually happens somewhere around the user list or password page. The box will reboot and restart connection process when user presses continue.

To solve this problem, have the customer try again to see if it was a momentary glitch, or something worse like a black hole router or similar problem. A pattern of failure in this mode might indicate problems with the POP, so exploring Vend-a-Telly (wtv-tricks menu) may be worthwhile. There may also be a problem with the service in general, which should be easy to test (make certain you are testing the correct service, as trial users do not run on the same service).

M-45 Error -- Disconnection Before Login Complete

```
Message: The line to WebTV has disconnected. Call 1-800-GO-WEBTV if
this continues to be a problem.
```

The box lost connection unexpectedly before completing the login process. This error is the same as the M-22 (Lost Connection) error except that the lost connection occurred before the login was complete, instead of after. The box will reboot and restart the connection process when the user presses continue.

Under Client 1.0, the box would **power off** instead of showing this error message.

The possible reasons for the disconnection are: call-waiting interruptions, POP problems, pick-ups on other extensions, and the usual list of suspects.

The possible solutions are: First, turn on audible dialing, to see precisely what's going on there (secret code 217). From there, turning off call waiting is often a good solution, as the box may drop connection if it hears line noise it thinks is an incoming call, or alternatively, drops for a real incoming call, but doesn't inform the user that there is an incoming call.

Other things to do include: checking to see if the user is contacting a bad POP, checking the logs to see if the user's connect speeds are sufficiently low to be dropped before the home page (see Service Operations and let them go through the logs to do this), checking call waiting sensitivity (turning it off is just plain easier), checking to see if the user is using a wireless phone jack, and verifying that no one is picking up other extensions on the user's line.

Other Errors

```
Message: The provider of that page responded in an odd way.
```

TBD

```
Message: The publisher of that page is too busy to use right now.
```

TBD

```
Message: There was a problem displaying the page on the screen.
```

TBD

Appendix B

The Service Log Files

For each service, the appendix presents log file format, examples of log files, errors and their interpretation, and other information you need to know to interpret the files.

This appendix covers the following topics:

- Crashrecord.log
- Flashromd.log
- Headwaiterd.log
- Logserverd.log
- Mailtod.log
- Preregistrationd.log
- Proxyd.log

Crashrecord.log

Crashrecord.log is a record of the logs that each client sends to the service after it recovers from a crash. Recorded by the logserver (see "Preregistrationd.log" on page 174), the logs are stored in binary format and aren't immediately useful until they're translated into a format we can actually read. The logs are stored in binary format because crash records are most useful when symbols from the ROM image can be displayed next to the crash data and it's not always possible to get the symbol data at the time the crash record is logged. Binary format also allows processing of the crash logs later and reprocessing in order to reformat some of the subfields or to use a different symbol file.

dumpcrash

`dumpcrash` is a tool that manipulates crash log data. Its first job is to try to find an appropriate symbol table. Symbols are appended to the end of some client ROM builds. If the ROM doesn't happen to end on an exact 1MB boundary, dumpcrash assumes that the symbols are appended at the end. This is verified by an internal comparison to information in the ROM file itself. When a crash record for a particular version of the client first appears, dumpcrash looks in the current directory for a ROM image for that version. If it finds an image with valid symbols, the symbol table is read and held for the lifetime of the dumpcrash process. If the symbol table isn't found, dumpcrash will look in the same directory every time that version is requested to see if an appropriate symbol table has been added in the interim.

dumpcrash understands three kinds of ROM files:

- Files < 1MB are assumed to contain nothing but symbols.
- Files > 1MB and < 2MB are assumed to be "approm" code-only images; dumpcrash looks for symbols one megabyte into the file.
- Files > 3MB and < 4MB are assumed to be "unirom" code/romfs images; dumpcrash looks for symbols three megabytes into the file.

The command syntax for using dumpcrash is:

```
% dumpcrash [-f] [-d symbol-table-dir] [-s symbol-table-override]
[crashrecord.log] ...
```

Executing `"dumpcrash -\?"` (which is "-?" with the question mark quoted) will give the usage information as well as the default symbol-table-dir and the ROM image file name search order.

The `[-f]` option causes dumpcrash to work similarly to `tail -f`. It processes the entire file, then checks every second for new input. If the log file is renamed and restarted—something that currently happens every night at midnight—dumpcrash will switch to the new file within 30 seconds after it appears (value is `kTailMaxCount`, in `utility/TailStdio.h`). This option is ignored if the input is from *stdin*.

The `[-d symbol-table-dir]` option tells dumpcrash to look for ROM images in the specified directory instead of the default directory (which is currently "."). The recommended way to see the symbols on a "private" (wtv-system-version: 0) build is to execute `ln -s approm approm.0` to establish an image for version 0 and use `-d` to tell dumpcrash to point into the directory where the ROM image is located.

The `[-s symbol-table-override]` option loads the specified file as the symbol table file for *every* ROM version. This option is handy if you want to use your own ROM image instead of the default; you need not rename the image to *`xxx`*`.version` and then pick it up with `-d`.

You can specify one or more crash records on the command line. dumpcrash will dump the information from each in turn. The file name "-" corresponds to *stdin*; if file names aren't specified, then dumpcrash assumes *stdin*.

Flashromd.log

Flashromd is responsible for downloading new versions of the client. The log file keeps a record of the requests it receives and the downloads it actually makes. The genrom tool (see “genrom” on page 75), which is associated with flashromd, splits a version of the client into appropriately sized pieces and compresses them. This procedure alters the format so that the Internet terminal can successfully download the new software and use it properly.

Examples

Routine download entry (split by comments)

A routine download entry, with comments, is shown in Example B-1.

Example B-1 Routine download entry with comments.

```
Unexpected nil value
     Channel_SetSessionKeys()   -/-
     Channel.c:80                flashromd[27463]                    02/04 00:42:08*

Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:557              flashromd[27463]                    02/04 00:42:08*
The “unexpected nil value” entries are flashromd logging that the connection isn’t
secure.  The bootrom doesn’t know how to request and utilize a secure connection.

Serving 81ef0c0000b00238
'content/sony-webtv-000/version-253/bf0app-part000.rom'
     FlashRom_Content()          -/81ef0c0000b00238
     FlashRom.c:840              flashromd[27463]                     02/04 00:42:08
This entry tells which version of the client is being downloaded, in this case,
build 253.  Build 253 also happens to be the final version of the 1.1 client.

FlashRom_SendBlock: sending block from 'content/sony-webtv-000/version-253/bf0app-
part000.rom'
     FlashRom_SendBlock()       -/81ef0c0000b00238
     FlashRom.c:1182             flashromd[27463]                     02/04 00:42:08

Pulled parts=16 out of bf0app rominfo file
     FlashUtil_ReadRomInfo()    -/81ef0c0000b00238
     FlashUtils.c:1263           flashromd[27463]                     02/04 00:42:08
Flashromd is sending information down to the client.
```

```
Unexpected nil value
     Channel_SetSessionKeys()   -/-
     Channel.c:80                flashromd[27463]                    02/04 00:42:39*

Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:557              flashromd[27463]                    02/04 00:42:39*
The client requests another part of the download.  Flashromd is still logging an
error because the connection isn't secure.

Serving 81ef0c0000b00238 'content/sony-webtv-000/version-253/bf0app-part001.rom'
     FlashRom_Content()          -/81ef0c0000b00238
     FlashRom.c:840               flashromd[27463]                     02/04 00:42:39

FlashRom_SendBlock: sending block from 'content/sony-webtv-000/version-253/bf0app-
part001.rom'
     FlashRom_SendBlock()       -/81ef0c0000b00238
     FlashRom.c:1182             flashromd[27463]                     02/04 00:42:39

Pulled parts=16 out of bf0app rominfo file
     FlashUtil_ReadRomInfo()    -/81ef0c0000b00238
     FlashUtils.c:1263           flashromd[27463]                     02/04 00:42:39
```

Flashromd sends more information down to the client, completing the download.

A loss of connectivity could possibly interrupt a download sequence. There isn't likely to be an explicit message in the log to indicate this condition. However, it is possible to determine if the condition has probably happened: The log would simply fail to record the entire download sequence, as seen above, and would instead stop in the middle.

Asking for a version of the client that doesn't exist or can't be found

The test code will complain if "current" doesn't exist for "normal" Internet terminals. This case is only likely to occur when downloading from Willie's in the wtv-tricks menu (See "WebTV Tricks Service" on page 118) or when an advanced services group isn't behaving properly. Example B-2 below shows an attempt to download build 10 of the client, which doesn't exist.

Example B-2 Attempt to download a non-existent build.

```
Handling:
 GET wtv-flashrom:/content/artemis-test-000/version-010/bf0app-part001.rom
     HandleGetOrPost()           -/-
     Services.c:953              flashromd[26379]                    02/04 15:57:18

target: content/artemis-test-000/version-010/bf0app-part001.rom
    wtv-client-serial-number: optional-serial
    wtv-client-rom-type: bf0app
     ServiceRequest_StartLogs()-/optional-serial
     Services.c:933              flashromd[26379]                    02/04 15:57:18

target: content/artemis-test-000/version-010/bf0app-part001.rom
     HandleGetOrPost()           -/optional-serial
     HandleGetOrPost()           -/optional-serial
     Services.c:1011             flashromd[26379]                    02/04 15:57:18

Serving optional-serial
'content/artemis-test-000/version-010/bf0app-part001.rom'
     FlashRom_Content()          -/optional-serial
     FlashRom.c:840              flashromd[26379]                    02/04 15:57:20

FlashRom_SendBlock: sending block from
'content/artemis-test-000/version-010/bf0app-part001.rom'
     FlashRom_SendBlock()        -/optional-serial
     FlashRom.c:1182             flashromd[26379]                    02/04 15:57:20

Unable to open rom info file 'bf0app-rominfo.txt' in
'content/artemis-test-000/version-010'
     FlashUtil_ReadRomInfo()    -/optional-serial
     FlashUtils.c:1242           flashromd[26379]                   02/04 15:57:20*

No such file or directory
     FlashUtil_GetNextURL()     -/optional-serial
     FlashUtils.c:1185           flashromd[26379]                   02/04 15:57:20*

No such file or directory
     FlashRom_SendBlock()        -/optional-serial
     FlashRom.c:1223             flashromd[26379]                   02/04 15:57:20*

No such file or directory
     FlashRom_Content()          -/optional-serial
     FlashRom.c:883              flashromd[26379]                   02/04 15:57:20*

No such file or directory
     FlashRom_Main()             -/optional-serial
     FlashRom.c:245              flashromd[26379]                   02/04 15:57:20*

No such file or directory
     HandleGetOrPost()           -/optional-serial
     Services.c:1026             flashromd[26379]                   02/04 15:57:20*

failed: GET
wtv-flashrom:/content/artemis-test-000/version-010/bf0app-part001.rom from
127.0.0.1
```

```
   wtv-client-serial-number: optional-serial
   wtv-client-rom-type: bf0app
    HandleGetOrPost()          -/optional-serial
    Services.c:1037            flashromd[26379]                    02/04 15:57:20*

Handled GET
wtv-flashrom:/content/artemis-test-000/version-010/bf0app-part001.rom
    HandleGetOrPost()          -/optional-serial
    Services.c:1053            flashromd[26379]                     02/04 15:57:20

GET couldn't handle: GET
wtv-flashrom:/content/artemis-test-000/version-010/bf0app-part001.rom: No such
file or directory
    HandleCommands()           -/-
    Protocol.c:234             flashromd[26379]                    02/04 15:57:20*
```

An example of a test log (uncommented)

About every 5 minutes, the service runs a test on flashromd, which brings a lot of clutter into the log. The main points to note are the productionID of “artemis-test-000” and the serial number of 0123456789. You can expect these “test” log entries to look like Example B-3:

Example B-3 Typical test log entries.

```
Using default rom type
    FlashRom_Main()            -/-
    FlashRom.c:193             flashromd[27463]                     02/04 00:01:43

Serving ask-for-upgrade
    FlashRom_AskForUpgrade()  -/-
    FlashRom.c:335             flashromd[27463]                     02/04 00:01:43

  Converting fake cookie to 'artemis-test-000/normal'
    FlashRom_AskForUpgrade()  -/-
    FlashRom.c:352             flashromd[27463]                     02/04 00:01:43

Yes button will get app/unirom only (mandatory)
    FlashRom_SendQueryTemplate()  -/-
    FlashRom.c:937             flashromd[27463]                     02/04 00:01:43

Using default rom type
    FlashRom_Main()            -/-
    FlashRom.c:193             flashromd[27463]                     02/04 00:01:43

Serving tell-me-to-flash
    FlashRom_TellMeToFlash()  -/-
    FlashRom.c:422             flashromd[27463]                     02/04 00:01:43
```

```
  Converting fake cookie to 'artemis-test-000/normal'
    FlashRom_TellMeToFlash()  -/-
    FlashRom.c:464              flashromd[27463]                    02/04 00:01:43

Using default rom type
    FlashRom_Main()             -/-
    FlashRom.c:193              flashromd[27463]                    02/04 00:01:43

Serving tell-me-to-flash-boot
    FlashRom_TellMeToFlashBoot()  -/-
    FlashRom.c:513              flashromd[27463]                    02/04 00:01:43

  Converting fake cookie to 'artemis-test-000/normal'
    FlashRom_TellMeToFlashBoot()  -/-
    FlashRom.c:551              flashromd[27463]                    02/04 00:01:43

Serving current
    FlashRom_Current()         -/0123456789
    FlashRom.c:701              flashromd[27463]                    02/04 00:01:43

Serving 0123456789 current
    FlashRom_Current()         -/0123456789
    FlashRom.c:746              flashromd[27463]                    02/04 00:01:43

Unknown customer
    FlashUtil_GetInfoFromAccount()     -/0123456789
    FlashUtilsDB.c:55          flashromd[27463]                    02/04 00:01:43*

  Account lookup by silicon serial failed - customer not found
    FlashRom_Current()         -/0123456789
    FlashRom.c:755             flashromd[27463]                    02/04 00:01:43*

  Flash path for artemis-prototype-000/normal is 'content/artemis-prototype-000/
version-253/bf0app-part000.rom'
    FlashRom_Current()         -/0123456789
    FlashRom.c:774              flashromd[27463]                    02/04 00:01:43

FlashRom_SendBlock: sending block from 'content/artemis-prototype-000/version-25
3/bf0app-part000.rom'
    FlashRom_SendBlock()       -/0123456789
    FlashRom.c:1182             flashromd[27463]                    02/04 00:01:43

Pulled parts=16 out of bf0app rominfo file
    FlashUtil_ReadRomInfo()   -/0123456789
    FlashUtils.c:1263          flashromd[27463]                    02/04 00:01:43

Serving current
    FlashRom_Current()         -/optional-serial
    FlashRom.c:701              flashromd[27463]                    02/04 00:01:43
```

```
Serving optional-serial current with version cookie 'cookie'
     FlashRom_Current()          -/optional-serial
     FlashRom.c:717              flashromd[27463]                    02/04 00:01:43

   Converting fake cookie to 'artemis-test-000/normal'
     FlashRom_Current()          -/optional-serial
     FlashRom.c:722              flashromd[27463]                    02/04 00:01:43

FlashRom_SendBootRom: sending boot image from 'content/artemis-test-000/version-
003/bf0app-boot.brom'
     FlashRom_SendBootRom()     -/optional-serial
     FlashRom.c:1268             flashromd[27463]                    02/04 00:01:43

Serving current
     FlashRom_Current()          -/optional-serial
     FlashRom.c:701              flashromd[27463]                    02/04 00:01:43

Serving optional-serial current with version cookie 'cookie'
     FlashRom_Current()          -/optional-serial
     FlashRom.c:717              flashromd[27463]                    02/04 00:01:43

   Converting fake cookie to 'artemis-test-000/normal'
     FlashRom_Current()          -/optional-serial
     FlashRom.c:722              flashromd[27463]                    02/04 00:01:43

FlashRom_SendBlock: sending block from 'content/artemis-test-000/version-007/bf0
app-part000.rom'
     FlashRom_SendBlock()       -/optional-serial
     FlashRom.c:1182             flashromd[27463]                    02/04 00:01:43

Pulled parts=2 out of bf0app rominfo file
     FlashUtil_ReadRomInfo()    -/optional-serial
     FlashUtils.c:1263           flashromd[27463]                    02/04 00:01:43

Serving optional-serial 'content/artemis-test-000/version-007/bf0app-part001.rom
'
     FlashRom_Content()          -/optional-serial
     FlashRom.c:840              flashromd[27463]                    02/04 00:01:43

FlashRom_SendBlock: sending block from 'content/artemis-test-000/version-007/bf0
app-part001.rom'
     FlashRom_SendBlock()       -/optional-serial
     FlashRom.c:1182             flashromd[27463]                    02/04 00:01:43

Pulled parts=2 out of bf0app rominfo file
     FlashUtil_ReadRomInfo()    -/optional-serial
     FlashUtils.c:1263           flashromd[27463]                    02/04 00:01:43
```

Headwaiterd.log

Headwaiterd is one of the centerpieces of the WebTV Service. Headwaiterd takes care of all the authentication associated with each client login. It also is responsible for giving the client a list of services it can access, based on the service group (e.g., alpha, beta, pioneer) and the build type (e.g.,kiosk demo or customer builds).

Anatomy of a Log Entry

As with the other WebTV service logs, most of the entries follow the standard three-line format for an entry, as shown in Example B-4:

Example B-4 Log entry three-line format.

```
<error-message>: <error-type>
<reporting-function>() <client-id>/<subscriber-id>
<reporting-file>:<line-number> <service>[<pid>] <date> <time><flag>
```

Note: This format is not a hard-and-fast rule, but merely a guideline. Many log entries have additional information, sometimes a large amount of it, between the first and second lines of the normal three-line log entry. Nevertheless, these same three lines appear in nearly every log entry—they just may not appear consecutively or alone.

Typical Log Entry

Example B-5 shows a typical log entry.

Example B-5 Typical log entry.

```
Gave services to ID 2395483, group normal:
     name=wtv-log host=207.76.180.73 port=1609 flags=0x00000001 connections=1
     name=wtv-log host=207.76.180.71 port=1609 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      -/8112500010b002fe
     ServiceConfig.c:584        headwaiterd[23321]                   01/31 00:00:06

currentBootVersion == minBootVersion == 0, no automatic download
     FlashUtil_CheckUpgradeImportance()          -/8112500010b002fe
     FlashUtils.c:838            headwaiterd[23321]                   01/31 00:00:08

Gave services to ID 2395483, group normal, user myles:
     reset
     name=wtv-flashrom host=207.76.180.62 port=1618
     name=wtv-flashrom host=207.76.180.63 port=1618
     name=wtv-head-waiter host=207.76.180.61 port=1601 flags=0x00000001 connections=1
     name=wtv-head-waiter host=207.76.180.62 port=1601 flags=0x00000001 connections=1
     name=wtv-register host=207.76.180.62 port=1607
     name=wtv-register host=207.76.180.63 port=1607
```

```
     name=wtv-cookie host=207.76.180.61 port=1619 connections=1
     name=wtv-cookie host=207.76.180.62 port=1619 connections=1
     name=wtv-favorite host=207.76.180.61 port=1611 connections=3
     name=wtv-favorite host=207.76.180.62 port=1611 connections=3
     name=wtv-setup host=207.76.180.61 port=1613 connections=3
     name=wtv-setup host=207.76.180.62 port=1613 connections=3
     name=wtv-explore host=207.76.180.61 port=1624 connections=3
     name=wtv-explore host=207.76.180.62 port=1624 connections=3
     name=wtv-guide host=207.76.180.71 port=1621 connections=3
     name=wtv-guide host=207.76.180.72 port=1621 connections=3
     name=wtv-log host=207.76.180.71 port=1609 flags=0x00000001 connections=1
     name=wtv-log host=207.76.180.72 port=1609 flags=0x00000001 connections=1
     name=wtv-home host=207.76.180.73 port=1612 connections=3
     name=wtv-home host=207.76.180.71 port=1612 connections=3
     name=wtv-channel host=207.76.180.71 port=1626 connections=3
     name=wtv-channel host=207.76.180.72 port=1626 connections=3
     name=wtv-aroundtown host=207.76.180.71 port=1628 connections=3
     name=wtv-aroundtown host=207.76.180.72 port=1628 connections=3
     name=wtv-content host=207.76.180.71 port=1629 connections=3
     name=wtv-content host=207.76.180.72 port=1629 connections=3
     name=wtv-tricks host=207.76.180.71 port=1602 flags=0x00000004
     name=wtv-tricks host=207.76.180.72 port=1602 flags=0x00000004
     name=wtv-mail host=207.76.180.81 port=1608 connections=1
     name=wtv-mail host=207.76.180.82 port=1608 connections=1
     name=mailto host=207.76.180.81 port=1608 connections=1
     name=mailto host=207.76.180.82 port=1608 connections=1
     name=http host=205.180.153.28 port=1650 connections=3
     name=http host=205.180.153.29 port=1650 connections=3
     name=ftp host=205.180.153.28 port=1650
     name=ftp host=205.180.153.29 port=1650
     name=gopher host=205.180.153.28 port=1650
     name=gopher host=205.180.153.29 port=1650
     name=wtv-notices host=207.76.180.16 port=1622 connections=1
     name=wtv-preregister host=207.76.180.97 port=1615
     name=wtv-preregister host=207.76.180.96 port=1615
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     name=wtv-star host=206.67.152.64 port=1603 flags=0x00000003
     AddServiceHeadersToDictionary()      -/8112500010b002fe
     ServiceConfig.c:584        headwaiterd[23321]                   01/31 00:00:08

For 8112500010b002fe at 007084855814, telly ID=1948225264, mod=853261025 (Tue Ja
n 14 08:57:05 1997)
     HeadWaiter_CheckTellyscript()    -/8112500010b002fe
     HeadWaiter.c:985            headwaiterd[23321]                   01/31 00:00:12

TS: Hashed 'base:14:-|ziplink:1:16302033355|cnc:2:16302030590|ziplink:1:16302033
355|cnc:2:16302030590|artemis:1:18006108918' into 0x741f8ef0
     TellyScript_Compute()      -/8112500010b002fe
     ScriptGen.c:350             headwaiterd[23321]                   01/31 00:00:12
```

The first entry in the log record above shows headwaiterd giving the client an address to talk to logserverd so it can upload its phone log to the service.

The second entry is a check on what version of the software the client has and whether the client needs to be forced into flashromd to receive a download.

The next, rather long entry, is a list of the services headwaiterd hands to the client. Note how there are two addresses for each service, so that a single failure will have no effect on the client's ability to communicate with the various elements of the service.

The fourth entry is headwaiterd checking what tellyscript the client currently has, including the last-modified date for that script.

The fifth entry is an abbreviated form of the tellyscript headwaiterd would now hand down to the client, based on the most current PhoneDB. In this instance, there was no need to send down a new tellyscript because the client's tellyscript and the current tellyscript from headwaiterd match.

Example B-6 below shows two entries that you would see if the tellyscripts didn't match, so that headwaiterd had to hand down a new tellyscript to the client.

Example B-6 Unmatched tellyscripts

```
TS: DEBUG: total generation time now 812/6 (135 msec avg)
     TellyScript_Generate()     -/812f300000b00202
     ScriptGen.c:570             headwaiterd[20117]                   01/31 11:54:49

Box posted 0xa5155160, new script is 0x7f3d54f1, sent 4018 bytes
     HeadWaiter_CheckTellyscript()    -/812f300000b00202
     HeadWaiter.c:1022           headwaiterd[20117]                   01/31 11:54:49
```

Typical Errors and Analysis

This section lists a number of typical errors that you may encounter, and an analysis of each error.

Couldn't find the customer in the database

Example B-7 Can't find customer in database.

```
Unknown customer
   FlashUtil_GetInfoFromAccount()     -/810c9d0000b00232
   FlashUtilsDB.c:55           headwaiterd[616]                     01/30 08:05:21*

Account lookup by silicon serial failed- customer not found
   GetProductIDAndCategory() -/810c9d0000b00232
   HeadWaiter.c:1601           headwaiterd[616]                     01/30 08:05:21*
```

The error in Example B-7 above indicates that headwaiterd was unable to find a record associated with the given silicon serial number in the customer database. The main reason a customer does not appear in the database is an error during the preregistration process. In some instances, preregistrationd fails to write client

information to the customer database and is not notified of the error, believing that it has made a successful write. Headwaiterd will cause the client to disconnect and force it back through the preregistration process, setting the user in an infinite loop because preregistrationd is still under the impression that a record has been written. This is ultimately a customer database problem that is being corrected for the next release of the service. These errors are most commonly encountered when there is a new release of the client or the service and something breaks unexpectedly.

Caused by simulator

Example B-8 Error caused by simulator use

```
TS: WARNING: using 0x00 as silicon serial CRC
     TellyScript_New()           -/DaveAnderson
     ScriptGen.c:238             headwaiterd[616]                    01/30 09:34:57
```

As you might imagine, when you see a name instead of a silicon serial number, it's because someone is using a simulator to test something in the service. In Example B-8, headwaiterd is trying to compare a tellyscript for a client that isn't actually a client and doesn't have a silicon serial number as expected. Hence, headwaiterd logs this case as a warning that something unexpected has occurred. This fragment is just one entry in a much longer record typical to simulator logins. The full record is listed at the end of this section. (see "Simulator login record (uncommented)" on page 152)

Expired ticket

Example B-9 Expired ticket.

```
Client ticket expired
     HeadWaiter_ValidateLogin()-/81a96f0000b002a0
     HeadWaiter.c:809            headwaiterd[23321]                  01/31 00:30:46*

Client ticket expired
     HeadWaiter_Main()           -/81a96f0000b002a0
     HeadWaiter.c:221            headwaiterd[23321]                  01/31 00:30:46*

Client ticket expired
     HandleGetOrPost()           -/81a96f0000b002a0
     Services.c:1014             headwaiterd[23321]                  01/31 00:30:46*

failed: POST wtv-head-waiter:/ValidateLogin from 206.173.144.87
    Referer: wtv-head-waiter:/ValidateLoginName
    wtv-incarnation: 57
```

```
   wtv-ticket: eP1fMERtYTY3mCLQhyuHwd4AZ85UV7o96vJlR0TkVG3NmJXzXl+6o2OqB/XitJwvh
jPWr6Nt8sPHkmzRZmA5mY/iCA1/wVaY/h5qgwbUzxIQ6KuOiP8t7hVBJxRfaHck/hSfyUG612xXQ48K4
f/W4IUs57LHqN3V55mjfAr0Rw2+JTP//Vv0kx4ArhHrQWbzXi7WfNdATKDvu7hwqU1jPHpcI/Va0YXSz
jyUiD+GZUENDhv0ZlBjqTvrkp537uHFPdcWxLRELiftJ0vgNcyHmLcBaLgSdTcj9pSBlH7jKQFEzDRcx
fA66V8vVdJ7b2B65z9WS6XJXGlYscCbYqNM7LMs/AfKzDhcfkmdM5PxVd2NG147q1Ho3HxsXbZbWktvZ
/R+78fCEt9NYi70eka7g8UEUs3zBS5zJek+A3F7pdMsh/0RO76TgZniNnx/ydjKbu/R94J+TIi0vnFoj
6Gf5I+bn6FdWxQMLGSyvajR2wwACsXhXt4P+lx5xynjuSginMF/nX58DN0=


   wtv-client-serial-number: 81a96f0000b002a0
   wtv-encryption: true
   wtv-system-version: 253
   wtv-client-rom-type: bf0app
   wtv-client-bootrom-version: 105
   wtv-system-chipversion: 16842752
   wtv-system-sysconfig: 736935823
   wtv-system-cpuspeed: 112791600
   wtv-script-id: -391924837
   wtv-script-mod: 854523396
   wtv-language-country: enUS
   wtv-tourist-enabled: true
   Content-type: application/x-www-form-urlencoded
   Content-length: 29
   password: RICK
   user-id: 2980076
   wtv-client-address: 206.173.144.87
    HandleGetOrPost()          -/81a96f0000b002a0
    Services.c:1025            headwaiterd[23321]                  01/31 00:30:46*
```

The series of messages in Example B-9 above refers to a client ticket having expired. The client was unable for some reason to complete the login process with a single TCP connection. Somewhere along the line, either the client or the service dropped the TCP connection, and headwaiterd is unwilling to validate a login that takes place over more than one TCP connection. This feature was implemented for security, to make it much more difficult for unauthorized access from the outside to occur.

Using client newer than current production client

Example B-10 Client is newer than current production client.

```
NOTE: box has newer flash version than server!
     FlashUtil_CheckUpgradeImportance()         -/011aea2501000094
     FlashUtils.c:819           headwaiterd[20117]                  01/31 05:10:20
```

Using the wtv-tricks menu (see “WebTV Tricks Service” on page 118), it is possible to download versions of the client that are newer than those available to regular customers. If this happens to normal customers, it may be because they are in a different service group, one designed to test new features of the WebTV

Service before release to the rest of the customer base. It's not clear whether that situation will be entered in the log as in Example B-10 above, but it's something to be aware of.

Unexpected EOF

Example B-11 Unexpected end of file.

```
Unexpected EOF
     WTVP_ReadResponseCode()    -/011aea2501000094
     HTTP.c:288                  headwaiterd[20117]                  01/31 05:13:34*

Unexpected EOF
     WTVP_ReadPage()              -/011aea2501000094
     HTTP.c:419                  headwaiterd[20117]                  01/31 05:13:34*

Unexpected EOF
     Notifier_MakeRequest()     -/011aea2501000094
     NotifyInterface.c:377      headwaiterd[20117]                  01/31 05:13:34*

Bad file number
     TimeoutStreamClose()       -/011aea2501000094
     BufferSocket.c:192          headwaiterd[20117]                  01/31 05:13:34*

BufferedStream close function 0 returned an error: Bad file number
     BufferedStreamClose()      -/011aea2501000094
     BufferSocket.c:531          headwaiterd[20117]                  01/31 05:13:34*

Unexpected EOF
     Notifier_TryRequest()      -/011aea2501000094
     NotifyInterface.c:422      headwaiterd[20117]                  01/31 05:13:34*
```

When a client logs in successfully, headwaiterd tries to contact notifyd to inform it. Notifyd is the process responsible for notifying users that new mail has arrived while they have been logged in, making it the only process that needs to keep accurate track of which clients are logged in at any given moment. The particular error message in Example B-11 indicates that somehow the connection between headwaiterd and notifyd was broken, maybe because the server that runs notifyd went down or network connectivity between the two servers was lost.

Simulator login record (uncommented)

A client simulator is available for the Macintosh platform. It enables the Macintosh to emulate the behavior of a given version of the client, duplicating how it would display pages on an Internet terminal. Since the simulator doesn't have a number of the features of a normal Internet terminal, there will be a number of unusual entries in the logs, peculiar to a simulator logging in.

A simulator login record is shown in.Example B-12.

Example B-12 Simulator login record (uncommented).

```
Gave services to ID 1603323, group normal:
     name=wtv-log host=207.79.34.4 port=1609 flags=0x00000001 connections=1
     name=wtv-log host=207.79.34.3 port=1609 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      -/DaveAnderson
     ServiceConfig.c:584        headwaiterd[20117]                   01/31 08:07:55

GLITCH: box failed to post wtv-client-bootrom-version (assuming 0)
     FlashUtil_GetBoxFlashVersions()      -/DaveAnderson
     FlashUtils.c:953           headwaiterd[20117]                   01/31 08:07:56*

Not taunting simulator with a flash ROM image
     CheckNeedsNewFlash()       -/DaveAnderson
     HeadWaiter.c:1474          headwaiterd[20117]                   01/31 08:07:56

Gave services to ID 1603323, group normal, user DaveSim4:
     name=wtv-register host=207.79.34.4 port=1607
     name=wtv-register host=207.79.34.3 port=1607
     AddServiceHeadersToDictionary()      -/DaveAnderson
     ServiceConfig.c:584        headwaiterd[20117]                   01/31 08:07:56

Gave services to ID 1603323, group normal:
     name=wtv-log host=207.79.34.3 port=1609 flags=0x00000001 connections=1
     name=wtv-log host=207.79.34.4 port=1609 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      -/DaveAnderson
     ServiceConfig.c:584        headwaiterd[20117]                   01/31 08:07:57

GLITCH: box failed to post wtv-client-bootrom-version (assuming 0)
     FlashUtil_GetBoxFlashVersions()      -/DaveAnderson
     FlashUtils.c:953           headwaiterd[20117]                   01/31 08:07:57*

Not taunting simulator with a flash ROM image
     CheckNeedsNewFlash()       -/DaveAnderson
     HeadWaiter.c:1474          headwaiterd[20117]                   01/31 08:07:57

Gave services to ID 1603323, group normal, user DaveSim4:
     reset
     name=wtv-head-waiter host=207.79.34.4 port=1601 flags=0x00000001 connections=1
     name=wtv-head-waiter host=207.79.34.3 port=1601 flags=0x00000001 connections=1
     name=wtv-preregister host=207.79.34.4 port=1615
     name=wtv-preregister host=207.79.34.3 port=1615
     name=wtv-register host=207.79.34.3 port=1607
     name=wtv-register host=207.79.34.4 port=1607
     name=http host=207.79.34.4 port=1650 connections=3
     name=http host=207.79.34.3 port=1650 connections=3
     name=ftp host=207.79.34.4 port=1650
     name=ftp host=207.79.34.3 port=1650
     name=gopher host=207.79.34.4 port=1650
     name=gopher host=207.79.34.3 port=1650
     name=mailto host=207.79.34.4 port=1608 connections=1
     name=mailto host=207.79.34.3 port=1608 connections=1
     name=wtv-mail host=207.79.34.4 port=1608 connections=1
     name=wtv-mail host=207.79.34.3 port=1608 connections=1
     name=wtv-cookie host=207.79.34.4 port=1619 connections=1
     name=wtv-cookie host=207.79.34.3 port=1619 connections=1
```

```
     name=wtv-guide host=207.79.34.4 port=1621 connections=3
     name=wtv-guide host=207.79.34.3 port=1621 connections=3
     name=wtv-flashrom host=207.79.34.4 port=1618
     name=wtv-flashrom host=207.79.34.3 port=1618
     name=wtv-log host=207.79.34.4 port=1609 flags=0x00000001 connections=1
     name=wtv-log host=207.79.34.3 port=1609 flags=0x00000001 connections=1
     name=wtv-explore host=207.79.34.4 port=1624 connections=3
     name=wtv-explore host=207.79.34.3 port=1624 connections=3
     name=wtv-favorite host=207.79.34.4 port=1611 connections=3
     name=wtv-favorite host=207.79.34.3 port=1611 connections=3
     name=wtv-setup host=207.79.34.4 port=1613 connections=3
     name=wtv-setup host=207.79.34.3 port=1613 connections=3
     name=wtv-home host=207.79.34.4 port=1612 connections=3
     name=wtv-home host=207.79.34.3 port=1612 connections=3
     name=wtv-channel host=207.79.34.4 port=1626 connections=3
     name=wtv-channel host=207.79.34.3 port=1626 connections=3
     name=wtv-aroundtown host=207.79.34.4 port=1628 connections=3
     name=wtv-aroundtown host=207.79.34.3 port=1628 connections=3
     name=wtv-tricks host=207.79.34.4 port=1602 flags=0x00000004
     name=wtv-tricks host=207.79.34.3 port=1602 flags=0x00000004
     name=wtv-news host=207.79.34.4 port=1605 connections=3
     name=wtv-news host=207.79.34.3 port=1605 connections=3
     name=wtv-* host=207.79.34.4 port=1603 flags=0x00000003
     name=wtv-* host=207.79.34.3 port=1603 flags=0x00000003
     name=wtv-content host=207.79.34.4 port=1629 connections=3
     name=wtv-content host=207.79.34.3 port=1629 connections=3
     name=wtv-star host=207.79.34.4 port=1603 flags=0x00000003
     name=wtv-star host=207.79.34.3 port=1603 flags=0x00000003
     name=wtv-notices host=207.79.34.4 port=1622 connections=1
     AddServiceHeadersToDictionary()      -/DaveAnderson
     ServiceConfig.c:584        headwaiterd[20117]                   01/31 08:07:58

Bad file number
     TimeoutStreamClose()       -/DaveAnderson
     BufferSocket.c:192         headwaiterd[20117]                   01/31 08:07:58*

BufferedStream close function 0 returned an error: Bad file number
     BufferedStreamClose()      -/DaveAnderson
     BufferSocket.c:531         headwaiterd[20117]                   01/31 08:07:58*

TS: WARNING: using 0x00 as silicon serial CRC
     TellyScript_New()           -/DaveAnderson
     ScriptGen.c:238             headwaiterd[20117]                   01/31 08:08:02

WARNING: CheckArtemisIP says 0xccfe4a44 (204.254.74.68) isn't artemis
     ClientUsingArtemisDialup()-/DaveAnderson
     Ani.c:219                   headwaiterd[20117]                   01/31 08:08:02*

For DaveAnderson at 994156145518, telly ID=-1525329568, mod=854726873 (Fri Jan 3
1 08:07:53 1997)
     HeadWaiter_CheckTellyscript()    -/DaveAnderson
     HeadWaiter.c:985            headwaiterd[20117]                   01/31 08:08:02

TS: Hashed 'base:15:-|cnc:2:6870610|ziplink:1:3279100|cnc:2:14156870610|ziplink:
1:14153279100|artemis:1:18006108918' into 0xa5155160
     TellyScript_Compute()      -/DaveAnderson
```

```
    ScriptGen.c:351            headwaiterd[20117]                  01/31 08:08:02
```

Logserverd.log

The original purpose of logserverd was to record events internally that the user need not know about (client warnings, errors, and assertions). Over time, logserverd has picked up a number of other tangential purposes and duties. Whenever a client crashes, logserverd records a wealth of information into the crashrecord.log (See page 139). It dumps the information in binary, although the dumpcrash tool (See "dumpcrash" on page 74) can format the log into a more readable text form. Also, logserverd now keeps a record, in phonelog.h, of the phone calls the service receives,.

Many of the log entries are of the same general three-line format seen in the other service log files. However, much of a normal phone log entry bears little resemblance to the standard format.

Anatomy of a Typical Three-line Log Entry

Example B-13 Three-log entry analysis.

```
<error-message>: <error-type>
<reporting-function>() <client-id>/<subscriber-id>
<reporting-file>:<line-number> <service>[<pid>] <date> <time><flag>
```

Note: This format is not a hard-and-fast rule, but merely a guideline. Many log entries have additional information, sometimes a large amount of it, between the first and second lines of the normal three-line log entry. Nevertheless, these same three lines appear in nearly every log entry—they just may not appear consecutively or alone.

As with many of the other service logs, there are more complex entries. These are generally the phone log entries.

Common Error Messages

Client error

Example B-14 Client error.

```
! >
    LogClientError()           1743537/0179e55e11b0025d
    LogServer.c:431            logserverd[17146]                   10/03 09:32:23*
This is a common error and isn't generally worth noting. The client probably
disconnected or carrier was somehow lost.
Post-timing error
Protocol error
```

```
     AcceptConnection()          -/-
     TCP_Server.c:525            logserverd[12070]                    10/04 02:20:12*
AcceptConnection() failed: Protocol error
     ReadyList_HandleControlAndAccept()          -/-
     TCP_Server.c:300            logserverd[12070]                    10/04 02:20:12*
--
Connection reset by peer
     AcceptConnection()          -/-
     TCP_Server.c:528            logserverd[12070]                    10/04 02:33:51*
AcceptConnection() failed: Connection reset by peer
     ReadyList_HandleControlAndAccept()          -/-
     TCP_Server.c:300            logserverd[12070]                    10/04 02:33:51*
```

The AcceptConnection failures happen because the client occasionally closes a connection to the server in an improper manner, causing the AcceptConnection function to log an error.

The dropped connection is the client attempting to post its phone connection log and changing its mind about the timing. This situation results in a race condition and an AcceptConnection error, where the client may check to see if it needs a new tellyscript before the phone log gets uploaded. Fortunately, later versions of the service have corrected this problem, but it remains as a useful condition to be aware of, as similar timing problems could conceivably pop up from time to time in other parts of the service.

A “good” connection looks like the following:

- Client starts to post phone log to logserver.
- Client finishes posting phone log.
- Client asks Headwaiter if it has the most current script.
- Headwaiter answers, using information from the latest phone log.

A “bad” connection looks like this:

- Client starts to post phone log to logserver.
- Client reverses the posting, drops the connection.
- Logserver reports AcceptConnection failures.
- Client asks headwaiter if it has the most current script.
- Headwaiter answers.
- Client posts phone log to logserver (with headwaiter not able to compare to latest phone log, i.e., the one just posted)

For version 1.1, the client shouldn't drop the logserver connection, which in turn should avoid both the race condition and the AcceptConnection problems. Again, the problem is shown as an illustration of the types of errors that you can look for, as newer versions of the client or service usually correct known bugs.

Less common error—used code tounregister client

Example B-15 Used code to unregister client.

```
0x03 GetDialInSuccess
       when=0x32554236 (Fri Oct   4 09:58:30 1996)
       dialString='3220489'
       callWaitingPrefix=''   dialOutsidePrefix=''
       accessNumber=''
       flags=0x04 (waittone)
       dialSpeed=1   dceRate=28800   dteRate=115200
       protocol=0, compression=2
     PhoneLog_Log()              1756313/016f96a7000000de
     PhoneLog.c:213              logserverd[12074]                    10/04 09:58:37

Failed on '

' (country=1): Phone number had wrong #of digits
     PhoneNumber_Init()         1756313/016f96a7000000de
     Phone.c:102                 logserverd[12074]                   10/04 09:58:37*

Phone number had wrong #of digits
     UpdateDialPatterns()       1756313/016f96a7000000de
     DialPattern.c:526           logserverd[12074]                   10/04 09:58:37*

Phone number had wrong #of digits
     DialPattern_Update()       1756313/016f96a7000000de
     DialPattern.c:121           logserverd[12074]                   10/04 09:58:37*

ERROR: dial patterns not updated: Phone number had wrong #of digits
     LogServer_PhoneLog()       1756313/016f96a7000000de
     LogServer.c:256             logserverd[12074]                   10/04 09:58:37*

ERROR: PhoneLog failed: Phone number had wrong #of digits
     LogServer_Main()            1756313/016f96a7000000de
    LogServer.c:115           logserverd[12074]                10/04 09:58:37*
```

Example B-15 shows what happens when the owner of the client may have used a code to forcibly unregister. This use has the effect of setting the ANI to zero, making it difficult to determine if headwaiterd should try to send down a new tellyscript. Unregistering is one of the functions available from the wtv-tricks menu. (see “WebTV Tricks Service” on page 118).

Anatomy of a “Simple” Phone Log Entry from Logserverd.log

Example B-16 below is an uncommented, simple example of a phone log entry from logserverd.log. Then, the elements of the individual parts are discussed.

Example B-16 Phone log entry from logserverd.log.

```
PhoneLog from 01abd92511b0026e (version=7)
  numPhoneBusy=0                        tcpInputPackets=18
  numPhoneNoAnswer=0                   tcpOutputPackets=25
  numPhoneNoConnection=0              tcpDuplicatePackets=0
  numPPPFailure=0                       tcpRetransPackets=0
  numAuthenticationFailure=0         tcpOutOfOrderPackets=0
  numSuccessfulConnections=0         pppPacketSent=38
  numRecords=4                          pppPacketReceived=27
                                           crcErrorCount=0
                                           tcpConnectionsOpened=2
                                           tcpConnectionsDropped=2
Records:
     0x05 Disconnection
       when=0x3254b5ca (Thu Oct  3 23:59:22 1996)
       disconnectionType=6 "redial"
0x01 RunScriptReport
       when=0x3254b5cb (Thu Oct  3 23:59:23 1996)
       id=0x17c1a4d1 modWhen=0x3254b639 (Fri Oct  4 00:01:13 1996)
0x02 GetDialInFailure
       when=0x3254b5f7 (Fri Oct  4 00:00:07 1996)
       resultCode=10 "BadPassword"
       dialString='14152330570'
       callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
       accessNumber='(null)'
       flags=0x00 ()
       dialSpeed=0
0x03 GetDialInSuccess
       when=0x3254b61f (Fri Oct  4 00:00:47 1996)
       dialString='14152330570'
       callWaitingPrefix=''  dialOutsidePrefix=''
       accessNumber=''
       flags=0x04 (waittone)
       dialSpeed=1  dceRate=26400  dteRate=115200
       protocol=0, compression=2
     PhoneLog_Log()              1684201/01abd92511b0026e
     PhoneLog.c:213              logserverd[12074]                    10/04 00:02:44


PhoneLog from 01abd92511b0026e (version=7)
  numPhoneBusy=0                        tcpInputPackets=18
  numPhoneNoAnswer=0                   tcpOutputPackets=25
  numPhoneNoConnection=0              tcpDuplicatePackets=0
  numPPPFailure=0                       tcpRetransPackets=0
  numAuthenticationFailure=0         tcpOutOfOrderPackets=0
  numSuccessfulConnections=0         pppPacketSent=38
  numRecords=4                          pppPacketReceived=27
                                           crcErrorCount=0
                                           tcpConnectionsOpened=2
                                           tcpConnectionsDropped=2
```

The top line of Example B-16 contains the silicon serial number and the version number of the service (7 = Version 1.0, 9 = Version 1.1 or 8 = 1.0 Simulators). The rest of the information below it is not reliably incremented and can't be trusted, except for the `numRecords` field, which tells how many recorded events appear in the log below. At present, the usefulness of the remaining fields is limited, but in future updates, these fields will be more reliably incremented and become more useful tools. Also note that the first line and the last two lines of the entire entry compose the "normal," three-line log entry seen in most of the service log files.

Example B-17 Disconnect by client.

```
Records:
     0x05 Disconnection
       when=0x3254b5ca (Thu Oct  3 23:59:22 1996)
       disconnectionType=6 "redial"
```

The lines shown in Example B-17 indicate a disconnection by the client. The `disconnectionType` value contains further information on the nature of the disconnection. (see "DisconnectionType value listing" on page 161.)

Example B-18 Comparison between tellyscript and client.

```
0x01 RunScriptReport
       when=0x3254b5cb (Thu Oct  3 23:59:23 1996)
       id=0x17c1a4d1 modWhen=0x3254b639 (Fri Oct  4 00:01:13 1996)
```

The part of the entry shown in Example B-18 indicates that the current tellyscript was compared to what the system thinks that client should have.

Example B-19 Dial-in failure.

```
0x02 GetDialInFailure
       when=0x3254b5f7 (Fri Oct  4 00:00:07 1996)
       resultCode=10 "BadPassword"
       dialString='14152330570'
       callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
       accessNumber='(null)'
       flags=0x00 ()
       dialSpeed=0
```

The part of the entry in Example B-19 indicates a dial-in failure. The `resultCode` can often shed light on the exact nature of the failure (see "Values for ResultCode field in a phone log" on page 160). Also take note of the last four lines, which all have values of zero or null. Version 7 (i.e., 1.0 of the service) doesn't fill in these fields, whereas version 9 (1.1 of the service) has meaningful data here.

Example B-20 Successful dial-in indication.

```
0x03 GetDialInSuccess
       when=0x3254b61f (Fri Oct  4 00:00:47 1996)
       dialString='14152330570'
       callWaitingPrefix=''  dialOutsidePrefix=''
       accessNumber=''
       flags=0x04 (waittone)
       dialSpeed=1  dceRate=26400  dteRate=115200
       protocol=0, compression=2
```

This portion of the entry indicates a successful dial-in, and the various fields tell something about the nature of how the call was made. It has a time/date stamp, as well as the phone number dialed. It also indicates the call-waiting prefix or any dialing prefixes (9 to get an outside line, for instance). The `flags` field has some useful information, including if the call was pulse dialing only, call waiting was disabled, the client waits for a dial tone (the flag active in the above example), and a variety of other similar data on the dialing options used. Lastly, the `dceRate` is the modem negotiation speed and can be an indicator of the quality of the customer's line. The `dteRate` is always 115200 for an Internet terminal and 57600 for a simulator.

Example B-21 Standard log entry.

```
PhoneLog_Log()              1684201/01abd92511b0026e
     PhoneLog.c:213              logserverd[12074]                  10/04 00:02:44
```

These last two lines of the record shown in Example B-21 are much the same as those found in other service logs and, when combined with the very first line of the log entry, they make up the standard three-line log entry common to the service logs as a whole.

Values for ResultCode field in a phone log

The following are codes that might appear in the `resultCode` field of a log entry, specifically, under a `GetDialInFailure`. Some nonobvious causes are explained.

0 ParseError

1 Connecting

2 LinkConnected

3 ConfigurationError—the tellyscript was corrupted and was rejected.

4 DialingError—the service was waiting for a result from the modem but never received one.

5 NoDialtone—The user’s phone cord wasn't plugged in or some similar problem.

6 NoAnswer—The remote side just rang and rang.

7 Busy—The remote side returned a busy signal to the client.

8 HandshakeFailure—Handshake failure errors are caused generally in one of three ways:

- The service was waiting for a result from the modem but never received one after the first. (If the service never sees the initial “OK” response, then it returns a “dialing error” result instead). This can happen if the modems start to train, but fail while negotiating v.42bis.
- The service gets garbage out of the modem when expecting a result code. If expecting the numeric result code for the connection rate and instead received "dkjfdkjfdkjf", it would return this code.
- If the service gets a “NoCarrier” after carrier was detected but before negotiating was complete. This error is somewhat like the previous one, but instead of garbage, it's a “no carrier” code.

9 UnknownError

10 BadPassword—Called up, PPP negotiations were working great, but there was an authentication failure. The user-side Radius server tried to call the service-side Radius server and either got an NAK or never got an answer at all.

11 PPPHandshakeFailure—Tried to negotiate PPP and failed. The service successfully connected to the remote modem but for some reason couldn't get through to the other side. Common causes are problems with the remote terminal server (i.e., the modem answered but there's nobody home) and bad modem connections (e.g., bad error correction and a ton of noise on the line).

12 NoCarrier—With the Rockwell chipset, this code means the client called the number, and the remote side answered but never produced carrier. This failure could be a defective modem, an “all circuits are busy” recording, a “you don't need to put a 1 in front of that” recording, or a live person.

13 Black Hole Router Problem—The client dialed up, negotiated PPP, Radius worked, and then tried to connect to the service but failed. This situation usually happens when the remote terminal server gives out a bad IP address and the packets travel from the Internet terminal to the server but can't find their way back from the server. This failure could also be caused by routing problems.

14 VerySlowConnect—This error is returned if the client connects at less than 14.4 Kbps. The previous behavior was to tell the modem to refuse to train at anything less than 14.4 Kbps, but that practice caused “no carrier”-ish behavior (i.e., the system would wait a full minute before trying the next POP and would return a “no carrier” status indication).

Three of these codes— “NoCarrier,” “Black Hole Router Problem,” and “VerySlowConnect”—are recent additions of version 1.1 of the client. As with the DisconnectionTypes (see below), it pays to be careful about overinterpreting the data here. For instance, the messages “Connecting” and “LinkConnected” don't sound like errors at all and probably aren't descriptive of the actual failure mode.

DisconnectionType value listing

An alternative to a GetDialInSuccess or a GetDialInFailure is a client disconnection. Just as a resultCode field is associated with a GetDialInFailure, a DisconnectionType field refers to the types of client disconnections. The possible codes are listed below:

0 Unknown

1 Power loss

2 User hangup

3 Carrier loss

4 Call waiting

5 Inactivity timeout

6 Redial

Be aware that these messages aren't always completely accurate. The only current difference between “carrier loss” and “call waiting” is that “call waiting” is returned if call waiting has been enabled on the client; if not, then “carrier loss” is returned.

“Power loss” might never be returned, as it's difficult in principle to write a record once power is lost, and the client isn't currently smart enough to try to write a log entry once it regains power.

The lesson in this discussion is to be careful what you try to interpret. Don't overinterpret either the DisconnectionTypes or resultCodes in the phone logs.

Obtaining Client Information

From the phone logs, you can obtain the subscriber ID or silicon serial number (a hexadecimal string). With this number, you have access to a wealth of information pertaining to that client, via the clientinfo {***Cross Reference***} command.

From one of the storeless machines, the command line

```
storeless-2:~$ clientinfo 81f2440010b002b4
```

yields a wealth of information on the client. A typical entry is given in the following example:

Example B-22 Can’t find customer in database.

```
ANI ........................ 00 817-751-0651 (WACO, TX)
  Shared Secret ............. 'vG3Li5rWjjg='
  Script locked? ............ no
  Fallback disallowed? ...... no
  Last successful connect ... 755-7212
```

The ANI is fairly self-explanatory, and Shared Secret is the string that the system looks for before allowing client access to the service.

Script Locked is set to ‘no’ as a default, meaning that if headwaiterd determines that a new tellyscript is necessary, it will be sent. The main reason Script Locked might be set ‘yes’ would be if the client has received a custom tellyscript and specifically doesn't want another.

Fallback disallowed specifies whether this client can fall back to the backup toll-free service. It defaults to 'no,' meaning that the client should be allowed into the toll-free service as a fallback measure. It can be reset to avoid abuses of the toll-free service. For instance, a customer in a toll-only zone could force the client to fall back to the toll-free service by unplugging the line at appropriate times.

Example B-23 Record of scripts sent to client

```
  Most recent script sent to client:
    Hash 0xedd51145, sent Fri Oct  4 15:34:55 1996
    v7  base/-
    v3  uunet/7557212
    v3  uunet/7557212
    v3  uunet/13186760748
    v1  artemis/18006138199
  Previous script sent to client:
    Hash 0xd15e0e41, sent Thu Oct  3 13:46:43 1996
    v7  base/-
    v3  uunet/7557212
    v3  uunet/18177557212
    v3  uunet/13186760748
    v1  artemis/18006138199
```

The above example details not only the current tellyscript sent to the client, but also the previous one. Note that a tellyscript has three main parts. First, there is the version of the base script (the versions tend to change more frequently than does provider information). Second, there is the provider information (for example, uunet, version 3 of their script). Third, there is the information the service generates, that is, the actual phone numbers that should be dialed.

Example B-24 Established dialing patterns.

```
Established dialing patterns:
     ANI 817-751-0651 + POP 817-755-7212 -- mode=7-digit
```

A combination of the ANI and the POP determines which mode of dialing the client will try first, whether 7, 10, or 11 digits.

Note: 8-digit dialing should no longer be used anywhere, having been phased out some number of years ago.

Example B-25 Dialing overrides.

```
Dialing overrides:
     (none)
  Current set of POPs for user's exchange:
    uunet 817-755-7212 (WACO, TX) [cost=$0.000/0.0mi  local]
      (7557212 or 18177557212)
    uunet 318-676-0748 (SHREVEPORT, LA) [cost=$4.800/209.1mi  TOLL]
      (13186760748)
```

This last example (Example B-25) details any dialing overrides that have been set, as well as detailing other POPs that might be available for that user's exchange but that, for load balancing reasons, have not been assigned.

Uncommented "Long" Phone Log Entry

Below is an example of a more complex phone log entry from the logserverd.log. A variety of disconnections and loss of carriers have occurred.

Example B-26 Complex phone log entry from logserverd.log.

```
PhoneLog from 81f2440010b002b4 (version=7)
  numPhoneBusy=14                      tcpInputPackets=208
  numPhoneNoAnswer=0                   tcpOutputPackets=271
  numPhoneNoConnection=3               tcpDuplicatePackets=0
  numPPPFailure=0                      tcpRetransPackets=1
  numAuthenticationFailure=0          tcpOutOfOrderPackets=0
  numSuccessfulConnections=0          pppPacketSent=333
  numRecords=60                          pppPacketReceived=260
                                            crcErrorCount=3
                                            tcpConnectionsOpened=9
                                            tcpConnectionsDropped=7
  Records:
    0x05 Disconnection
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
      disconnectionType=2 "user hangup"
    0x06 NVRAMWrite
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
    0x01 RunScriptReport
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
      id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
    0x02 GetDialInFailure
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
      resultCode=5 "NoDialtone"
      dialString=''
      callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
      accessNumber='(null)'
      flags=0x00 ()
      dialSpeed=0
    0x01 RunScriptReport
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
      id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
    0x02 GetDialInFailure
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
      resultCode=5 "NoDialtone"
      dialString=''
      callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
      accessNumber='(null)'
      flags=0x00 ()
      dialSpeed=0
    0x06 NVRAMWrite
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
    0x01 RunScriptReport
      when=0x00000000 (Wed Dec 31 16:00:00 1969)
      id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
    0x05 Disconnection
```

```
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  disconnectionType=2 "user hangup"
0x06 NVRAMWrite
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=5 "NoDialtone"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=5 "NoDialtone"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=8 "HandshakeFailure"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x05 Disconnection
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  disconnectionType=2 "user hangup"
0x06 NVRAMWrite
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=8 "HandshakeFailure"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
```

```
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
```

```
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
```

```
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=7 "Busy"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x02 GetDialInFailure
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  resultCode=8 "HandshakeFailure"
  dialString=''
  callWaitingPrefix='(null)'  dialOutsidePrefix='(null)'
  accessNumber='(null)'
  flags=0x00 ()
  dialSpeed=0
0x01 RunScriptReport
```

```
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0x00000000 modWhen=0x00000000 (Wed Dec 31 16:00:00 1969)
0x03 GetDialInSuccess
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  dialString=''
  callWaitingPrefix=''  dialOutsidePrefix='9'
  accessNumber=''
  flags=0x14 (extern waittone)
  dialSpeed=1  dceRate=24000  dteRate=115200
  protocol=0, compression=2
0x05 Disconnection
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  disconnectionType=6 "redial"
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0xd15e0e41 modWhen=0x32542633 (Thu Oct  3 13:46:43 1996)
0x03 GetDialInSuccess
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  dialString='7557212'
  callWaitingPrefix=''  dialOutsidePrefix='9'
  accessNumber=''
  flags=0x14 (extern waittone)
  dialSpeed=1  dceRate=21600  dteRate=115200
  protocol=0, compression=2
0x05 Disconnection
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  disconnectionType=2 "user hangup"
0x06 NVRAMWrite
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
0x01 RunScriptReport
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  id=0xd15e0e41 modWhen=0x32542633 (Thu Oct  3 13:46:43 1996)
0x03 GetDialInSuccess
  when=0x00000000 (Wed Dec 31 16:00:00 1969)
  dialString='7557212'
  callWaitingPrefix=''  dialOutsidePrefix='9'
  accessNumber=''
  flags=0x14 (extern waittone)
  dialSpeed=1  dceRate=21600  dteRate=115200
  protocol=0, compression=2
PhoneLog_Log()              1855636/81f2440010b002b4
PhoneLog.c:213              logserverd[14087]                    10/04 15:34:56
```

Mailtod.log

At present, there are 10 mail services, which record to one log file. This log file can be found on production mailtod machines in the directory:

`~production/webtv/log/mailtod.log`

The mailtod.log records the transactions associated with customer mail usage. It details any rewriting of tags when an HTML-based message is sent, problems with character set recognition, and a variety of other information useful for tracing errors in the mail system (broken pipes caused by client disconnections, and the like).

Anatomy of a Typical Log Entry

The examples below analyze a typical service log entry. Note that not all of the fields are necessarily present in any one entry. The <flag>, an asterisk after the date and time stamp, indicates an error, as opposed to an informational message or a warning.

Example B-27 Analysis of typical service log entry.

```
<error-message>: <error-type>
<reporting-function>() <client-id>/<subscriber-id>
<reporting-file>:<line-number> <service>[<pid>] <date> <time><flag>
```

An Example

Example B-28 Example 1of typical service log entry.

```
Error reading headers from client: Unexpected EOF
HandleCommands() -/-
Protocol.c:203 mailtod[235] 10/03 12:46:22*
```

Or

Example B-29 Example 2 of typical service log entry.

```
Invalid parameter
ReadMail() 1844144/81470a0000b00259
 ReadMail.c:184 mailtod[235] 10/03 10:16:50*
```

Note: This format is not a hard-and-fast rule, but merely a guideline. Many log entries have additional information, sometimes a large amount of it, between the first and second lines of the normal three-line log entry. Nevertheless, these same three lines appear in nearly every log entry—they just may not appear consecutively or alone.

It's fairly typical to have more than one log entry for a given error. This duplication happens as the error codes percolate up the stack (there's an example of this later).

Anatomy of a More Complex Log Entry

Example B-30 below is a log entry with significantly more information in it, but the format is still basically the same. It's just like the simple log entry, except that there is now a set of ticket headers added just after the title line.

Example B-30 A more complex log entry.

```
failed: GET wtv:/readmail?message_delete=true&message_id=13&selected=delete from
153.35.243.238
pragma: no-cache
wtv-incarnation: 23
wtv-ticket: 8KnR2Uxxov1JaTTAZbH3GeFKImZirPAiA/CVUkhKUf9ib96sN6jYltgIujAFx...
wtv-client-serial-number: 0118d55e0100002d
wtv-encryption: true
wtv-system-version: 105
wtv-client-rom-type: bf0app
wtv-client-bootrom-version: 105
wtv-system-chipversion: 16842752
wtv-system-sysconfig: 736935851
wtv-system-cpuspeed: 112789920
wtv-script-id: 1934556714
wtv-script-mod: 844291539
wtv-tourist-enabled: true
wtv-user-name-hack:
message_delete: true
message_id: 13
selected: delete
wtv-user-name: Zarkov
wtv-client-time-zone: PDT -0700

HandleGetOrPost() 1679597/0118d55e0100002d
Services.c:819 mailtod[235] 10/03 14:43:01*
```

Things to note on more complex entries:

- The `wtv-ticket` can be quite long. It's an encrypted string, which will often include an entire mail message.
- The `wtv-system-version` and `wtv-system-client-bootrom`-version refer to the build number, not the current version of the software it is running.

What was the meaning of that whole entry? The mail system was trying to access a deleted message and could not do so. Because the message shouldn't have been an option on screen, it is now a problem that needs attention. Problems like this typically show up now and again, usually corrected in service updates.

Examples of “Safe” Warnings and Error Messages

Note the absence of “*” in the rightmost column for most of these.

Example B-31 Example 1 of safe warning and error messages.

```
MIME charset "iso-2022-jp" is not supported, treating as ISO-8859-1
WriteMimeBodyAsHTML() 1782189/816c0f0000b002c5

MimeHTML.c:450 mailtod[228] 10/03 12:19:32
```

In Example B-31, the system tried to handle an unsupported character set in a mail message—in this case, a Japanese character set. The system does translate the character fairly well and thus is not a real cause for concern.

Example B-32 Example 2 of safe warning and error messages.

```
Threw away unsafe HTML tag HEAD

WriteSafeHTMLSubset() 1792955/813a530010b002a7
SafeHTML.c:315 mailtod[228] 10/03 11:03:54
```

In Example B-32, the system is filtering out "bad" tags from HTML-based messages. In other words, it's cutting out `<HEAD>...</HEAD>` from the messages. Since this kind of filtering is routine, it is not a cause for concern.

Example B-33 Example 3 of safe warning and error messages.

```
Implementation error
Event_AddString() -/-
UserEvents.c:415 mailtod[5617] 09/29 23:19:27*
```

The message in Example B-33 just says that an event needs to be started before getting an eventlog. It's a harmless error that should be gone in the present version of the service.

Example B-34 Example 4 of safe warning and error messages.

```
write() failed in TimeoutWrite(): Broken pipe
TimeoutWrite() -/-
TimeoutSocket.c:226 mailtod[12528] 09/29 09:37:27*
```

The error message in Example B-34 is typical if the client disconnects while receiving data; hence, the "broken pipe" warning.

A Not-So-Safe Warning

Example B-35 Example 1 of an unsafe warning.

```
Mailbox3 index out of bounds: -1 [0..23]
Mailbox3_GetMessageBytesAtOffsetLength() 1672084/813a1d0010b002ec
Mailbox3.c:961 mailtod[228] 10/03 04:29:33

Invalid parameter
Mailbox3_GetMessageBytesAtOffsetLength() 1672084/813a1d0010b002ec
Mailbox3.c:963 mailtod[228] 10/03 04:29:33*

Invalid parameter
Mailbox3_GetMessageAt() 1672084/813a1d0010b002ec
Mailbox3.c:943 mailtod[228] 10/03 04:29:33*
```

In Example B-35, the system attempted to go to deleted email, but it shouldn't have offered this as an option to the user in the first place. These sorts of problems point to software bugs. Note that the first error message lacked a flag, whereas the second two were interpreted as errors. These messages are obviously from the same event, as both the process id and the exact time of the event match up. This type of warning can be a useful diagnostic.

A Serious Problem

Every so often there is a really bad message; that is, something that can't immediately be identified. Note in Example B-36 below how the problem was not flagged as an error, but that the message did hint at a serious problem.

Example B-36 A serious problem.

```
MIME multipart message is empty
WriteMimeBodyAsHTML() 1679298/0174eb2501000089
MimeHTML.c:402 mailtod[12528] 09/29 06:15:34
```

The above event actually indicated that part of a multipart message was cut off. Certain messages were being truncated, which was a serious problem. The important thing to note is that the problem wasn't even flagged—another warning that severity of the problem does not necessarily correlate with whether an event is flagged in the log.

Preregistrationd.log

The preregistrationd.log is a general record of what happens when someone calls either the toll-free number or the local number (415-329-0511) to preregister. Preregistration consists of receiving ANI, producing a tellyscript based on the ANI, comparing that script to the client's current tellyscript, and if different, handing the new tellyscript down to the client. Aside from the "normal" happenings, the log records anything abnormal that might have happened. Telling the difference between what is bad and what is simply routine-but-odd-looking isn't altogether straightforward.

Anatomy of a Preregistration Log Entry

Many of the entries in preregistrationd.log are similar in format to the other service log files, in that its entries are routinely three lines long and contain the same information as that seen in the other service logs. The anatomy of a simple log entry is pictured in Example B-37 below:

Example B-37 Analysis of preregistration log entry.

```
<error-message>: <error-type>
<reporting-function>() <client-id>/<subscriber-id>
<reporting-file>:<line-number> <service>[<pid>] <date> <time><flag>
```

Note: This format is not a hard-and-fast rule, but merely a guideline. Many log entries have additional information, sometimes a large amount of it, between the first and second lines of the normal three-line log entry. Nevertheless, these same three lines appear in nearly every log entry—they just may not appear consecutively or alone.

General Errors

Many of the errors that require attention are related to ANI failures of one sort or another, and those entries will generally have "Sending manual ANI request page" as the first line of the entry. (See "ANI and Associated Problems" on page 180.)

Startup and its typical messages

Example B-38 Startup messages.

```
** starting **
     InitLog()                    -/-
     Log.c:571                    preregistrationd[207]              09/30 03:17:08

Successfully mapped PhoneDB (ver=8)
       Built Fri Sep 27 15:29:13 1996 by uid=1057
PhoneDB_Map()                -/-
     PhoneDB.c:234                preregistrationd[207]              09/30 03:17:10
Allocated space for 100 connections (20 children, 80 clients)
     Descriptors_New()           -/-
```

```
     DescriptorControl.c:241    preregistrationd[207]                09/30 03:17:10
    The first action the system should take on startup is to create an InitLog
.   If you don't see this entry, it may be time to worry.  The next typical action
    is to  map a PhoneDB.  Looking at the date the PhoneDB was built can be helpful
    for some diagnostics.  The system should then allocate space for some
    number of connections.
```

Reclaiming idle file descriptors

Example B-39 Idle file descriptions.

```
Reclaimed idle fd 94 (idle for 143085 seconds)
     Descriptors_ReclaimIdle() -/-
     DescriptorControl.c:173    preregistrationd[207]                10/04 13:25:20
```

The first line of the entry in Example B-39is the only important one, and it has two aspects of note. First, the fd number is important, because if the number reaches 100 (at least in the example configuration), the system crashes.

The other notable element in the first line is the idle time. High numbers are good, indicating that load is acceptably low and that the process was allowed to remain idle a long time before being reclaimed. Numbers in the range of 10–15 seconds are getting toward overloading the system resources and may indicate a problem.

Oracle errors

Example B-40 Oracle errors.

```
SQL error: ORA-12203: TNS:unable to connect to destination (err num -12203)
     ConnectToDatabase()        -/01a2e85e11b0028d
     DBCalls.c:0                 headwaiterd[307]                    10/04 14:51:07*

SQL error: Error while trying to retrieve text for error ORA-12203 (err num - 12203)
     ConnectToDatabase()        -/01db7d5e010000e6
     DBCalls.c:0                 preregistrationd[222]               09/30 03:28:12*
```

The first entry in Example B-40 is something to be concerned about, as error message ORA-12203 indicates that, by design or accident, the system wasn't able to contact the database. This sort of error tends to need further looking into. The second message merely indicates that the system was unable to retrieve the appropriate text for the error message itself.

Example B-41 Database error.

```
SQL error: ORA-03113: end-of-file on communication channel (err num -3113, retrying)
     CustomerDB_NewClientFromID()  1832776/0124eb5e11b0029b
     DBCalls.c:0                 preregistrationd[222]              10/04 14:49:35*

TS: Hashed 'uunet:3:915-498-2003|uunet:3:915-498-2003|artemis:1:18006138199' into
0xab821aa2
     TellyScript_ComputeSimple() 1832776/0124eb5e11b0029b
     ScriptGen.c:330             preregistrationd[222]              10/04 14:50:04
ORA-03113 is a suspicious error, whose exact meaning is unclear, and hence needs to
be looked at further.  "TS" refers to something embedded in the tellyscript
generation code. All the log messages emanating from the tellyscript library are
prefaced with TS.
Database error
     LogServer_Main()            1832776/0124eb5e11b0029b
     LogServer.c:106             logserverd[14089]                  10/04 14:50:08*

Database error
     HandleGetOrPost()           1832776/0124eb5e11b0029b
     Services.c:808              logserverd[14089]                  10/04 14:50:08*
```

Database error is a generic term that offers little help in determining exactly what happened. To trace these problems, check the user event log for the machine in question, (`~production/webtv/event_log/user-events.log`), noting the process and timestamp.

Server dying errors

Example B-42 Server dying errors.

```
ReceiveFD: connection closed by server
     ReceiveFD()                  -/-
     ReceiveFD.c:51              preregistrationd[253]              09/30 02:02:49*

Implementation error
     RunChild()                   -/-
     ProcessControl.c:239       preregistrationd[253]              09/30 02:02:49*

Exiting Child
     RunChild()                   -/-
     ProcessControl.c:270       preregistrationd[253]              09/30 02:02:49*

ReceiveFD: connection closed by server
     ReceiveFD()                  -/-
     ReceiveFD.c:51              preregistrationd[251]              09/30 02:02:49*

Implementation error
     RunChild()                   -/-
     ProcessControl.c:239       preregistrationd[251]              09/30 02:02:49*
```

```
Exiting Child
     RunChild()                    -/-
     ProcessControl.c:270       preregistrationd[251]               09/30 02:02:49*

ReceiveFD: connection closed by server
     ReceiveFD()                   -/-
     ReceiveFD.c:51              preregistrationd[250]               09/30 02:02:49*
[...]

** starting **
     InitLog()                     -/-
     Log.c:571                    preregistrationd[207]               09/30 03:17:08
```

"Server dying" is a rather broad term and can be either a routine process or a problem. Example B-42 above is not completely shown because it is rather long, and is a routine process shutting down. The important elements to note are the first entry, which indicates that a connection has been closed by the server, and the last entry, which should be a typical InitLog sequence if the machine has come back up gracefully.

A Typical Successful Preregistration Sequence

Below is a typical preregistration sequence. Then, we discuss the elements of the individual parts.

Example B-43 Preregistration sequence.

```
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 13:23:35
Unexpected nil value
     Channel_SetSessionKeys()  -/-
     Channel.c:80                preregistrationd[222]               10/04 13:23:35*
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]               10/04 13:23:35*
DEBUG: returning '006123212109' from port 16 on ani-2:2000
     GetPeerANI()                 -/818f210010b002a5
     PreRegistration.c:1734    preregistrationd[222]               10/04 13:23:35
TS: Hashed 'base:7:-|uunet:3:3217960|cnc:2:8210575|uunet:3:3217960|cnc:2:161
28210575|artemis:1:18006138199' into 0x7146a372
     TellyScript_Compute()      -/818f210010b002a5
     ScriptGen.c:263             preregistrationd[222]               10/04 13:23:35
TS: DEBUG: total generation time now 142133/1836 (77 msec avg)
     TellyScript_Generate()     -/818f210010b002a5
     ScriptGen.c:420             preregistrationd[222]               10/04 13:23:35
Sending tellyscript 0x7146a372 (3107 bytes)
     SendTellyscript()           -/818f210010b002a5
     PreRegistration.c:2349    preregistrationd[222]               10/04 13:23:35
Gave services to ID 818f210010b002a5, group (nil):
```

```
     reset
    name=wtv-head-waiter host=207.76.180.63 port=1601 flags=0x00000001 connections=1
    name=wtv-head-waiter host=207.76.180.61 port=1601 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      -/818f210010b002a5
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:23:35
Gave services to ID 818f210010b002a5, group (nil):
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     AddServiceHeadersToDictionary()      -/818f210010b002a5
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:23:35
Gave services to ID 818f210010b002a5, group (nil):
     name=wtv-flashrom host=207.76.180.81 port=1618
     name=wtv-flashrom host=207.76.180.82 port=1618
     AddServiceHeadersToDictionary()      -/818f210010b002a5
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:23:35
```

Discussion of individual parts

Example B-44 Preregistration sequence, part 1.

```
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 13:23:35
Unexpected nil value
     Channel_SetSessionKeys()   -/-
     Channel.c:80                preregistrationd[222]               10/04 13:23:35*
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]               10/04 13:23:35*
```

These three entries (two of which are flagged as errors) are nothing to worry about. It's just system security being alert, as the Internet terminal doesn't yet have an identity with the service.

Example B-45 Preregistration sequence, part 2.

```
DEBUG: returning '006123212109' from port 16 on ani-2:2000
     GetPeerANI()                 -/818f210010b002a5
     PreRegistration.c:1734     preregistrationd[222]               10/04 13:23:35
```

The DEBUG entry is good. It is returning the raw ANI information, as well as the port number and the ANI server it is on. Ideally, the raw ANI information is returned with two zeroes as headers for the actual phone number.

Example B-46 Preregistration sequence, part 3.

```
TS: Hashed 'base:7:-|uunet:3:3217960|cnc:2:8210575|uunet:3:3217960|cnc:2:161
28210575|artemis:1:18006138199' into 0x7146a372
     TellyScript_Compute()      -/818f210010b002a5
     ScriptGen.c:263             preregistrationd[222]               10/04 13:23:35
```

TS: Hashed is an indication that the current tellyscript the client has is being compared to what the service thinks the client should have. The string is an abbreviated version of the information actually in the tellyscript. Base refers to the version of the Base script in use, and the text separated by pipes is the provider, followed by the version number of their script and the actual POP number to dial. The last string is a code for that tellyscript, which is what the server is comparing against.

Example B-47 Preregistration sequence, part 4.

```
TS: DEBUG: total generation time now 142133/1836 (77 msec avg)
     TellyScript_Generate()     -/818f210010b002a5
     ScriptGen.c:420             preregistrationd[222]               10/04 13:23:35
Sending tellyscript 0x7146a372 (3107 bytes)
     SendTellyscript()           -/818f210010b002a5
     PreRegistration.c:2349     preregistrationd[222]               10/04 13:23:35
```

These two messages indicate a new tellyscript being generated and sent to the client.

Example B-48 Preregistration sequence, part 5.

```
Gave services to ID 818f210010b002a5, group (nil):
     reset
     name=wtv-head-waiter host=207.76.180.63 port=1601 flags=0x00000001 connections=1
     name=wtv-head-waiter host=207.76.180.61 port=1601 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      -/818f210010b002a5
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:23:35
Gave services to ID 818f210010b002a5, group (nil):
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     AddServiceHeadersToDictionary()      -/818f210010b002a5
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:23:35
Gave services to ID 818f210010b002a5, group (nil):
     name=wtv-flashrom host=207.76.180.81 port=1618
     name=wtv-flashrom host=207.76.180.82 port=1618
     AddServiceHeadersToDictionary()      -/818f210010b002a5
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:23:35
```

These final three messages are indications that the system has allowed access for the Internet terminal with the given subscriber ID.

Notice that once preregistrationd has successfully gotten a subscriber id, it stops flagging further messages as errors. System security has been satisfied that the Internet terminal has a right to be connected. In the future, the first three messages will probably be filtered out of the log.

ANI and Associated Problems

The single message that tends to point out ANI problems is **"Sending manual ANI request page."** This message indicates that the server has prompted users to manually input their phone number, meaning that some sort of failure in ANI has occurred. Some of these failures are not worrisome at all, internal users, for instance; however, others indicate a significant error.

Raw ANI information normally has *00 as the header before the phone number. At times, it will have *06 or *07, in which case the ANI server will not recognize the ANI as valid.

The headers *99, *98, and *97 have special internal uses, as discussed below. These prefixes appear in the customer database, so Customer Care will be able to see them.

*97—User used “cnc-palo” or “uunet-palo” code (useful for foreign countries where tollfree 800 service to the US is unavailable, and preregistration must take place using a non-tollfree line)

*98—User used “xyzzy” code (back door into tollfree service)

*97—User entered ANI manually

No matching ANI server

Example B-49 No matching ANI server.

```
NOTE: IP address 0xccfe4ad3 does not map to any ANI server
       Not doing ANI lookup from server
     GetPeerANI()                -/ptrump
     PreRegistration.c:1639     preregistrationd[222]               10/04 15:43:35*
Sending manual ANI request page
     AskUserForANI()             -/ptrump
     PreRegistration.c:704      preregistrationd[222]               10/04 15:43:35
```

Generally, this error is generated when someone within the company has tried to access WebTV internally. Note the subscriber id is "ptrump," as opposed to the expected hexadecimal string. Alternatively, this message can be the result of some external agent trying to telnet into our system. Either way, the IP address does not map to one of the ports on either of the ANI servers, and this is what produces the error.

"Bad" ANI

Example B-50 Bad ANI.

```
Got non-00 ANI '*078133483716#*', returning error
     GetPeerANI()                -/81c6180010b00231
     PreRegistration.c:1725     preregistrationd[222]               10/04 07:54:45
Sending manual ANI request page
     AskUserForANI()             -/81c6180010b00231
     PreRegistration.c:704      preregistrationd[222]               10/04 07:54:45
[...]
Got non-00 ANI '*078133483716#*', returning error
     GetPeerANI()                -/81c6180010b00231
     PreRegistration.c:1725     preregistrationd[222]               10/04 07:57:02
Sending manual ANI request page
     AskUserForANI()             -/81c6180010b00231
     PreRegistration.c:704      preregistrationd[222]               10/04 07:57:02
[...]
Failed on '' (country=1): Phone number had wrong #of digits
     PhoneNumber_Init()         -/81c6180010b00231
     Phone.c:102                 preregistrationd[222]               10/04 08:00:29*
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 08:00:32
Unexpected nil value
     Channel_SetSessionKeys()   -/-
     Channel.c:80                preregistrationd[222]               10/04 08:00:32*
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]               10/04 08:00:32*
Got non-00 ANI '*078133483716#*', returning error
     GetPeerANI()                -/81c6180010b00231
     PreRegistration.c:1725     preregistrationd[222]               10/04 08:00:32
Sending manual ANI request page
     AskUserForANI()             -/81c6180010b00231
     PreRegistration.c:704      preregistrationd[222]               10/04 08:00:32
(813-348-3716 is in Tampa, FL; user manually entered 813-348-3716
```

Bad ANI can take a number of forms. Most commonly, the ANI received will not begin with the expected *00 and instead will begin with *07 or *06. The precise differences between non-*00 entries aren't well known, but they might come from switchboards, to indicate that calling this number will not connect the caller directly to the person who originated the call but that an intermediary is involved. At present, the ANI server will not recognize as valid any ANI that *does not* have *00 as its first three digits.

The ANI may also have the incorrect number of digits, causing the number to be thrown out by the ANI server.

It makes sense to be conservative in the acceptance of ANI information because an error at this point will often cause a nonlocal number to be assigned as the primary POP, even though a local POP is available. This misassignment can

produce large, unnecessary phone bills for the customer or may regularly cause fallover to the backup tollfree service. Both of these errors are costly and thus to be avoided.

Internal user, probably using “xyzzy”

Example B-51 User inputting xyzzy.

```
NOTE: IP address 0xccfe4a0e does not map to any ANI server
       Not doing ANI lookup from server
     GetPeerANI()                -/JohnMatheny
     PreRegistration.c:1639    preregistrationd[222]               10/04 09:50:12*
Sending manual ANI request page
     AskUserForANI()             -/JohnMatheny
     PreRegistration.c:704     preregistrationd[222]               10/04 09:50:12
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 09:50:13
Unexpected nil value
     Channel_SetSessionKeys()  -/-
     Channel.c:80                preregistrationd[222]               10/04 09:50:13*
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]               10/04 09:50:13*
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 09:50:13
Unexpected nil value
     Channel_SetSessionKeys()  -/-
     Channel.c:80                preregistrationd[222]               10/04 09:50:13*
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]               10/04 09:50:13*
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 09:50:27
Unexpected nil value
     Channel_SetSessionKeys()  -/-
     Channel.c:80                preregistrationd[222]               10/04 09:50:27*
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]               10/04 09:50:27*
Making fake ticket
     AddFakeTicket()             -/-
     Services.c:340              preregistrationd[222]               10/04 09:50:33
Unexpected nil value
     Channel_SetSessionKeys()  -/-
     Channel.c:80                preregistrationd[222]               10/04 09:50:33*
```

```
Unexpected nil value
     ServiceRequest_Authorize()-/-
     Services.c:454              preregistrationd[222]              10/04 09:50:33*
WARNING: ANI failed (000000000000), using artemis/18006138199
     SendTellyscript()          -/JohnMatheny
     PreRegistration.c:2324     preregistrationd[222]              10/04 09:50:33*
```

Note how the log in Example B-51 corresponds closely to the error listing for "No Matching ANI Server" (see page 180). The difference is that when prompted for their phone number, users can input "xyzzy," which will allow them to call in on the tollfree number. It is, in effect, like a back door into the fallover tollfree service. Other than an internal access via "xyzzy," the only other way to get here is by an Internet terminal with a corrupt section of flash ROM, the section with the code in it. In this case, no user interface is presented, so the client isn't able to post the manual ANI screen. Thus, the only time users will get the toll-free service is if their flash ROM became corrupted *and* the service failed to get their ANI.

Dialup on local 415-329-0511 or equivalent

Example B-52 ANI out of acceptable range.

```
ERROR: request for ANI port 27; min is 1, max is 24
     GetPeerANI()                -/81520c0010b0023f
     PreRegistration.c:1647     preregistrationd[222]              10/03 00:12:31*
Sending manual ANI request page
     AskUserForANI()             -/81520c0010b0023f
     PreRegistration.c:704      preregistrationd[222]              10/03 00:12:31
```

Foreign customers have to call in to the local 415 number because the normal 800 preregistration number isn't valid out of the United States. Since ANI is provided only on the tollfree lines, the ANI server will necessarily fail to find it and prompt users to input their phone number. The important failure mode to note here is that the client requested an ANI port out of the acceptable range (27, when 1–24 was expected). Specifically, the call came in on the 415 number instead of the tollfree number, inasmuch as ports 1–24 are reserved for the tollfree preregistration modem banks.

ANI server failing

Example B-53 ANI server failure.

```
Ignoring ANI server junk ('866*004083289335#*')
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1720     preregistrationd[222]               10/03 15:55:26
ERROR: got EOF from ANI server (errno=145, eof=0, err=1), closing connection
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1695     preregistrationd[222]               10/03 15:55:34*
Retrying ANI server connection
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1699     preregistrationd[222]               10/03 15:55:34
Connection to ANI server ani-2:2000 opened, took 0 seconds
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1669     preregistrationd[222]               10/03 15:55:34
Ignoring ANI server junk ('220 ani-server 1.1(UX) on ani2.webtv.net at Thu Oct
3 15:50:39 1996')
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1720     preregistrationd[222]               10/03 15:55:34
Ignoring ANI server junk ('')
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1720     preregistrationd[222]               10/03 15:55:34
Ignoring ANI server junk ('866*004083289335#*')
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1720     preregistrationd[222]               10/03 15:55:34
ERROR: got EOF from ANI server (errno=145, eof=0, err=1), closing connection
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1695     preregistrationd[222]               10/03 15:55:42*
Unexpected EOF
     GetPeerANI()                -/813d180000b002f1
     PreRegistration.c:1703     preregistrationd[222]               10/03 15:55:42*
Sending manual ANI request page
     AskUserForANI()            -/813d180000b002f1
     PreRegistration.c:704      preregistrationd[222]               10/03 15:55:42
```

The ANI server will try once, fail, and then try once more before it prompts the user for manual ANI. In this case, the raw ANI data had digits before the expected *0 in the ANI, hence the reference to “ignoring server junk.” When the server fails to find a line in the ANI that begins with the *0, it reads to the end of available input and finds an unexpected end-of-file. When this happens twice, the server prompts the user to input the phone number manually.

Missing Tellyscript Fragment

Example B-54 Missing tellyscript fragment.

```
TS: Unable to get mod time on '/usr/home/production/webtv/content/tellyscript/
midwest.tsf': No such file or directory
     GetScriptModWhen()         1671069/01fad82501b002ba
     ScriptCache.c:174          preregistrationd[222]               10/02 00:13:55*
No such file or directory
     LoadIntoCache()             1671069/01fad82501b002ba
     ScriptCache.c:193          preregistrationd[222]               10/02 00:13:55*
No such file or directory
     ScriptCache_GetByName()    1671069/01fad82501b002ba
     ScriptCache.c:90            preregistrationd[222]               10/02 00:13:55*
TS: Failed on 'midwest': No such file or directory
     ScriptCache_GetByName()    1671069/01fad82501b002ba
     ScriptCache.c:138          preregistrationd[222]               10/02 00:13:55*
No such file or directory
     TellyScript_ComputeSimple() 1671069/01fad82501b002ba
     ScriptGen.c:309             preregistrationd[222]               10/02 00:13:55*
No such file or directory
     HandlePOPTour()             1671069/01fad82501b002ba
     PreRegistration.c:1296     preregistrationd[222]               10/02 00:13:55*
No such file or directory
     Handler()                    1671069/01fad82501b002ba
     PreRegistration.c:357      preregistrationd[222]               10/02 00:13:55*
No such file or directory
     HandleGetOrPost()          1671069/01fad82501b002ba
     Services.c:808              preregistrationd[222]               10/02 00:13:55*
```

In Example B-54, the server is looking for a tellyscript pertaining to a provider the service doesn't use. Hence, it can't find the files or directories it's looking for. This is only a problem when (instead of Midwest) the carrier is one we *do have*, such as CNC, UUNET, BBN, etc. Then, it's time to worry, as some missing files need to be replaced.

Client Problems

Example B-55 Client disconnect during preregistration.

```
Unexpected EOF
     ReadMimeHeaders()          -/-
     MimeHeaders.c:171          preregistrationd[222]               10/02 22:29:44*
Error reading headers from client: Unexpected EOF
     HandleCommands()            -/-
     Protocol.c:203              preregistrationd[222]               10/02 22:29:44*
```

```
Implementation error
     Event_AddString()           -/-
     UserEvents.c:415            preregistrationd[222]               10/02 22:29:44*
```

The problem in Example B-55 results from a single client disconnecting from the service during the preregistration process. These errors aren't worrisome as single entities, but when they start showing up in volume, they signify a greater problem. Multiple instances from a single user would point at either a bad PPP interface in the client, or a provider error (UUNET, CNC, etc.). As a rule of thumb, it's a good idea to track these down further when they show up in groups because they can also indicate a bad USR modem unit or a more general problem with a provider.

POP Tours

Example B-56 Pop tours 1.

```
POP tour 01abd92511b0026e succeeded (276 cnc/415-233-0570)  IP=206.173.223.34
     HandlePOPTour()             1684201/01abd92511b0026e
     PreRegistration.c:1151     preregistrationd[222]               10/04 13:25:55
Replacing 'last' with fixed-1 (274)
     HandlePOPTour()             1684201/01abd92511b0026e
     PreRegistration.c:1186     preregistrationd[222]               10/04 13:25:55
Gave services to ID 01abd92511b0026e, group (nil):
     reset
     name=wtv-preregister host=207.76.180.96 port=1615
     name=wtv-preregister host=207.76.180.97 port=1615
     AddServiceHeadersToDictionary()      1684201/01abd92511b0026e
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:25:55
Gave services to ID 01abd92511b0026e, group (nil):
     name=wtv-head-waiter host=207.76.180.61 port=1601 flags=0x00000001 connections=1
     name=wtv-head-waiter host=207.76.180.62 port=1601 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      1684201/01abd92511b0026e
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:25:55
Gave services to ID 01abd92511b0026e, group (nil):
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     name=wtv-* host=206.67.152.64 port=1603 flags=0x00000003
     AddServiceHeadersToDictionary()      1684201/01abd92511b0026e
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:25:55
Gave services to ID 01abd92511b0026e, group (nil):
     name=wtv-log host=207.76.180.71 port=1609 flags=0x00000001 connections=1
     name=wtv-log host=207.76.180.72 port=1609 flags=0x00000001 connections=1
     AddServiceHeadersToDictionary()      1684201/01abd92511b0026e
     ServiceConfig.c:519        preregistrationd[222]               10/04 13:25:55
TS: Hashed 'cnc:2:14152330570|cnc:2:14152330570|artemis:1:18006138199' into
0x17c1a4d1
     TellyScript_ComputeSimple() 1684201/01abd92511b0026e
     ScriptGen.c:330             preregistrationd[222]               10/04 13:25:55
```

```
TS: DEBUG: total generation time now 142225/1837 (77 msec avg)
     TellyScript_Generate()     1684201/01abd92511b0026e
     ScriptGen.c:420              preregistrationd[222]               10/04 13:25:56
POP tour 01abd92511b0026e now trying (276 cnc/415-233-0570)
     HandlePOPTour()              1684201/01abd92511b0026e
     PreRegistration.c:1304     preregistrationd[222]               10/04 13:25:56
```

POP tours are a type of diagnostic whereby an Internet terminal is programmed to call and verify the functionality of a large number of our available POPs. This option is available from the wtv-tricks menu (see"WebTV Tricks Service" on page 118). POP tours are a common activity that tends to be going on irregularly behind the scenes. They are generally distinguishable from other entries by the presence of the words **POP Tour** in the first entry and in entries where the POP tour switches POPs. Notice the general similarity with a successful preregistration sequence in the entries where the server is "giving services" to the client as well as verifying the new tellyscript in the TS: Hashed entry.

Example B-57 Pop tours 2.

```
Array end
     PhoneDB_GetPopByIndex()    1671069/01fad82501b002ba
     PhoneDB.c:765                preregistrationd[222]               10/02 14:17:17*
POP tour for 01fad82501b002ba complete
     HandlePOPTour()              1671069/01fad82501b002ba
     PreRegistration.c:1197     preregistrationd[222]               10/02 14:17:17
```

The entries in Example B-57 indicate the successful end of a POP tour. This is by no means as common as initiation of a POP tour, as POP tours frequently fail before completion.

Proxyd.log

The proxy servers act as links between the client and the Internet. They relay all the page requests, send back the desired URLs, and often modify them so they load faster on the client, enhancing perceived performance. Their other job is to cache frequently accessed pages, eliminating the need to access an external server to process every request. This also speeds up the loading process, again enhancing perceived performance.

Proxyd Log Entries

The logs contain a wealth of information pertaining to all the file transfers a proxy executes. User ID, the URL loaded, duration of each step in the loading process, and details about any rewriting the proxy may have done are all present in the log files. This is by no means all that is recorded, but it gives an idea of the types of information available. The logs can be found on the various proxy machines at:

`~production/webtv/log/proxyd.log`

In addition to data on normal (successful) transfers, any unexplained errors the system encounters are also logged. The information in the error messages is similar to that in the other WebTV service logs, in both format and content; the format is shown in Example B-58.

Example B-58 Proxyd log entry.

```
<error-message>: <error-type>
<reporting-function>() <client-id>/<subscriber-id>
<reporting-file>:<line-number> <service>[<pid>] <date> <time><flag>
```

Note: This format is not a hard-and-fast rule, but merely a guideline. Many log entries have additional information, sometimes a large amount of it, between the first and second lines of the normal three-line log entry. Nevertheless, these same three lines appear in nearly every log entry—they just may not appear consecutively or alone.

Typical user-events.log Entries

Below are two typical log entries from the `user-events.log` on a proxy machine (one through an external server, the other through the internal cache), and a listing of what each element means. The file `user-events.log` is in the directory `~production/webtv/event_log`. Proxyd.log itself tends to log error messages rather than routine transactions. Examples of some of these errors are in the next section.

Note: Most log entries will not have all the possible fields. Also, note the salient points on the first line of each example, detailing which machine, the type of transfer (in this case, a typical URL via http), and the date/time stamp.

Example B-59 User-event logs.

```
proxyd@stirrup                http                    10/07/96 15:05:37 -0700
 user-id=102003  pid=6630  ip=204.254.74.224
 serial-number=JayLogue visit-url=http://www.nbc.com/sports/index.html
 cache-miss-reason=NotFound
 connect-duration=7438 response-duration=1083590
 orig-content-length=3155
 html-rewritten=true rewrite-duration=88509 image-tags=15
 image-dims-missing=5 image-dims-added=0 caching=true
 content-type=text/html relay-duration=2011 content-len=3152
 bytes-from-server=3316 bytes-served=3251 bytes-received=738
 duration=1.464146
```

```
proxyd@stirrup                    http                        10/07/96 15:08:39 -0700
 user-id=102003  pid=6630  ip=204.254.74.224
 serial-number=JayLogue visit-url=http://www.nbc.com/sports/index.html
 orig-content-length=3152
 html-rewritten=true rewrite-duration=35680 image-tags=15
 image-dims-missing=5 image-dims-added=5 content-type=text/html
 relay-duration=1591 content-len=3249 cache-served=true
 bytes-served=3348 bytes-received=738 duration=0.152238
```

Descriptions of the fields of a typical log entry

audo-redirects
The number of auto-redirections performed by the proxy. This field is only present when its value is greater that zero.

bytes-from-server
The total number of bytes received from the remote server. Presence implies that the response was served from the Internet.

bytes-received
The total number of bytes received from the client.

bytes-served
The total number of bytes sent to the client. Value is approximately equal to the content length plus the length of response headers.

cache-miss-reason
The reason why the response was not served from cache. Implies that the response was either served from the Internet or was an immediately returned error.

cache-served
An indication that the response was served from the page cache (i.e., without going to a remote server). The only possible value is “true”; absence indicates that the response either originated from the source server or was a locally generated error.

caching
An indication that the response was stored in the proxy's page cache. The only possible value is “true”; absence indicates that the response was *not* cached.

connection-duration
The amount of time taken to lookup the server IP address in DNS and establish a connection. The field is only present if the request was served from the net. Values are in microseconds.

content-len
The length of the response content as returned to the client (i.e., after any performed transformations). Value does not include the length of the response headers. A value of -1 indicates that the length of the response content was unknown at the time it was sent.

content-type
The type of the response content, as returned to the client (i.e., after any performed transformations).

duration
The amount of time taken to service the response, expressed in seconds. The duration is measured from the time the request arrives at the proxy to the time the last byte of the response is pushed into the kernel. Note that because of network delays and buffering, the value is an inaccurate representation of client-observed latency.

html-rewritten
An indication that HTML rewriting was performed. The only possible value is “true”; absence of the field implies that HTML rewriting was not performed.

image-dims-added
The number of <IMG> tags for which size hints were added by the rewriting process.

image-dims-missing
The number of <IMG> tags without size hints.

image-tags
The total number of <IMG> tags in the response. Only present if HTML rewriting is performed.

ip
The client’s IP address.

not-cached-reason
The reasons why the response was not stored in the page cache; multiple reasons listed are comma separated. The presence of this field implies that the response originated from the net. (see “Proxy 'Not Cached' reasons” on page 194)

orig-content-length
The original length of the response content prior to transformation (i.e., HTML rewriting or image transcoding). The field is only present if the response was transformed.

orig-content-type
The type of the response content prior to transformation. The field is only present for image responses in cases where the image is transformed.

pid
The process ID of the proxy child process that serviced the request.

referer
The URL of page from which the visit-url was selected. Sometimes incorrect because of a bug in early versions client.

relay-duration
The amount of time taken to push the response back to the client. Inaccurate for small responses due to buffering in the network. Value is in microseconds.

response-duration
The amount of time taken to forward the request to the remote server and receive its response. Only present if the request was served from the Internet. (Value in microseconds).

responseCode
The HTTP response code. The field is only present if response is an error (i.e., other than "200 Ok." see "Proxy response codes and messages" on page 193).

rewrite-duration
The amount of time spent rewriting HTML. Value is in microseconds.

serial-number
The Internet terminal (client) serial number.

transcoding
Indicates the result of attempting transcoding on an image response. Possible values are:

performed—the image was transcoded.

unnecessary—no transcoding of the image was necessary.

inappropriate —transcoding the image would have produced unacceptable results (e.g., growth in the image size beyond a certain threshold).

unsupported—the image's type and/or encoding were unsupported by the proxy's transcoding mechanism.

user-id
User's id.

visit-url
The URL that was the target of the request.

Things to note when analyzing a log entry

Perhaps the simplest element we can draw from an entry is whether the page requested was in the cache, has been put in the cache now, or has not been cached. This information can be found by looking first at the *cache-miss-reason*. If it's there, the page wasn't in the internal cache to begin with. If the field `caching` is true, it means that the proxy has now cached the page (or image). If the system failed to cache the data, the reason will be presented in the field `not-cached-reason`. (See end of this section.)

Another interesting thing we can draw from these log entries is whether the external server is being slow. Time-outs can often be a source of frustration for the customer, and by analyzing the `connect-duration`, `response-duration`, and `duration` fields, we can at least begin to determine whether the external server is answering, answered slowly, or transmitted slowly.

By looking at the transcoding and rewriting types of fields, we can determine what alterations, if any, the proxy made to the page or image before (or after) it transmitted it to the client.

The `response-code` field is only present when there is an error, and the precise value of this field gives a good indication of what went wrong and where the error occurred.

Typical Error Examples

Here are some examples of typical error logs. Notice how identification of a pid (process-id) with a date-time stamp can show you which entries in a log belong to the same event and are a product of the error percolating up the stack.

Example B-60 Typical error logs.

```
OS BUG: select() cried wolf!: Resource temporarily unavailable
     TimeoutRead()               1794342/8145410010b002e4
     TimeoutSocket.c:170        proxyd[26321]                        10/07 14:58:49*

Or:
Unable to resolve host www.jerkyboys.com: DNS lookup timed out
     GetHostByName()             1867152/817f050010b00230
     DNS.c:33                    proxyd[26318]                        10/07 15:12:51*
Unable to connect to server: DNS lookup timed out
     ConnectToServer()           1867152/817f050010b00230
     ServeNet.c:117              proxyd[26318]                        10/07 15:12:51*


DNS lookup timed out
     Proxy_ServeFromNet()       1867152/817f050010b00230
     ServeNet.c:729              proxyd[26318]                        10/07 15:12:51*
DNS lookup timed out
     Proxy_ServeReq()            1867152/817f050010b00230
     Serve.c:209                 proxyd[26318]                        10/07 15:12:51*

Or:
ReadGIFHeader failed: Invalid parameter
     GetImageStats()             1877142/8195090010b002f7
     Transcode.c:815             proxyd[26317]                        10/07 14:33:16*
Invalid parameter
     TranscodeImage()            1877142/8195090010b002f7
     Transcode.c:424             proxyd[26317]                        10/07 14:33:16*
Transcoding failed: Invalid parameter
     DoImageTranscode()          1877142/8195090010b002f7
     Transform.c:311             proxyd[26317]                        10/07 14:33:16*

Or:
Protocol error
     AcceptConnection()          -/-
     TCP_Server.c:525            proxyd[6148]                         10/06 19:39:28*
AcceptConnection() failed: Protocol error
     ReadyList_HandleControlAndAccept()         -/-
     TCP_Server.c:300            proxyd[6148]                         10/06 19:39:28*

Or:
Ignoring malformed header (no colon): Error. Unable to launch ACGI application.
     ParseMimeHeadersLengthAndMerge()       1875876/81cd480010b0025c
     MimeHeaders.c:85            proxyd[6154]                          10/06 15:34:41
```

Note that the "Error. Unable to launch ACGI application" is not an internally generated descriptor, but rather a chunk of the page. This can be useful to note for diagnostic purposes.

Proxy response codes and messages

Error codes with only three digits are messages from the external server (i.e., the site from which the proxy is trying to get a URL). Response codes with four digits are refinements offered by the proxy to give a more complete explanation of the error.

Bad Request Errors (x400)

400 Bad Request—No message.

1400 Host Missing—"The requested address is missing a host name."

2400 Scheme Missing—"The requested address is missing the first part, like http:."

3400 Unsupported Scheme—"WebTV is not capable of going to the requested page."

Forbidden Errors (x403)

403 Forbidden—"The publisher refuses to give you access to the requested page."

1403 Content Blocked—"Access to the requested page is restricted."

Not Found Errors (x404)

404 Not Found—"The publisher <*publisher*> couldn't find the requested page."

1404 Host Unknown—"The publisher <*publisher*> is unknown."

2404 Inaccessible—"The publisher <*publisher*> couldn't be reached."

3404 Communication Problem—"A problem came up while communicating with the publisher of the requested page."

4404 Unexpected Response—"The publisher <*publisher*> responded in an odd way."

5404 Response too long—"The item chosen contains so much information that WebTV can't use it."

Internal Server Errors (x500)

500 Internal Server Error—"The publisher <*publisher*> ran into a technical problem. Please try again."

1500 Proxy Inaccessible—"WebTV technical problem: the proxy <*publisher*> couldn't be reached."

2500 Proxy Internal Failure—"WebTV ran into a technical problem. Please try again."

Service Unavailable Errors (x503)

503 Service Unavailable—"The publisher <*publisher*> ran into a technical problem. Please try again."

1503 Inaccessible or Too Busy—"Proxy Replay Error: Response not found in cache."

2503 Captured Response Not Found—"Proxy Replay Error: Response not found in cache."

3503 Busy—"The publisher <*publisher*> is too busy to use right now."

Proxy 'Not Cached' reasons

NotGet
The request was not an HTTP get.

Cookie
The request contained a Cookie header.

SetCookie
The response contained a Set-Cookie header.

Auth
The request contained an Authorization header.

Error
The response was an uncacheable error (i.e., not 200, 301 or 302).

NoExipresInfo
The response contained neither an Expires header nor a Last-Modified header.

Expires0
The response contained an Expires: 0 header.

Expired
The response has already expired.

LifeTooSmall
The lifetime of the response (as determined by Expires or Last-Modified headers) is below the configured threshold.

Collision
The attempt to cache the response collided with a similar attempt by another proxy process.

Appendix C

Choosing the Right POP

There's an uncomfortably wide variety of supported dialing patterns and places which may have zero, one, or several local POPs. The object here is to figure out the appropriate way of handling every case, both to minimize the customer's costs, and achieve an effective load balancing, which minimizes our costs as well.

This appendix describes the details involved in selecting the correct POP, including script generation, dialing, and load balancing. The following topics are covered:

- Dialing Patterns
- Choosing the Right POP to Dial
- Learning What to Do
- Remembering What to Do
- Script Descriptions
- Deciding if the Client Needs a New Script
- Access Number
- Panic Mode
- Interactions Between Dialing Options
- 1-800 Addicts
- Customer Care Issues
- Flat Rate and Interleaved Providers
- Load Balancing

Dialing Patterns

There are three ways to dial any particular number: 7-digit, 10-digit, and 11-digit. Which the client needs to use is based on whether the POP is in the same area code and whether it's a local call. There are four possible combinations, CCMI calls them: HL (home area code, local call), HT (home area, toll call), FL (foreign area code, local call) and FT (foreign area code, toll call).

HL and FT are easy; everybody supports 7-digit for the former and requires 11-digit for the latter. FL is almost always 11-digit, but according to the CCMI tables some places **require** 7-digit dialing (which is odd, and at times difficult to deal with). HT varies the most.

The CCMI tables aren't always right, both because of inaccuracies in the databases (not commonly observed, no demonstrated instances of it as yet, although area code splits would cause problems until the database is updated) and inconsistencies in the phone system (try calling MCI's POP at 1-415-482-2900... it's the only 415 number we know of that **requires** you to use only 7 digits from the 415 area code).

So, for every POP the client dials, it may have to try more than dialing pattern. We can maintain a list of which patterns work in the server, and reconstruct the tellyscripts so it only dials the numbers that work.

Choosing the Right POP to Dial

Choosing the numbers to include in the script is easy. `rgenphonedb` (see “rgenphonedb” on page 102) originally found the two cheapest calls for each exchange, identifying them as local or toll and calculating an estimate of cost and distance. We now look at more than two POPs for each exchange, as this becomes necessary for load balancing, (“Load Balancing” on page 202) as more providers are added.

Choosing which of the two numbers the client should dial first is a little harder. Some of the POPs will be more desirable than others, either because they're cheaper for the customer or they're cheaper for us. This may depend on the time of day, and particularly, the type of service they offer (flat rate charge to us, for example).

We want to re-use the last phone number we called, because it works, but only if it's the cheapest POP. There's not much value in saying "only retry it if N minutes have passed", because most of the failures seen with CNC have been transient anyway.

If all of the local POPs fail, or if there simply aren't any local POPs, the customer may have to be sent to an expensive (for them) POP. This should be accompanied by an alert message telling them that they're about to dial a non-local number. (I'd like to include the number to be dialed, so they can determine for themselves if it's prohibitively expensive.) The alert will only appear the first time they try that number during a particular connection, i.e. if they try 2330570 and then 4152330570, they'll only see a message when they try the first pattern.

It may be desirable to retry the same number if it fails the first time. This is especially useful with CNC, which occasionally doesn't answer the phone or rejects the PAP password. If both numbers are local, they can use round-robin, but if one is toll they should hit the local one a couple of times before moving on.

We have a tollfree number to fail over to in the event that all IAPs fail. This will is a separate bank from the preregistration modems, which will be on a somewhat isolated network.

Learning What to Do

The only way to get the dialing patterns right is to have the client try different patterns until one works. Storing negative information isn't as useful, since we can't know that the failure occurred because the dialing pattern was wrong.

Once we know what to do, we want to generate tellyscripts that only have the correct patterns in them. This means that we will want to rewrite the scripts after every time we learn something new.

The set of working patterns shouldn't change often. There are only 2 logical reasons that a dialing pattern might change. The first would be a split in an area code, and the other would be a new calling plan, although it isn't clear that a customer's calling plan should have any affect on the dialing pattern. We should be able to detect such changes statistically (lots of ring-no-answers for POPs that work fine for other users), but the customer will probably become rather annoyed if the secondary is a toll call, and the secondary is the only one that ever works.

An astute customer will turn audible dialing on (or call customer service who will tell them to turn audible dialing on) and hear the operator yelling at him when the pattern shifts, so even if we don't find out about it from harvesting connection data, we're still likely to find out from the customer service side.

Remembering What to Do

We can't use any persistent storage on the client itself, because there isn't any. Instead, the information about which numbers work and which don't will be stored in the server, and communicated to the client by sending down an updated tellyscript. The data is sent up in the connection logs to the logserver (see Appendix B, "The Service Log Files").

Having the data collected by the logserver isn't so great, since the servers that vend tellyscripts are preregistrationd and headwaiterd. The data will have to be stored in the "mini" database currently used to hold the ANI and shared secret information from preregistrationd.

There are two general approaches to collecting connection logs:

1. Have something other than logserverd collect the connection logs. This is less useful than it sounds, since connection logs are asynchronous: the server advertises the wtv-phone-log-url, and the client responds by POSTing the logs. However, we've seen the client instantly close the logserver connection during the initial login because the homed wants to preload the world (note the relatively high proportion of "accept() failed: protocol error" error messages in the logserverd.log), so I'm not sure if we can expect to do a request-->reply for the phone logs. Besides which, the client doesn't say anything if it has nothing to say, so if we wait for the logs we might be waiting for a while. Might as well just have the logserver do it.

Let the logserver vend tellyscripts. Since this will all be embedded in a library, it's not substantially more difficult to do things this way. It does, however, mean that both headwaiterd and logserverd could give the client a new tellyscript while the client is logging in. If we assume that the client will almost always post a

connection log, we could send the headwaiter tellyscript with a "don't redial immediately", and the logserverd tellyscript with "do redial immediately", so that the client would only redial once during a login.

2. Let the logserver collect the connection logs, but don't bother sending down a new tellyscript. Less than ideal, but it'll work, and it avoids race conditions.

The user interface ramifications of redialing at a semi-random point in the login process aren't pleasant. Bruce has suggested that we neither redial nor notify the user that a new script has been sent down. This is the easiest approach, and should be acceptable in most cases, but seems suboptimal from a customer happiness perspective if their local POP has died and the only way to get a new script is to make a toll call.

They'll have to be told this, via an expensive customer service call, but it seems there's no easy way around this.

Sometime soon the headwaiter should force an immediate redial if the customer comes in on our 800 number and has an ANI different from what's in the database. This is meant to reduce modem congestion.

We're planning to store 8 sets of numbers, in LRU order. The mini database is organized by box serial number, not caller ANI, so we need to specify the number that the person was calling from when the call worked. If they alternate between a couple of different numbers, we'll retain the dialing pattern knowledge for both. Each element will look like:

```
number calling FROM (i.e. ANI)
POP number called (full 10-digit form, for compares)
dialing pattern used (7, 10, 11)
```

An entry will be added each time a successful connection is made to a previously unknown {ANI, POP} combination. If a previous entry matches {ANI, POP}, the dialing pattern will be replaced if different. If the new knowledge means the box needs a different tellyscript, the new script will be sent down immediately, but the client won't be told to redial.

Both preregistrationd and headwaiterd will use this data when a client needs a new script.

Deciding if the Client Needs a New Script

Two 32-bit values are embedded in the header of a tellyscript, and are sent up to every server. The first is the time in GMT when the script was generated, which allows us to tell at a glance if the script is an out of date, buggy version. The second is a 32-bit hash of:

- the names and version numbers of every script component included in the generated script
- every phone number that the client could potentially dial, in the order that they'd be tried

When the client connects, we figure out which components and phone numbers it should dial, but don't actually generate a script. We compare the hash of what we think it should have against the hash that it does have. If they differ, we generate a new script and send it down.

We don't actually know what POPs are currently in use, but we can figure that out by looking at the phone numbers in the connection logs and the caller's ANI. The string from which the hash value is computed will be stored in the mini database in case we want to do something to the set of users who dial a particular POP, or we want to see what numbers a user is dialing.

Script Descriptions

These are pseudo-code outlines of how each script will look. In every case we will put up an alert before the first time we call a non-local number. Since we have no NV storage on the box, this will be displayed every time the box is powered on as a reminder.

In every script, we bail out when we connect successfully or if we are unable to detect dialtone before dialing. We use an "explorer" script if either or both of the numbers lacks an established dialing pattern.

If we only have one POP:

try number

If we have a secondary dial pattern, try it

retry number

call 1-800

If both POPs have the same cost (i.e. both are local or both are toll with the same estimated cost, and both will cost US the same amount):

try pop#1

try pop#2

retry pop#1, using secondary dial pattern if it exists

retry pop#2, using secondary dial pattern if it exists

call 1-800

If one POP is more expensive than the other (which is expected to be the more common case):

try pop#1

retry pop#1, using secondary dial pattern if it exists

try pop#2

If we have a secondary dial pattern for pop#2, try it

call 1-800

In no case do we try more than 5 numbers, and we don't try the more expensive number more than once unless we're trying to figure out what the correct dialing pattern is.

Some test Pop Tours (see “POP Tours” on page 186) we ran with retries suggest that retrying the POP immediately after a failure will greatly increase the number of successes.

Access Number

One of the things you can do from the "dialing options" screen is enter an access number that overrides whatever else the client might think to dial. This is supremely useful if for some reason we can't use the 1-800 number burned into the mask ROM, but also comes in handy when visiting foreign countries.

The tellyscript has no way of knowing what IAP is associated with the access number, so it will be assumed that the person enters a manual ANI value that gets him the IAP of choice. The generated script will invoke the chat function for the primary IAP, i.e. the one listed first in the script. The access number overrides the usual calling pattern stuff.

Panic Mode

A different sort of situation arises if a customer calls in from an invalid number. This could be because an exchange is brand new and hasn't been entered into the CCMI database, because of a gap in the CCMI database, or because the user entered the number manually and simply erred.

In these cases, the customer is assigned to the “global default” POP, which will be our tollfree POP. An “unknown-exchange” tag is added to the user-event log “Typical user-events.log Entries” on page 188) so we can track them.

Interactions Between Dialing Options

The way that some of the dialing options behave under certain circumstances isn't immediately obvious. This should clear up a few things.

The "remove leading 1" setting available on client 1.1 or later doesn't affect access numbers.

Turning on "audible dialing" on client 1.1 or later also enables "visible dialing", which shows the phone number to the user as the phone is dialed. The strings change in certain situations:

audible dialing off, all cases:

"Dialing WebTV"

audible dialing on, normal case:

"Dialing 14156145539"

audible dialing on, normal case, with a prefix of "9":

"Dialing 9,14156145539"

audible dialing on, dialing a toll-free POP (e.g. the fallover)

"Dialing WebTV..."

audible dialing on, access number used:

"Dialing access number"

Toll-free POPs have numbers starting with “1800” or “1888”. (Yes, it's checked before the “remove leading 1” part.) If someone puts in an override with clientpopedit (“PhoneDB Tools” on page 92) that starts with “1-800” instead of “1800”, the user is going to be able to see the number. Appropriately nasty warnings have been added to clientpopedit.

The interaction of the "prefix" and "LD prefix" (the latter of which is found under "spooky dialing options" on 1.1 or later clients) works like this:

prefix=(none), LD prefix=(none)

local=6145539 long=18005551212

prefix=9, LD prefix=(none)

local=9,6145539 long=9,18005551212

prefix=(none), LD prefix=8

local=6145539 long=8,18005551212

prefix=9, LD prefix=8

local=9,6145539 long=8,18005551212

The determination of "local" or "long" is made by the tellyscript generator. The fallback number is always considered long-distance, as are numbers entered with Vend-A-Telly or clientpopedit (if you're using the latter two methods, you shouldn't need a long-distance prefix anyway). Ditto for tellyscripts handed out when "IgnoreANI" is set in the config file (e.g. all development servers).

Local calls and ExLocal calls use the local prefix, Toll calls use the long prefix if it exists or the local prefix if it doesn't. Perversely, Local calls don't get the "this may be a toll call" dialog, but both ExLocal and Toll calls do. The Vend-A-Telly and fallback numbers are considered long-distance, but don't get the dialog either. (The code gets real interesting in spots.)

1-800 Addicts

Sooner or later, someone will discover that, if he disrupts the dialing process while it's calling the assigned POPs, he can get toll-free calling. This is bad because it costs the company money and ties up a line on the tollfree fallover modem rack.

The only way to deal with this is identification of repeat offenders, attempts to “help” them with their dialing problem, and eventually removal of the tollfree fallback from their tellyscripts. This should be under the control of Customer Care.

Customer Care Issues

If a POP dies, it may be useful to disable it for short periods of time. This wouldn't be used for temporary outages unless the failure mode prevents the customer from ever getting into the service (e.g. the modem answers, PPP is negotiated, but all outbound packets fall into a black hole... something we've seen CNC do).

It may be desirable for customer service to be able to modify the dialing patterns held in the database, though in practice they shouldn't have to.

We may also want to have per-user POP overrides in the database. The mini-db is probably a good place to put these. The overrides would only specify the POP; the dialing patterns would be entered in the usual way.

Flat Rate and Interleaved Providers

We don't want a CNC or UUNET outage to cause a major denial of service for WebTV customers, so it's in our best interests to interleave providers whenever possible. If a customer has two local UUNET POPs and one local CNC POP, all of which cost the customer the same amount, we want to give them one from each provider.

We don't want to change the primary POP, which was chosen with things like how much the provider charges us in mind, but we may be able to swap the alternate POP with one from a different provider that costs the customer the same amount.

When doing this we need to take into account that some providers charge us a fixed amount per month, whether the customer shows up once for 5 minutes or 50 times a day every day. We don't want to use these POPs as secondary numbers because we'll get charged for a full month if the primary provider fails just once. Flat-rate POPs that end up in the secondary slot will be moved into the primary when possible, or into the 3rd or 4th slot if not.

There are some rough edges to this. It's not the end of the world if we can't find a good secondary POP from a different provider, but it could get downright expensive for us if we use a flat-rate POP as a secondary. If the 1st and 2nd POPs are both flat-rate, we have to swap the 2nd with the 3rd, even if the 3rd is more expensive. This is the only place in the POP selection algorithm where a more expensive POP could be chosen over a less expensive one.

Another situation is when the 1st POP has an hourly or per-port charge and the second POP is flat-rate, but is a more expensive call. We could swap the 1st and 2nd to save ourselves money, but the customer would end up paying more, which seems a poor strategy.

Load Balancing

We can't send all traffic to one provider, because of business arrangements and provider capacity. Dealing with this requires load-balancing the POP assignments. The initial implementation is a little crude and is in two parts.

The first part is in the PhoneDB file. POPs that are local to the customer and cost the same as the primary are marked as **can be primary**. POPs from flat-rate IAPs are marked as **can't be secondary**. These flags define the parameters that the load balancing can work inside. Any new arrangement of POPs must have a **can be primary** in the 1st slot and must not have a **can't be secondary** in the 2nd slot.

The second part is in the tellyscript generator. Given a set of load balancing parameters and the data in the PhoneDB file, it figures out whether the default POP assignment is acceptable, and if not, tries to rearrange the POPs into a more desirable order.

This is where things get fuzzy. A set of configuration options determines the load that should be placed on each provider (for the sake of example, assume cnc=3, uunet=1, psi=1). This would try to give 60% of traffic to CNC, 20% to UUNET, and 20% to PSI.

The values are normalized and used to divide up a range of 256, which would come out to 0-152=CNC, 153-204=UUNET, and 205-256=PSI. These are compared against the last byte of the silicon serial number, which is a CRC of the previous digits (it appears to be reasonably well distributed).

If the serial CRC fell within the UUNET range, the tellyscript generator would do everything possible to make UUNET the primary provider. If UUNET wasn't already primary, it would try to find a UUNET POP marked with "can be primary". Once found, the job isn't over, since we need to have a secondary POP as well. If the current primary has **can't be secondary** set, then a third POP with **can be primary** would have to be found and used as the secondary.

For example, suppose there were three IAPs:

- PSI (can be primary, can't be secondary)
- CNC (can be primary)
- UUNET (can be primary)

This would get rearranged to be UUNET, then CNC, then PSI. If, on the other hand, the arrangement was:

- PSI (can be primary, can't be secondary)
- UUNET (can be primary)
- some toll number

This would be impossible to rearrange, because PSI can’t be the secondary, and we don't want to give them a toll number when they have two local ones. (Side note: if UUNET had been preferred over PSI, the default arrangement would still look like the above, because the only alternatives are to use PSI as a secondary or use the first toll number as secondary, neither of which are acceptable.)

Refusing to rearrange POPs like the above could lead to situations where a flat-rate provider receives a much heavier load in a certain area than we'd like. To deal with this, the configuration file allows a "tenacity" setting to be adjusted. The primary can be left alone, moved into the secondary slot, or swapped with a more expensive toll call. This decision applies globally.

A substantial percentage (around 75%) have at least two local POPs, so in many cases it should be possible to do the balancing. While the above scheme may not be ideal, it should address most problems.

Bear in mind that the occasional inability to rearrange POPs is not unique to this scheme. Any algorithm will have to take into account that some POPs can't be primary and some can't be secondary.

As an added bonus, if there's more than one can-be-primary POP from the same provider, the code will choose from them based on the silicon serial CRC byte. This should help balance areas that have two local POPs by sending half the traffic to one and half the traffic to the other. Won't help for CNC-style SuperPOPs though.

Here's a real-life example from “lookuppop” (see “PhoneDB Tools” on page 92):

Phone number 510-799-0000 from HERCULSROD,CA has 4 numbers to try:

In or near OAKLAND,CA (cost=240/1040 [10.1mi]) LOCAL*

psi 8450326

psi 15108450326

In or near OAKLAND,CA (cost=240/1040 [10.1mi]) LOCAL

uunet 7048599

uunet 15107048599

Next-best POPs:

In or near ORINDA,CA (cost=240/1129 [10.8mi]) LOCAL*

psi 2539500

psi 15102539500

In or near CONCORD,CA (cost=420/1780 [13.3mi]) local*

psi 6870679

psi 15106870679

Four POPs were found. The 1st, 2nd, and 3rd say "LOCAL" instead of "local", which means that they can be swapped in with the primary. The 1st, 3rd, and 4th have an asterisk after the word "local", meaning that they can't be put into the secondary position.

This satisifes the POP interleave rules (1st and 2nd provider are from different POPs), the flat-rate rule (2nd provider isn't flat-rate), and the can-be-primary rule (4th POP is more expensive than the other three, therefore isn't eligible).

If the server wanted to use CNC or UUNET as the primary, it would fail, because there's no CNC POP and there's no POP eligible for use as a backup if UUNET were moved into the first position. If the server wanted to use PSI, and the serial checksum was in the top half of the PSI range, the tellyscript generator would use the 3rd POP as primary instead of the 1st.

Giving one provider 100% of the traffic has roughly the same result as building the PhoneDB with that provider favored. It's not quite the same though, since provider interleaving only affects the first two slots. The 3rd and 4th positions in the PhoneDB will be filled by POPs from the preferred provider first, so an area with a glut of local POPs will tend to have more of the preferred provider's POPs available for rotation or balancing. This shouldn't usually matter though.

Glossary

This is an alphabetic glossary of current WebTV terms.

Symbols and Numbers

A

active links

In Remedy applications, active links are events triggered by a user action, tied to a list of tasks that are then performed. For example, a command to show active tickets would be designed using an active link. Unlike *Macros* (see page 206), they are server-side operations.

ANI (Automatic number identification)

A technology for determining the caller's phone number. For more information on ANI, see Andy Rubin's "WebTV Communications Technology" document.

black list

A list of sites that Surfwatch-restricted users should NOT be able to access.

Client

The hardware box or its internal software being used to access the WebTV service.

DNS (Domain Name System)

The on-line distributed database system used to map machine names into IP addresses. DNS servers throughout the Internet use this hierarchical namespace to allow sites freedom in assigning alphanumeric names and numeric addresses to machines.

escalations

In Remedy applications, escalations are events triggered by the passage of time.

event

Events in Remedy are actions that can cause data to change or to move between schemas, trigger alerts, change ticket status, or otherwise cause actions to the Remedy database and associated data. Remedy event types include active links, escalations, and filters, each of which is triggered by a different activity.

fields

In Remedy applications, fields store information, such as integers, date and time, character strings. They can be configured with different lengths and properties.

filters

In Remedy applications, filters are events triggered by data events. For example, when a ticket is changed to closed, some portion of its fields may be logged for future reference. Filters are server-side operations.

firewall

A combination of hardware and software used to separate one or more parts of a network from the rest for security purposes.

HTTP (HyperText Transfer Protocol)

HTTP is an application-level protocol idea for distributed, collaborative, hypermedia information systems.

IAP (Internet Access Provider)

A company providing WebTV with an Internet information feed.

IP (Internet Protocol)

A networking protocol, used to pass information in discrete units called packets, from one machine on the Internet to another. See also TCP/IP and TCP

ISP (Internet Service Provider)

A company providing the hardware and software point-of-presence (POP) permitting the client to connect to the WebTV service.

Kid-friendly

Kid-friendly users ONLY have access to sites on the World Wide Web that SurfWatch deems acceptable (see also the "white list" on page 209). ALL other sites are blocked. *(Note: are whole sites blocked, or are only individual pages blocked?)* This designation replaces the earlier "Yahooligans-restricted" description of this user category.

macros

In Remedy, macros are client-side action sequences, triggered by the user to perform a series of operations quickly and efficiently. Macros can be recorded by any user of the Remedy application.

MIME (Multipurpose Internet Mail Exchange)

A standardized method for passing non-text information in email.

NPA

area code (the first 3 digits of a 10-digit number in the USA).

NXX

prefix a.k.a. exchange (the first 3 digits of a 7-digit number in the USA).

PAP

See "Password Authentication Protocol" on page 207.

Password Authentication Protocol

Abbreviated PAP. Only used for non-scripted logins.

"phase 2" configuration options

"phase 2" configuration options are those components of the */src/config/arcadia.cf* configuration file that can be modified without closing the service to users.

POP (Point-of-Presence)

The local access point to the Internet.

PPP (Point-to-Point Protocol)

A communication protocol that allows a user to connect a modem to the Internet. PPP uses TCP/IP to pass information to and receive information from the Internet.

RAID (Redundant Array of Inexpensive Disks)

A RAID is a magnetic storage device consisting of a number of identical hard drives, all storing the same information. The controller, part of the RAID, ensures that if something happens to the data on one disk, the others maintain the original information.

Rate Center

When you place a call, the cost of the call is based on the distance between your rate center and the rate center of the place you're calling.

RBOC

Regional Bell Operating Company, e.g. the local PacBell office.

Schema

In Remedy applications, a database and some combination of Remedy objects.

Server

A server can either be a machine or a software package running on a particular machine. In either case, the server acts to provide a particular service (for example, email or access to the World Wide Web) to a client. One server machine can run multiple types of server software, providing a client with a large number of services.

Service

A function provided to a client by a server. Example services include email, World

Wide Web access, and so forth.

Silicon Serial Number

A unique serial number that is hard-wired into the WebTV box. Useful for identification purposes of individual units, particularly for purposes of tracing information in the customer database of the log files.

Subscriber

The person who signs up for WebTV service.

SurfWatch-restricted users

SurfWatch-restricted users have access to the entirety of the World Wide Web, EXCEPT for a set of pages (see also the “black list” on page 205) that contain inappropriate material, as defined by SurfWatch.

Telco

telephone company

Tellyscripts

The programs used by the WebTV box to control its functions and customize operating parameters. For more information on tellyscripts, see Andy Rubin's "WebTV Communications Technology" document.

TCP (Transmission Control Protocol)

A networking protocol, used to pass information in discrete units called packets, from one machine on the Internet to another. See also TCP/IP and IP

TCP/IP

A suite, or collection, of communication protocols used by machines on the Internet to communicate with each other and pass information. See also IP and TCP.

tmpl

tmpl is HTML-like language with some additional extensions that can substitute the user's name. It is analogous to HTML in the same way WTVP is analogous to HTTP.

URL (Uniform Resource Locator)

A URL is a standardized, universal format for pointing to the location of a file on a local or remote system. They typically have the format “http://machine.domain.type/directory1/../directoryn/filename”. On the World Wide Web, they are used as pointers to Web pages.

User

The person currently using the WebTV service.

User Interface

The part of any software that is presented to the user and allows the user to enter data, make decisions, or otherwise influence the software or hardware's actions.

V&H

Vertical & Horizontal coordinates of the rate center, used for distance calculations. It's possible, but complicated, to transform these into latitude & longitude.

WebTV Internet Terminal

The WebTV hardware, software, and user interface (home page).

WebTV Network

The service that provide e-mail and Internet access.

WebTV HTML interface

The software inside the Internet Terminal includes an HTML interpreter that can handle tags and attributes from HTML standards—HTML 2.0 and HTML 3.2—as well as many extensions offered by Netscape® Navigator™ 3.0 and Microsoft® Internet Explorer™ 3.0. The combination of standard HTML support with the WebTV extensions**.**

white list

A list of the ONLY sites that Yahooligans-restricted users can access.

WTVP (WebTV Protocol)

A modified form of HTTP for use by WebTV devices.

Yahooligans-restricted users

Yahooligans-restricted users ONLY have access to sites on the World Wide Web that SurfWatch deems acceptable (see also the “white list” on page 209). ALL other sites are blocked. *(Note: are whole sites blocked, or are only individual pages blocked?)*

Index

A

access code 20
ANI 70
    definition of 25
    general connectivity schematic 25
    server problems 26
    troubleshooting protocols 25
audible dialing 19
automatic number identification
    see ANI 25

B

billing 17
    verification issues 46
    verification of IAP 35
black list 30
black list and white list additions 31
bulkmail 56

C

Cabletron SPECTRUM 34
Cabletron Spectrum 34
cat-event 58
clientinfo 60
CMR 70
comparepops 36
computil 62
configuration file 47
connection errors 20
contacts
    Customer Care 15
content
    delivery of new 50
CS-Ticket 66, 68
    schematic 67
customer
    problems 16
Customer Care
    and customer contact 15
    center 14
    contact volume 15
    contacts 15
    customer requests 16
    duties 1
    educating consumers 16
    facilities 15
    hours 14
    knowledge base 70
    organization 6, 13
    training of specialists 14
customer database 70
    overview 42
Customer Master Record tool
    see CMR 70
customer support 16
customers 15
    problem users 32

D

database
    components 46
    connection configuration 51
database administration 41
database structure 45

database system
controller/disk layout 44
hardware configuration 42
hardware configuration schematic 43
RAID layout schematic 45
delegation 2
Dial Pattern Editor
see dpedit 70
dialing
infinite loop 24
Dialing Options 18
distill-log 73
downtime 51
dpedit 70
dumpcrash 74

E

Empire 34
error messages 28
escalations 69
Etherpage 75

F

facilities
Customer Care 15
fetchpops 36

I

installation problems 17
installpops 36
IR keyboard
shortcuts 29

J

jumpstarting 51

K

Kidfriendly 30
KidsProtection Administration 7
knowledge base 16, 70

L

log files
problems in 62

M

mail merge 56
maintenance 35
makepops 36
messages
normal connection 21
normal preregistration connection 21
Monitord 35
monitord 75
monitoring
see network monitoring 34

N

NetApp 39
network monitoring 34
Network Systems 41
duties 2
responsibilities 41
staffing 9

O

OFA 45
Operations Group
delegation schematic 3
duties 1
organization 5
organizational chart 6
staffing 5
Oracle Flexible Architecture
see OFA 45

P

PhoneDB 36
phonetool 16
POP
balancing 37
listings 16
Postmaster
duties 46
preregistration 21
preregistrationd 26
problem notification 1

R

radiusd 26
Remedy
applications 69
remote control 29

S

sar 34
secret codes 18
sendmail 46
service groups 47

Service Operations 7, 33
Center 33
duties 33
IAP/ISP Operations 33
shutdown procedure 40
shutting down the service 39
staffing 8
Unix System Operations 33
WebTV Network Operations 34
Service Operations Center
duties 2
service request system
see SRS 66
service roll-out 47
shut down 39
Simple Network Management Protocol
see SNMP 34
SNMP 34
SNMP traps 34
SRS 66
workflow 67
workflow diagram, incoming telephone calls 68
workflow diagram, incoming/outgoing e-mail 68
staffing xiii, 5
SurfWatch 30
system activity reporter 34
see sar 34
system backups 35

T

terms of service 1
tools 55
bulkmail 56
cat-event 58
clientinfo 60
CMR 70
computil 62
CS-Ticket 66
distill-log 73
dpedit 70
dumpcrash 74
troubleshooting 17
connectivity problems 18
Dialing Options menu 18
dialing problem workarounds 19
failure to connect 21

U

updates xiii

V

version
client xiii
service xiii

W

WebTV Network
service rollout 47
WebTV Tricks 51
white list 30